

User Guide — nsight-systems

329–419 minutes

NVIDIA Nsight Systems user guide

Profiling from the CLI[#]

Installing the CLI on Your Target[#]

The Nsight Systems CLI provides a simple interface to collect on a target without using the GUI. The collected data can then be copied to any system and analyzed later.

The CLI is distributed in the Target directory of the standard Nsight Systems download package.

If you wish to run the CLI without root (recommended mode), you will want to install in a directory where you have full access.

Note

You must run the CLI on Windows as administrator.

Command Line Options[#]

The Nsight Systems command lines can have one of two forms:

`nsys [global_option]`

or

`nsys [command_switch][optional command_switch_options][application] [optional application_options]`

All command line options are case-sensitive. For command switch options, when short options are used, the parameters should follow the switch after a space; e.g., `-s process-tree`. When long options are used, the switch should be followed by an equal sign and then the parameter(s); e.g., `--sample=process-tree`.

For this version of Nsight Systems, if you launch a process from the command line to begin analysis, the launched process will be terminated when collection is complete, including runs with `--duration` set, unless the user specifies the `--kill none` option (details below). The exception is that if the user uses NVTX, `cudaProfilerStart/Stop`, or hotkeys to control the duration, the application will continue unless `--kill` is set.

The Nsight Systems CLI supports concurrent analysis by using sessions. Each Nsight Systems session is defined by a sequence of CLI commands that define one or more collections (e.g., when and what data is collected). A session begins with either a start, launch, or profile command. A session ends with a shutdown command, when a profile command terminates, or, if requested, when all the process tree(s) launched in the session exit. Multiple sessions can run concurrently on the same system.

CLI Global Options#

Short	Long	Description
-h	--help	Help message providing information about available command switches and their options.
-v	--version	Output Nsight Systems CLI version information.

CLI Command Switches#

The Nsight Systems command line interface can be used in two modes. You may launch your application and begin analysis with options specified to the `nsys profile` command. Alternatively, you can control the launch of an

application and data collection using interactive CLI commands.

Command	Description
analyze	Post process existing Nsight Systems result, either in .nsys-rep or SQLite format, to generate expert systems report.
cancel	Cancels an existing collection started in interactive mode. All data already collected in the current collection is discarded.
export	Generates an export file from an existing .nsys-rep file. For more information about the exported formats see the /documentation/nsys-exporter directory in your Nsight Systems installation directory.
launch	In interactive mode, launches an application in an environment that supports the requested options. The launch command can be executed before or after a start command.
nvprof	Special option to help with transition from legacy NVIDIA nvprof tool. Calling nsys nvprof [options] will provide the best available translation of nvprof [options] See the Migrating from NVIDIA nvprof topic for details. No additional functionality of nsys will be available when using this option.
profile	A fully formed profiling description requiring and accepting no further input. The command switch options used (see below table) determine when the collection starts, stops, what collectors are used (e.g., API trace, IP sampling, etc.), what processes are monitored, etc.
recipe	Post process multiple existing Nsight Systems results to generate statistical information and create various plots. See the Multi-Report Analysis topic for details.

Command	Description
sessions	Gives information about all sessions running on the system.
shutdown	Disconnects the CLI process from the launched application and forces the CLI process to exit. If a collection is pending or active, it is canceled.
start	Starts a collection in interactive mode. The start command can be executed before or after a launch command.
stats	Post process existing Nsight Systems result, either in .nsys-rep or SQLite format, to generate statistical information.
status	Reports on the status of a CLI-based collection or the suitability of the profiling environment.
stop	Stops a collection that was started in interactive mode. When executed, all active collections stop, the CLI process terminates but the application continues running.

CLI Profile Command Switch Options#

After choosing the `profile` command switch, the following options are available. Usage:

`nsys [global-options] profile [options] [application] [application-arguments]`

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--accelerator-trace</code>	none, tegra-accelerators	none	Collect other accelerators workload trace from the h Available in Nsight Systems Embedded Platforms E

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--auto-report-name</code>	true, false	false	Derive report file name from collected data using default graphics application. Format: [Process Name] [Core Resolution] [Graphics API] Timestamp . automatically generate report file names.
-b	<code>--backtrace</code>	auto,fp,lbr,dwarf, none		Select the backtrace method to use while sampling. Intel(c) Corporation's Last Branch Record registers, Intel(c) CPUs codenamed Haswell and later. The option and assumes that frame pointers were enabled during option dwarf uses DWARF's CFI (Call Frame Information) value to none can reduce collection overhead.
-c	<code>--capture-range</code>	none, cudaProfilerApi, hotkey, nvtx	none	When <code>--capture-range</code> is used, profiling will start at appropriate start API or hotkey is invoked. If <code>--capture-range</code> none, start/stop API calls and hotkeys will be ignored. Note Hotkey works for graphic applications only.
	<code>--capture-range-end</code>	none, stop, stop-shutdown, repeat[:N], repeat-shutdown:N		Default is stop-shutdown. Specify the desired behavior when capture range ends. Applicable only when used along with the <code>--capture-range</code> option. If none, capture range end will be ignored. If stop, stop at the capture range end. Any subsequent capture range end will be ignored. The target app will continue running. If stop-shutdown, collection will stop at the capture range end and session will be shutdown. If repeat[:N], collection will stop at capture range end and repeat the capture range for N times.

Sh	Long	Possible Parameters	Default	Switch Description
				capture ranges will trigger more collections. The option specifies the maximum number of capture ranges to be honored. Any additional ranges will be ignored once N capture ranges are collected. For repeat-shutdown:N, the same behavior as repeat:N but shutdown after N ranges. For stop-shutdown and shutdown:N, as always, use the --kill option to ensure the target app should be terminated when shutting down.
	--clock-frequency-changes	true, false	false	Collect clock frequency changes. Available only in Nsight Systems Embedded Platforms Edition and Arm server (SBS).
	--command-file	< filename >	none	Open a file that contains profile switches and parse additional switches on the command line will override. This flag can be specified more than once.
	--cpu-cluster-events	0x16, 0x17, ..., none	none	Collect per-cluster Uncore PMU counters. Multiple values can be specified separated by commas only (no spaces). Use the --events=help switch to see the full list of values. Arm Systems Embedded Platforms Edition only.
	--cpu-core-events (Nsight Systems Embedded Platforms Edition)	0x11, 0x13, ..., none	none	Collect per-core PMU counters. Multiple values can be specified separated by commas only (no spaces). Use the --cpu-core switch to see the full list of values.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--cpu-core-events</code> (not Nsight Systems Embedded Platforms Edition)	'help' or the end users selected events in the format 'x,y'	'2'	Default is Instructions Retired. Select the CPU Core the <code>--cpu-core-events=help</code> switch to see the the number of events that can be collected simultaneously can be selected, separated by commas only (no spaces). Use the <code>event-sample</code> switch to enable.
	<code>--cpu-core-metrics</code>	0,1,2,...,none	none	Collect metrics on the CPU core. Multiple values can be separated by commas only (no spaces). Use the <code>--metrics=help</code> switch to see the full list of values. Use the <code>sample</code> switch to enable. Note Only available on Grace.
	<code>--cpu-socket-events</code> (Nsight Systems Embedded Platforms Edition)	0x2a,0x2c,...,none	none	Collect per-socket Uncore PMU counters. Multiple values can be separated by commas only (no spaces). Use the <code>--events=help</code> switch to see the full list of values. Available on Nsight Systems Embedded Platforms Edition only.
	<code>--cpu-socket-events</code> (not Nsight Systems Embedded Platforms Edition)	'help' or the end users selected events in the format 'x,y'	none	Select the Uncore CPU Socket events to sample. Use the <code>socket-events=help</code> switch to see the full list of events that can be collected simultaneously. Multiple events can be selected, separated by commas only (no spaces). Use the <code>sample</code> switch to enable.
	<code>--cpu-socket-</code>	0,1,2,...,none	none	Collect Uncore metrics on the CPU socket. Multiple

Sh	Long	Possible Parameters	Default	Switch Description
	<code>metrics</code>			separated by commas only (no spaces). Use the <code>--metrics=help</code> switch to see the full list of values. <code>sample</code> switch to enable. Note Only available on Grace.
	<code>--cpuctxsw</code>	process-tree, system-wide, none		Default is process-tree. Trace OS thread scheduling disable tracing CPU context switches. Depending on values may require admin or root privileges. Note If the <code>--sample</code> switch is set to a value other than <code>none</code>, the setting is hardcoded to the same value as the <code>--sample</code> switch. <code>sample=none</code> and a target application is launched with <code>process-tree</code>, otherwise the default is <code>none</code>. Related to <code>trigger=perf</code> switch in Nsight Systems Embedded.
	<code>--cuda-event-trace</code>	auto, true, false	false	Trace CUDA Event completion on the device side, and support among CUDA Event APIs. Applicable only if <code>cudaEventDisableTiming</code> is enabled. "CUDA Event" refers to the synchronization (cudaEventRecord, cudaStreamWaitEvent etc.). Enabling this switch will increase runtime overhead and the likelihood of false positives. <code>CUDA Streams</code>, similar to CUDA Event's timing function. <code>cudaEventDisableTiming</code> is not disabled. <code>auto</code> will enable <code>trace</code> if a target process has <code>CUDA_DEVICE_MAX_COMPUTE_CAPABILITY</code>.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--cuda-flush-interval</code>	milliseconds		Set the interval when buffered CUDA data is automatically saved in milliseconds. The CUDA data buffer saves may overflow. Buffer save behavior can be controlled with this switch. If the flush interval is set to 0 on systems running CUDA 11.0 or later, buffers are saved when they fill. If a flush interval is set to a non-zero value on older systems, buffers are saved only when the flush interval expires and the profiler runs out of available buffers. If a flush interval is set and the profiler runs out of available buffers, additional buffers will be allocated and saved. Setting a flush interval can reduce buffer save overhead and memory use by the profiler. If the flush interval is set to 0 on older versions of CUDA, buffers are saved at the end of the run. If the profiler runs out of available buffers, additional buffers will be allocated and saved. If a flush interval is set to a non-zero value on older versions of CUDA, buffers are saved when the flush interval expires. A cuCtxPushCurrent call must be inserted into the workflow before the buffers are saved. Setting a flush interval can reduce application overhead. In this case, setting a flush interval can reduce memory use by the profiler but may increase save overhead. If the flush interval is set to 0, an interval of 10 seconds is recommended. If the flush interval is set to a non-zero value, an interval of 10000 for Nsight Systems Embedded Platforms Edition is recommended.
	<code>--cuda-graph-trace</code>	graph, node	graph	If graph is selected, CUDA graphs will be traced as activities will not be collected. This will reduce overhead. If node is selected, activities will be collected, but CUDA graphs will not be collected. This requires CUDA driver version 515.43 or higher. If no value is specified, activities will be collected, but CUDA graphs will not be collected. This may cause significant runtime overhead. Default is graph.

Sh	Long	Possible Parameters	Default	Switch Description
				otherwise the default is node.
	--cuda-memory-usage	true, false	false	Track the GPU memory usage by CUDA kernels. A CUDA tracing is enabled. Note This feature may cause significant runtime overhead.
	--cuda-trace-all-apis	true, false	false	By default, Nsys skips some CUDA APIs that are not relevant to performance analysis. If enabled, Nsys will trace all CUDA APIs, including those less relevant to performance analysis. Note that this may cause significant runtime overhead. Default is 'false'.
	--cuda-um-cpu-page-faults	true, false	false	This switch tracks the page faults that occur when a memory page that resides on the device. Note that this may cause significant runtime overhead. Not available on Nsight Systems Platforms Edition.
	--cuda-um-gpu-page-faults	true, false	false	This switch tracks the page faults that occur when a memory page that resides on the host. Note that this may cause significant runtime overhead. Not available on Nsight Systems Platforms Edition.
	--cudabacktrace	all, none, kernel, memory, sync, other	none	When tracing CUDA APIs, enable the collection of a backtrace when a CUDA API is invoked. Significant runtime overhead may be incurred. Backtraces can be combined using ' , '. Each value except none may be followed by a threshold after ' : '. The threshold is duration, in nanoseconds.

Sh	Long	Possible Parameters	Default	Switch Description
				APIs must execute before backtraces are collected. The default value for each threshold is 1000ns (1us) Note CPU sampling must be enabled.
-y	--delay	< seconds >	0	Collection start delay in seconds.
-d	--duration	< seconds >	NA	Collection duration in seconds; duration must be greater than 0. The process launched process will be terminated when the specification expires unless the user specifies the --kill none
	--duration-frames	60 <= integer		Stop the recording session after this many frames have been recorded. When it is selected, command cannot include any of the other options specified, the default is disabled.
	--dx-force-declare-adapter-removal-support	true, false	false	The Nsight Systems trace initialization involves creating and then discarding it. Enabling this flag makes a call to <code>DXGIDeclareAdapterRemovalSupport()</code> before the trace is created. Requires DX11 or DX12 trace to be enabled.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--dx12-gpu-workload</code>	true, false, individual, batch, none		Default is individual. If individual or true, trace each workload's activity individually. If batch, trace DX12 workloads' ExecuteCommandLists call batches. If none or false, trace workloads' GPU activity. Note that this switch is applicable only if <code>--trace=dx12</code> is specified. This option is only supported on Windows targets.
	<code>--dx12-wait-calls</code>	true, false	true	If true, trace wait calls that block on fences for DX12. This switch is applicable only when <code>--trace=dx12</code> is specified. This option is only supported on Windows targets.
	<code>--xhv-vm-symbols</code>	<filepath kernel_symbols.json>	none	XHV sampling config file. Available in Nsight System Edition only.
-e	<code>--env-var</code>	A=B	NA	Set environment variable(s) for the application process. Environment variables should be defined as A=B. Multiple variables can be specified as A=B,C=D.
	<code>--enable</code>	<plugin_name> [,arg1,arg2,...]	NA	Use the specified plugin. The option can be specified multiple times to enable multiple plugins. Plugin arguments are separated by commas (no spaces). Commas can be escaped with a backslash. The backslash itself can be escaped by another backslash \\\\. To specify a plugin argument, enclose the argument in double quotes. To disable all plugins, use the <code>--enable=help</code> command.
	<code>--etw-provider</code>	"<name>,<guid>", or	none	Add custom ETW trace provider(s). If you want to specify multiple providers, use the <code>--etw-provider=provider1,provider2</code> command.

Sh	Long	Possible Parameters	Default	Switch Description
		path to JSON file		than Name and GUID, provide a JSON configuration below. This switch can be used multiple times to add Note: Only available for Windows targets.
	--event-sample	system-wide, none	none	Use the --cpu-core-events=help and the --cpu-core switches to see the full list of events. If event sampling events are selected, the CPU Core event 'Instruction by default. Not available on Nsight Systems Embedded
	--event-sampling-interval	Integers from 1 to 1000 milliseconds	10	The interval between each event sample collection. sampling interval is 1 mSec. Maximum event sampling mSec. Not available in Nsight Systems Embedded
	--export	arrow, arrowdir, hdf, json, parquetdir, sqlite, text, none	none	Create additional output file(s) based on the data collection be given more than once. Warning If the collection captures a large amount of data, creation take several minutes to complete.
	--flush-on-cudaprofilerstop	true, false	true	If set to true, any call to cudaProfilerStop() will flush buffers to be flushed. Note that the CUDA trace buffers the collection ends, regardless of the value of this switch
-f	--force-overwrite	true, false	false	If true, overwrite all existing result files with same output .sqlite, .h5, .txt, .json, .arrows, _arrowdir, _pqtdir).

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--ftrace</code>			Collect ftrace events. Argument should list events to event1, subsystem2/event2. Requires root. No ftrace by default.
	<code>--ftrace-keep-user-config</code>			Skip initial ftrace setup and collect already configured the ftrace configuration.
	<code>--gpu-metrics-devices</code>	GPU ID, help, all, none	none	Collect GPU Metrics from specified devices. Determine <code>--gpu-metrics-devices=help</code> switch.
	<code>--gpu-metrics-frequency</code>	integer	10000	Specify GPU Metrics sampling frequency. Minimum 10 (Hz). Maximum supported frequency is 200000 (Hz).
	<code>--gpu-metrics-set</code>	alias, file:<file name>		Specify metric set for GPU Metrics. The argument name aliases reported by <code>--gpu-metrics-set=help</code> or metric config file prefixed by <code>file:</code> . The default is the set that supports all selected GPUs.
	<code>--gpu-video-device</code>	help, <id1,id2,...>, all, none	none	Analyze video devices. <code>--help</code> gives a list of supported and unsupported devices and IDs. <id1,id2,...> to analyze specified devices only.
	<code>--gpuctxsw</code>	true,false	false	Trace GPU context switches. See the GPU Context Switches section.
	<code>--help</code>	<tag>	none	Print the help message. The option can take one option to be used as a tag. If a tag is provided, only options related to the tag are shown.

Sh	Long	Possible Parameters	Default	Switch Description
				printed.
	--hotkey-capture	'F1' to 'F12'	'F12'	Hotkey to trigger the profiling session. Note that this when --capture-range=hotkey is specified.
	--ib-net-info-devices	<NIC names>	none	A comma-separated list of NIC names. The NICs will be used for networks discovery. This option creates the files for collecting network information. Example value: mlx5_0,mlx5_1. If the --ib-net-info-output option is set then Nsight will write network information at that path. Otherwise it will be written to the current directory and will be discarded after processing. If more than one path is specified, only the last network information file will be used. Note that this option should not be used together with the --ib-net-info-files option.
	--ib-net-info-files	<file paths>	none	A comma-separated list of file paths. Paths of an existing files, containing networks information data. Nsight will read networks' information from these files. Don't use ~ as a path. Note that this option should not be used together with the --ib-net-info-devices option.
	--ib-net-info-output	<directory path>	none	Sets the path of a directory into which ibdiagnet network information will be written. Use this option together with the --ib-net-info-files option. Don't use ~ alias within the path.
	--ib-switch-	<IB switch GUIDs>	none	A comma-separated list of InfiniBand switch GUIDs

Sh	Long	Possible Parameters	Default	Switch Description
	congestion - device			switch congestion events from switches identified by <code>--ib-switch-congestion-nic-device</code> . This switch can be used multiple times. System scope. <code>--ib-switch-congestion-nic-device</code> , <code>--ib-switch-congestion-percent</code> , and <code>--ib-switch-congestion-threshold-high</code> switches to further control how congestion events are reported.
	<code>--ib-switch-congestion-nic-device</code>	<NIC name>	none	The name of the NIC (HCA) through which InfiniBand is accessed. By default, the first active NIC will be used. The NIC's name is via the <code>ibnetdiscover --Hca_1 \$(hostname)</code> command. Example usage: <code>--ib-switch-congestion-nic-device=mlx5_3</code>
	<code>--ib-switch-congestion-percent</code>	1 <= integer <= 100	50	Percent of InfiniBand switch congestion events to be reported. <code>--ib-switch-congestion-percent</code> enables reducing the network bandwidth consumed by congestion events.
	<code>--ib-switch-congestion-threshold-high</code>	1 < integer <= 1023	75	High threshold percentage for InfiniBand switch egress congestion. Before a packet leaves an InfiniBand switch, it is stored in the switch's buffer. The buffer's size is checked and if it exceeds the specified percentage, a congestion event is reported. The percentage can be greater than 100.
	<code>--ib-switch-metrics-device</code>	<IB switch GUIDs>	none	A comma-separated list of InfiniBand switch GUIDs to monitor for congestion on the specified InfiniBand switches. This switch can be used multiple times. System scope.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--ib-switch-metrics-nic-device</code>	<NIC name>	none	The name of the NIC (HCA) through which InfiniBand is accessed for performance metrics. By default, the file <code>/dev/infiniband/rdma_vend</code> is used. One way to find a NIC's name is via the <code>ibstat</code> command. Example usage: <code>--ib-switch-metrics-nic-device=mlx5_0</code>
-n	<code>--inherit-environment</code>	true, false	true	When true, the current environment variables and their values will be specified for the launched process. When false, the tool's environment variables will be specified for the launched process.
	<code>--injection-use-detours</code>	true, false	true	Use detours for injection. If false, process injection via Windows hooks which allows it to bypass anti-cheat software.
	<code>--isr</code>	true, false	false	Trace Interrupt Service Routines (ISRs) and Deferred Procedure Calls (DPCs). Requires administrative privileges. Available on Windows devices.
	<code>--kill</code>	none, sigkill, sigterm, signal number	sigterm	Send signal to the target application's process group. <code>duration</code> or range markers.
	<code>--mpi-impl</code>	openmpi, mpich	openmpi	When using <code>--trace=mpi</code> to trace MPI APIs, use <code>--mpi-impl</code> to specify which MPI implementation the application is using. If no implementation is specified, nsys tries to automatically detect the implementation. If this fails, openmpi is used. <code>--mpi-impl</code> without <code>--trace=mpi</code> is not supported.
	<code>--nic-metrics</code>	true, false	false	Collect metrics from supported NIC/HCA devices. Supported devices are listed in the <code>ibstat</code> command output.

Sh	Long	Possible Parameters	Default	Switch Description
				available on Nsight Systems Embedded Platforms
-p	--nvtx-capture	range@domain, range, range@*	none	Specify NVTX range and domain to trigger the profile. This switch is applicable only when used along with --capture
	--nvtx-domain-exclude	default, <domain_names>		Choose to exclude NVTX events from a comma separated list of domains. default excludes NVTX events without a domain name or commas in a domain name must be escaped. Note Only one of --nvtx-domain-include and --nvtx-domain-exclude can be used. This option is only applicable when --capture is specified.
	--nvtx-domain-include	default, <domain_names>		Choose to only include NVTX events from a comma separated list of domains. default filters the NVTX default domain name or commas in a domain name must be escaped. Note Only one of --nvtx-domain-include and --nvtx-domain-exclude can be used. This option is only applicable when --capture is specified.
	--python-functions-trace	<json_file>		Specify the path to the JSON file containing the required annotations.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--opengl-gpu-workload</code>	true, false	true	If true, trace the OpenGL workloads' GPU activity. Not applicable only when <code>--trace=opengl</code> is specified.
	<code>--os-events</code>	'help' or the end users selected events in the format 'x,y'		Select the OS events to sample. Use the <code>--os-events</code> switch to see the full list of events. Multiple values can be selected separated by commas only (no spaces). Use the <code>--event-sample</code> switch to set the sample rate. Not available on Nsight Systems Embedded Platform.
	<code>--osrt-backtrace-depth</code>	integer	24	Set the depth for the backtraces collected for OS runtime.
	<code>--osrt-backtrace-stack-size</code>	integer	6144	Set the stack dump size, in bytes, to generate backtraces for libraries calls.
	<code>--osrt-backtrace-threshold</code>	nanoseconds	80000	Set the duration, in nanoseconds, that all OS runtime functions execute before backtraces are collected.
	<code>--osrt-threshold</code>	<nanoseconds>	1000 ns	Set the duration, in nanoseconds, that Operating System APIs must execute before they are traced. Values smaller than 1000 may cause significant overhead and result in empty trace files. Note This setting is ignored for APIs that interact with files if file access is set to true.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--osrt-file-access</code>	true, false	false	Collect file access data when tracing Operating Sys that interact with files. Note When this setting is set to true the <code>--osrt-thres</code> for APIs that interact with files.
-o	<code>--output</code>	< filename >	report#	Set the report file name. Any %q{ENV_VAR} pattern substituted with the value of the environment variable. filename will be substituted with the hostname of the in the filename will be substituted with the PID of the PID of the root process if there is a process tree. Any filename will be substituted with %. Default is report# rep,.sqlite,.h5,.txt,.arrows,_arwdir,_pqtdir,.json} in th
	<code>--process-scope</code>	main, process-tree, system-wide	main	Select which process(es) to trace. Available in Nsigl Platforms Edition only. Nsight Systems Workstation system-wide in this version of the tool.
	<code>--python-backtrace</code>	cuda, none	none	Collect Python backtrace event when tracing the se option is supported on Arm server (SBSA) platforms. Note: tracing and backtraces of the selected API an be enabled. For example, <code>--cudabackt race mus python-backt race=cuda</code> .
	<code>--python-</code>	true, false	false	Collect Python backtrace sampling events. This opt

Sh	Long	Possible Parameters	Default	Switch Description
	sampling			server (SBSA) platforms, x86 Linux and Windows to profiling Python-only workflows, consider disabling the option to reduce overhead.
	--python-sampling-frequency	1 < integers < 2000	1000	Specify the Python sampling frequency. The minimum is 1Hz. The maximum supported frequency is 2KHz if the --python-sampling option is set to false.
	--pytorch	autograd-nvtx, autograd-shapes-nvtx, functions-trace, none	none	Enable automatic annotations of PyTorch functions.
	--dask	functions-trace, none	none	Enable automatic annotations of Dask functions
	--qnx-kernel-events	class/event,event, class/event:mode, class:mode,help,none	none	Multiple values can be selected, separated by comma. See the --qnx-kernel-events-mode switch description for format. Use the --qnx-kernel-events=help switch for values. Example: --qnx-kernel-events=8/1:system:wide,_NT0_TRACE_THR,_NT0_TRACE_KERCALLENTER/___KER_BAD,_NT0_TRACE_COMM,13. Collect Q
	--qnx-kernel-events-mode	system,process,fast,wide		Default is system:fast. Values are separated by a colon. system and process cannot be specified at the same time. wide cannot be specified at the same time. Please

Sh	Long	Possible Parameters	Default	Switch Description
				documentation to determine when to select the fast mode. Specify the default mode for QNX kernel events collection.
	--resolve-symbols	true,false	true	Resolve symbols of captured samples and backtraces.
	--retain-etw-files	true, false	false	Retain ETW files generated by the trace, merge and output directory.
	--run-as	< username >	none	Run the target application as the specified username. If no username is specified, the target application will be run by the same user as the user running the tool. Not available for Windows targets. Available for Linux targets only.
-s	--sample	process-tree, system-wide, xhv, xhv-system-wide, none		Default is process-tree. Select how to collect CPU information. If none is selected, CPU sampling is disabled. Depending on the platform, some values may require admin or root privileges. Select system-wide to enable Cross-Hypervisor (XHV) sampling, which requires root privileges. If a target application is launched, the default is process-tree; otherwise, the default is none. Note system-wide is not available on all platforms. Note If set to none, CPU context switch data will still be collected if the cputctxsw switch is set to none.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--samples-per-backtrace</code>	integer <= 32	1	The number of CPU IP samples collected for every sample collected. For example, if set to 4, on the fourth sample collected, a backtrace will also be collected. Lower values result in a smaller amount of data collected. Higher values can reduce the number of CPU IP samples dropped and reduce the number of CPU IP samples dropped. If samples are collected, the default is 4, otherwise the default is 1. This option is available on Nsight Systems Embedded Platforms Edition targets.
	<code>--sampling-frequency</code>	100 < integers < 8000	1000	Specify the sampling/backtracing frequency. The minimum frequency is 100 Hz. The maximum supported frequency is 8000 Hz. This option is supported only on QNX, Linux for Tegra, and Windows for Tegra.
	<code>--sampling-period</code> (Nsight Systems Embedded Platforms Edition)	integer		Default is determined dynamically. The number of CPU cycles counted before a CPU instruction pointer (IP) sample is collected. If configured, backtraces may also be collected. The smaller the period, the higher the sampling rate. Note that smaller periods increase overhead and significantly increase the size of the data collected. Requires the <code>--sampling-trigger=perf</code> switch.
	<code>--sampling-period</code> (not Nsight Systems Embedded Platforms Edition)	integer		Default is determined dynamically. The number of CPU cycles counted before a CPU instruction pointer (IP) sample is collected. The period for the collection of a sample is determined dynamically. On non-ARM based platforms, it will probably be "Reference Cycles". On ARM based platforms, "CPU Cycles". If configured, backtraces may also be collected.

Sh	Long	Possible Parameters	Default	Switch Description
				The smaller the sampling period, the . higher the sa smaller sampling periods will increase overhead an the size of the result file(s). This option is available o
	--sampling - trigger	timer, sched, perf, cuda	timer sched	Specify backtrace collection trigger. Multiple APIs ca separated by commas only (no spaces). Available o Embedded Platforms Edition targets only.
	--session-new	[a-Z][0-9,a-Z,spaces]		Default is profile-<id>-<application>. Name the sess command. Name must start with an alphabetical ch printable or space characters. Any %q{ENV_VAR} p substituted with the value of the environment variab substituted with the hostname of the system. Any %s substituted with %.
-w	--show-output	true, false	true	If true, send the target process's stdout and stderr st console and stdout/stderr files which are added to th only send the target process stdout and stderr strea files which are added to the report file.
	--soc-metrics	true,false	false	Collect SoC Metrics. Available in Nsight Systems E Edition only.
	--soc-metrics - frequency	integer	10000	Specify SoC Metrics sampling frequency. Minimum '100' (Hz). Maximum supported frequency is '10000 Nsight Systems Embedded Platforms Edition only.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--soc-metrics-set</code>	alias, file:<file name>		Specify metric set for SoC Metrics. The argument may be an alias or a path to a metric config file prefixed by <code>file:</code> . Available in Nsight Embedded Platforms Edition only.
	<code>--start-frame-index</code>	1 <= integer		Start the recording session when the frame index reaches the number preceding the start frame index. Note when used, it must include any other start options. If not specified, the collection starts at the beginning of the trace.
-Y	<code>--start-later</code>	true, false	false	Delays collection indefinitely until the nsys start command is received. This session. Enabling this option overrides the <code>--duration</code> option.
	<code>--stats</code>	true, false	false	Generate summary statistics after the collection. Warning When set to true, an SQLite database will be created to store the data. If the collection captures a large amount of data, creation of the database may take several minutes to complete.
-x	<code>--stop-on-exit</code>	true, false	true	If true, stop collecting automatically when the launch process exits or when the duration expires - whichever occurs first. If false, the collection stops only when the duration expires. Systems does not officially support runs longer than 10 minutes.
	<code>--syscall (beta)</code>	process-tree, pid-namespace, none	none	Collect system calls. The value defines the collection mode. <code>process-tree</code> makes it tracing the application processes and their child processes in the current PID namespace and its child namespaces.

Sh	Long	Possible Parameters	Default	Switch Description
				to the system-wide mode of other features).
-t	--trace	cuda, cuda-hw, cudnn cublas cublas- verbose cusparse- verbose, cudla, cudla- verbose, cusolver, opengl, cusolver- verbose, openssl- annotations openacc, openmp, osrt mpi, nvvideo, nvtx, dx11,dx11- annotations dx12,dx12- annotations tegra- accelerators, ucx, openxr, oshmem, openxr-annotations, python-gil, gds(beta) wddm, vulkan, vulkan-annotations, none	cuda opengl nvtx osrt	Select the API(s) to be traced. The osrt switch controls libraries tracing. Multiple APIs can be selected, separated by spaces (no spaces). Since OpenACC and cuXXX APIs are available on CUDA, selecting one of those APIs will automatically enable tracing of the corresponding API. For example, selecting cublas, cudla, cusparse and cusolver all have XXX-annotations available. Reflex SDK latency markers will be automatically enabled if DX or vulkan API trace is enabled. See information on the osrt switch below if mpi is selected. If <api>-annotations is selected, the corresponding API will also be traced. If the none option is selected, no APIs are traced and no other API can be selected. Note cuDNN is not available on Windows target. Note cuda-hw is not available for hardware before Blackwell before 11.0

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--trace-fork-before-exec</code>	true, false	false	If true, trace any child process after fork and before the <code>exec</code> functions. Beware, tracing in this interval relies on user mode and might cause your application to crash or deadlock. This option is available on Linux target platforms.
	<code>--vsync</code>	true, false	false	Collect vsync events. If collection of vsync events is enabled, <code>display_scanline</code> ftrace events will also be captured. Systems Embedded Platforms Edition only.
	<code>--vulkan-gpu-workload</code>	true, false, batch, individual, none		Default is individual. If individual or true, trace each Vulkan activity individually. If batch, trace Vulkan workloads as <code>vkQueueSubmit</code> call batches. If none or false, do not trace workloads' GPU activity. Note that this switch is applicable only if <code>trace=vulkan</code> is specified. This option is not supported on Windows targets.
	<code>--wait</code>	primary,all	all	If primary, the CLI will wait on the application process to exit. If all, the CLI will additionally wait on re-parented processes to exit.
	<code>--wddm-additional-events</code>	true, false	true	If true, collect additional range of ETW events, including allocations, sync wait and signal events, etc. Note that this option is applicable only when <code>--trace=wddm</code> is specified. This option is supported on Windows targets.
	<code>--wddm-backtraces</code>	true, false	false	If true, collect backtraces of WDDM events. Disabling this option can reduce overhead for certain target applications.

Sh	Long	Possible Parameters	Default	Switch Description
				applicable only when <code>--trace=wddm</code> is specified. supported on Windows targets.
	<code>--xhv-trace</code>	< filepath pct.json >	none	Collect hypervisor trace. Available in Nsight System Edition only.
	<code>--xhv-trace - events</code>	all, none, core, sched, irq, trap	all	Available in Nsight Systems Embedded Platforms E

CLI Analyze Command Switch Options#

The `nsys analyze` command generates and outputs a report to the terminal using expert system rules on existing results. Reports are generated from an SQLite export of a `.nsys-rep` file. If a `.nsys-rep` file is specified, Nsight Systems will look for an accompanying SQLite file and use it. If no SQLite export file exists, one will be created.

After choosing the `analyze` command switch, the following options are available. Usage:

```
nsys [global-options] analyze [options] [input-file]
```

Short	Long	Possible Parameters	Default	Switch Description
	<code>--help</code>	<tag>	none	Print the help message. The option can take one optional argument that will be used as a tag. If a tag is provided, only options relevant to the tag will be printed.

Short	Long	Possible Parameters	Default	Switch Description
-f	--format	column, table, csv, tsv, json, hdoc, htable, .		Specify the output format. The special name “.” indicates the default format for the given output. The default format for console is column, while files and process outputs default to csv. This option may be used multiple times. Multiple formats may also be specified using a comma-separated list (<name[:args...][,name[:args...]]...>). See Report Formatters for options available with each format.
	--force-export	true, false	false	Force a re-export of the SQLite file from the specified .nsys-rep file, even if an SQLite file already exists.
	--force-overwrite	true, false	false	Overwrite any existing output files.
	--help-formats	<format_name>, ALL, [none]	none	With no argument, list a summary of the available output formats. If a format name is given, a more detailed explanation of the the format is displayed. If ALL is given, a more detailed explanation of all available formats is displayed.
	--help-rules	<rule_name>, ALL, [none]	none	With no argument, list available rules with a short description. If a rule name is given, a more detailed explanation of the rule is displayed. If ALL is given, a more detailed explanation of all available rules is displayed.

Short	Long	Possible Parameters	Default	Switch Description
-o	--output	-, @<command>, <basename>, .	–	Specify the output mechanism. There are three output mechanisms: print to console, output to file, or output to command. This option may be used multiple times. Multiple outputs may also be specified using a comma-separated list. If the given output name is “-”, the output will be displayed on the console. If the output name starts with “@”, the output designates a command to run. The nsys command will be executed and the analysis output will be piped into the command. Any other output is assumed to be the base path and name for a file. If a file basename is given, the filename used will be: <basename>_<analysis&args>.<output_format>. The default base (including path) is the name of the SQLite file (as derived from the input file or --sqlite option), minus the extension. The output “.” can be used to indicate the analysis should be output to a file, and the default basename should be used. To write one or more analysis outputs to files using the default basename, use --output. If the output starts with “@”, the nsys command output is piped to the given command. The command is run, and the output is piped to the command’s stdin (standard-input). The command’s stdout and stderr remain attached to

Short	Long	Possible Parameters	Default	Switch Description
				the console, so any output will be displayed directly to the console. Be aware there are some limitations in how the command string is parsed. No shell expansions (including *, ?, [], and ~) are supported. The command cannot be piped to another command, nor redirected to a file using shell syntax. The command and command arguments are split on whitespace, and no quotes (within the command syntax) are supported. For commands that require complex command line syntax, it is suggested that the command be put into a shell script file, and the script designated as the output command.
-q	--quiet			Do not display verbose messages, only display errors.
-r	--rule	cuda_memcpy_async, cuda_memcpy_sync, cuda_memset_sync, cuda_api_sync, gpu_gaps, gpu_time_util, dx12_mem_ops	all	Specify the rules(s) to execute, including any arguments. This option may be used multiple times. Multiple rules may also be specified using a comma-separated list. See Expert Systems section and --help-rules switch for details on all rules.
	--sqlite	<file.sqlite>		Specify the SQLite export filename. If this file exists, it will be used. If this file doesn't exist (or if

Short	Long	Possible Parameters	Default	Switch Description
				<code>--force-export</code> was given) this file will be created from the specified <code>.nsys-rep</code> file before processing. This option cannot be used if the specified input file is also an SQLite file.
	<code>--timeunit</code>	nsec, nanoseconds, usec, microseconds, msec, milliseconds, seconds	nanoseconds	Set basic unit of time. The argument of the switch is matched by using the longest prefix matching. Meaning that it is not necessary to write a whole word as the switch argument. It is similar to passing a “:time=<unit>” argument to every formatter, although the formatter uses more strict naming conventions. See <code>nsys analyze --help-formats column</code> for more detailed information on unit conversion.

CLI Cancel Command Switch Options#

After choosing the `cancel` command switch, the following options are available. Usage:

`nsys [global-options] cancel [options]`

Short	Long	Possible Parameters	Default	Switch Description

Short	Long	Possible Parameters	Default	Switch Description
	--help	<tag>	none	Print the help message. The option can take one optional argument that will be used as a tag. If a tag is provided, only options relevant to the tag will be printed.
	--session	<session identifier>	none	Cancel the collection in the given session. The option argument must represent a valid session name or ID as reported by <code>nsys sessions list</code> . Any <code>%q{ENV_VAR}</code> pattern in the option argument will be substituted with the value of the environment variable. Any <code>%h</code> pattern in the option argument will be substituted with the hostname of the system. Any <code>%%</code> pattern in the option argument will be substituted with <code>%</code> .

CLI Export Command Switch Options#

After choosing the `export` command switch, the following options are available. Usage:

`nsys [global-options] export [options] [nsys-rep-file]`

Short	Long	Possible Parameters	Default	Switch Description
	--append			This option only applies to “directory of files” output formats with existing export files. If this option is given, an error will not be reported and the existing output files will not be over-written.

Short	Long	Possible Parameters	Default	Switch Description
-f	--force-overwrite	true, false	false	If true, overwrite all existing result files with same output filename (nsys-rep, SQLITE, HDF, TEXT, JSON, ARROW, ARROWDIR, PARQUETDIR).
	--help	<tag>	none	Print the help message. The option can take one optional argument that will be used as a tag. If a tag is provided, only options relevant to the tag will be printed.
	--include-blobs	true, false	false	Controls if NVTX extended payloads are exported as binary data. This option affects SQLite, Arrow, and Arrow/Parquet directory exports only.
	--include-json	true, false	false	Controls if repetitive JSON blocks are included in an export or not. Some events contain dynamically defined payloads. These payloads are often exported as JSON blocks to preserve their free-form structure. Unfortunately, blocks of JSON text are not an efficient way to represent data, and can cause the export files to become quite large. To address this, some classes of events (such as GENERIC_EVENT data) were extended to export payload data in the native export

Short	Long	Possible Parameters	Default	Switch Description
				format. For those events that have an export-native representation, this flag will enable or disable the export of the equivalent JSON blocks. Note that this does not suppress all JSON output. Some tables, like META_DATA_* tables and TARGET_INFO_* tables may contain a small number of JSON strings. This flag will not suppress those. Additionally, some classes of events (such as ETW events and NVTX events with user-defined payloads) do not have a native export representation. For events where the JSON block is the only export format, it will always be included. Please note that this flag has nothing to do with JSON exports, (i.e., --type=json), nor does it alter the JSON export output.
-l	--lazy	true, false	true	Controls if table creation is lazy or not. When true, a table will only be created when it contains data. This option will be deprecated in the future, and all exports will be non-lazy. This affects SQLite, HDF5, Arrow, and Arrow/Parquet directory exports only.

Short	Long	Possible Parameters	Default	Switch Description
-o	--output	<filename>	<inputfile.ext>	Set the .output filename. The default is the input filename with the extension for the chosen format.
-q	--quiet	true, false	false	If true, do not display progress bar.
	-- separate- strings	true,false	false	Output stored strings and thread names separately, with one value per line. This affects JSON and text output only.
-t	--type	sqlite, hdf, text, json, info, arrow, arrowdir, parquetdir	sqlite	Export format type. HDF format is supported only on x86_64 Linux and Windows.
	--tables	<pattern>[,<pattern>...]		Value is a comma-separated list of search patterns (no spaces). This option can be given more than once. If set, only tables that match one or more of the patterns will be exported. If not set, all tables will be exported. This feature applies to SQLite, HDFS, Arrow, and Arrow/Parquet directory exports only. The patterns are case-insensitive POSIX basic regular expressions. Note This is an advanced feature intended for

Short	Long	Possible Parameters	Default	Switch Description
				expert users. This option does not enforce any type of dependency or relationship between tables and will truly export only the listed tables. If partial exports are used with analytics features such as <code>nsys stats</code> or <code>nsys analyze</code> , it is the responsibility of the user to ensure all required tables are exported.
	<code>--times</code>	<code><timerange>[,<timerange>...]</code>		Value is a comma-separated list of time ranges (no spaces). This option can be given more than once. If set, only events that fall within at least one of the given ranges will be exported. If not set, all events will be exported. This feature applies to SQLite, HDFS, Arrow, and Arrow/Parquet directory exports only. Note This is an advanced feature intended for expert users. This option does not enforce any type of dependency or relationship between related events (such as CUDA launch APIs and CUDA kernel executions). If filtered exports are used with analytics features such as generated due to missing data, and unexpected or

Short	Long	Possible Parameters	Default	Switch Description
				misleading results may be generated. It is the responsibility of the user to ensure all relevant and interrelated events are exported. The format of a time-range is: [:] [<start-time>] / [<end-time>] [:] A single time range is defined by a pair of time values, separated by a slash. At least one time value is required. Any omitted time value will default to the minimum or maximum value (approximately +/- 290 years from the zero-point). The start time must be less than or equal to the end time. The time values are a series of integer or floating-point values followed by an optional unit. If no unit is given, the number is assumed to be in nanoseconds. Positive and negative values are supported, as well as scientific e notation. More than one value/unit can be given as long as there are no spaces. The units do not need to be given in any order and can even repeat. The following units are understood: ns, nsec : nanosecond us, usec : microsecond

Short	Long	Possible Parameters	Default	Switch Description
				ms, msec : millisecond s, sec : second m, min : minute (60 seconds) h, hour : hour (3600 seconds) For example, the value 1s2ms3us4ns would indicate 1,002,003,004 nanoseconds. 2ns5us2 would be 5004 nanoseconds (2 nanoseconds plus 5 microseconds plus 2 nanoseconds). A floating-point value is converted as a 64-bit double and is subject to the precision limitations of that format. By default, the time ranges have strict boundaries. The presence of a : character at the beginning and/or end of a time range makes that boundary non-strict, meaning the filtered events are allowed to cross the boundary. In essence, if both boundaries are strict, the event must fully exist <i>within</i> the defined range, but if both boundaries are non-strict, the event must exist <i>during</i> the defined range. Given the following timeline, with a single filter range (marked START and END), the given events (marked with = characters)

Short	Long	Possible Parameters	Default	Switch Description
				would be considered a match (T) or not (F), depending on the strictness of the filter's start/endboundaries. START END S/E :S/E S/E: :S/ E: ===== T T T T ===== F T F T ===== F F T T ===== F F F T ===== or ===== F F F F While many events have both a start and end time, some events only have a single timestamp. These types of events are treated as an event with a start time equal to the end time. If an event's end time is before the start time, the end time is adjusted to the start time. If used in conjunction with the <code>--ts-normalize</code> and/or <code>--ts-shift</code> options, the time filter is applied after the event's time values have been adjusted.

Short	Long	Possible Parameters	Default	Switch Description
	<code>--ts-normalize</code>	true, false	false	If true, all timestamp values in the report will be shifted to UTC wall-clock time, as defined by the UNIX epoch. This option can be used in conjunction with the <code>--ts-shift</code> option, in which case both adjustments will be applied. If this option is used to align a series of reports from a cluster or distributed system, the accuracy of the alignment is limited by the synchronization precision of the system clocks. For detailed analysis, the use of PTP or another high-precision synchronization methodology is recommended. NTP is unlikely to produce desirable results. This option only applies to SQLite, HDF5, Arrow, and Arrow/Parquet directory exports.
	<code>--ts-shift</code>	signed integer, in nanoseconds	0	If given, all timestamp values in the report will be shifted by the given amount. This option can be used in conjunction with the <code>--ts-normalize</code> option, in which case both adjustments will be applied. This option can be used to “hand-align” report files captured at different times, or reports captured on distributed systems with

Short	Long	Possible Parameters	Default	Switch Description
				poorly synchronized system clocks. This option only applies to SQLite, HDF5, Arrow, and Arrow/Parquet directory exports.

CLI Launch Command Switch Options#

After choosing the launch command switch, the following options are available. Usage:

nsys [global-options] launch [options] <application> [application-arguments]

Sh	Long	Possible Parameters	Default	Switch Description
-b	--backtrace	auto,fp,lbr,dwarf, none		Select the backtrace method to use while sampling. The Intel(c) Corporation's Last Branch Record registers, available on Intel(c) CPUs codenamed Haswell and later. The option <code>auto</code> and assumes that frame pointers were enabled during compilation. The option <code>dwarf</code> uses DWARF's CFI (Call Frame Information) value to none can reduce collection overhead.
	--clock-frequency-changes	true, false	false	Collect clock frequency changes. Available only in Nsight Embedded Platforms Edition and Arm server (SBSA) platform.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--cpu-cluster-events</code>	0x16, 0x17, ..., none	none	Collect per-cluster Uncore PMU counters. Multiple values separated by commas only (no spaces). Use the <code>--cpu-events=help</code> switch to see the full list of values. Available Systems Embedded Platforms Edition only.
	<code>--command-file</code>	<filename>	none	Open a file that contains profile switches and parse the switches. Any additional switches on the command line will override switches in the file. This flag can be specified more than once.
	<code>--cpu-core-events</code> (Nsight Systems Embedded Platforms Edition)	0x11, 0x13, ..., none	none	Collect per-core PMU counters. Multiple values can be separated by commas only (no spaces). Use the <code>--cpu-core-events=help</code> switch to see the full list of values.
	<code>--cpu-core-events</code> (not Nsight Systems Embedded Platforms Edition)	'help' or the end users selected events in the format 'x,y'	'2'	Default is Instructions Retired. Select the CPU Core events using the <code>--cpu-core-events=help</code> switch to see the full list of events. The number of events that can be collected simultaneously can be selected, separated by commas only (no spaces). Use the <code>event-sample</code> switch to enable.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--cpu-socket-events</code> (Nsight Systems Embedded Platforms Edition)	0x2a,0x2c,...,none	none	Collect per-socket Uncore PMU counters. Multiple values separated by commas only (no spaces). Use the <code>--cpu-events=help</code> switch to see the full list of values. Available Systems Embedded Platforms Edition only.
	<code>--cpu-socket-events</code> (not Nsight Systems Embedded Platforms Edition)	'help' or the end users selected events in the format 'x,y'	none	Select the Uncore CPU Socket events to sample. Use the <code>socket-events=help</code> switch to see the full list of events of events that can be collected simultaneously. Multiple events selected, separated by commas only (no spaces). Use the <code>sample</code> switch to enable.
	<code>--cpuctxsw</code>	process-tree, system-wide, none		Default is process-tree. Trace OS thread scheduling activity and disable tracing CPU context switches. Depending on the values may require admin or root privileges. Note If the <code>--sample</code> switch is set to a value other than none, the <code>cpuctxsw</code> setting is hardcoded to the same value as the <code>--sample</code> switch. If <code>sample=none</code> and a target application is launched, the <code>process-tree</code> is used, otherwise the default is none. Requires the <code>trigger=perf</code> switch in Nsight Systems Embedded Platforms Edition.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--cuda-event-trace</code>	auto, true, false	false	Trace CUDA Event completion on the device side, and support among CUDA Event APIs. Applicable only when enabled. "CUDA Event" refers to the synchronization mechanism (cudaEventRecord, cudaStreamWaitEvent etc.). Enabling this switch will increase runtime overhead and the likelihood of false device events. CUDA Streams, similar to CUDA Event's timing function, are not affected. cudaEventDisableTiming is not disabled. auto will auto-enable trace if a target process has CUDA_DEVICE_MAX_CONNECTIONS > 1.
	<code>--cuda-flush-interval</code>	milliseconds		Set the interval when buffered CUDA data is automatically flushed to the host in milliseconds. The CUDA data buffer saves may cause memory pressure. Buffer save behavior can be controlled with this switch. If the flush interval is set to 0 on systems running CUDA 11.0 or newer, buffers are saved when they fill. If a flush interval is set to a non-zero value on newer systems, buffers are saved only when the flush interval expires. If the flush interval is set and the profiler runs out of available buffers, additional buffers will be allocated as needed. Setting a flush interval can reduce buffer save overhead and memory use by the profiler. If the flush interval is set to 0 on older versions of CUDA, buffers are saved at the end of the profiling session. If the profiler runs out of available buffers, additional buffers are allocated as needed. If a flush interval is set to a non-zero value on older versions, buffers are saved when the flush interval expires. A cuCtxSync can be inserted into the workflow before the buffers are saved to reduce application overhead. In this case, setting a flush interval can reduce memory use by the profiler but may increase save overhead.

Sh	Long	Possible Parameters	Default	Switch Description
				over 30 seconds, an interval of 10 seconds is recommended. 10000 for Nsight Systems Embedded Platforms Edition.
	<code>--cuda-memory-usage</code>	true, false	false	Track the GPU memory usage by CUDA kernels. Application CUDA tracing is enabled. Note This feature may cause significant runtime overhead.
	<code>--cuda-trace-all-apis</code>	true, false	false	By default, Nsys skips some CUDA APIs that are not critical for performance analysis. If enabled, Nsys will trace all CUDA APIs, including those less relevant to performance analysis. Note that enabling this feature may cause significant runtime overhead. Default is "false".
	<code>--cuda-um-cpu-page-faults</code>	true, false	false	This switch tracks the page faults that occur when CPU accesses a memory page that resides on the device. Note that this feature may cause significant runtime overhead. Not available on Nsight Systems Embedded Platforms Edition.
	<code>--cuda-um-gpu-page-faults</code>	true, false	false	This switch tracks the page faults that occur when GPU accesses a memory page that resides on the host. Note that this feature may cause significant runtime overhead. Not available on Nsight Systems Embedded Platforms Edition.
	<code>--cudabacktrace</code>	all, none, kernel, memory, sync, other	none	When tracing CUDA APIs, enable the collection of a backtrace when a CUDA API is invoked. Significant runtime overhead may be incurred. Values may be combined using ' '. Each value except none may be

Sh	Long	Possible Parameters	Default	Switch Description
				threshold after ' : '. The threshold is duration, in nanoseconds. The threshold is the duration of APIs must execute before backtraces are collected; e.g. 1000ns (1us). The default value for each threshold is 1000ns (1us). Note CPU sampling must be enabled.
	--cuda-graph -trace	graph, node	graph	If graph is selected, CUDA graphs will be traced as a whole. If node is selected, only the activities will not be collected. This will reduce overhead. This switch requires CUDA driver version 515.43 or higher. If node is selected, only the activities will be collected, but CUDA graphs will not be traced. This may cause significant runtime overhead. Default is graph. Otherwise the default is node.
	--dx-force- declare- adapter- removal- support	true, false	false	The Nsight Systems trace initialization involves creating and then discarding it. Enabling this flag makes a call to <code>DXGIDeclareAdapterRemovalSupport()</code> before creating the trace. Requires DX11 or DX12 trace to be enabled.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--dx12-gpu-workload</code>	true, false, individual, batch, none		Default is individual. If individual or true, trace each DX12 activity individually. If batch, trace DX12 workloads' GPU ExecuteCommandLists call batches. If none or false, workloads' GPU activity. Note that this switch is applicable only when <code>--trace=dx12</code> is specified. This option is only supported on Windows targets.
	<code>--dx12-wait-calls</code>	true, false	true	If true, trace wait calls that block on fences for DX12. Not applicable only when <code>--trace=dx12</code> is specified. This option is supported on Windows targets.
-e	<code>--env-var</code>	A=B	NA	Set environment variable(s) for the application process. Environment variables should be defined as A=B. Multiple variables can be specified as A=B,C=D.
	<code>--gpu-video-device</code>	help, <id1,id2,...>, all, none	none	Analyze video devices. <code>--help</code> gives a list of supported and unsupported devices and IDs. <code><id1, id2, ...></code> turns on specified devices only.
	<code>--help</code>	<tag>	none	Print the help message. The option can take one option which can be used as a tag. If a tag is provided, only options relevant to the tag are printed.
	<code>--hotkey-capture</code>	'F1' to 'F12'	'F12'	Hotkey to trigger the profiling session. Note that this switch is applicable only when <code>--capture-range=hotkey</code> is specified.

Sh	Long	Possible Parameters	Default	Switch Description
-n	<code>--inherit-environment</code>	true, false	true	When true, the current environment variables and the tool's environment variables will be specified for the launched process. When false, the tool's environment variables will be specified for the launched process.
	<code>--injection-use-detours</code>	true, false	true	Use detours for injection. If false, process injection will be performed using windows hooks which allows it to bypass anti-cheat software.
	<code>--isr</code>	true, false	false	Trace Interrupt Service Routines (ISRs) and Deferred Procedure Calls (DPCs). Requires administrative privileges. Available on Windows devices.
	<code>--mpi-impl</code>	openmpi, mpich	openmpi	When using <code>--trace=mpi</code> to trace MPI APIs, use <code>--mpi-impl</code> to specify which MPI implementation the application is using. If no implementation is specified, nsys tries to automatically determine the implementation from the dynamic linker's search path. If this fails, openmpi is used. <code>mpi-impl</code> without <code>--trace=mpi</code> is not supported.
-p	<code>--nvtx-capture</code>	range@domain, range, range@*	none	Specify NVTX range and domain to trigger the profiling session. This switch is applicable only when used along with <code>--capture-range</code> .
	<code>--nvtx-domain-exclude</code>	default, <domain_names>		Choose to exclude NVTX events from a comma separated list of domain names. <code>default</code> excludes NVTX events without a domain. A domain name or commas in a domain name must be escaped with <code>\</code> . Note Only one of <code>--nvtx-domain-include</code> and <code>--nvtx-domain-exclude</code> can be used.

Sh	Long	Possible Parameters	Default	Switch Description
				can be used. This option is only applicable when <code>--trace</code> is specified.
	<code>--nvtx-domain-include</code>	default, <domain_names>		Choose to only include NVTX events from a comma separated list of domains. default filters the NVTX default domain. A domain name or commas in a domain name must be escaped with backslashes. Note Only one of <code>--nvtx-domain-include</code> and <code>--nvtx-domain-exclude</code> can be used. This option is only applicable when <code>--trace</code> is specified.
	<code>--python-functions-trace</code>	<json_file>		Specify the path to the JSON file containing the requested function annotations.
	<code>--opengl-gpu-workload</code>	true, false	true	If true, trace the OpenGL workloads' GPU activity. Note this option is only applicable only when <code>--trace=opengl</code> is specified.
	<code>--os-events</code>	'help' or the end users selected events in the format 'x,y'		Select the OS events to sample. Use the <code>--os-events-help</code> switch to see the full list of events. Multiple values can be selected using commas only (no spaces). Use the <code>--event-sample-rate</code> switch to set the sampling rate. Not available on Nsight Systems Embedded Platforms.
	<code>--osrt-backtrace-depth</code>	integer	24	Set the depth for the backtraces collected for OS runtime events.

Sh	Long	Possible Parameters	Default	Switch Description
	depth			
	--osrt-backtrace-stack-size	integer	6144	Set the stack dump size, in bytes, to generate backtrace libraries calls.
	--osrt-backtrace-threshold	nanoseconds	80000	Set the duration, in nanoseconds, that all OS runtime lib execute before backtraces are collected.
	--osrt-threshold	< nanoseconds >	1000 ns	Set the duration, in nanoseconds, that Operating System APIs must execute before they are traced. Values significant 1000 may cause significant overhead and result in extreme files. Note This setting is ignored for APIs that interact with files whose access is set to true.
	--osrt-file-access	true, false	false	Collect file access data when tracing Operating System that interact with files. Note When this setting is set to true the --osrt-threshold for APIs that interact with files.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--python-backtrace</code>	cuda, none	none	Collect Python backtrace event when tracing the selected option is supported on Arm server (SBSA) platforms and Note: tracing and backtraces of the selected API and CFI be enabled. For example, <code>--cudabacktrace</code> must be <code>python-backtrace=cuda</code> .
	<code>--python-sampling</code>	true, false	false	Collect Python backtrace sampling events. This option is server (SBSA) platforms, x86 Linux and Windows target profiling Python-only workflows, consider disabling the CFI option to reduce overhead.
	<code>--python-sampling-frequency</code>	1 < integers < 2000	1000	Specify the Python sampling frequency. The minimum is 1Hz. The maximum supported frequency is 2KHz. The <code>--python-sampling</code> option is set to false.
	<code>--pytorch</code>	autograd-nvtx, autograd-shapes-nvtx, functions-trace, none	none	Enable automatic annotations of PyTorch functions.
	<code>--dask</code>	functions-trace, none	none	Enable automatic annotations of Dask functions
	<code>--qnx-kernel-events</code>	class/event,event, class/event:mode, class:mode,help,none	none	Multiple values can be selected, separated by commas. See the <code>--qnx-kernel-events-mode</code> switch description format. Use the <code>--qnx-kernel-events=help</code> switch values. Example: <code>--qnx-kernel-</code>

Sh	Long	Possible Parameters	Default	Switch Description
				events=8/1:system:wide,_NT0_TRACE_THREAD _NT0_TRACE_KERCALLENTER/ __KER_BAD,_NT0_TRACE_COMM,13. Collect QNX k
	--qnx-kernel -events-mode	system,process,fast, wide		Default is system:fast. Values are separated by a colon (colon). system and process cannot be specified at the same time. wide cannot be specified at the same time. Please check the documentation to determine when to select the fast or wide mode. Specify the default mode for QNX kernel events collection.
	--resolve-symbols	true,false	true	Resolve symbols of captured samples and backtraces.
	--retain-etw-files	true, false	false	Retain ETW files generated by the trace, merge and move them to the output directory.
	--run-as	<username>	none	Run the target application as the specified username. If the target application will be run by the same user as Nsight System, root privileges. Available for Linux targets only.
-s	--sample	process-tree, system-wide, xhv, xhv-system-wide, none		Default is process-tree. Select how to collect CPU IP/batches. If none is selected, CPU sampling is disabled. Depending on the selected values, some values may require admin or root privileges. Select system-wide to enable Cross-Hypervisor (XHV) sampling. If a target application is launched, the default is process-tree. Otherwise, the default is none.

Sh	Long	Possible Parameters	Default	Switch Description
				Note system-wide is not available on all platforms. Note If set to none, CPU context switch data will still be collected. The <code>cpuctxsw</code> switch is set to none.
	<code>--samples-per-backtrace</code>	integer <= 32	1	The number of CPU IP samples collected for every CPU sample collected. For example, if set to 4, on the fourth CPU sample collected, a backtrace will also be collected. Lower values result in less amount of data collected. Higher values can reduce collection overhead and reduce the number of CPU IP samples dropped. If <code>backtrace</code> is not collected, the default is 4, otherwise the default is 1. This option is available on Nsight Systems Embedded Platforms Edition targets.
	<code>--sampling-frequency</code>	100 < integers < 8000	1000	Specify the sampling/backtracing frequency. The minimum supported frequency is 100 Hz. The maximum supported frequency is 8000 Hz. This option is supported only on QNX, Linux for Tegra, and Windows.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--sampling-period</code> (Nsight Systems Embedded Platforms Edition)	integer		Default is determined dynamically. The number of CPU counted before a CPU instruction pointer (IP) sample is configured, backtraces may also be collected. The smaller the period, the higher the sampling rate. Note that smaller sampling periods will increase overhead and significantly increase the size of the result file(s). Requires the <code>--sampling-trigger=perf</code> switch.
	<code>--sampling-period</code> (not Nsight Systems Embedded Platforms Edition)	integer		Default is determined dynamically. The number of event CPU instruction pointer (IP) sample is collected. The event the collection of a sample is determined dynamically. For based platforms, it will probably be "Reference Cycles" and platforms, "CPU Cycles". If configured, backtraces may be collected. The smaller the sampling period, the higher the sampling rate. The smaller sampling periods will increase overhead and significantly increase the size of the result file(s). This option is available only on Nsight Systems Embedded Platforms Edition targets only.
	<code>--sampling-trigger</code>	timer, sched, perf, cuda	timer sched	Specify backtrace collection trigger. Multiple APIs can be separated by commas only (no spaces). Available on Nsight Systems Embedded Platforms Edition targets only.
	<code>--session</code>	session identifier	none	Launch the application in the indicated session. The option can represent a valid session name or ID as reported by <code>nsys list</code> . Any <code>%q{ENV_VAR}</code> pattern will be substituted with the value of the environment variable. Any <code>%h</code> pattern will be substituted with the host name.

Sh	Long	Possible Parameters	Default	Switch Description
				of the system. Any %% pattern will be substituted with %.
	--session-new	[a-Z][0-9,a-Z,spaces]		Default is profile-<id>-<application>. Name the session command. Name must start with an alphabetical character or space characters. Any %q{ENV_VAR} pattern substituted with the value of the environment variable. A substituted with the hostname of the system. Any %% pattern substituted with %.
-w	--show-output	true, false	true	If true, send the target process's stdout and stderr streams to console and stdout/stderr files which are added to the report. If false, only send the target process stdout and stderr streams to the report files which are added to the report file.
-t	--trace	cuda, cuda-hw, cudnn, cublas, cublas-verbose, cusparse-verbose, cudla, cudla-verbose, cusolver, opengl, cusolver-verbose, opengl-annotations, openacc, openmp, osrt, mpi, nvvideo, nvtx, dx11, dx11-annotations	cuda, opengl, nvtx, osrt	Select the API(s) to be traced. The osrt switch controls the libraries tracing. Multiple APIs can be selected, separated by spaces (no spaces). Since OpenACC and cuXXX APIs are tightly coupled with CUDA, selecting one of those APIs will automatically enable the corresponding API. For example, selecting cublas, cudla, cusparse and cusolver all have XXX-verbose available. Reflex SDK latency markers will be automatically enabled if any of the above APIs are selected. If DX or vulkan API trace is enabled. See information on --reflex below if mpi is selected. If <api>-annotations is selected, the corresponding API will also be traced. If the none option is selected, no APIs are traced and no other API can be selected. Note

Sh	Long	Possible Parameters	Default	Switch Description
		dx12,dx12-annotations tegra-accelerators, ucx, openxr, oshmem, openxr-annotations, python-gil, gds(beta) wddm, vulkan, vulkan-annotations, none		cuDNN is not available on Windows target. Note cuda-hw is not available for hardware before Blackwell, before 11.0
	--trace-fork -before-exec	true, false	false	If true, trace any child process after fork and before they functions. Beware, tracing in this interval relies on undef might cause your application to crash or deadlock. This available on Linux target platforms.
	--vulkan-gpu -workload	true, false, batch, individual, none		Default is individual. If individual or true, trace each Vulkan activity individually. If batch, trace Vulkan workloads' GPU vkQueueSubmit call batches. If none or false, do not trace workloads' GPU activity. Note that this switch is applicable t trace=vulkan is specified. This option is not supported
	--wait	primary,all	all	If primary, the CLI will wait on the application process the CLI will additionally wait on re-parented processes of application.
	--wddm-	true, false	true	If true, collect additional range of ETW events, including

Sh	Long	Possible Parameters	Default	Switch Description
	additional-events			allocations, sync wait and signal events, etc. Note that this is applicable only when <code>--trace=wddm</code> is specified. This is supported on Windows targets.
	<code>--wddm-backtraces</code>	true, false	false	If true, collect backtraces of WDDM events. Disabling this can reduce overhead for certain target applications. Note that this is applicable only when <code>--trace=wddm</code> is specified. This is supported on Windows targets.

CLI Sessions Command Switch Subcommands#

After choosing the `sessions` command switch, the following subcommands are available. Usage:

`nsys [global-options] sessions [subcommand]`

Subcommand	Description
list	List all active sessions including ID, name, and state information

CLI Sessions List Command Switch Options#

After choosing the `sessions list` command switch, the following options are available. Usage:

`nsys [global-options] sessions list [options]`

Short	Long	Possible	Default	Switch Description
-------	------	----------	---------	--------------------

		Parameters		
	--help	<tag>	none	Print the help message. The option can take one optional argument that will be used as a tag. If a tag is provided, only options relevant to the tag will be printed.
-p	--show-header	true, false	true	Controls whether a header should appear in the output.
-f	--output-format	plain, json	true	Output format used for session list.

CLI Shutdown Command Switch Options#

After choosing the shutdown command switch, the following options are available. Usage:

nsys [global-options] shutdown [options]

Short	Long	Possible Parameters	Default	Switch Description
	--help	<tag>	none	Print the help message. The option can take one optional argument that will be used as a tag. If a tag is provided, only options relevant to the tag will be printed.
	--kill	On Linux: one, sigkill, sigterm, signal number On Windows:	On Linux: sigterm On Windows:	Send signal to the target application's process group when shutting down session.

Short	Long	Possible Parameters	Default	Switch Description
		true, false	true	
	--session	session identifier	none	Shutdown the indicated session. The option argument must represent a valid session name or ID as reported by <code>nsys sessions list</code> . Any <code>%q{ENV_VAR}</code> pattern will be substituted with the value of the environment variable. Any <code>%h</code> pattern will be substituted with the hostname of the system. Any <code>%%</code> pattern will be substituted with <code>%</code> .

CLI Start Command Switch Options#

After choosing the `start` command switch, the following options are available. Usage:

`nsys [global-options] start [options]`

Sh	Long	Possible Parameters	Default	Switch Description
	--accelerator-trace	none, tegra-accelerators	none	Collect other accelerators workload trace from the hardware engine units. Available in Nsight Systems Embedded Platforms Edition only.
-b	--backtrace	auto,fp,lbr,dwarf, none		Select the backtrace method to use while sampling. The option <code>lbr</code> uses Intel(c) Corporation's Last Branch Record registers, available only with Intel(c) CPUs codenamed Haswell and later. The option <code>fp</code> is frame pointer

Sh	Long	Possible Parameters	Default	Switch Description
				and assumes that frame pointers were enabled during compilation. The option <code>dwarf</code> uses DWARF's CFI (Call Frame Information). Setting the value to <code>none</code> can reduce collection overhead.
-c	--capture-range	none, cudaProfilerApi, hotkey, nvtx	none	When <code>--capture-range</code> is used, profiling will start only when an appropriate start API or hotkey is invoked. If <code>--capture-range</code> is set to <code>none</code>, start/stop API calls and hotkeys will be ignored. Note Hotkey works for graphic applications only.
	--capture-range -end	none, stop, stop-shutdown, repeat[:N], repeat-shutdown:N		Default is stop-shutdown. Specify the desired behavior when a capture range ends. Applicable only when used along with the <code>--capture-range</code> option. If <code>none</code>, capture range end will be ignored. If <code>stop</code>, collection will stop at the capture range end. Any subsequent capture ranges will be ignored. The target app will continue running. If <code>stop-shutdown</code>, collection will stop at the capture range end and session will be shutdown. If <code>repeat[:N]</code>, collection will stop at capture range end and subsequent capture ranges will trigger more collections. The

Sh	Long	Possible Parameters	Default	Switch Description
				optional :N specifies the max number of capture ranges to be honored. Any subsequent capture ranges will be ignored once N capture ranges are collected. If repeat-shutdown:N, the same behavior as repeat:N but session will be shutdown after N ranges. For stop-shutdown and repeat-shutdown:N, as always, use the --kill option to specify whether the target app should be terminated when shutting down the session.
	--cpu-core-events (Nsight Systems Embedded Platforms Edition)	0x11,0x13,...,none	none	Collect per-core PMU counters. Multiple values can be selected, separated by commas only (no spaces). Use the --cpu-core-events=help switch to see the full list of values.
	--cpu-core-events (not Nsight Systems Embedded Platforms Edition)	'help' or the end users selected events in the format 'x,y'	'2'	Default is Instructions Retired. Select the CPU Core events to sample. Use the --cpu-core-events=help switch to see the full list of events and the number of events that can be collected simultaneously. Multiple values can be selected, separated by commas only (no spaces). Use the --event-sample switch to enable.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--cpu-core-metrics</code>	0,1,2,...,none	none	Collect metrics on the CPU core. Multiple values can be selected, separated by commas only (no spaces). Use the <code>--cpu-core-metrics=help</code> switch to see the full list of values. Use the <code>--event-sample</code> switch to enable. Note Only available on Grace.
	<code>--cpu-socket-events</code> (Nsight Systems Embedded Platforms Edition)	0x2a,0x2c,...,none	none	Collect per-socket Uncore PMU counters. Multiple values can be selected, separated by commas only (no spaces). Use the <code>--cpu-socket-events=help</code> switch to see the full list of values. Available in Nsight Systems Embedded Platforms Edition only.
	<code>--cpu-socket-events</code> (not Nsight Systems Embedded Platforms Edition)	'help' or the end users selected events in the format 'x,y'	none	Select the Uncore CPU Socket events to sample. Use the <code>--cpu-socket-events=help</code> switch to see the full list of events and the number of events that can be collected simultaneously. Multiple values can be selected, separated by commas only (no spaces). Use the <code>--event-sample</code> switch to enable.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--cpu-socket-metrics</code>	0,1,2,...,none	none	Collect Uncore metrics on the CPU socket. Multiple values can be selected, separated by commas only (no spaces). Use the <code>--cpu-socket-metrics=help</code> switch to see the full list of values. Use the <code>--event-sample</code> switch to enable. Note Only available on Grace.
	<code>--cpuctxsw</code>	process-tree, system-wide, none		Default is process-tree. Trace OS thread scheduling activity. Select none to disable tracing CPU context switches. Depending on the platform, some values may require admin or root privileges. Note If the <code>--sample</code> switch is set to a value other than none, the <code>--cpuctxsw</code> setting is hardcoded to the same value as the <code>--sample</code> switch. If <code>--sample=none</code> and a target application is launched, the default is process-tree, otherwise the default is none. Requires <code>--sampling-trigger=perf</code> switch in Nsight Systems Embedded Platforms Edition

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--enable</code>	<code><plugin_name></code> <code>[,arg1,arg2,...]</code>	NA	Use the specified plugin. The option can be specified multiple times to enable multiple plugins. Plugin arguments are separated by commas only (no spaces). Commas can be escaped with a backslash <code>\</code> . The backslash itself can be escaped by another backslash <code>\\</code> . To include spaces in an argument, enclose the argument in double quotes <code>"</code> . To list all available plugins, use the <code>--enable=help</code> command.
	<code>--etw-provider</code>	<code>"<name>,<guid>"</code> , or path to JSON file	none	Add custom ETW trace provider(s). If you want to specify more attributes than Name and GUID, provide a JSON configuration file as outlined below. This switch can be used multiple times to add multiple providers. Note: Only available for Windows targets.
	<code>--event-sample</code>	system-wide, none	none	Use the <code>--cpu-core-events=help</code> and the <code>--os-events=help</code> switches to see the full list of events. If event sampling is enabled and no events are selected, the CPU Core event 'Instructions Retired' is selected by default. Not available on Nsight Systems Embedded Platforms Edition.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--event-sampling-interval</code>	Integers from 1 to 1000 milliseconds	10	The interval between each event sample collection. Minimum event sampling interval is 1 mSec. Maximum event sampling interval is 1000 mSec. Not available in Nsight Systems Embedded Platforms Edition.
	<code>--export</code>	arrow, arrowdir, hdf, json, parquetdir, sqlite, text, none	none	Create additional output file(s) based on the data collected. This option can be given more than once. Warning If the collection captures a large amount of data, creating the export file may take several minutes to complete.
	<code>--flush-on-cudaprofilerstop</code>	true, false	true	If set to true, any call to <code>cudaProfilerStop()</code> will cause the CUDA trace buffers to be flushed. Note that the CUDA trace buffers will be flushed when the collection ends, regardless of the value of this switch.
-f	<code>--force-overwrite</code>	true, false	false	If true, overwrite all existing result files with same output filename (.nsys-rep, .sqlite, .h5, .txt, .json, .arrows, _arwdir, _pqtdir).
	<code>--ftrace</code>			Collect ftrace events. Argument should list events to collect as: subsystem1/

Sh	Long	Possible Parameters	Default	Switch Description
				event1, subsystem2/event2. Requires root. No ftrace events are collected by default.
	--ftrace-keep-user-config			Skip initial ftrace setup and collect already configured events. Default resets the ftrace configuration.
	--gpu-metrics-devices	GPU ID, help, all, none	none	Collect GPU Metrics from specified devices. Determine GPU IDs by using --gpu-metrics-devices=help switch.
	--gpu-metrics-frequency	integer	10000	Specify GPU Metrics sampling frequency. Minimum supported frequency is 10 (Hz). Maximum supported frequency is 200000 (Hz).
	--gpu-metrics-set	alias, file:<file name>		Specify metric set for GPU Metrics. The argument must be one of the aliases reported by --gpu-metrics-set=help switch, or a path to a metric config file prefixed by file:. The default is the first metric set that supports all selected GPUs.
	--gpu-video-device	help, <id1,id2,...>, all, none	none	Analyze video devices. --help gives a list of supported devices, reason for unsupported devices and IDs. <id1,id2,...> turns on the feature for the specified devices only.

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--gpuctxsw</code>	true,false	false	Trace GPU context switches. See the GPU Context Switch topic for details.
	<code>--help</code>	<tag>	none	Print the help message. The option can take one optional argument that will be used as a tag. If a tag is provided, only options relevant to the tag will be printed.
	<code>--ib-net-info-devices</code>	<NIC names>	none	A comma-separated list of NIC names. The NICs which ibdiagnet will use for networks discovery. This option creates the ibdiagnet files to be used for collecting network information. Example value: mlx5_0,mlx5_1. If the <code>--ib-net-info-output</code> option is set then Nsight Systems will store the network information at that path. Otherwise it will be created at a temporary path and will be discarded after processing. If more than one NIC was specified, only the last network information file will be saved. Note that this option should not be used together with the <code>--ib-net-info-files</code> option.
	<code>--ib-net-info-files</code>	<file paths>	none	A comma-separated list of file paths. Paths of an existing ibdiagnet db_csv files, containing networks information data. Nsight Systems will read the networks' information from these files.

Sh	Long	Possible Parameters	Default	Switch Description
				Don't use ~ alias within the path. Note that this option should not be used together with the <code>--ib-net-info-devices</code> option.
	<code>--ib-net-info-output</code>	<directory path>	none	Sets the path of a directory into which ibdiagnet network discovery data will be written. Use this option together with the <code>--ib-net-info-devices</code> option. Don't use ~ alias within the path.
	<code>--ib-switch-congestion-device</code>	<IB switch GUIDs>	none	A comma-separated list of InfiniBand switch GUIDs. Collect InfiniBand switch congestion events from switches identified by the specified GUIDs. This switch can be used multiple times. System scope. Use the <code>--ib-switch-congestion-nic-device</code> , <code>--ib-switch-congestion-percent</code> , and <code>--ib-switch-congestion-threshold-high</code> switches to further control how congestion events are collected.
	<code>--ib-switch-congestion-nic-device</code>	<NIC name>	none	The name of the NIC (HCA) through which InfiniBand switches will be accessed. By default, the first active NIC will be used. One way to find a NIC's name is via the <code>ibnetdiscover --Hca_list grep "\$(hostname)"</code> command. Example usage: <code>--ib-switch-</code>

Sh	Long	Possible Parameters	Default	Switch Description
				congestion-nic-device=mlx5_3
	--ib-switch-congestion-percent	1 <= integer <= 100	50	Percent of InfiniBand switch congestion events to be collected. This option enables reducing the network bandwidth consumed by reporting congestion events.
	--ib-switch-congestion-threshold-high	1 < integer <= 1023	75	High threshold percentage for InfiniBand switch egress port buffer size. Before a packet leaves an InfiniBand switch, it is stored at an egress port buffer. The buffer's size is checked and if it exceeds the given threshold percentage, a congestion event is reported. The percentage can be greater than 100.
	--ib-switch-metrics-device	<IB switch GUIDs>	none	A comma-separated list of InfiniBand switch GUIDs. Collect metrics from the specified InfiniBand switches. This switch can be used multiple times. System scope.
	--ib-switch-metrics-nic-device	<NIC name>	none	The name of the NIC (HCA) through which InfiniBand switches will be accessed for performance metrics. By default, the first active NIC will be used. One way to find a NIC's name is via the <code>ibstat -l`</code> command. Example usage: <code>--ib-switch-metrics-nic-device=mlx5_3</code>

Sh	Long	Possible Parameters	Default	Switch Description
	<code>--isr</code>	true, false	false	Trace Interrupt Service Routines (ISRs) and Deferred Procedure Calls (DPCs). Requires administrative privileges. Available only on Windows devices.
	<code>--nic-metrics</code>	true, false	false	Collect metrics from supported NIC/HCA devices. System scope. Not available on Nsight Systems Embedded Platforms Edition.
	<code>--os-events</code>	'help' or the end users selected events in the format 'x,y'		Select the OS events to sample. Use the <code>--os-events=help</code> switch to see the full list of events. Multiple values can be selected, separated by commas only (no spaces). Use the <code>--event-sample</code> switch to enable. Not available on Nsight Systems Embedded Platforms Edition.
-o	<code>--output</code>	< filename >	report#	Set the report file name. Any %q{ENV_VAR} pattern in the filename will be substituted with the value of the environment variable. Any %h pattern in the filename will be substituted with the hostname of the system. Any %p pattern in the filename will be substituted with the PID of the target process or the PID of the root process if there is a process tree. Any %% pattern in the filename will be substituted with %. Default is report#{.nsys-rep,.sqlite,.h5,.txt,.arrows,

Sh	Long	Possible Parameters	Default	Switch Description
				_arwdir,_pqtdir,.json} in the working directory.
	--process-scope	main, process-tree, system-wide	main	Select which process(es) to trace. Available in Nsight Systems Embedded Platforms Edition only. Nsight Systems Workstation Edition will always trace system-wide in this version of the tool.
	--retain-etw-files	true, false	false	Retain ETW files generated by the trace, merge and move the files to the output directory.
-s	--sample	process-tree, system-wide, xhv, xhv-system-wide, none		Default is process-tree. Select how to collect CPU IP/backtrace samples. If none is selected, CPU sampling is disabled. Depending on the platform, some values may require admin or root privileges. Select ``xhv`` or xhv-system-wide to enable Cross-Hypervisor (XHV) sampling, requires root privileges. If a target application is launched, the default is process-tree; otherwise, the default is none. Note system-wide is not available on all platforms. Note If set to none, CPU context switch data will still be collected unless the --cpuctxsw switch is

Sh	Long	Possible Parameters	Default	Switch Description
				set to none.
	--samples-per-backtrace	integer <= 32	1	The number of CPU IP samples collected for every CPU IP/backtrace sample collected. For example, if set to 4, on the fourth CPU IP sample collected, a backtrace will also be collected. Lower values increase the amount of data collected. Higher values can reduce collection overhead and reduce the number of CPU IP samples dropped. If DWARF backtraces are collected, the default is 4, otherwise the default is 1. This option is not available on Nsight Systems Embedded Platforms Edition or on non-Linux targets.
	--sampling-frequency	100 < integers < 8000	1000	Specify the sampling/backtracing frequency. The minimum supported frequency is 100 Hz. The maximum supported frequency is 8000 Hz. This option is supported only on QNX, Linux for Tegra, and Windows targets.
	--sampling-period (Nsight Systems Embedded Platforms Edition)	integer		Default is determined dynamically. The number of CPU Cycle events counted before a CPU instruction pointer (IP) sample is collected. If configured, backtraces may also be collected. The smaller the sampling period, the higher the sampling rate. Note that smaller sampling

Sh	Long	Possible Parameters	Default	Switch Description
				periods will increase overhead and significantly increase the size of the result file(s). Requires the <code>--sampling-trigger=perf</code> switch.
	<code>--sampling-period</code> (not Nsight Systems Embedded Platforms Edition)	integer		Default is determined dynamically. The number of events counted before a CPU instruction pointer (IP) sample is collected. The event used to trigger the collection of a sample is determined dynamically. For example, on Intel based platforms, it will probably be "Reference Cycles" and on AMD platforms, "CPU Cycles". If configured, backtraces may also be collected. The smaller the sampling period, the . higher the sampling rate Note that smaller sampling periods will increase overhead and significantly increase the size of the result file(s). This option is available only on Linux targets.
	<code>--sampling-trigger</code>	timer, sched, perf, cuda	timer sched	Specify backtrace collection trigger. Multiple APIs can be selected, separated by commas only (no spaces). Available on Nsight Systems Embedded Platforms Edition targets only.
	<code>--session-new</code>	[a-Z][0-9,a-Z,spaces]		Default is profile-<id>-<application>. Name the session created by the command. Name must start with an alphabetical character followed by printable or space characters. Any

Sh	Long	Possible Parameters	Default	Switch Description
				%q{ENV_VAR} pattern will be substituted with the value of the environment variable. Any %h pattern will be substituted with the hostname of the system. Any %% pattern will be substituted with %.
-w	--show-output	true, false	true	If true, send the target process's stdout and stderr streams to both the console and stdout/stderr files which are added to the report file. If false, only send the target process stdout and stderr streams to the stdout/stderr files which are added to the report file.
	--soc-metrics	true,false	false	Collect SoC Metrics. Available in Nsight Systems Embedded Platforms Edition only.
	--soc-metrics-frequency	integer	10000	Specify SoC Metrics sampling frequency. Minimum supported frequency is '100' (Hz). Maximum supported frequency is '1000000' (Hz). Available in Nsight Systems Embedded Platforms Edition only.
	--soc-metrics-set	alias, file:<file name>		Specify metric set for SoC Metrics. The argument must be one of the aliases reported by --soc-metrics-set=help switch, or a path to a metric config file prefixed by file:. Available in Nsight Systems Embedded

Sh	Long	Possible Parameters	Default	Switch Description
				Platforms Edition only.
	--stats	true, false	false	Generate summary statistics after the collection. Warning When set to true, an SQLite database will be created after the collection. If the collection captures a large amount of data, creating the database file may take several minutes to complete.
-x	--stop-on-exit	true, false	true	If true, stop collecting automatically when the launched process has exited or when the duration expires - whichever occurs first. If false, duration must be set and the collection stops only when the duration expires. Nsight Systems does not officially support runs longer than 5 minutes.
	--syscall (beta)	process-tree, pid-namespace, none	none	Collect system calls. The value defines the collection scope: process-tree makes it tracing the application processes only, pid-namespace - all processes in the current PID namespace and its child namespaces (similar to the system-wide mode of other features).
	--vsync	true, false	false	Collect vsync events. If collection of vsync events is enabled, display/display_scanline

Sh	Long	Possible Parameters	Default	Switch Description
				ftrace events will also be captured. Available in Nsight Systems Embedded Platforms Edition only.
	--xhv-trace	< filepath pct.json >	none	Collect hypervisor trace. Available in Nsight Systems Embedded Platforms Edition only.
	--xhv-trace - events	all, none, core, sched, irq, trap	all	Available in Nsight Systems Embedded Platforms Edition only.
	--xhv-vm-symbols	<filepath kernel_symbols.json>	none	XHV sampling config file. Available in Nsight Systems Embedded Platforms Edition only.

CLI Stats Command Switch Options#

The `nsys stats` command generates a series of summary or trace reports. These reports can be output to the console, or to individual files, or piped to external processes. Reports can be rendered in a variety of different output formats, from human readable columns of text, to formats more appropriate for data exchange, such as CSV.

Reports are generated from an SQLite export of a `.nsys-rep` file. If a `.nsys-rep` file is specified, Nsight Systems will look for an accompanying SQLite file and use it. If no SQLite file exists, one will be exported and created.

Individual reports are generated by calling out to scripts that read data from the SQLite file and return their report data in CSV format. Nsight Systems ingests this data and formats it as requested, then displays the data to the console, writes it to a file, or pipes it to an external process. Adding new reports is as simple as writing a script that can read the SQLite file and generate the required CSV output. See the shipped scripts as an example. Both reports and formatters may take arguments to tweak their processing. For details on shipped scripts and formatters, see [Statistical Analysis](#).

Reports are processed using a three-tuple that consists of:

1. The requested report (and any arguments),
2. The presentation format (and any arguments), and
3. The output (filename, console, or external process).

The first report specified uses the first format specified, and is presented via the first output specified. The second report uses the second format for the second output, and so forth. If more reports are specified than formats or outputs, the format and/or output list is expanded to match the number of provided reports by repeating the last specified element of the list (or the default, if nothing was specified).

`nsys stats` is a very powerful command and can handle complex argument structures, please see the topic below on Example Stats Command Sequences.

After choosing the `stats` command switch, the following options are available. Usage:

`nsys [global-options] stats [options] [input-file]`

Short	Long	Possible Parameters	Default	Switch Description
	<code>--help</code>	<code><tag></code>	none	Print the help message. The option can take one optional argument that will be used as a tag. If a tag is provided, only options relevant to the tag will be printed.
<code>-f</code>	<code>--format</code>	column, table, csv, tsv, json, hdoc, htable, .		Specify the output format. The special name “.” indicates the default format for the given output. The default format for console is column, while files and process outputs default to csv. This option may be used multiple times. Multiple formats may also be specified using a comma-separated list (<code><name [:args...] [, name [:args...] ...]></code>). See

Short	Long	Possible Parameters	Default	Switch Description
				Report Scripts for options available with each format.
	--force-export	true, false	false	Force a re-export of the SQLite file from the specified .nsys rep file, even if an SQLite file already exists.
	--force-overwrite	true, false	false	Overwrite any existing report file(s).
	--help-formats	<format_name>, ALL, [none]	none	With no argument, give a summary of the available output formats. If a format name is given, a more detailed explanation of that format is displayed. If ALL is given, a more detailed explanation of all available formats is displayed.
	--help-reports	<report_name>, ALL, [none]	none	With no argument, list a summary of the available summary and trace reports. If a report name is given, a more detailed explanation of the report is displayed. If ALL is given, a more detailed explanation of all available reports is displayed.
-o	--output	-, @<command>, <basename>, .	-	Specify the output mechanism. There are three output mechanisms: print to console, output to file, or output to command. This option may be used multiple times. Multiple outputs may also be specified using a comma-separated list. If the given output name is "-", the output will be displayed on the console. If the output name starts with "@", the output designates a command to run. The nsys command will be

Short	Long	Possible Parameters	Default	Switch Description
				executed and the analysis output will be piped into the command. Any other output is assumed to be the base path and name for a file. If a file basename is given, the filename used will be: <basename>_<analysis&args>.<output_format> The default base (including path) is the name of the SQLite file (as derived from the input file or <code>--sqlite</code> option), minus the extension. The output “.” can be used to indicate the analysis should be output to a file, and the default basename should be used. To write one or more analysis outputs to files using the default basename, use the option: <code>--output</code>. If the output starts with “@”, the <code>nsys</code> command output is piped to the given command. The command is run and the output is piped to the command’s stdin (standard-input). The command’s stdout and stderr remain attached to the console, so any output will be displayed directly to the console. Be aware there are some limitations in how the command string is parsed. No shell expansions (including <code>?</code>, <code>[]</code>, and <code>~</code>) are supported. The command cannot be piped to another command, nor redirected to a file using shell syntax. The command and command arguments are split on whitespace, and no quotes (within the command syntax) are supported. For commands that require complex command line syntax, it is suggested that the command be put in a shell script file, and that be designated as the output command.

Short	Long	Possible Parameters	Default	Switch Description
-q	--quiet			Do not display verbose messages, only display errors.
-r	--report	See Report Scripts		Specify the report(s) to generate, including any arguments. This option may be used multiple times. Multiple reports may also be specified using a comma-separated list (<name [:args...] [, name [:args...] ...]>). If no reports are specified, the following will be used as the default report set: nvtx_sum, osrt_sum, cuda_api_sum, cuda_gpu_kern_sum, cuda_gpu_mem_time_sum, cuda_gpu_mem_size_sum, openmp_sum, opengl_khr_range_sum, opengl_khr_gpu_range_sum, vulkan_marker_sum, vulkan_gpu_marker_sum, dx11_pix_sum, dx12_gpu_marker_sum, dx12_pix_sum, wddm_queue_sum, um_sum, um_total_sum, um_cpu_page_faults_sum, openacc_sum. See Report Scripts for details about existing built-in scripts and how to make your own.
	--report-dir	<path>		Add a directory to the path used to find report scripts. This is usually only needed if you have one or more directories with personal scripts. This option may be used multiple times. Each use adds a new directory to the end of the path. A search path can also be defined using the environment variable NSYS_STATS_REPORT_PATH. Directories added this way will be added after the application flags. The last two entries in the path will always be the current working

Short	Long	Possible Parameters	Default	Switch Description
				directory, followed by the directory containing the shipped nsys reports.
	<code>--sqlite</code>	<file.sqlite>		Specify the SQLite export filename. If this file exists, it will be used. If this file doesn't exist (or if <code>--force-export</code> was given) this file will be created from the specified .nsys-rep file before processing. This option cannot be used if the specified input file is also an SQLite file.
	<code>--timeunit</code>	nsec, nanoseconds, usec, microseconds, msec, milliseconds, seconds	nanoseconds	Set basic unit of time. The argument of the switch is matched by using the longest prefix matching, meaning that it is not necessary to write a whole word as the switch argument. It is similar to passing a <code>:time=<unit></code> argument to every formatter, although the formatter uses more strict naming conventions. See <code>nsys stats --help-formats</code> column for more detailed information on unit conversion.

CLI Status Command Switch Options#

The `nsys status` command returns the current state of the CLI. After choosing the `status` command switch, the following options are available. Usage:

`nsys [global-options] status [options]`

Short	Long	Possible Parameters	Default	Switch Description
	--all			Prints information for all the available profiling environments.
-e	--environment			Returns information about the system regarding suitability of the profiling environment.
	--help	<tag>	none	Print the help message. The option can take one optional argument that will be used as a tag. If a tag is provided, only options relevant to the tag will be printed.
-n	--network			Returns information about the system regarding suitability of the network profiling environment.
	--session	session identifier	none	Print the status of the indicated session. The option argument must represent a valid session name or ID as reported by <code>nsys sessions list</code> . Any <code>%{ENV_VAR}</code> pattern will be substituted with the value of the environment variable. Any <code>%h</code> pattern will be substituted with the hostname of the system. Any <code>%%</code> pattern will be substituted with <code>%</code> .

CLI Stop Command Switch Options#

After choosing the `stop` command switch, the following options are available. Usage:

`nsys [global-options] stop [options]`

Short	Long	Possible Parameters	Default	Switch Description
	<code>--help</code>	<code><tag></code>	none	Print the help message. The option can take one optional argument that will be used as a tag. If a tag is provided, only options relevant to the tag will be printed.
	<code>--keep</code>	time in seconds	0	Indicate how many seconds of collected data previous to the stop command should be retained in the result file. Zero is treated as a special setting that retains all of the data.
	<code>--session</code>	session identifier	none	Stop the indicated session. The option argument must represent a valid session name or ID as reported by <code>nsys sessions list</code> . Any <code>%q{ENV_VAR}</code> pattern will be substituted with the value of the environment variable. Any <code>%h</code> pattern will be substituted with the hostname of the system. Any <code>%%</code> pattern will be substituted with <code>%</code> .

Example Single Command Lines#

Version Information

`nsys -v`

Effect: Prints tool version information to the screen.

Run with elevated privilege

`sudo nsys profile <app>`

Effect: Nsight Systems CLI (and target application) will run with elevated privilege. This is necessary for some features, such as FTrace or system-wide CPU sampling. If you don't want the target application to be elevated, use

--run-as option.

Default analysis run

```
nsys profile <application>  
[application-arguments]
```

Effect: Launch the application using the given arguments. Start collecting immediately and end collection when the application stops. Trace CUDA, OpenGL, NVTX, and OS runtime libraries APIs. Collect CPU Instruction Pointer (IP) sampling information and thread scheduling information. With Nsight Systems Embedded Platforms Edition this will only analysis the single process. With Nsight Systems Workstation Edition this will trace the process tree. Generate the report#`.nsys-rep` file in the default location, incrementing the report number if needed to avoid overwriting any existing output files.

Limited trace only run

```
nsys profile --trace=cuda,nvtx -d 20  
--sample=none --cpuctxsw=none -o my_test <application>  
[application-arguments]
```

Effect: Launch the application using the given arguments. Start collecting immediately and end collection after 20 seconds or when the application ends. Trace CUDA and NVTX APIs. Do not collect CPU sampling information or thread scheduling information. Profile any child processes. Generate the output file as `my_test.nsys-rep` in the current working directory.

Delayed start run

```
nsys profile -e TEST_ONLY=0 -y 20  
<application> [application-arguments]
```

Effect: Set environment variable `TEST_ONLY=0`. Launch the application using the given arguments. Start collecting after 20 seconds and end collection at application exit. Trace CUDA, OpenGL, NVTX, and OS runtime libraries APIs. Collect CPU sampling and thread schedule information. Profile any child processes. Generate the report#`.nsys-rep` file in the default location, incrementing if needed to avoid overwriting any existing output files.

Collect ftrace events

```
nsys profile --ftrace=drm/drm_vblank_event  
-d 20
```

Effect: Collect ftrace `drm_vblank_event` events for 20 seconds. Generate the `report#.nsys-rep` file in the current working directory. Note that ftrace event collection requires running as root. To get a list of ftrace events available from the kernel, run the following:

```
sudo cat /sys/kernel/debug/tracing/available_events
```

Run GPU metric sampling on one TU10x

```
nsys profile --gpu-metrics-devices=0  
--gpu-metrics-set=tu10x-gfxt <application>
```

Effect: Launch application. Collect default options and GPU metrics for the first GPU (a TU10x), using the `tu10x-gfxt` metric set at the default frequency (10 kHz). Profile any child processes. Generate the `report#.nsys-rep` file in the default location, incrementing if needed to avoid overwriting any existing output files.

Run GPU metric sampling on all GPUs at a set frequency

```
nsys profile --gpu-metrics-devices=all  
--gpu-metrics-frequency=20000 <application>
```

Effect: Launch application. Collect default options and GPU metrics for all available GPUs using the first suitable metric set for each and sampling at 20 kHz. Profile any child processes. Generate the `report#.nsys-rep` file in the default location, incrementing if needed to avoid overwriting any existing output files.

Collect CPU IP/backtrace and CPU context switch

```
nsys profile --sample=system-wide --duration=5
```

Effect: Collects both CPU IP/backtrace samples using the default backtrace mechanism and traces CPU context switch activity for the whole system for 5 seconds. Note that it requires root permission or a Linux paranoid level of 0 or less to run. No hardware or OS events are sampled. Post processing of this collection will take longer due to the

large number of symbols to be resolved caused by system-wide sampling.

Get list of available CPU core events

```
nsys profile --cpu-core-events=help
```

Effect: Lists the CPU events that can be sampled and the maximum number of CPU events that can be sampled concurrently.

Collect system-wide CPU events and trace application

```
nsys profile --event-sample=system-wide  
--cpu-core-events='1,2' --event-sampling-interval=5 <app> [app args]
```

Effect: Collects CPU IP/backtrace samples using the default backtrace mechanism, traces CPU context switch activity, and samples each CPU's "CPU Cycles" and "Instructions Retired" event every 5 ms for the whole system. Note that it requires root permission or a Linux paranoid level of 0 or less to run. Note that CUDA, NVTX, OpenGL, and OSRT within the app launched by Nsight Systems are traced by default while using this command. Post processing of this collection will take longer due to the large number of symbols to be resolved caused by system-wide sampling.

Collect custom ETW trace using configuration file

```
nsys profile --etw-provider=file.JSON
```

Effect: Configure custom ETW collectors using the contents of file.JSON. Collect data for 20 seconds. Generate the report#.nsys-rep file in the current working directory.

A template JSON configuration file is located at in the Nsight Systems installation directory as \\target-windows-x64\\etw_providers_template.json. This path will show up automatically if you call the following:

```
nsys profile --help
```

The **level** attribute can only be set to one of the following:

- TRACE_LEVEL_CRITICAL

- TRACE_LEVEL_ERROR
- TRACE_LEVEL_WARNING
- TRACE_LEVEL_INFORMATION
- TRACE_LEVEL_VERBOSE

The **flags** attribute can only be set to one or more of the following:

- EVENT_TRACE_FLAG_ALPC
- EVENT_TRACE_FLAG_CSWITCH
- EVENT_TRACE_FLAG_DBGPRINT
- EVENT_TRACE_FLAG_DISK_FILE_IO
- EVENT_TRACE_FLAG_DISK_IO
- EVENT_TRACE_FLAG_DISK_IO_INIT
- EVENT_TRACE_FLAG_DISPATCHER
- EVENT_TRACE_FLAG_DPC
- EVENT_TRACE_FLAG_DRIVER
- EVENT_TRACE_FLAG_FILE_IO
- EVENT_TRACE_FLAG_FILE_IO_INIT
- EVENT_TRACE_FLAG_IMAGE_LOAD
- EVENT_TRACE_FLAG_INTERRUPT
- EVENT_TRACE_FLAG_JOB
- EVENT_TRACE_FLAG_MEMORY_HARD_FAULTS
- EVENT_TRACE_FLAG_MEMORY_PAGE_FAULTS

- EVENT_TRACE_FLAG_NETWORK_TCPIP
- EVENT_TRACE_FLAG_NO_SYSCONFIG
- EVENT_TRACE_FLAG_PROCESS
- EVENT_TRACE_FLAG_PROCESS_COUNTERS
- EVENT_TRACE_FLAG_PROFILE
- EVENT_TRACE_FLAG_REGISTRY
- EVENT_TRACE_FLAG_SPLIT_IO
- EVENT_TRACE_FLAG_SYSTEMCALL
- EVENT_TRACE_FLAG_THREAD
- EVENT_TRACE_FLAG_VAMAP
- EVENT_TRACE_FLAG_VIRTUAL_ALLOC

Typical case: profile a Python script that uses CUDA

```
nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx  
--delay=60 python my_dnn_script.py
```

Effect: Launch a Python script and start profiling it 60 seconds after the launch, tracing CUDA, cuDNN, cuBLAS, OS runtime APIs, and NVTX as well as collecting CPU IP samples and thread schedule information.

Typical case: profile an app that uses Vulkan

```
nsys profile --trace=vulkan,osrt,nvtx  
--delay=60 ./myapp
```

Effect: Launch an app and start profiling it 60 seconds after the launch, tracing Vulkan, OS runtime APIs, and NVTX as well as collecting CPU sampling and thread schedule information.

Example Interactive CLI Command Sequences#

Collect from beginning of application, end manually

```
nsys start --stop-on-exit=false
```

```
nsys launch --trace=cuda,nvtx --sample=none <application> [application-arguments]
```

```
nsys stop
```

Effect: Create interactive CLI process and set it up to begin collecting as soon as an application is launched. Launch the application, set up to allow tracing of CUDA and NVTX as well as collection of thread schedule information. Stop only when explicitly requested. Generate the report#.nsys-rep in the default location.

Note

If you start a collection and fail to stop the collection (or if you are allowing it to stop on exit, and the application runs for too long), your system's storage space may be filled with collected data causing significant issues for the system.

Nsight Systems will collect a different amount of data/sec depending on options, but in general Nsight Systems does not support runs of more than 5 minutes duration.

Run application, begin collection manually, run until process ends

```
nsys launch -w true <application> [application-arguments]
```

```
nsys start
```

Effect: Create interactive CLI and launch an application set up for default analysis. Send application output to the terminal. No data is collected until you manually start collection at area of interest. Profile until the application ends. Generate the report#.nsys-rep in the default location.

Note

If you launch an application and that application and any descendants exit before start is called, Nsight Systems will create a fully formed .nsys-rep file containing no data.

Run application, name the session, keep only the last seconds

```
nsys start --session-new=mysession
```

```
nsys launch --session=mysession myapp [application-arguments]
```

```
nsys stop --session=mysession --keep=3
```

Effect: Create named interactive CLI process and launch your app with default collection options. Manually stop that session and keep only the last three seconds of data.

Note

Currently Nsight Systems will collect all the data and then trim the data at stop time. In the future we will add an option that does the collection in a ring buffer, so that if the user knows ahead of time how many seconds of data they wish to save we can avoid using unneeded memory.

Run application, start/stop collection using cudaProfilerStart/Stop

```
nsys start -c cudaProfilerApi
```

```
nsys launch -w true <application> [application-arguments]
```

Effect: Create interactive CLI process and set it up to begin collecting as soon as a `cudaProfilerStart()` is detected. Launch application for default analysis, sending application output to the terminal. Stop collection at next call to `cudaProfilerStop`, when the user calls `nsys stop`, or when the root process terminates. Generate the report `#.nsys-rep` in the default location.

Note

If you call `nsys launch` before `nsys start -c cudaProfilerApi` and the code contains a large number of short duration `cudaProfilerStart/Stop` pairs, Nsight Systems may be unable to process them correctly, causing a fault. This will be corrected in a future version.

Note

Use the Nsight Systems CLI option `--capture-range-end-repeat` to capture a separate report file for each capture range defined by calls to `cudaProfilerStart/Stop`. To avoid overwriting report files unexpectedly, Nsight Systems will ignore the `--force-overwrite` option in this case.

Run application, start/stop collection using NVTX

```
nsys start -c nvtx
```

```
nsys launch -w true -p MESSAGE@DOMAIN <application> [application-arguments]
```

Effect: Create interactive CLI process and set it up to begin collecting as soon as an NVTX range with a given message in a given domain (capture range) is opened. Launch application for default analysis, sending application output to the terminal. Stop collection when all capture ranges are closed, when the user calls `nsys stop`, or when the root process terminates. Generate the `report#.nsys-rep` in the default location.

Note

The Nsight Systems CLI only triggers the profiling session for the first capture range.

NVTX capture range can be specified:

- `Message@Domain`: All ranges with given message in given domain are capture ranges. For example:

```
nsys launch -w true -p profiler@service ./app
```

This would make the profiling start when the first range with message “profiler” is opened in domain “service.”

- `Message@*`: All ranges with given message in all domains are capture ranges. For example:

```
nsys launch -w true -p profiler@* ./app
```

This would make the profiling start when the first range with message “profiler” is opened in any domain.

- `Message`: All ranges with given message in default domain are capture ranges. For example:

```
nsys launch -w true -p profiler ./app
```

This would make the profiling start when the first range with message “profiler” is opened in the default domain.

- By default, only messages provided by NVTX registered strings are considered to avoid additional overhead. To enable non-registered strings check, launch your application with `NSYS_NVTX_PROFILER_REGISTER_ONLY=0` environment:

```
nsys launch -w true -p profiler@service -e NSYS_NVTX_PROFILER_REGISTER_ONLY=0 ./app
```

Note

The separator ‘@’ can be escaped with backslash ‘\’. If multiple separators without escape character are specified, only the last one is applied, all others are discarded.

Run application, start/stop collection multiple times

The interactive CLI supports multiple sequential collections per launch.

```
nsys launch <application> [application-arguments]
```

```
nsys start
```

```
nsys stop
```

```
nsys start
```

```
nsys stop
```

```
nsys shutdown --kill sigkill
```

Effect: Create interactive CLI and launch an application set up for default analysis. Send application output to the terminal. No data is collected until the start command is executed. Collect data from start until stop requested, generate `report#.qstrm` in the current working directory. Collect data from second start until the second stop request, generate `report#.nsys-rep` (incremented by one) in the current working directory. Shutdown the interactive CLI and send sigkill to the target application's process group.

Note

Calling `nsys cancel` after `nsys start` will cancel the collection without generating a report.

Example Stats Command Sequences[#]

Display default statistics

```
nsys stats report1.nsys-rep
```

Effect: Export an SQLite file named `report1.sqlite` from `report1.nsys-rep` (assuming it does not already exist). Print the default reports in column format to the console.

Note: The following two command sequences should present very similar information:

```
nsys profile --stats=true <application>
```

or

```
nsys profile <application>
```

```
nsys stats report1.nsys-rep
```

Display specific data from a report

```
nsys stats --report cuda_gpu_trace report1.nsys-rep
```

Effect: Export an SQLite file named `report1.sqlite` from `report1.nsys-rep` (assuming it does not already exist). Print the report generated by the `cuda_gpu_trace` script to the console in column format.

Generate multiple reports, in multiple formats, output multiple places

```
nsys stats --report cuda_gpu_trace --report cuda_gpu_kern_sum --report cuda_api_sum --format csv,column --output .,- report1.nsys-rep
```

Effect: Export an SQLite file named `report1.sqlite` from `report1.nsys-rep` (assuming it does not already exist). Generate three reports. The first, the `cuda_gpu_trace` report, will be output to the file `report1_cuda_gpu_trace.csv` in CSV format. The other two reports, `cuda_gpu_kern_sum` and `cuda_api_sum`, will be output to the console as columns of data. Although three reports were given, only two formats and outputs are given. To reconcile this, both the list of formats and outputs is expanded to match the list of reports by repeating the last element.

Submit report data to a command

```
nsys stats --report cuda_api_sum --format table \ --output @"grep -E (-|Name|cudaFree" test.sqlite
```

Effect: Open `test.sqlite` and run the `cuda_api_sum` script on that file. Generate table data and feed that into the command `grep -E (-|Name|cudaFree)`. The `grep` command will filter out everything but the header, formatting, and the `cudaFree` data, and display the results to the console.

Note

When the output name starts with '@,' it is defined as a command. The command is run, and the output of the report is piped to the command's stdin (standard-input). The command's stdout and stderr remain attached to the console, so any output will be displayed directly to the console.

Be aware there are some limitations in how the command string is parsed. No shell expansions (including *, ?, [], and ~) are supported. The command cannot be piped to another command, nor redirected to a file using shell syntax. The command and command arguments are split on whitespace, and no quotes (within the command syntax) are supported. For commands that require complex command line syntax, it is suggested that the command be put into a shell script file, and the script designated as the output command

Example Output from `--stats` Option[#]

The `nsys stats` command can be used post analysis to generate specific or personalized reports. For a default fixed set of summary statistics to be automatically generated, you can use the `--stats` option with the `nsys profile` or `nsys start` command to generate a fixed set of useful summary statistics.

If your run traces CUDA, these include CUDA API, Kernel, and Memory Operation statistics:

Generating cuda API Statistics...

cuda API Statistics

Time(%)	Time (ns)	Calls	Avg (ns)	Min (ns)	Max (ns)	Name
73.0	1858829425	404	4601062.9	131864	18705795	cudaMemcpy
11.3	287212369	1	287212369.0	287212369	287212369	cudaMalloc3DArray
4.3	108862768	2215	49148.0	3478	15493937	cudaGraphicsMapResources
3.3	84097966	202	416326.6	258148	2046180	cudaMalloc
3.0	75687195	201	376553.2	167486	1559709	cudaFree
2.1	54669996	2215	24681.7	3261	17194720	cudaGraphicsUnmapResources
1.5	37697367	4221	8930.9	5532	71517	cudaLaunch
1.4	36258561	202	179497.8	5441	737046	cudaMemcpyToSymbol
0.1	1961207	5	392241.4	350245	490291	cudaGraphicsGLRegisterBuffer
0.0	661494	4221	156.7	94	4855	cudaConfigureCall
0.0	469750	1	469750.0	469750	469750	cudaMemcpy3D
0.0	6513	1	6513.0	6513	6513	cudaBindTextureToArray

Generating cuda Kernel and Memory Operation Statistics...

cuda Kernel Statistics

Time(%)	Time (ns)	Instances	Avg (ns)	Min (ns)	Max (ns)	Name
38.2	20957543	1206	17377.7	9152	42272	DeviceRadixSortDownsweepKernel
36.3	19951318	1206	16543.4	15808	20961	RadixSortScanBinsKernel
13.4	7381869	1206	6121.0	3936	11776	DeviceRadixSortUpsweepKernel
12.0	6605490	603	10954.4	1920	25536	_kernel_agent

cuda Memory Operation Statistics (time)

Time(%)	Time (ns)	Operations	Avg (ns)	Min (ns)	Max (ns)	Name
71.9	1080910	606	1783.7	832	91361	[CUDA memcpy HtoD]
28.1	421799	1	421799.0	421799	421799	[CUDA memcpy HtoA]

cuda Memory Operation Statistics (bytes)

Total Bytes (KB)	Operations	Avg (KB)	Min (bytes)	Max (KB)	Name
5184.0	606	8.5551	52	1024.0	[CUDA memcpy HtoD]
4096.0	1	4096.0	4194304	4096.0	[CUDA memcpy HtoA]

If your run traces OS runtime events or NVTX push-pop ranges:

Generating Operating System Runtime API Statistics...

Operating System Runtime API Statistics

Time(%)	Time (ns)	Calls	Avg (ns)	Min (ns)	Max (ns)	Name
33.8	7780422146	388	20052634.4	1021	101325794	poll
32.5	7486252249	84	89122050.6	18165	100621271	sem_timedwait
30.4	7001017913	14	500072708.1	500054528	500094119	pthread_cond_timedwait
3.0	691921867	2879	240334.1	1000	16503430	ioctl
0.1	20746589	2156	9622.7	4703	43645	fgets
0.1	15236506	275	55405.5	1021	14452991	recvmsg
0.0	5341120	456	11713.0	1122	258129	fopen
0.0	3961960	284	13950.6	1000	91521	mmap
0.0	3660301	435	8414.5	1457	27680	fclose
0.0	1959097	246	7963.8	2252	69097	munmap
0.0	1020789	194	5261.8	2068	19845	open64
0.0	841520	489	1720.9	1000	16808	sched_yield
0.0	623388	40	15584.7	1007	50469	read
0.0	582336	158	3685.7	1289	78529	recv
0.0	279456	80	3493.2	1111	18551	writew
0.0	149645	64	2338.2	1214	10598	open
0.0	144462	5	28892.4	22780	39774	pthread_create
0.0	139762	15	9317.5	1118	77744	fread
0.0	52949	13	4073.0	1341	9112	mprotect
0.0	38777	9	4308.6	2443	10141	write
0.0	22994	4	5748.5	4763	6798	socket
0.0	21060	4	5265.0	4674	5925	sendmsg
0.0	18287	4	4571.7	2795	7277	socketpair
0.0	16881	3	5627.0	2390	7615	connect
0.0	12617	5	2523.4	1157	3926	mmap64
0.0	11368	3	3789.3	2270	5849	pipe2
0.0	11014	2	5507.0	4484	6530	pthread_cond_signal
0.0	5121	1	5121.0	5121	5121	fopen64
0.0	5118	3	1706.0	1086	2945	fcntl
0.0	4102	1	4102.0	4102	4102	shutdown
0.0	3587	1	3587.0	3587	3587	lockf
0.0	1744	1	1744.0	1744	1744	bind
0.0	1007	1	1007.0	1007	1007	fflush

Generating NVTX Push-Pop Range Statistics...

NVTX Push-Pop Range Statistics

Time(%)	Time (ns)	Instances	Avg (ns)	Min (ns)	Max (ns)	Range
-----	-----	-----	-----	-----	-----	-----
93.2	6856491504	201	34111898.0	6935189	285693359	frame
6.8	499693190	201	2486035.8	1874225	31362835	render

If your run traces graphics debug markers these include DX11 debug markers, DX12 debug markers, Vulkan debug markers or KHR debug markers:

Running [D:\src\output_host\Built\Bin\QuadD-Release\target-windows-x64\reports\vulkanmarkerssum.py D:\src\output_host\Built\Bin\QuadD-Release\target-windows-x64\marker_test\ue4_infiltrator_vulkan_markers.sqlite]...

Time(%)	Total Time (ns)	Instances	Average (ns)	Minimum (ns)	Maximum (ns)	StdDev	Range
-----	-----	-----	-----	-----	-----	-----	-----
37.2	2335018401	136	17169252.9	13692647	28504631	2729172.8	SendAllEndOfFrameUpdates
10.7	670082802	137	4891115.3	1711561	283744644	24611148.8	BasePass
5.0	311897629	1507	206965.9	6198	3407757	215626.1	BeginRenderingGBuffer
3.2	198279698	1644	120608.1	3128	952634	152230.7	BeginRenderingTranslucency
3.0	186977118	137	1364796.5	1066597	4088690	395358.6	Lights
3.0	186626496	137	1362237.2	1064426	4084804	395189.6	DirectLighting
1.7	104681274	137	764096.9	599535	2607777	238923.8	NonShadowedLights
1.5	95597952	136	702926.1	535868	1944513	269677.7	PostProcessing
1.4	85533857	137	624334.7	432949	2350711	243159.6	PrePass DDM_AllOpaque (Forced by DBuffer)
1.3	83827427	2820	29726.0	1514	755373	43994.7	BeginRenderingPrePass
1.3	81223414	137	592871.6	442856	1876611	186288.9	ShadowedLights
1.3	81100377	1777	45638.9	4951	1541354	115928.7	StandardDeferredLighting
1.3	80751508	2462	32799.2	2085	1529342	99702.1	BeginRenderingSceneColor
1.1	70866836	276	256763.9	16485	2145669	291073.5	BeginSeparateTranslucency
1.0	61329202	19591	3130.5	1228	319829	4991.0	M_FogSheet_Master_SM_FogSheet_Plane
0.8	52655480	1914	27510.7	2400	734665	60946.1	InjectTranslucentLightArray
0.8	50038202	14125	3542.5	1362	281286	5669.2	M_GodRay_MASTER_SM_GodRay_Plane
0.7	43025473	137	314054.5	231792	1002868	103224.0	InjectNonShadowedTranslucentLighting
0.6	37774169	96	393480.9	323525	550130	37746.8	UpdateGPUScene PrimitivesToUpdate and Offset = 48 0
0.6	37313346	548	68090.0	23621	252291	39130.9	GPUParticles_SimulateAndClear
0.6	37119090	273	135967.4	10130	4930980	389044.2	RenderVelocities
0.6	34849906	136	256249.3	192556	597954	92451.7	DOF(Alpha=No)
0.5	32790709	332	98767.2	13863	1703854	107206.8	VIS_ArtDemo_Screw.SpotLightStationary_7
0.5	29069102	137	212183.2	161573	539013	56026.7	GPUParticles_PreRender
0.4	25741368	411	62631.1	28555	342351	38492.3	ShadowProjectionOnOpaque
0.4	24154169	136	177604.2	130608	596279	88681.5	Bloom
0.4	23310271	548	42537.0	9120	201856	30514.7	ParticleSimulation
0.4	23121491	411	56256.7	24857	333976	35675.2	RenderShadowProjection
0.4	22939125	137	167438.9	137646	492316	45670.7	LightCompositionTasks_PreLighting
0.4	22779500	6850	3325.5	1093	203231	4304.5	Environment_Wire_MASTER_SK_Wires_02
0.3	21179479	137	154594.7	107090	1582286	133950.6	ViewOcclusionTests 0
0.3	21058892	274	76857.3	13489	447949	74050.6	VIS_ArtDemo_Screw.SM_Infil_Underground_Pipe01_439
0.3	20158748	137	147144.1	104452	1754719	149998.6	ShadowDepths
0.3	19368869	410	47241.1	5325	435358	62128.4	VelocityRendering
0.3	18867829	137	137721.4	100037	1745961	146450.6	Atlas0 2048x2048
0.3	17336627	820	21142.2	10371	119580	13048.1	InjectTranslucentVolume
0.3	17278564	137	126120.9	83753	1551792	130703.7	IndividualQueries
0.3	15776024	274	57576.7	21847	210442	33293.9	ParticleSimulationCommands
0.2	15631413	274	57049.0	13915	364140	49448.6	VIS_ArtDemo_Screw.SpotLightStationary_30
0.2	13147425	685	19193.3	10046	303482	16959.3	ShadowMapAtlases
0.2	12826154	136	94310.0	60204	1305019	109278.5	Distortion
0.2	12029306	821	14652.0	8142	237658	13625.1	Skinning000 Chunk=0 InStreamStart=0 OutStart=0 Vert=8021 Morph=0/0
0.2	11313758	2740	4129.1	1377	54848	4497.1	M_CloudSheetMaster_EditorPlane
0.2	11120575	135	82374.6	61389	313120	33641.8	VIS_ArtDemo_Keyhole.SpotLightStationary_2
0.2	10845277	1507	7196.6	3515	174162	6173.2	GBuffer
0.2	10647885	25	425915.4	338156	962796	122138.9	UpdateGPUScene PrimitivesToUpdate and Offset = 47 0
0.2	10420894	137	76064.9	57156	206479	20830.7	ComputeLightGrid

Recipes for these statistics as well as documentation on how to create your own metrics will be available in a future version of the tool.

System Wide API Trace on Windows#

On Windows, Nsight Systems can trace certain APIs (currently supported: DX11, DX12 and Vulkan) in already-running applications, by way of system-wide API trace from the CLI.

To initiate system-wide API tracing, run the Nsight Systems CLI with the `--trace` option including one or more of the supported APIs, the `--system-wide` option set to `true`, and without specifying a target application. System-wide API tracing may be combined with trace types that are always system-wide such as `--trace=wddm`.

To trace a DX11 or DX12 target application, it must gain the system focus, the user must either click on the application window or use Alt+Tab to select it.

For example, to trace multiple DX12 applications with PIX markers and GPU workload trace, as well as WDDM events for the next 20 seconds, run the command:

```
nsys profile --trace=dx12-annotations,wddm --dx12-gpu-workload=individual
--duration=20
```

Then click each of the target applications' windows to give them focus.

To trace a Vulkan target application, it must be launched after the `nsys profile` command has been executed.

Opening Command Line Results Files for Visualization#

Open CLI results in GUI

The CLI will generate an `.nsys-rep` file after analysis is complete. This file can be opened in any GUI that is the same version or a more recent version.

The opening of really large, multi-gigabyte, `.nsys-rep` files may take up all of the memory on the host computer and lock up the system. This will be fixed in a later version.

Importing Windows ETL files

For Windows targets, ETL files captured with Xperf or the `log.cmd` command supplied with GPUView in the Windows Performance Toolkit can be imported to create reports as if they were captured with Nsight Systems's "WDDM trace" and "Custom ETW trace" features. Simply choose the `.etl` file from the Import dialog to convert it to a

.nsys-rep file.

Handling Application Launchers (mpirun, deepspeed, etc)<#>

Nsight Systems has built-in API trace support for various communication APIs, such as MPI, OpenSHMEM, UCX, NCCL and NVSHMEM. To execute respective programs on multiple different machines (compute nodes), usually launchers are used, e.g., mpirun/mpiexec (MPI), shmemrun/oshrun (OpenSHMEM), srun (SLURM) or deepspeed.

In **single-node profiling** sessions, the Nsight Systems CLI can be prefixed before the program (binary) or the launcher. In the latter case, the execution of the launcher will also be profiled and all processes will be recorded into the same report file, e.g. for mpirun

```
{nsys profile [nsys args] mpirun [mpirun args] ...
```

In **multi-node profiling** sessions, the Nsight Systems CLI has to be prefixed before the application, but not before the launcher command, e.g. for mpirun

```
{mpirun [mpirun args] nsys profile [nsys args] ...
```

You can use %q{OMPI_COMM_WORLD_RANK} (Open MPI), %q{PMI_RANK} (MPICH), %q{SLURM_PROCID} (Slurm) or %p in the -o | --output option to include the rank or process ID into the report file name.

Warning

An error will occur if several processes want to write to the same report file at the same time.

Profile a Single Process or a Subset of Processes<#>

It might be reasonable to profile only a single or a few representative processes of a program run, e.g., to reduce the amount of measurement data.

To achieve this, a wrapper script can be used. The following script called *nsys_profile.sh* uses nsys to profile MPI rank 0 only.

```
#!/bin/bash
```

```
# Use $PMI_RANK for MPICH and $SLURM_PROCID with srun.  
if [ $OMPI_COMM_WORLD_RANK -eq 0 ]; then  
    nsys profile -e NSYS_MPI_STORE_TEAMS_PER_RANK=1 -t mpi "$@"  
else  
    "$@"  
fi
```

You can change the profiling options accordingly and execute with `mpirun [mpirun options] ./nsys_profile.sh ./myapp [app options]`. The above code can be easily adapted for OpenSHMEM and SLURM launchers.

Note

If only a subset of MPI ranks is profiled, set the environment variable `NSYS_MPI_STORE_TEAMS_PER_RANK=1` to store all members of custom MPI communicators per MPI rank. Otherwise, the execution might hang or fail with an MPI error.

DeepSpeed#

DeepSpeed provides a parallel launcher, which launches a Python script on multiple nodes and/or GPUs. For multi-node runs, a simple wrapper script (*nsys_profile.sh*) is required to profile with Nsight Systems.

```
#!/bin/bash  
nsys profile -t cuda,mpi,nvtx,cudnn -o rname.%p python ...
```

This above script has to be used with the `--no-python` option:

```
deepspeed --no_python [deepspeed args] ./nsys_profile.sh
```

Torchrun/Pytorch#

Here is an example of using Nsight Systems to selectively profile GPUs in a multi-gpu system using torchrun.

```
$ cat run.py
```

```
import subprocess
import sys
import os
local_rank = int(os.environ["LOCAL_RANK"])

args = sys.argv[1:]
args_string = ''.join(args)

#Define the command to execute
print(f"Profile local rank {local_rank} only")
if local_rank == 0:
    command = "nsys profile -t cuda,nvtx -o test_run python " + args_string
else:
    command = "python " + args_string

#Run the command
subprocess.run(command, shell=True)
```

```
$ torchrun --nnodes=1 --nproc-per-node=8 run.py target_python_script.py
```

GPU and NIC metrics collection<#>

If multiple instances of `nsys profile` are executed concurrently on the same node, and GPU and/or NIC metrics collection is enabled, each process will collect metrics for all available NICs and tries to collect GPU metrics for the specified devices. This can be avoided with a simple bash script similar to the following:

```
#!/bin/bash
```

```
# Use $SLURM_LOCALID with srun.  
if [ $OMPI_COMM_WORLD_LOCAL_RANK -eq 0 ]; then  
    nsys profile --nic-metrics=true --gpu-metrics-devices=all "$@"  
else  
    nsys profile "$@"  
fi
```

This above script will collect NIC and GPU metrics only for one rank, the node-local rank 0. Alternatively, if one rank per GPU is used, the GPU metrics devices can be specified based on the node-local rank in a wrapper script as follows:

```
#!/bin/bash
```

```
# Use $SLURM_LOCALID with srun.  
nsys profile -e CUDA_VISIBLE_DEVICES=${OMPI_COMM_WORLD_LOCAL_RANK}\  
--gpu-metrics-devices=${OMPI_COMM_WORLD_LOCAL_RANK} "$@"
```

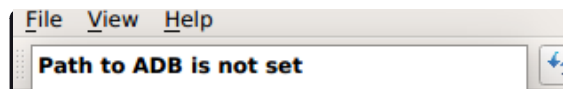
Profiling from the GUI#

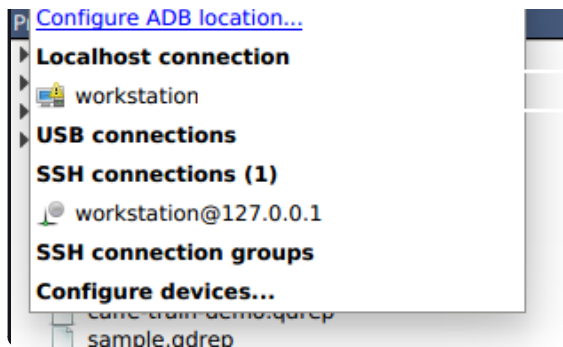
Profiling Linux Targets from the GUI#

Connecting to the Target Device#

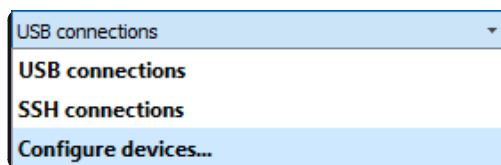
Nsight Systems provides a simple interface to profile on localhost or manage multiple connections to Linux or Windows based devices via SSH. The network connections manager can be launched through the device selection dropdown:

On x86_64:

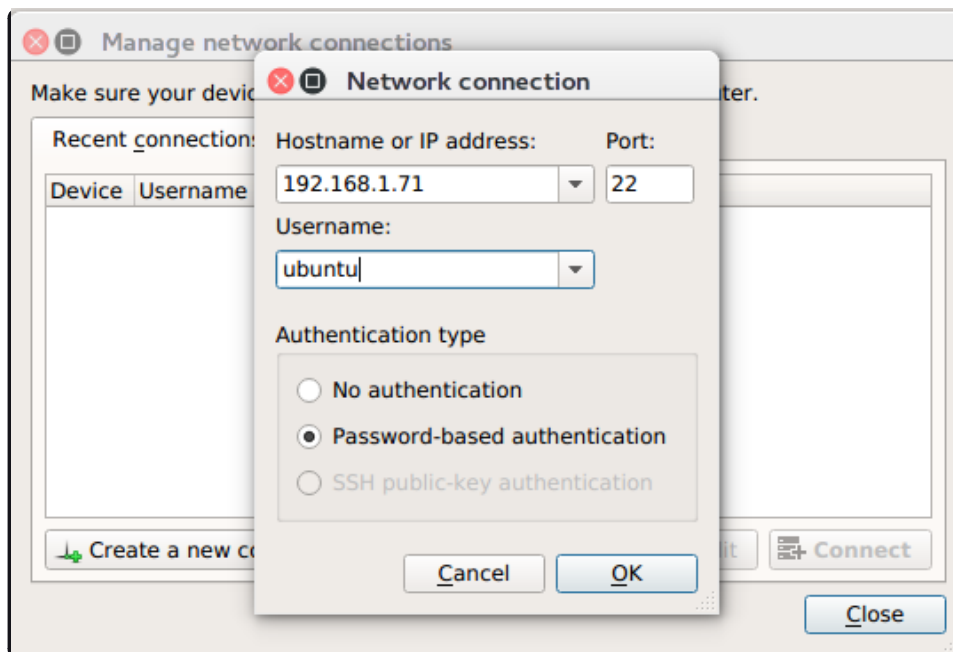




On Tegra:



The dialog has simple controls that allow adding, removing, and modifying connections:



Security notice: SSH is only used to establish the initial connection to a target device, perform checks, and upload necessary files. The actual profiling commands and data are transferred through a raw, unencrypted socket. Nsight

Systems should not be used in a network setup where attacker-in-the-middle attack is possible, or where untrusted parties may have network access to the target device.

While connecting to the target device, you will be prompted to input the user's password. Note that if you choose to remember the password, it will be stored in plain text in the configuration file on the host. Stored passwords are bound to the public key fingerprint of the remote device.

The **No authentication** option is useful for devices configured for passwordless login using `root` username. To enable such a configuration, edit the file `/etc/ssh/sshd_config` on the target and specify the following option:

```
PermitRootLogin yes
```

Then set empty password using `passwd` and restart the SSH service with `service ssh restart`.

Open ports: The Nsight Systems daemon requires port 22 and port 45555 to be open for listening. You can confirm that these ports are open with the following command:

```
sudo firewall-cmd --list-ports --permanent  
sudo firewall-cmd --reload
```

To open a port use the following command, skip `--permanent` option to open only for this session:

```
sudo firewall-cmd --permanent --add-port 45555/tcp  
sudo firewall-cmd --reload
```

Likewise, if you are running on a cloud system, you must open port 22 and port 45555 for ingress.

Kernel Version Number - To check for the version number of the kernel support of Nsight Systems on a target device, run the following command on the remote device:

```
cat /proc/quadd/version
```

Minimal supported version is 1.82.

Additionally, presence of Netcat command (`nc`) is required on the target device. For example, on Ubuntu this package can be installed using the following command:

```
sudo apt-get install netcat-openbsd
```

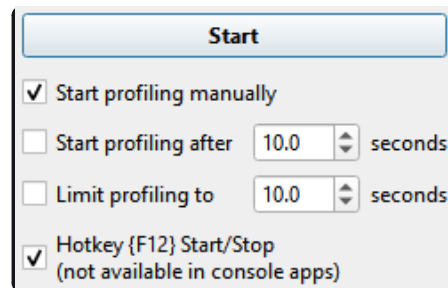
System-Wide Profiling Options#

Target Sampling Options#

Target sampling behavior is somewhat different for Nsight Systems Workstation Edition and Nsight Systems Embedded Platforms Edition.

Hotkey Trace Start/Stop#

Nsight Systems Workstation Edition can use hotkeys to control profiling. Press the hotkey to start and/or stop a trace session from within the target application's graphic window. This is useful when tracing games and graphic applications that use fullscreen display. In these scenarios, switching to Nsight Systems' UI would unnecessarily introduce the window manager's footprint into the trace. To enable the use of Hotkey, check the Hotkey checkbox in the project settings page:



Start

☒ Start profiling manually

☐ Start profiling after 10.0 seconds

☐ Limit profiling to 10.0 seconds

☒ Hotkey {F12} Start/Stop
(not available in console apps)

The default hotkey is F12.

Launching Processes#

Nsight Systems can launch new processes for profiling on target devices. The profiler ensures that all environment variables are set correctly to successfully collect trace information

Specify process launch options below

Command line with arguments: [Edit arguments](#)

Working directory:

The **Edit arguments...** link will open an editor window, where every command line argument is edited on a separate line. This is convenient when arguments contain spaces or quotes.

Profiling Windows Targets from the GUI#

Profiling on Windows devices is similar to the profiling on Linux devices. Please refer to the [Profiling Linux Targets from the GUI](#) section for the detailed documentation and connection information. The major differences on the platforms are listed below:

Remoting to a Windows Based Machine#

To perform remote profiling to a target Windows based machines, [install and configure an OpenSSH Server on the target machine](#).

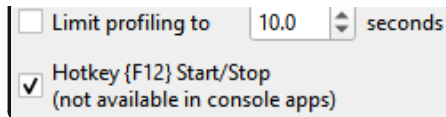
Hotkey Trace Start/Stop#

Nsight Systems Workstation Edition can use hotkeys to control profiling. Press the hotkey to start and/or stop a trace session from within the target application's graphic window. This is useful when tracing games and graphic applications that use fullscreen display. In these scenarios, switching to Nsight Systems' UI would unnecessarily introduce the window manager's footprint into the trace. To enable the use of Hotkey, check the Hotkey checkbox in the project settings page:

Start

☒ Start profiling manually

☐ Start profiling after seconds



☐ Limit profiling to 10.0 seconds

☒ Hotkey {F12} Start/Stop
(not available in console apps)

The default hotkey is F12.

Changing the Default Hotkey Binding - A different hotkey binding can be configured by setting the `HotKeyIntValue` configuration field in the `config.ini` file.

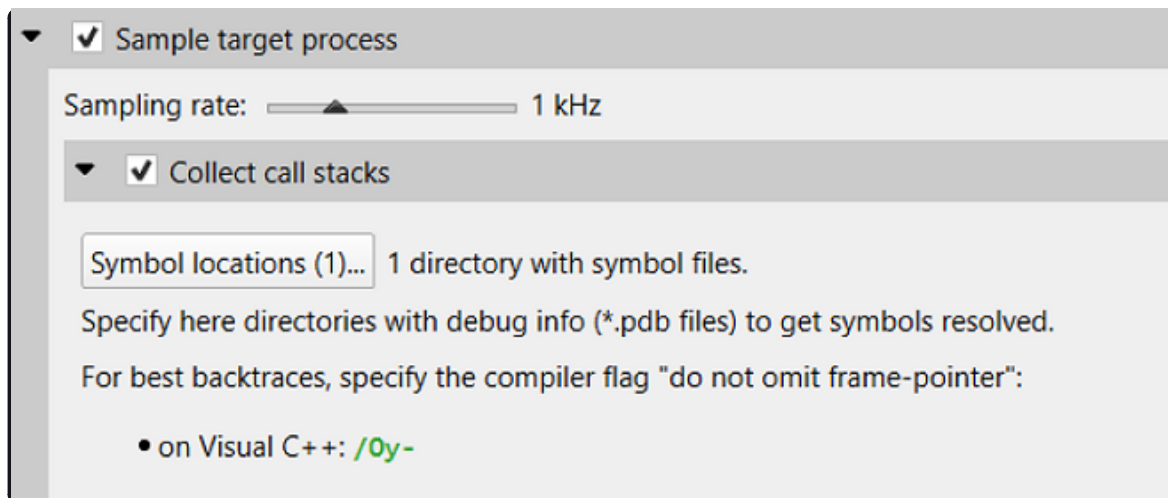
Set the decimal numeric identifier of the hotkey you would like to use for triggering start/stop from the target app graphics window. The default value is 123 which corresponds to 0x7B, or the F12 key.

Virtual key identifiers are detailed in [MSDN's Virtual-Key Codes](#).

Note that you must convert the hexadecimal values detailed in this page to their decimal counterpart before using them in the file. For example, to use the F1 key as a start/stop trace hotkey, use the following settings in the `config.ini` file:

`HotKeyIntValue=112`

Target Sampling Options on Windows#



☒ Sample target process

Sampling rate: 1 kHz

☒ Collect call stacks

Symbol locations (1)... 1 directory with symbol files.

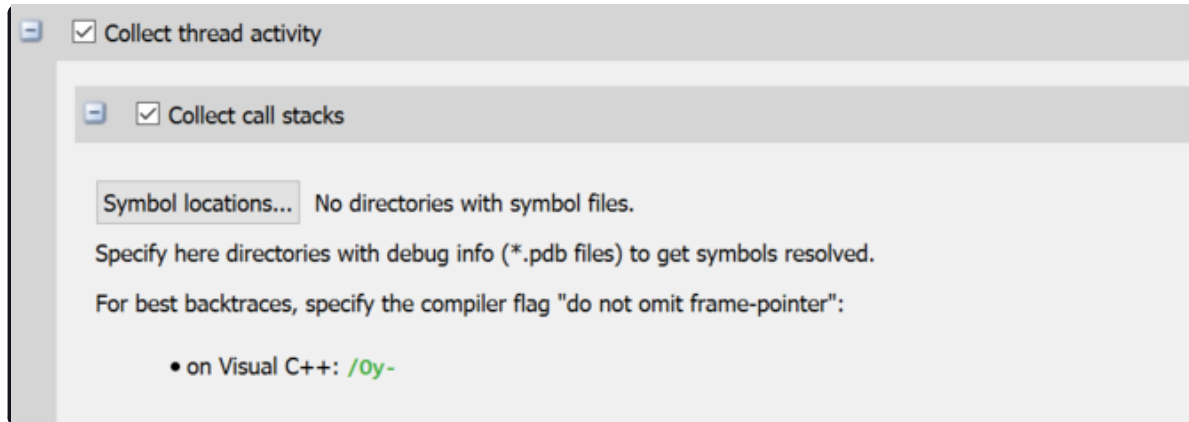
Specify here directories with debug info (*.pdb files) to get symbols resolved.

For best backtraces, specify the compiler flag "do not omit frame-pointer":

- on Visual C++: `/Oy-`

Nsight Systems can sample one process tree. Sampling here means interrupting each processor periodically. The

sampling rate is defined in the project settings and is either 100Hz, 1KHz (default value), 2KHz, 4KHz, or 8KHz.



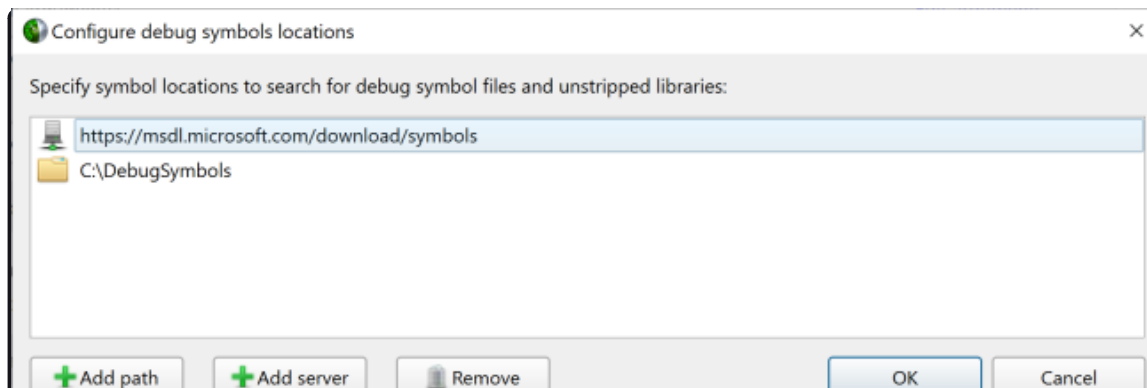
On Windows, Nsight Systems can collect thread activity of one process tree. Collecting thread activity means that each thread context switch event is logged and (optionally) a backtrace is collected at the point that the thread is scheduled back for execution. Thread states are displayed on the timeline.

If it was collected, the thread backtrace is displayed when hovering over a region where the thread execution is blocked.

Symbol Locations#

Symbol resolution happens on host, and therefore does not affect performance of profiling on the target.

Press the **Symbol locations...** button to open the **Configure debug symbols location** dialog.





Local symbols cache: C:\SymCache Change

Use this dialog to specify:

- Paths of PDB files
- Symbols servers
- The location of the local symbol cache

To use a symbol server:

1. Install **Debugging Tools for Windows**, a part of the [Windows 10 SDK](#).
2. Add the symbol server URL using the **Add Server** button.

Information about Microsoft's public symbol server, which enables getting Windows operating system related debug symbols can be found [here](#).

Profiling QNX Targets from the GUI#

Profiling on QNX devices is similar to the profiling on Linux devices. Please refer to the [Profiling Linux Targets from the GUI](#) section for the detailed documentation. The major differences on the platforms are listed below:

- Backtrace sampling is not supported. Instead backtraces are collected for long OS runtime libraries calls. Please refer to the [OS Runtime Libraries Trace](#) section for the detailed documentation.
- CUDA support is limited to CUDA 9.0+.
- Filesystem on QNX device might be mounted read-only. In that case Nsight Systems is not able to install target-side binaries, required to run the profiling session. Please make sure that target filesystem is writable before connecting to QNX target. For example, make sure the following command works:

```
echo XX > /xx && ls -l /xx
```

Profiling within JupyterLab[#]

The JupyterLab Nsight extension integrates Nsight Systems profiling into JupyterLab for profiling of Jupyter notebook cells. CUDA kernels launched by the cells as well as CUDA and Python code execution can be profiled and analyzed.

For more information and to install the extension, go to [JupyterLab Nsight extension on PyPI](#)

Example - How to use Nsight Systems to profile code in individual cells of a Jupyter notebook.

- Launch jupyter-lab with Nsight Systems using the desired trace options. For example:

```
(nsys launch --trace=cuda,nvtx,cublas,cudnn jupyter-lab
```
- (optional) Add NVTX ranges to the important operations in the notebook using `range_push` and `range_pop`. These NVTX ranges add information but are not used to define the profiling capture.
- To profile a cell, add a shell command to `nsys start` at the top of the cell and a shell command to `nsys stop` at the bottom of the cell. We recommend using the absolute path to `nsys` on your system to make sure it is found.
- Save the notebook.
- Run all the cells required for the code you want to profile, then run the cell you want to profile.
- Each time the cell with `nsys start` and `nsys stop` is run, a new `.nsys-rep` file will be generated.
- Open the `nsys-rep` file in `nsys-ui`.

Container and Scheduler Support[#]

Collecting Data Within a Container[#]

While examples in this section use Docker container semantics, other containers work much the same.

The following information assumes the reader is knowledgeable regarding Docker containers. For further information about Docker use in general, see the [Docker documentation](#).

We strongly recommend using the CLI to profile in a container. Best container practice is to split services across containers when they do not require colocation. The Nsight Systems GUI is not needed to profile and brings in many dependencies, so the CLI is recommended. If you wish, the GUI can be in a separate side-car container you use to view your report. All you need is a shared folder between the containers. See section on [GUI VNC container](#) for more information.

Enable Docker Collection#

When starting the Docker to perform a Nsight Systems collection, additional steps are required to enable the `perf_event_open` system call. This is required in order to utilize the Linux kernel's `perf` subsystem which provides sampling information to Nsight Systems.

There are three ways to enable the `perf_event_open` syscall. You can enable it by using the `--privileged=true` switch, adding `--cap-add=SYS_ADMIN` switch to your docker run command file, or you can enable it by setting the seccomp security profile if your system meets the requirements.

Secure computing mode (seccomp) is a feature of the Linux kernel that can be used to restrict an application's access. This feature is available only if the kernel is enabled with seccomp support. To check for seccomp support:

```
$ grep CONFIG_SECCOMP= /boot/config-$(uname -r)
```

The official Docker documentation says:

"Seccomp profiles require seccomp 2.2.1 which is not available on Ubuntu 14.04, Debian Wheezy, or Debian Jessie. To use seccomp on these distributions, you must download the latest static Linux binaries (rather than packages)."

Download the default seccomp profile file, `default.json`, relevant to your Docker version. If `perf_event_open` is already listed in the file as guarded by `CAP_SYS_ADMIN`, then remove the `perf_event_open` line. Add the following lines under "syscalls" and save the resulting file as `default_with_perf.json`.

```
{  
  "name": "perf_event_open",
```

```
"action": "SCMP_ACT_ALLOW",  
"args": []  
},
```

Then you will be able to use the following switch when starting the Docker to apply the new seccomp profile.

```
--security-opt seccomp=default_with_perf.json
```

Launch Docker Collection#

Here is an example command that has been used to launch a Docker for testing with Nsight Systems:

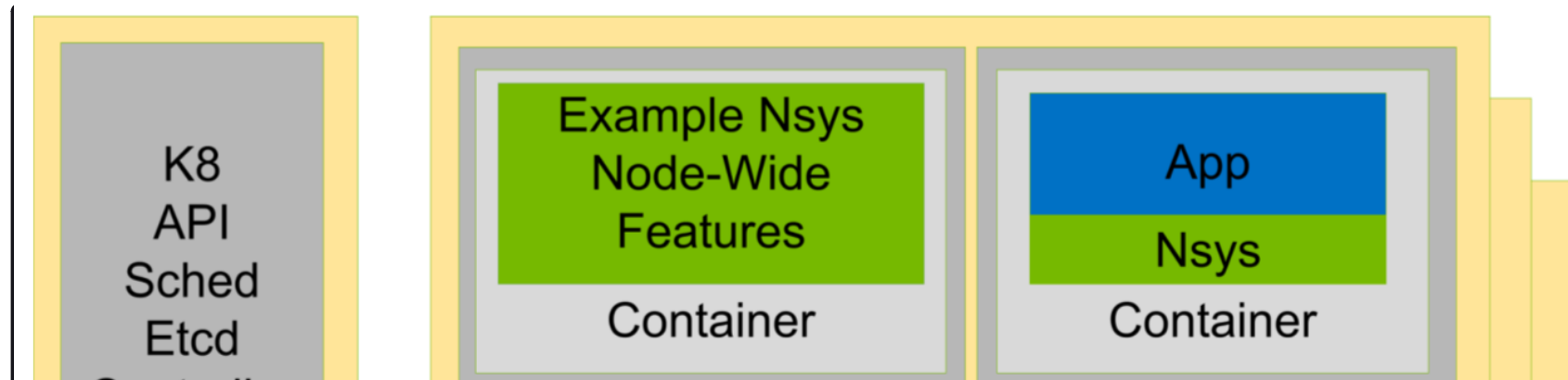
```
sudo nvidia-docker run --network=host --security-opt seccomp=default_with_perf.json --rm -ti caffe-demo2 bash
```

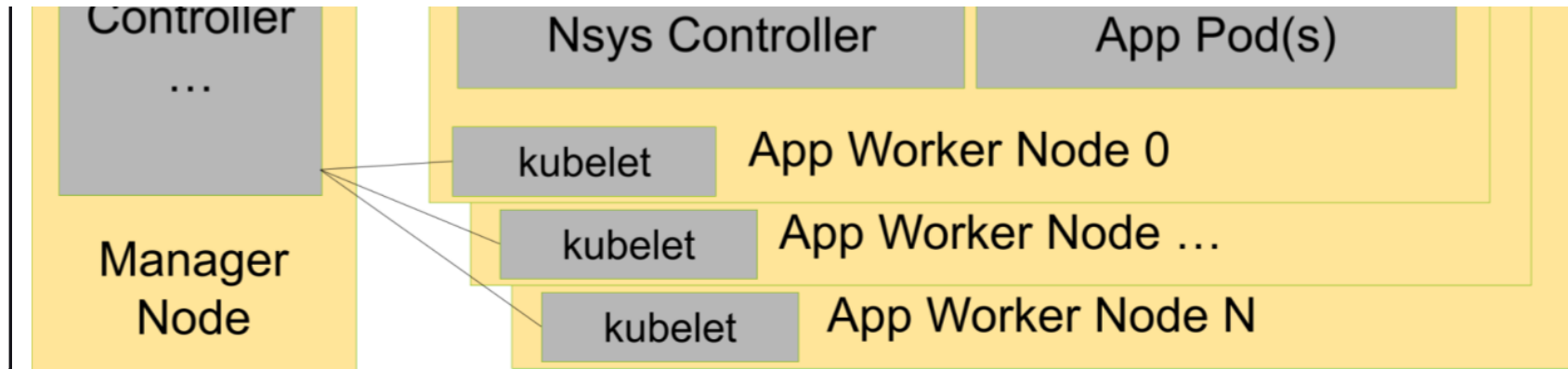
There is a known issue where Docker collections terminate prematurely with older versions of the driver and the CUDA Toolkit. If collection is ending unexpectedly, please update to the latest versions.

After the Docker has been started, use the Nsight Systems CLI to launch a collection within the Docker. The resulting file can be imported into the Nsight Systems host like any other CLI result.

Profiling Services Launched via Kubernetes#

Nsight Systems now can provide profiling via sidecar injection without need to modify your containers or k8/helm specs.





Once the sidecar is enabled, the data collected data can be filtered by namespace or pod using Kubernetes labels, or within a container process by using command-line regex.

This functionality is compatible with various cloud service provider's in-house managed Kubernetes variants including AKS, EKS, GKE, and OKE.

Documentation and download for this sidecar is available at [NGC Nsight Operator](#).

Nsight Streamer for Nsight Systems#

A self-hosted NVIDIA Nsight Systems GUI running inside a Docker container enables remote access through a web browser. This configuration is particularly useful for analyzing data on remote servers or clusters.

For more information and instructions on running the container, visit: [Nsight Streamer for Nsight Systems on NGC](#)

GUI VNC container#

Nsight Systems provides a build script to build a self-isolated Docker container with the Nsight Systems GUI and VNC server.

You can find the `build.py` script in the `host-linux-x64/Scripts/VncContainer` directory (or similar on other architectures) under your Nsight Systems installation directory. You will need to have [Docker](#), and Python 3.5 or later.

Available Parameters

Short Name	Full Name	Description
	<code>--vnc-password</code>	(optional) Default password for VNC access (at least 6 characters). If it is specified and empty user will be asked during the build. Can be changed when running a container.
<code>-aba</code>	<code>--additional-build-arguments</code>	(optional) Additional arguments, which will be passed to the docker build command.
<code>-hd</code>	<code>--nsys-host-directory</code>	(optional) The directory with Nsight Systems host binaries (with GUI).
<code>-td</code>	<code>--nsys-target-directory</code>	(optional, repeatable) The directory with Nsight Systems target binaries (can be specified multiple times).
	<code>--tigervnc</code>	(optional) Use TigerVNC instead of x11vnc.
	<code>--http</code>	(optional) Install noVNC in the Docker container for HTTP access.
	<code>--rdp</code>	(optional) Install xRDP in the Docker for RDP access.
	<code>--geometry</code>	(optional) Default VNC server resolution in the format WidthxHeight (default 1920x1080).
	<code>--build-directory</code>	(optional) The directory to save temporary files (with the write access for the current user). By default, script or tmp directory will be used.

Ports

These ports can be published from the container to provide access to the Docker container:

Port	Purpose	Condition
TCP 5900	Port for VNC access	
TCP 80 (optional)	Port for HTTP access to noVNC server	Container is build with <code>--http</code> parameter
TCP 3389 (optional)	Port for RDP access	Container is build with <code>--rdp</code> parameter

Volumes

Docker folder	Purpose	Description
/mnt/host	Root path for shared folders	Folder owned by the Docker user (inner content can be accessed from Nsight Systems GUI)
/mnt/host/Projects		Folder with projects and reports, created by Nsight Systems UI in container
/mnt/host/logs	Folder with inner services logs	May be useful to send reports to developers

Environment variables

Variable Name	Purpose
VNC_PASSWORD	Password for VNC access (at least 6 characters)

Variable Name	Purpose
NSYS_WINDOW_WIDTH	Width of VNC server display (in pixels)
NSYS_WINDOW_HEIGHT	Height of VNC server display (in pixels)

Examples

With VNC access on port 5916:

```
sudo docker run -p 5916:5900/tcp -ti nsys-ui-vnc:1.0
```

With VNC access on port 5916 and HTTP access on port 8080:

```
sudo docker run -p 5916:5900/tcp -p 8080:80/tcp -ti nsys-ui-vnc:1.0
```

With VNC access on port 5916, HTTP access on port 8080, and RDP access on port 33890:

```
sudo docker run -p 5916:5900/tcp -p 8080:80/tcp -p 33890:3389/tcp -ti nsys-ui-vnc:1.0
```

With VNC access on port 5916, shared “HOME” folder from the host, VNC server resolution 3840x2160, and custom VNC password:

```
sudo docker run -p 5916:5900/tcp -v $HOME:/mnt/host/home -e NSYS_WINDOW_WIDTH=3840 -e  
NSYS_WINDOW_HEIGHT=2160 -e VNC_PASSWORD=7654321 -ti nsys-ui-vnc:1.0
```

With VNC access on port 5916, shared “HOME” folder from the host, and the projects folder to access reports created by the Nsight Systems GUI in container:

```
sudo docker run -p 5916:5900/tcp -v $HOME:/mnt/host/home -v /opt/NsysProjects:/mnt/host/Projects -ti nsys-ui-  
vnc:1.0
```

Migrating from NVIDIA nvprof#

Using the Nsight Systems CLI nvprof Command#

The `nvprof` command of the Nsight Systems CLI is intended to help former `nvprof` users transition to `nsys`. Many `nvprof` switches are not supported by `nsys`, often because they are now part of NVIDIA Nsight Compute.

The full `nvprof` documentation can be found at <https://docs.nvidia.com/cuda/profiler-users-guide>.

The `nvprof` transition guide for Nsight Compute can be found at <https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html#nvprof-guide>.

Any `nvprof` switch not listed below is not supported by the `nsys nvprof` command. No additional `nsys` functionality is available through this command. New features will not be added to this command in the future.

CLI `nvprof` Command Switch Options#

After choosing the `nvprof` command switch, the following options are available. When you are ready to move to using Nsight Systems CLI directly, see [Command Line Options](#) documentation for the `nsys` switch(es) given below. Note that the `nsys` implementation and output may vary from `nvprof`.

Usage.

`nsys nvprof [options]`

Switch	Parameters (Default in Bold)	nsys switch	Switch Description
<code>--annotate-mpi</code>	off , <code>openmpi</code> , <code>mpich</code>	<code>--trace=mpi</code> AND <code>--mpi-impl</code>	Automatically annotate MPI calls with NVTX markers. Specify the MPI implementation installed on your machine. Only OpenMPI and MPICH implementations are supported.
<code>--cpu-thread-</code>	<code>on</code> , off	<code>--trace=osrt</code>	Collect information about CPU thread API activity.

Switch	Parameters (Default in Bold)	nsys switch	Switch Description
tracing			
-- profile- api-trace	none, runtime, driver, all	--trace=cuda	Turn on/off CUDA runtime and driver API tracing. For Nsight Systems there is no separate CUDA runtime and CUDA driver trace, so selecting runtime or driver is equivalent to selecting all.
-- profile- from- start	on , off	if off use --capture- range=cudaProfilerApi	Enable/disable profiling from the start of the application. If disabled, the application can use {cu,cuda}Profiler{Start,Stop} to turn on/off profiling.
-t -- timeout	<nanoseconds> default= 0	--duration=seconds	If greater than 0, stop the collection and kill the launched application after timeout seconds. nvprof started counting when the CUDA driver is initialized. nsys starts counting immediately.
--cpu- profiling	on, off	--sampling=cpu	Turn on/off CPU profiling
-- openacc- profiling	on , off	--trace=openacc to turn on	Enable/disable recording information from the OpenACC profiling interface. Note: OpenACC profiling interface depends on the presence of the OpenACC runtime. For

Switch	Parameters (Default in Bold)	nsys switch	Switch Description
			supported runtimes, see the CUDA Trace section of the documentation .
-o -- export- profile	<filename>	--output={filename} and/or --export=sqlite	Export named file to be imported or opened in the Nsight Systems GUI. %q{ENV_VAR} in string will be replaced with the value of the environment variable. If not set this is an error. %h is replaced with the system hostname. % % in the string is replaced with %. %p in the string is not supported currently. Any other character following % is illegal. The default is report1, with the number incrementing to avoid overwriting files, in the user's working directory.
-f -- force- overwrite		--force-overwrite=true	Force overwriting all output files with same name.
-h --help		--help	Print Nsight Systems CLI help
-V -- version		--version	Print Nsight Systems CLI version information

Next Steps#

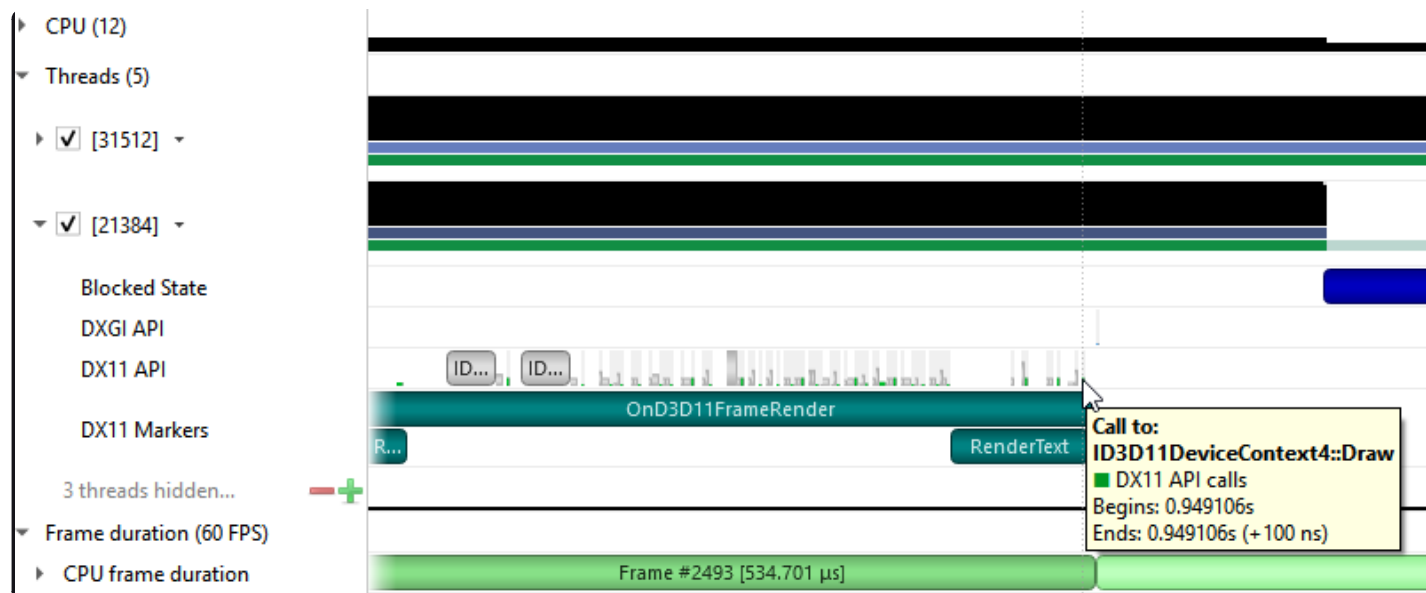
NVIDIA Visual Profiler (NVVP) and NVIDIA nvprof are deprecated. New GPUs and features will not be supported by those tools. We encourage you to make the move to Nsight Systems now. For additional information, suggestions, and rationale, see the blog series in [Other Resources](#).

Direct3D Trace#

Nsight Systems has the ability to trace both the Direct3D 11 API and the Direct3D 12 API on Windows targets.

D3D11 API trace#

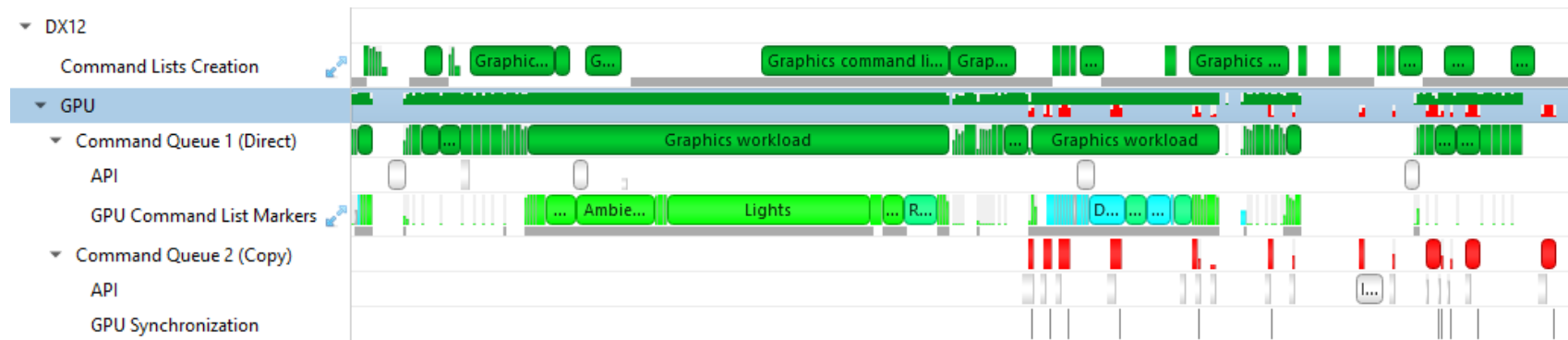
Nsight Systems can capture information about Direct3D 11 API calls made by the profiled process. This includes capturing the execution time of D3D11 API functions, performance markers, and frame durations.



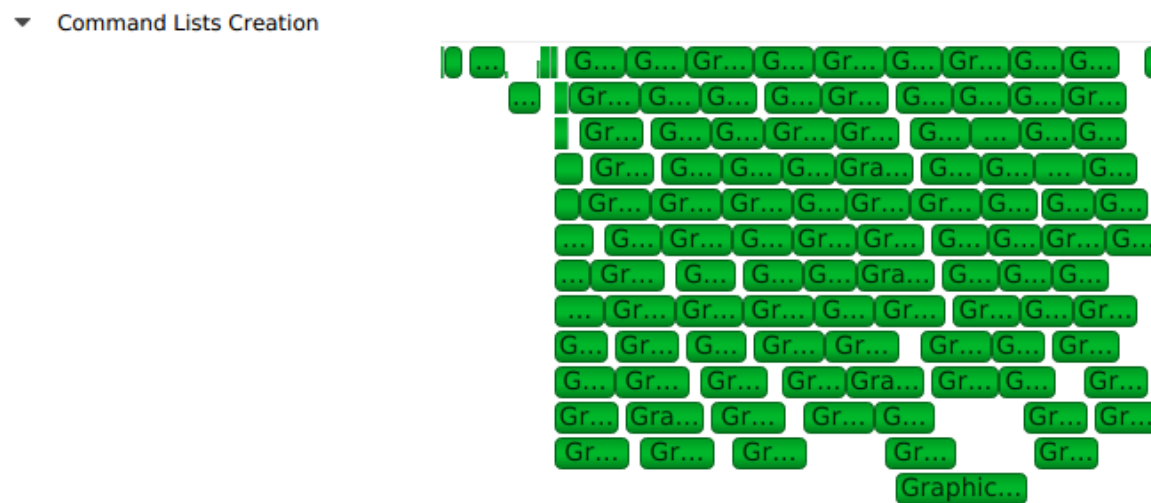
D3D12 API Trace#

Direct3D 12 is a low-overhead 3D graphics and compute API for Microsoft Windows. Information about Direct3D 12 can be found at the [Direct3D 12 Programming Guide](#).

Nsight Systems can capture information about Direct3D 12 usage by the profiled process. This includes capturing the execution time of D3D12 API functions, corresponding workloads executed on the GPU, performance markers, and frame durations.



The Command List Creation row displays time periods when command lists were being created. This enables developers to improve their application's multi-threaded command list creation. Command list creation time period is measured between the call to `ID3D12GraphicsCommandList::Reset` and the call to `ID3D12GraphicsCommandList::Close`.



The GPU row shows a compressed view of the D3D12 queue activity, color-coded by the queue type. Expanding it will show the individual queues and their corresponding API calls.

A Command Queue row is displayed for each D3D12 command queue created by the profiled application. The row's header displays the queue's running index and its type (Direct, Compute, Copy).

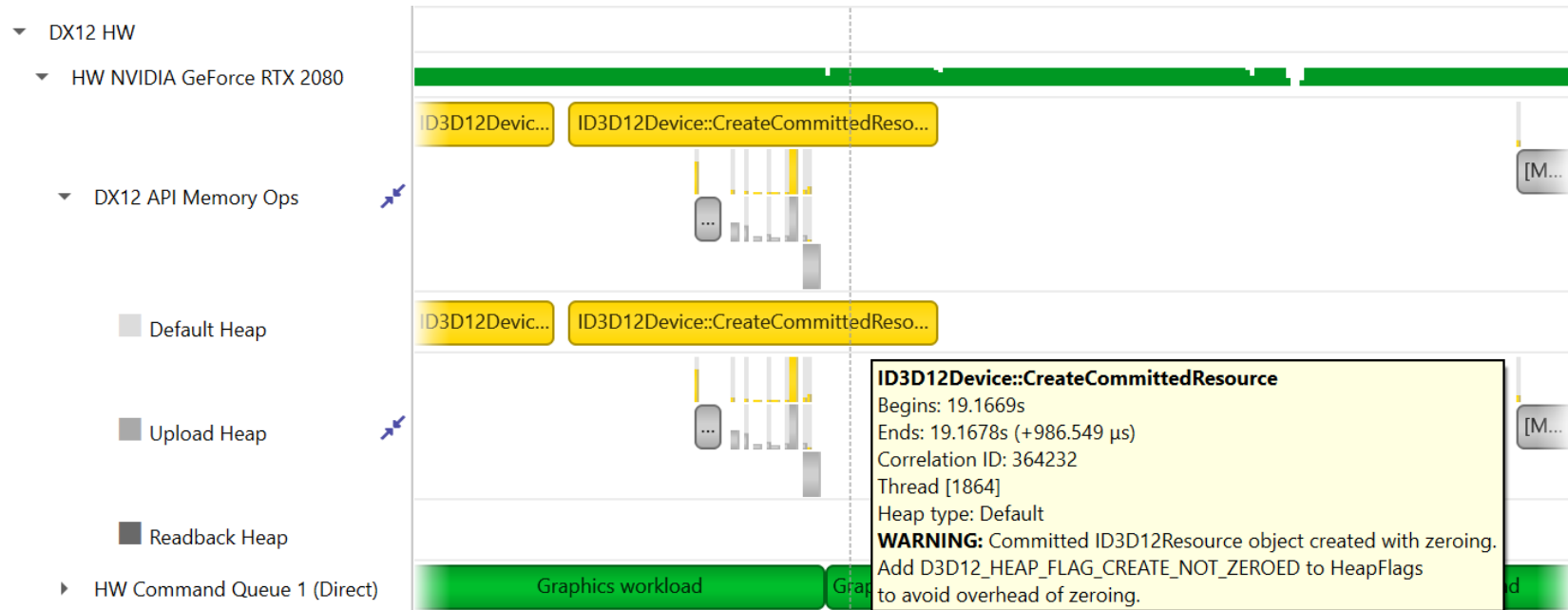
- ▶ Command Queue 0 (Compute)
- ▶ Command Queue 1 (Direct)

The DX12 API Memory Ops row displays all API memory operations and non-persistent resource mappings. Event ranges in the row are color-coded by the heap type they belong to (Default, Readback, Upload, Custom, or CPU-Visible VRAM), with usage warnings highlighted in yellow. A breakdown of the operations can be found by expanding the row to show rows for each individual heap type.

The following operations and warnings are shown:

- Calls to `ID3D12Device::CreateCommittedResource`, `ID3D12Device4::CreateCommittedResource1`, and `ID3D12Device8::CreateCommittedResource2`
- A warning will be reported if `D3D12_HEAP_FLAG_CREATE_NOT_ZEROED` is not set in the method's `HeapFlags` parameter.
- Calls to `ID3D12Device::CreateHeap` and `ID3D12Device4::CreateHeap1`
- A warning will be reported if `D3D12_HEAP_FLAG_CREATE_NOT_ZEROED` is not set in the `Flags` field of the method's `pDesc` parameter
- Calls to `ID3D12Resource::ReadFromSubResource`
- A warning will be reported if the read is to a `D3D12_CPU_PAGE_PROPERTY_WRITE_COMBINE` CPU page or from a `D3D12_HEAP_TYPE_UPLOAD` resource.
- Calls to `ID3D12Resource::WriteToSubResource`
- A warning will be reported if the write is from a `D3D12_CPU_PAGE_PROPERTY_WRITE_BACK` CPU page or to a `D3D12_HEAP_TYPE_READBACK` resource.
- Calls to `ID3D12Resource::Map` and `ID3D12Resource::Unmap` will be matched into [Map, Unmap] ranges for

non-persistent mappings. If a mapping range is nested, only the most external range (reference count = 1) will be shown.



The API row displays time periods where `ID3D12CommandQueue::ExecuteCommandLists` was called. The GPU Workload row displays time periods where workloads were executed by the GPU. The workload's type (Graphics, Compute, Copy, etc.) is displayed on the bar representing the workload's GPU execution.



In addition, you can see the PIX command queue CPU-side performance markers, GPU-side performance markers, and the GPU Command List performance markers, each in their row.





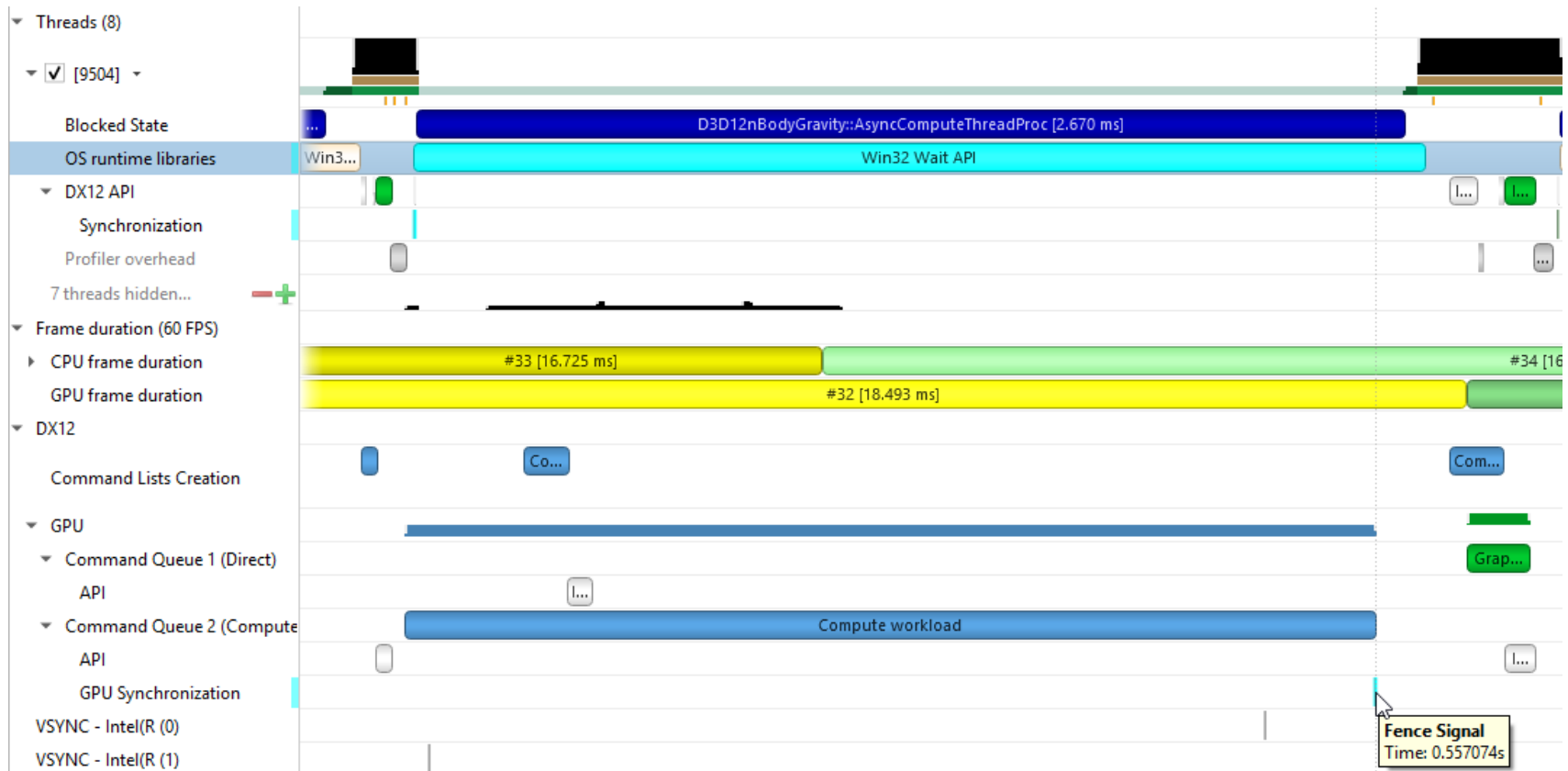
Clicking on a GPU workload highlights the corresponding `ID3D12CommandQueue::ExecuteCommandLists`, `ID3D12GraphicsCommandList::Reset` and `ID3D12GraphicsCommandList::Close` API calls, and vice versa.



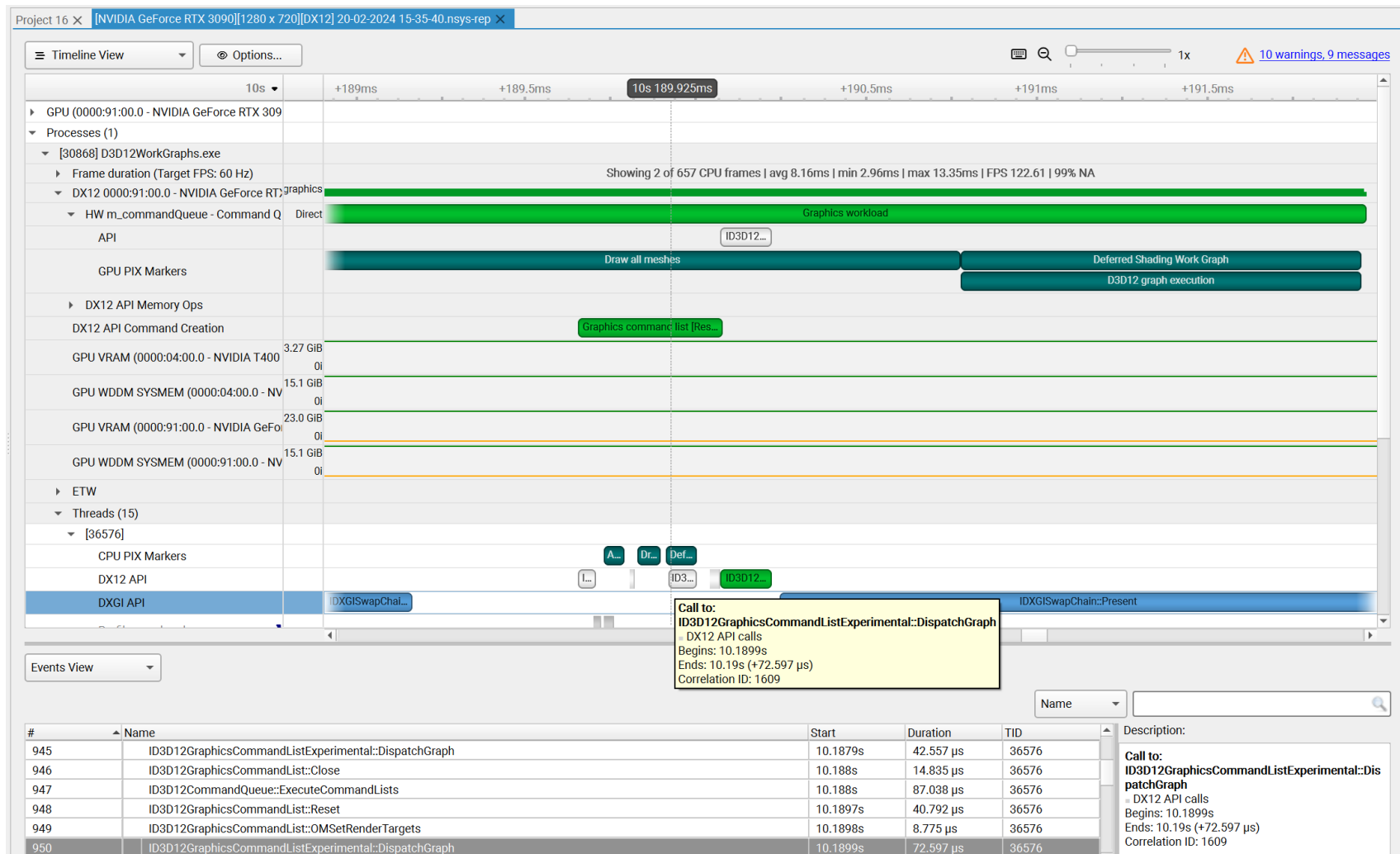
Detecting which CPU thread was blocked by a fence can be difficult in complex apps that run tens of CPU threads. The timeline view displays the 3 operations involved:

- The CPU thread pushing a signal command and fence value into the command queue. This is displayed on the DX12 Synchronization sub-row of the calling thread.
- The GPU executing that command, setting the fence value and signaling the fence. This is displayed on the GPU Queue Synchronization sub-row.
- The CPU thread calling a Win32 wait API to block-wait until the fence is signaled. This is displayed on the Thread's OS runtime libraries row.

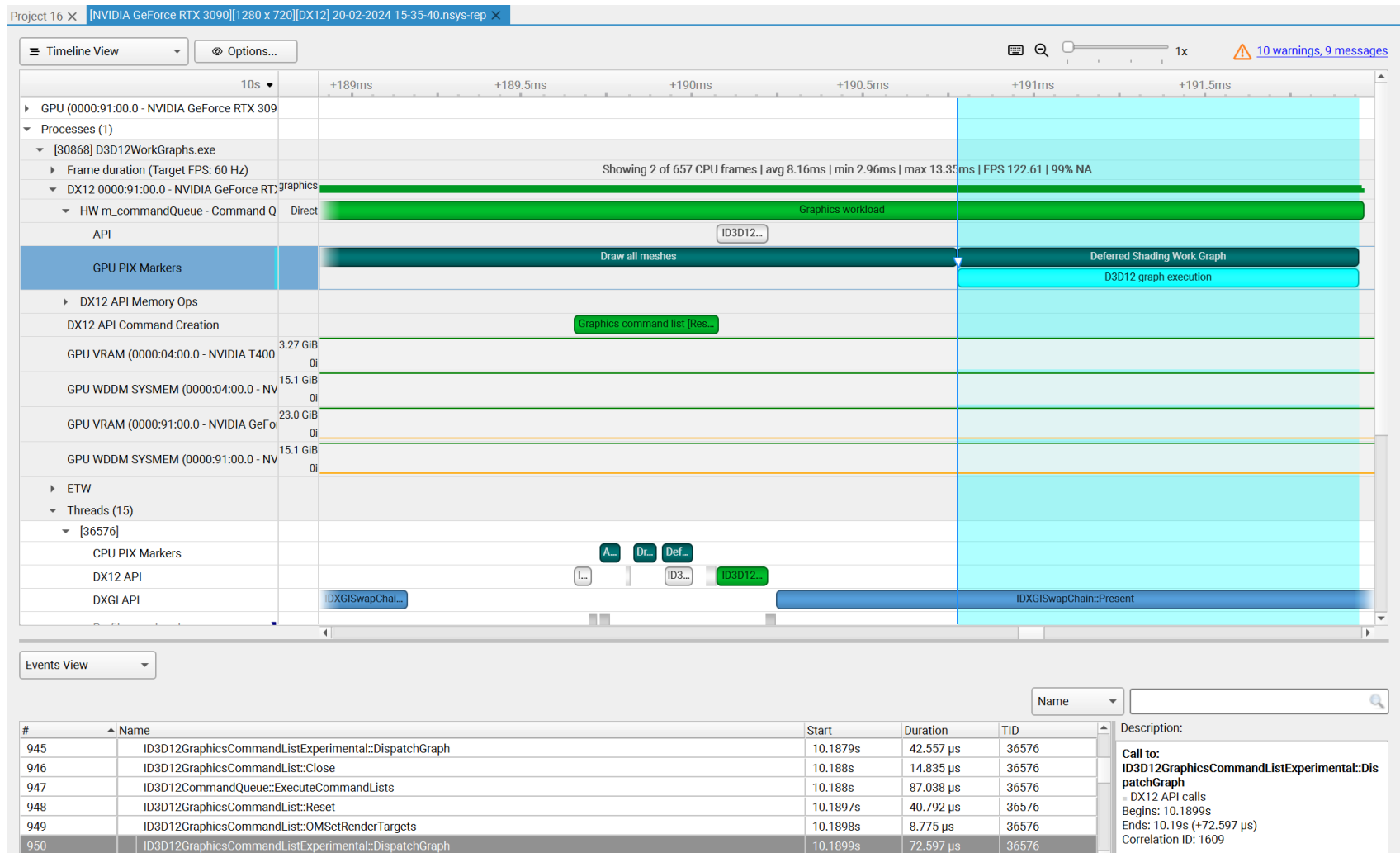
Clicking one of these will highlight it and the corresponding other two calls.



Nsight Systems D3D12 trace captures D3D12 Work Graphs dispatch calls to `DispatchGraph` and time boundaries of the GPU execution of the work graph.



The DX12 API row displays ID3D12GraphicsCommandList10::DispatchGraph calls. The GPU PIX Markers row marks graph execution by the GPU with a custom marker captioned “D3D12 graph execution.”

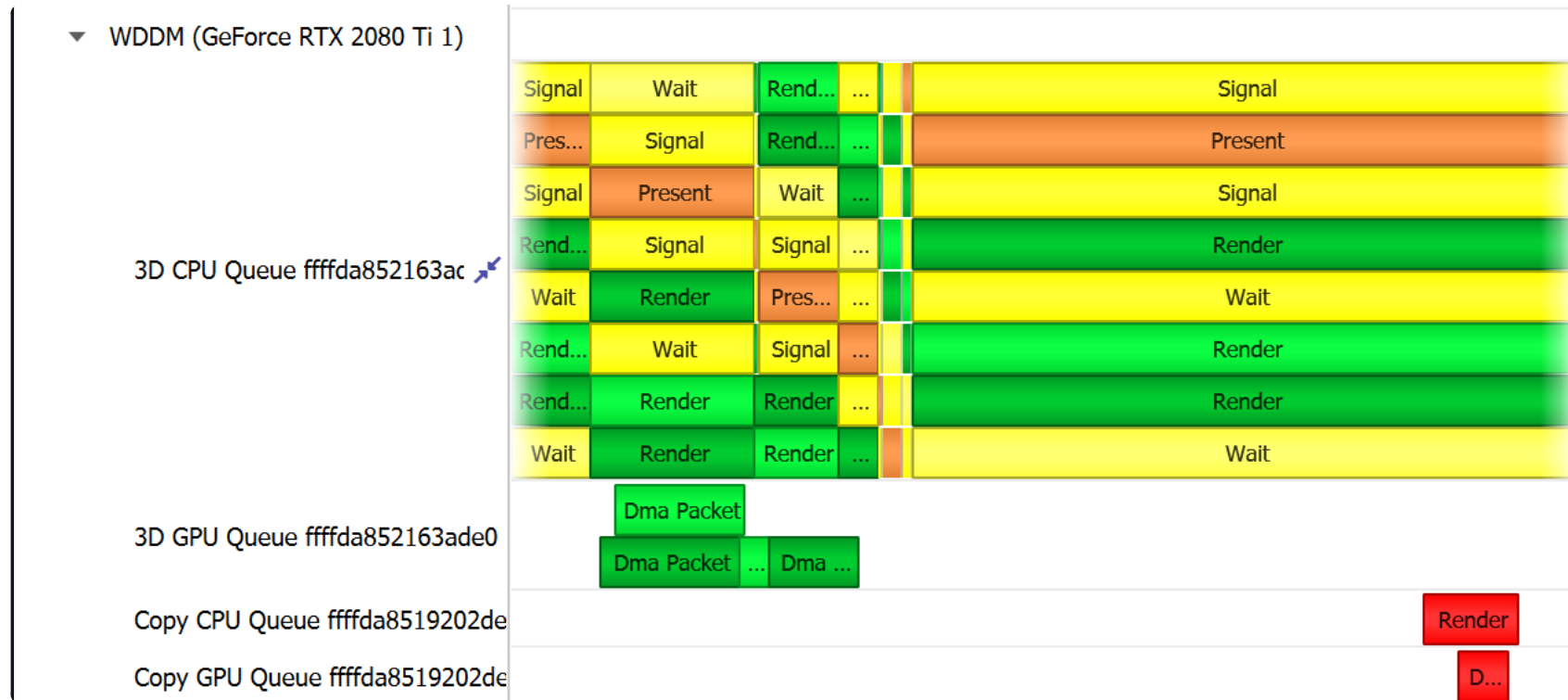


WDDM Queues#

The Windows Display Driver Model (WDDM) architecture uses queues to send work packets from the CPU to the GPU. Each D3D device in each process is associated with one or more contexts. Graphics, compute, and copy commands that the profiled application uses are associated with a context, batched in a command buffer, and pushed into the relevant queue associated with that context.

Nsight Systems can capture the state of these queues during the trace session.

Enabling the “Collect additional range of ETW events” option will also capture extended DxgKrnl events from the Microsoft-Windows-DxgKrnl provider, such as context status, allocations, sync wait, signal events, etc.



A command buffer in a WDDM queues may have one the following types:

- Render
- Deferred
- System
- MMIOFlip
- Wait
- Signal

- Device
- Software

It may also be marked as a Present buffer, indicating that the application has finished rendering and requests to display the source surface.

See the Microsoft documentation for the WDDM architecture and the `DXGKETW_QUEUE_PACKET_TYPE` enumeration.

To retain the .etl trace files captured, so that they can be viewed in other tools (e.g. GPUView), change the “Save ETW log files in project folder” option under “Profile Behavior” in Nsight Systems’s global Options dialog. The .etl files will appear in the same folder as the .nsys-rep file, accessible by right-clicking the report in the Project Explorer and choosing “Show in Folder...”. Data collected from each ETW provider will appear in its own .etl file, and an additional .etl file named “Report XX-Merged-*.etl”, containing the events from all captured sources, will be created as well.

WDDM HW Scheduler#

When GPU Hardware Scheduling is enabled in Windows 10 or newer, the Windows Display Driver Model (WDDM) uses the `DxgKrnl` ETW provider to expose report of NVIDIA GPUs’ hardware scheduling context switches.

Nsight Systems can capture these context switch events, and display under the GPUs in the timeline rows titled WDDM HW Scheduler - [HW Queue type]. The ranges under each queue will show the process name and PID associated with the GPU work during the time period.

The events will be captured if GPU Hardware Scheduling is enabled in the Windows System Display settings, and “Collect WDDM Trace” is enabled in the Nsight Systems Project Settings.

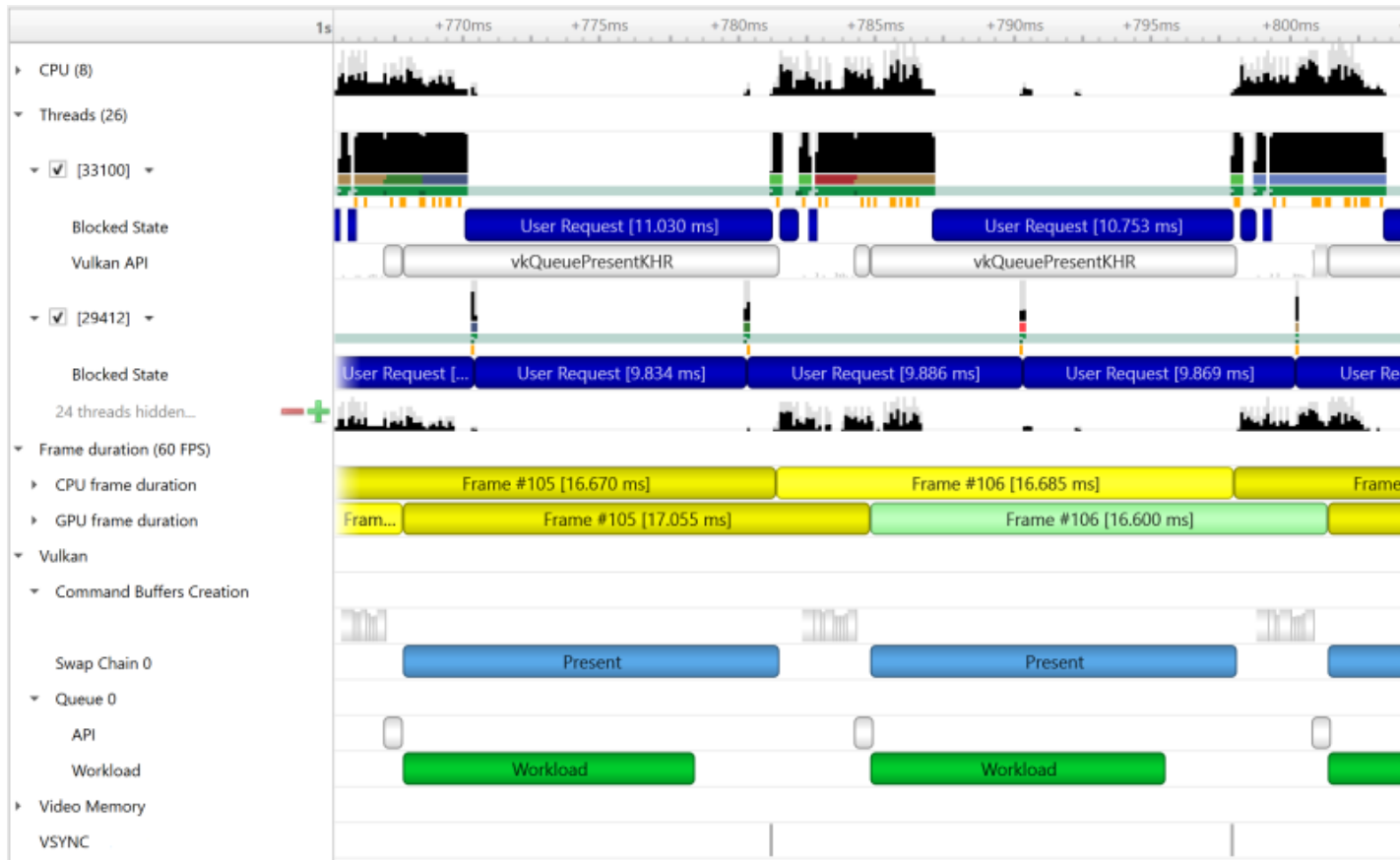


Vulkan API Trace#

Vulkan Overview#

Vulkan is a low-overhead, cross-platform 3D graphics and compute API, targeting a wide variety of devices from PCs to mobile phones and embedded platforms. The Vulkan API is defined by the Khronos Group. Information about Vulkan and the Khronos Group can be found at the [Khronos Vulkan Site](https://www.khronos.org/vulkan/).

Nsight Systems can capture information about Vulkan usage by the profiled process. This includes capturing the execution time of Vulkan API functions, corresponding GPU workloads, debug util labels, and frame durations. Vulkan profiling is supported on both Windows and x86 Linux operating systems.



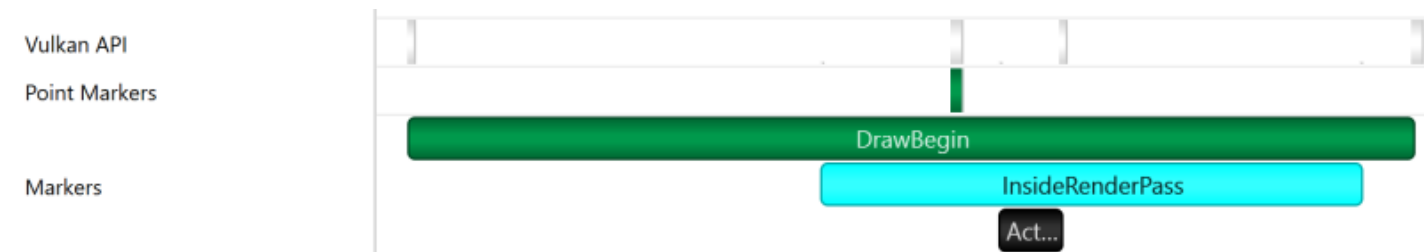
The Command Buffer Creation row displays time periods when command buffers were being created. This enables developers to improve their application's multi-threaded command buffer creation. Command buffer creation time period is measured between the call to `vkBeginCommandBuffer` and the call to `vkEndCommandBuffer`.



A Queue row is displayed for each Vulkan queue created by the profiled application. The API sub-row displays time periods where `vkQueueSubmit` was called. The GPU Workload sub-row displays time periods where workloads were executed by the GPU.



In addition, you can see [Vulkan debug util labels](#) on both the CPU and the GPU.

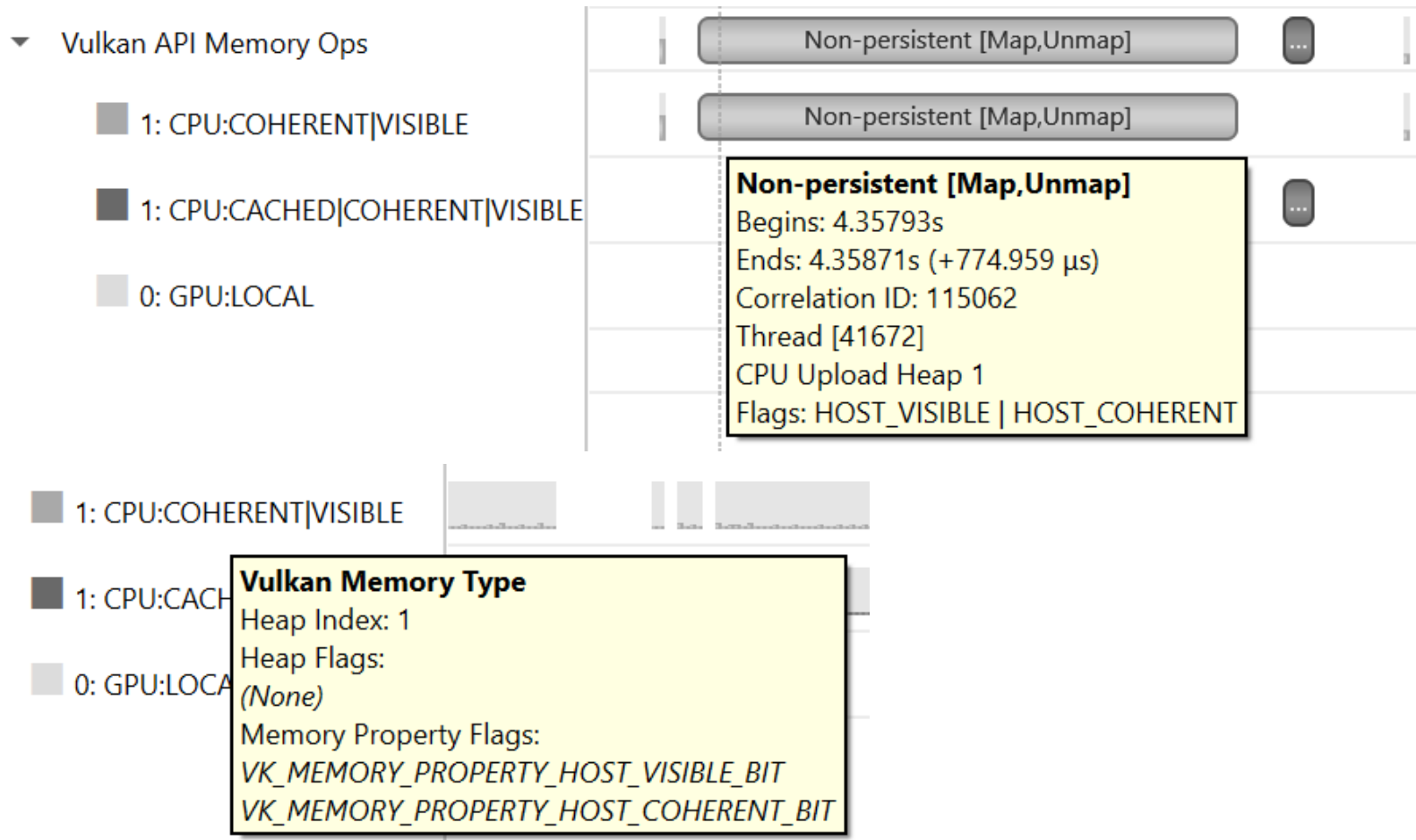


Clicking on a GPU workload highlights the corresponding `vkQueueSubmit` call, and vice versa.



The Vulkan Memory Operations row contains an aggregation of all the Vulkan host-side memory operations, such as host-blocking writes and reads or non-persistent map-unmap ranges.

The row is separated into sub-rows by heap index and memory type - the tooltip for each row and the ranges inside show the heap flags and the memory property flags.

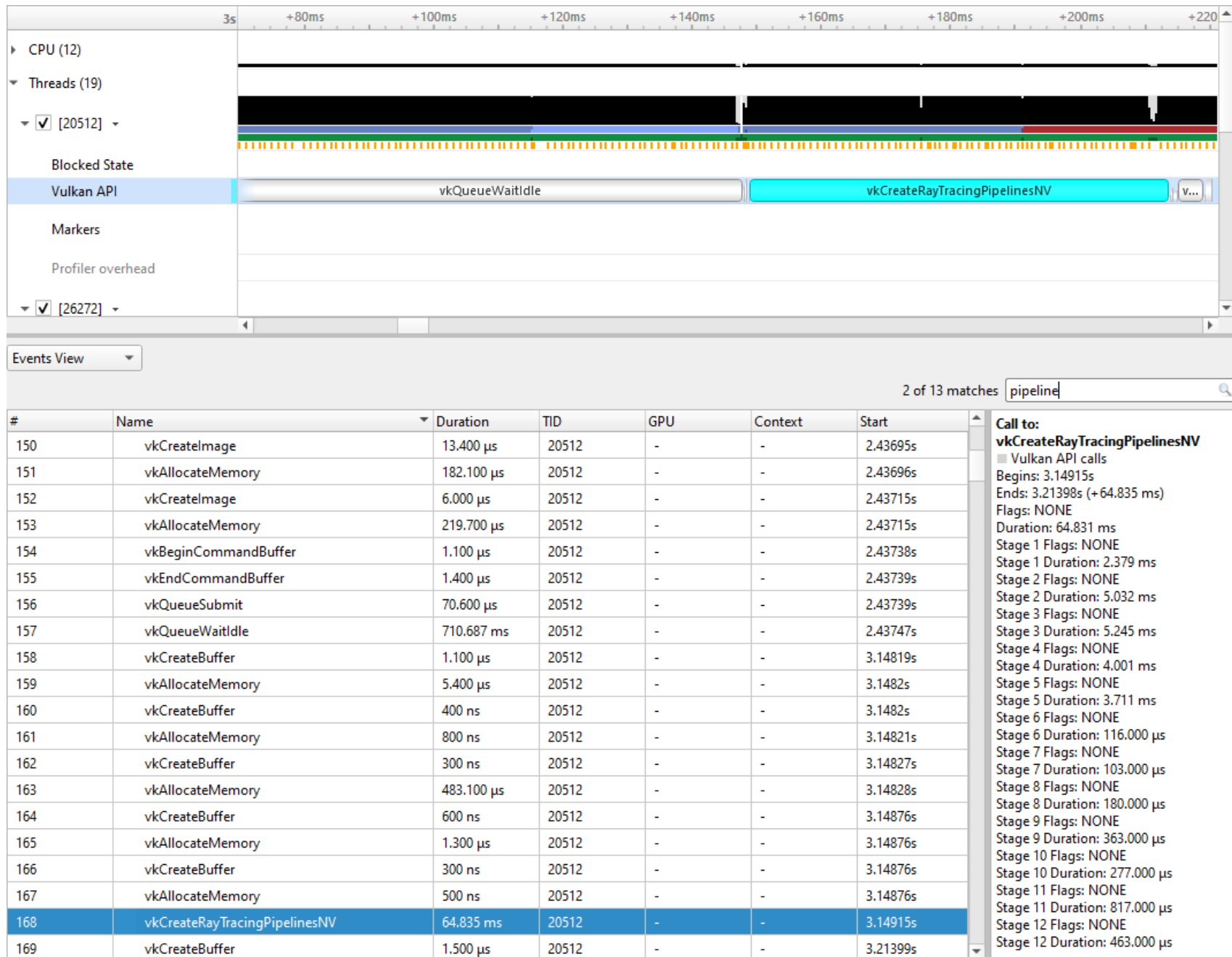


Pipeline Creation Feedback#

When tracing target application calls to Vulkan pipeline creation APIs, Nsight Systems leverages the Pipeline Creation Feedback extension to collect more details about the duration of individual pipeline creation stages.

See [Pipeline Creation Feedback extension](#) for details about this extension.

Vulkan pipeline creation feedback is available on NVIDIA driver release 435 or later.



Vulkan GPU Trace Notes#

- Vulkan GPU trace is available only when tracing apps that use NVIDIA GPUs.
- The endings of Vulkan Command Buffers execution ranges on Compute and Transfer queues may appear earlier on the timeline than their actual occurrence.

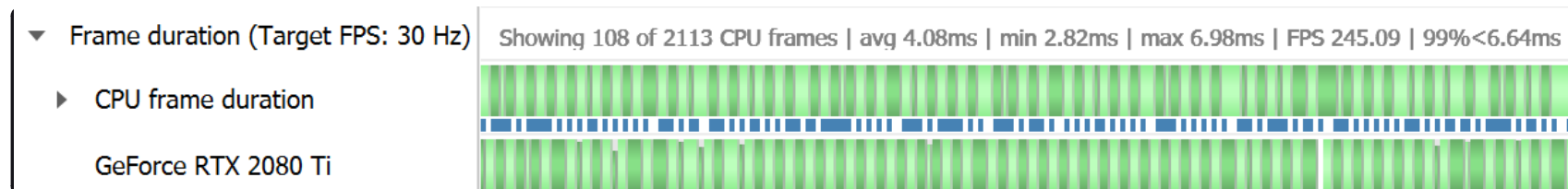
Stutter Analysis#

Stutter Analysis Overview

Nsight Systems on Windows targets displays stutter analysis visualization aids for profiled graphics applications that use either OpenGL, D3D11, D3D12 or Vulkan, as detailed below in the following sections.

FPS Overview#

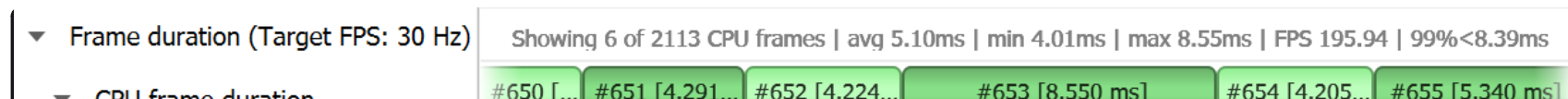
The Frame Duration section displays frame durations on both the CPU and the GPU.



The frame duration row displays live FPS statistics for the current timeline viewport. Values shown are:

1. Number of CPU frames shown of the total number captured.
2. Average, minimal, and maximal CPU frame time of the currently displayed time range.
3. Average FPS value for the currently displayed frames.
4. The 99th percentile value of the frame lengths (such that only 1% of the frames in the range are longer than this value).

The values will update automatically when scrolling, zooming or filtering the timeline view.





The stutter row highlights frames that are significantly longer than the other frames in their immediate vicinity.

The stutter row uses an algorithm that compares the duration of each frame to the median duration of the surrounding 19 frames. Duration difference under 4 milliseconds is never considered a stutter, to avoid cluttering the display with frames whose absolute stutter is small and not noticeable to the user.

For example, if the stutter threshold is set at 20%:

1. Median duration is 10 ms. Frame with 13 ms time will not be reported (relative difference $> 20\%$, absolute difference < 4 ms).
2. Median duration is 60 ms. Frame with 71 ms time will not be reported (relative difference $< 20\%$, absolute difference > 4 ms).
3. Median duration is 60 ms. Frame with 80 ms is a stutter (relative difference $> 20\%$, absolute difference > 4 ms, both conditions met).

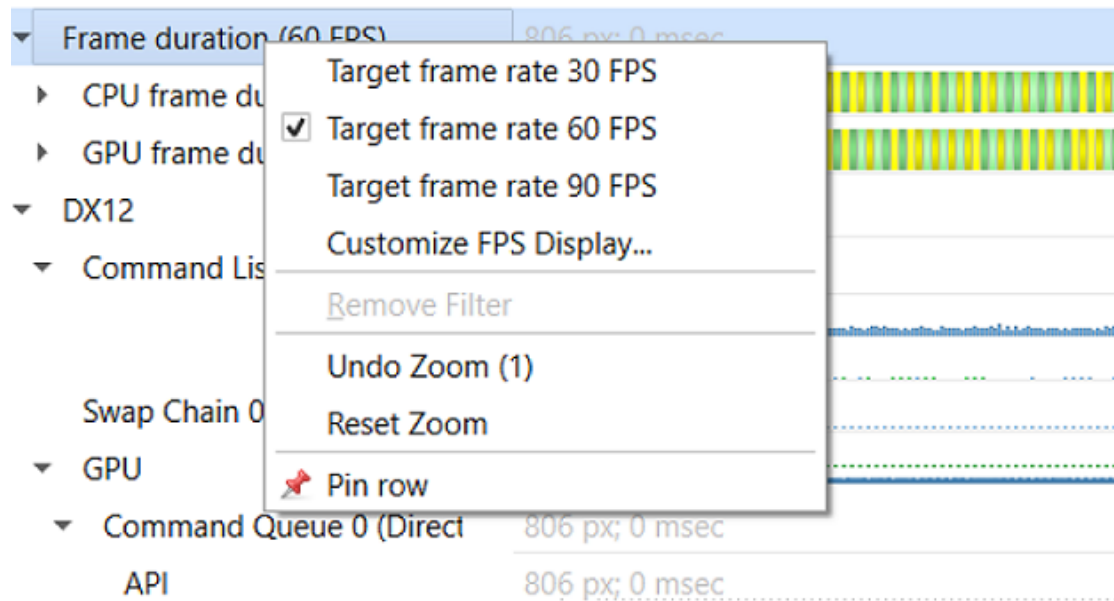
OSC detection

The “19 frame window median” algorithm by itself may not work well with some cases of “oscillation” (consecutive fast and slow frames), resulting in some false positives. The median duration is not meaningful in cases of oscillation and can be misleading.

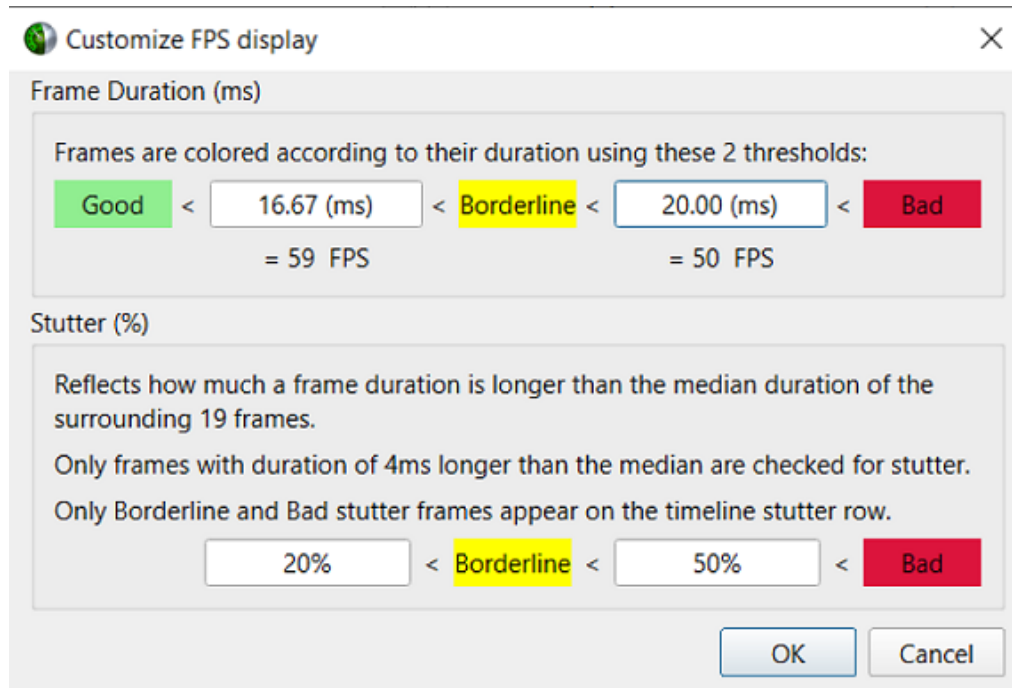
To address the issue and identify if oscillating frames, the following method is applied:

1. For every frame, calculate the median duration, 1st and 3rd quartiles of 19-frames window.
2. Calculate the delta and ratio between 1st and 3rd quartiles.
3. If the 90th percentile of 3rd - 1st quartile delta array > 4 ms AND the 90th percentile of 3rd/1st quartile array > 1.2 (120%) then mark the results with “OSC” text.

Right-clicking the Frame Duration row caption lets you choose the target frame rate (30, 60, 90 or custom frames per second).



By clicking the **Customize FPS Display** option, a customization dialog pops up. In the dialog, you can now define the frame duration threshold to customize the view of the potentially problematic frames. In addition, you can define the threshold for the stutter analysis frames.



Frame duration bars are color-coded:

- Green, the frame duration is shorter than required by the target FPS ratio.
- Yellow, duration is slightly longer than required by the target FPS rate.
- Red, duration far exceeds that required to maintain the target FPS rate.



The CPU Frame Duration row displays the CPU frame duration measured between the ends of consecutive frame boundary calls:

- The OpenGL frame boundaries are `glSwapBuffers/glxSwapBuffers/SwapBuffers` calls.
- The D3D11 and D3D12 frame boundaries are `IDXGISwapChainX::Present` calls.
- The Vulkan frame boundaries are `vkQueuePresentKHR` calls.

The timing of the actual calls to the frame boundary calls can be seen in the blue bar at the bottom of the CPU frame duration row

The GPU Frame Duration row displays the time measured between:

- The start time of the first GPU workload execution of this frame.
- The start time of the first GPU workload execution of the next frame.

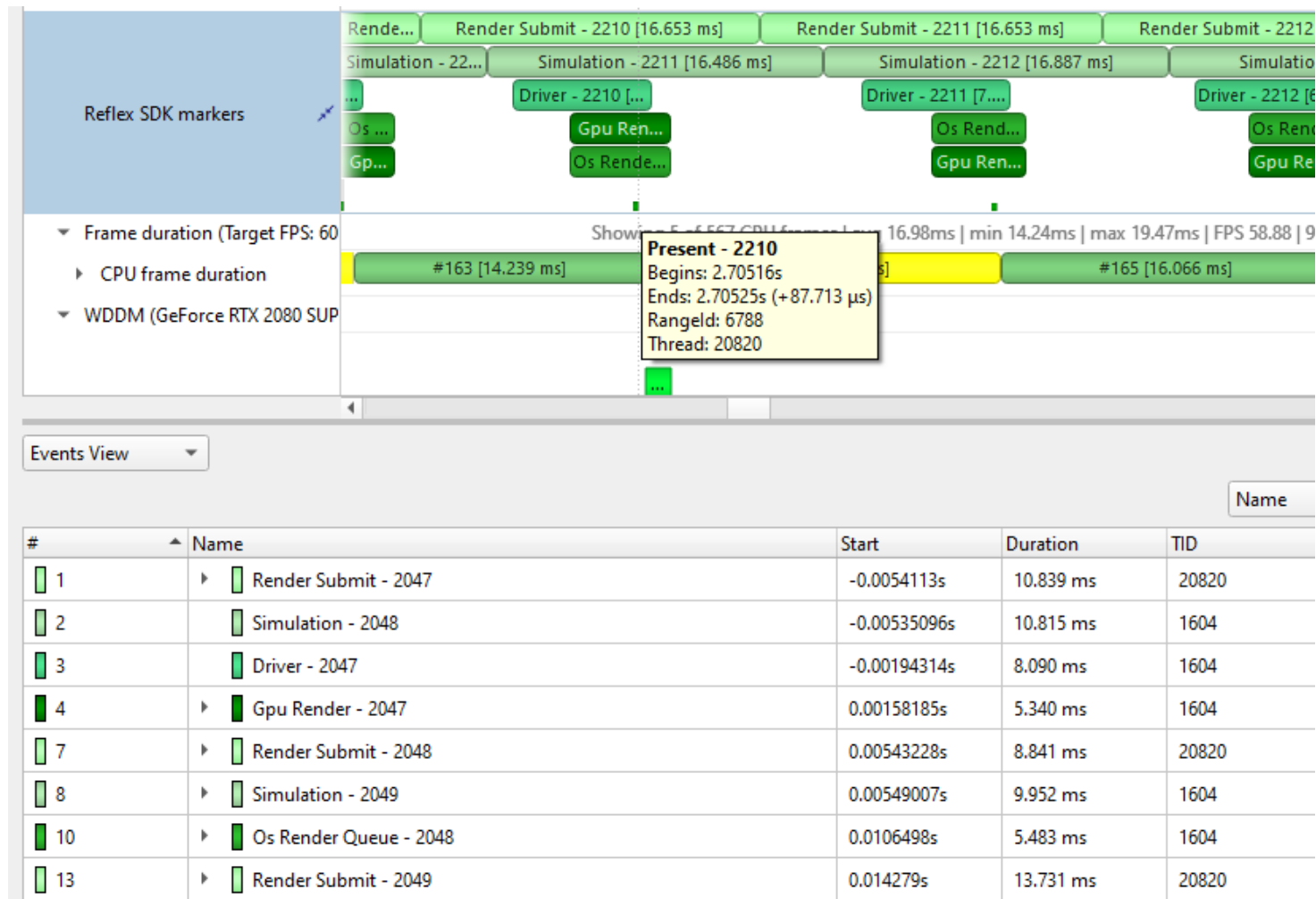
Reflex SDK

NVIDIA Reflex SDK is a series of NVAPI calls that allow applications to integrate the Ultra Low Latency driver feature more directly into their game to further optimize synchronization between simulation and rendering stages and lower the latency between user input and final image rendering. For more details about Reflex SDK, see the [Reflex SDK Site](#).

Nsight Systems will automatically capture NVAPI functions when either Direct3D 11, Direct3D 12, or Vulkan API trace are enabled.

The Reflex SDK row displays timeline ranges for the following types of latency markers:

- RenderSubmit
- Simulation
- Present
- Driver
- OS Render Queue
- GPU Render



Performance Warnings row

This row shows performance warnings and common pitfalls that are automatically detected based on the enabled capture types. Warnings are reported for:

- ETW performance warnings.
- Vulkan calls to `vkQueueSubmit` and D3D12 calls to `ID3D12CommandQueue::ExecuteCommandList` that take a longer time to execute than the total time of the GPU workloads they generated.
- [D3D12 Memory Operation warnings](#).

- Usage of Vulkan API functions that may adversely affect performance.
- Creation of a Vulkan device with memory zeroing, whether by physical device default or manually.
- Vulkan command buffer barrier which can be combined or removed, such as subsequent barriers or read-to-read barriers.

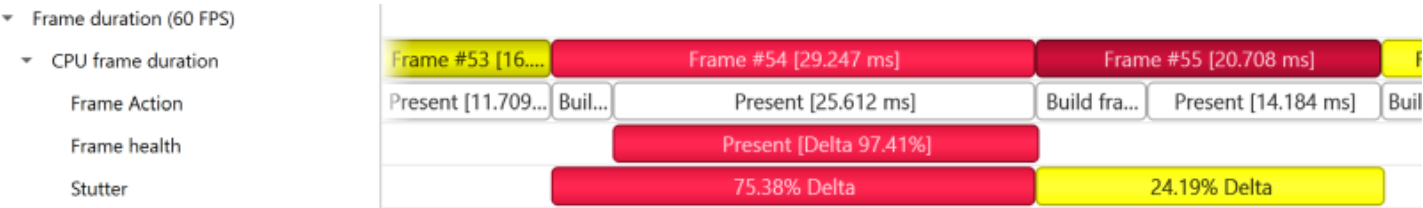
Warnings



Frame Health#

The Frame Health row displays actions that took significantly a longer time during the current frame, compared to the median time of the same actions executed during the surrounding 19-frames. This is a great tool for detecting the reason for frame time stuttering. Such actions may be: shader compilation, present, memory mapping, and more. Nsight Systems measures the accumulated time of such actions in each frame. For example: calculating the accumulated time of shader compilations in each frame and comparing it to the accumulated time of shader compilations in the surrounding 19 frames.

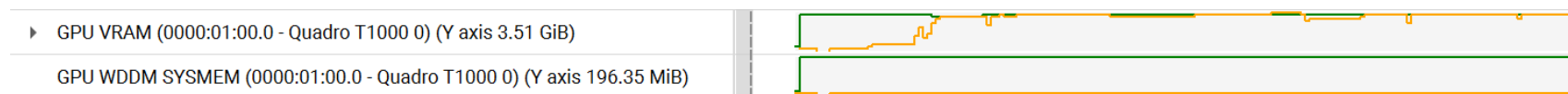
Example of a Vulkan frame health row:



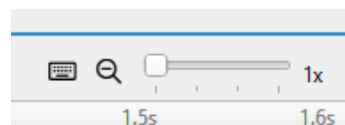


Windows GPU Memory Utilization#

Each GPU has two rows detailing its memory utilization: **GPU VRAM**, showing the memory consumed on the device, and **GPU WDDM SYMMEM**, showing the memory consumed on the host computer RAM.



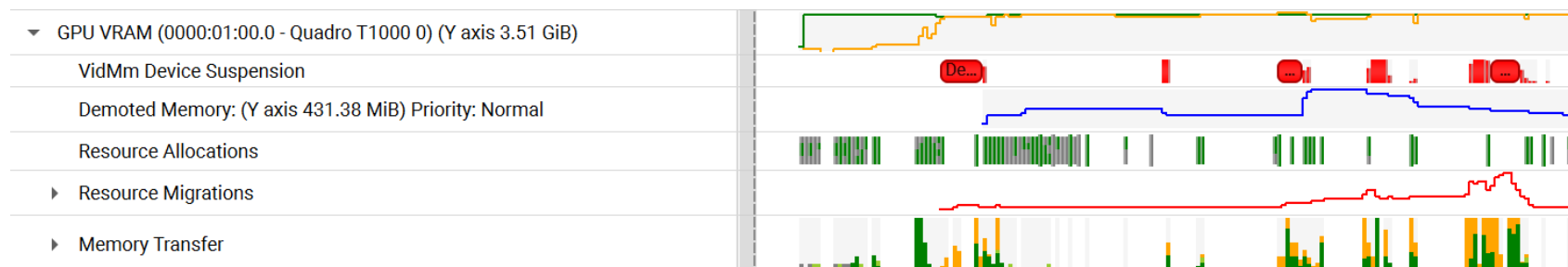
These rows show a green-colored line graph for the memory budget for this memory segment, and an orange-colored line graph for the actual amount of memory used. Note that these graphs are scaled to fit the highest value encountered, as indicated by the “Y axis” value in the row header. You can use the vertical zoom slider in the top-right of the timeline view to make the row taller and view the graph in more detail.



Note that the value in the GPU VRAM row is not the same as the CUDA kernel memory allocation graph, see [CUDA GPU Memory Allocation Graph](#) for that functionality.

The GPU VRAM row also has several child rows, accessed by expanding the row in the tree view

The events will be captured if “Collect WDDM Trace” and “Collect additional range of ETW events, including context status, allocations, sync wait and signal events, etc.” are enabled in the Nsight Systems Project Settings.



VidMm Device Suspension

This row displays time ranges when the GPU memory manager suspended all memory transfer operations, pending the completion of a single memory transfer.

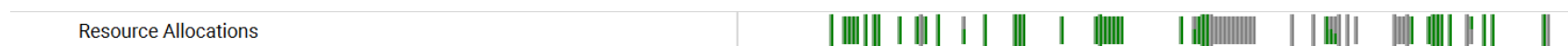
The events will be captured if “Collect WDDM Trace” and “Collect additional range of ETW events, including context status, allocations, sync wait and signal events, etc.” are enabled in the Nsight Systems Project Settings.

Demoted Memory

This row displays the amount of VRAM that was demoted from GPU local memory to non-local memory (possibly due to exceeding the VRAM budget) as a blue-colored line graph. High amounts of demoted memory could be indicative of video memory leaks or poor memory management. Note that the Demoted memory row is scaled to its highest value, similar to the GPU VRAM and GPU WDDM SYSMEM rows.

The events will be captured if “Collect WDDM Trace” and “Collect additional range of ETW events, including context status, allocations, sync wait and signal events, etc.” are enabled in the Nsight Systems Project Settings.

Resource Allocations

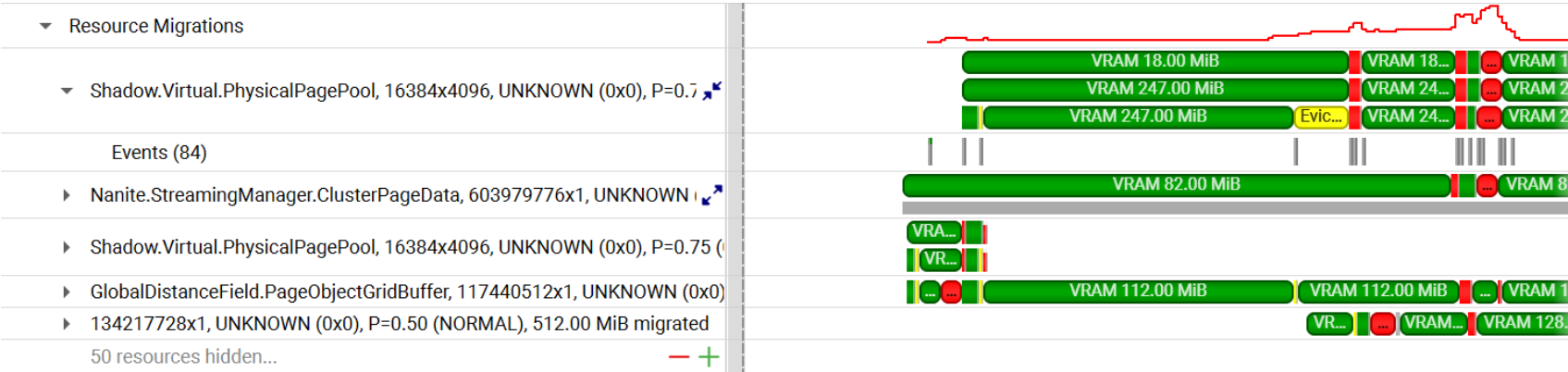


This row shows markers indicating resource allocation events. VRAM resources are shown as green markers while SYSMEM resources are shown in gray. Hovering over a marker or selecting it in the [Events view](#) will display all the

allocation parameters as well as the call stack that led to the allocation event.

The events will be captured if “Collect WDDM Trace” and “Collect additional range of ETW events, including context status, allocations, sync wait and signal events, etc.” are enabled in the Nsight Systems Project Settings.

Resource Migrations



This row displays a breakdown of resources’ movement between VRAM and SYSMEM, focusing on resource evictions. The main row shows a timeline of total evicted resource memory and count as a red-colored line graph.

Each child row displays a timeline of the status of each resource, as reflected by WDDM events related to it. If the object has been named using PIX or ID3D11object::SetName / ID3D12object::SetName, the name will be shown in the row title. Whether named or not, the row title will also show the resource dimensions, format, priority, and the memory size migrated. If the resource was migrated in parts using subresources, the row can be expanded to show the status for each subresource at any given time.

Expanding the row for a resource will show the individual WDDM events relevant to it and the call stacks that led to each event.

By default, the resources are sorted by Relevance (most / largest migrations). Right-clicking the main Resource Migrations row header allows choosing between the following sorting options:

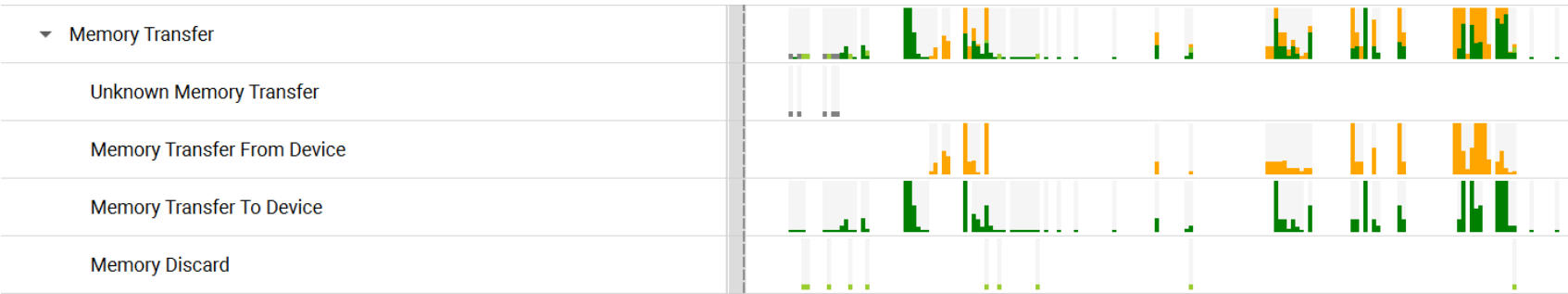
- Relevance
- Name

- Format
- Priority
- Earliest allocation timestamp (order of appearance on the host)
- Earliest migration timestamp (order of appearance on the device)

The top 5 resources are shown initially. If more than 5 resources exist, a row showing the number of hidden resources and buttons allowing to show more or fewer of them will appear below them. Right-click this row and select “show all” or “show all collapsed” to display all the resources at once.

The events will be captured if “Collect WDDM Trace” and “Collect additional range of ETW events, including context status, allocations, sync wait and signal events, etc.” are enabled in the Nsight Systems Project Settings. Additionally, to correlate Graphics API debug name events with resource migration events, the “Collect DX12” or “Collect Vulkan” option should be enabled.

Memory Transfer



This row shows an overview of all memory transfer operations. Device-to-host transfers are shown in orange, host-to-device transfers are shown in green, discarded device memory is shown in light green, and unknown events are shown in dark gray. The height of each event marker corresponds to the amount of memory that the event affected. Hovering over the marker will show the exact amount.

Expanding the row will show a breakdown of the events by each specific type.

The events will be captured if “Collect WDDM Trace” and “Collect additional range of ETW events, including context status, allocations, sync wait and signal events, etc.” are enabled in the Nsight Systems Project Settings.

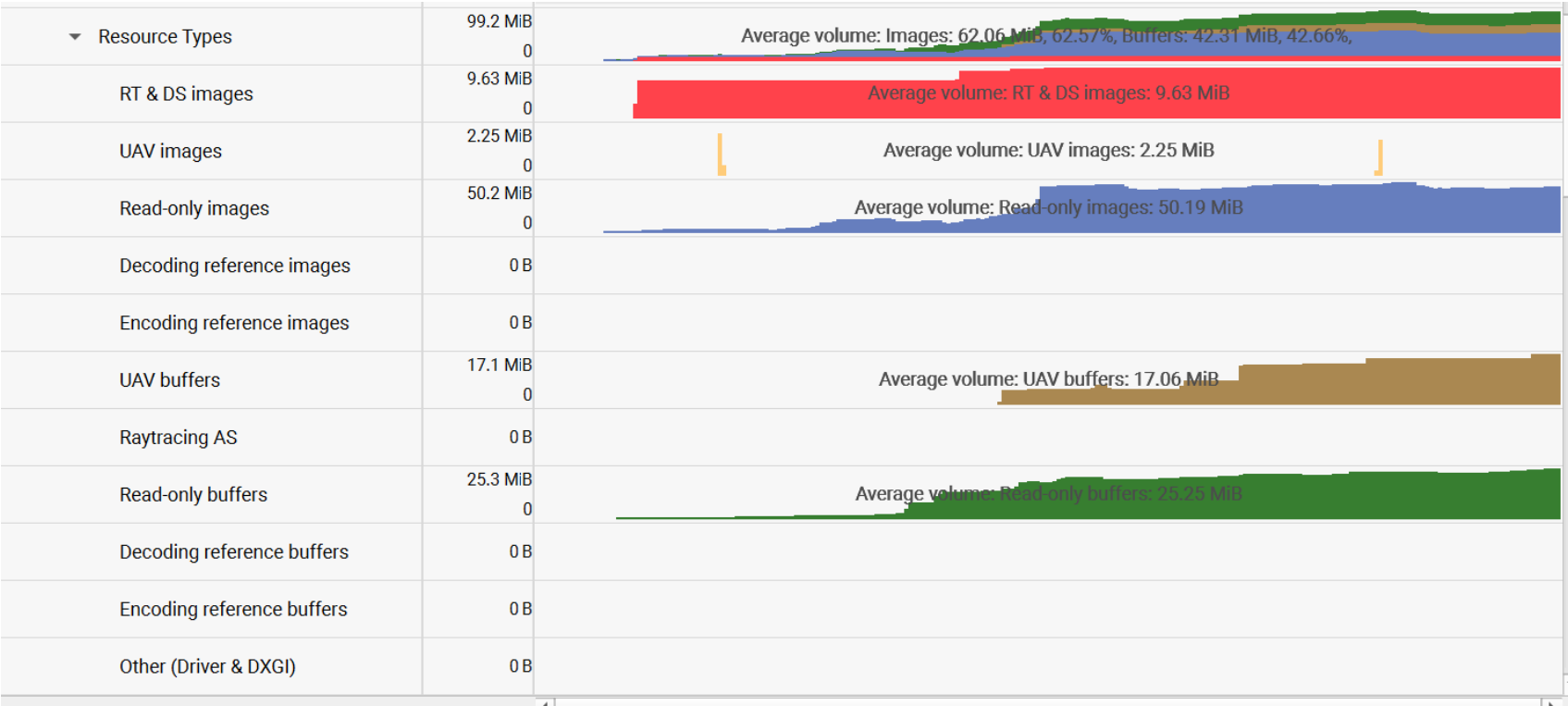
System Committed VRAM



This row represents the total size of committed VRAM by all processes currently using the GPU. The stacked chart displays colored layers. Each layer corresponds to the VRAM commitment of a specific process.

To track VRAM commitment, enable the “Collect WDDM Trace” and “Collect additional range of ETW events, including context status, allocations, sync wait and signal events, etc.” in Nsight Systems Project Settings.

VRAM Resource Types Distribution



This row shows the distribution of VRAM usage across different resource types per process. it is color-coded to show the different resource types, and the height of each segment corresponds to the amount of VRAM used by that resource type. Expand the chart’s parent row to expose detailed separate rows for individual resource categories.

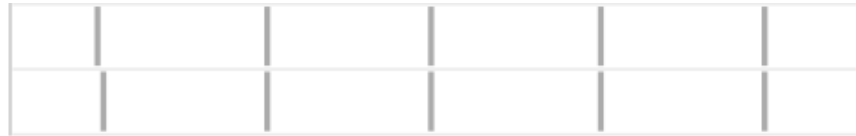
The events will be captured if “Collect WDDM Trace” and “Collect additional range of ETW events, including context status, allocations, sync wait and signal events, etc.” are enabled in the Nsight Systems Project Settings. Additionally, to correlate Graphics API debug name events with resource migration event, the “Collect DX12” or “Collect Vulkan” option should be enabled.

Vertical Synchronization#

The VSYNC rows display when the monitor’s vertical synchronizations occur.

VSYNC - iGPU (0)

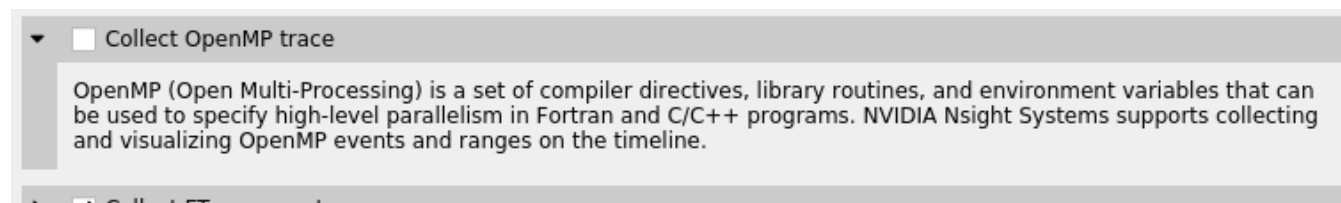
VSYNC - dGPU 1 (0)



OpenMP Trace#

Nsight Systems for Linux is capable of capturing information about OpenMP events. This functionality is built on the OpenMP Tools Interface (OMPT), full support is available only for runtime libraries supporting tools interface defined in OpenMP 5.0 or greater.

As an example, LLVM OpenMP runtime library partially implements tools interface. If you use PGI compiler <= 20.4 to build your OpenMP applications, add the `-mp=libomp` switch to use LLVM OpenMP runtime and enable OMPT based tracing. If you use Clang, make sure the LLVM OpenMP runtime library you link to was compiled with tools interface enabled.



Only a subset of the OMPT callbacks are processed:

`ompt_callback_parallel_begin`

ompt_callback_parallel_end
ompt_callback_sync_region
ompt_callback_task_create
ompt_callback_task_schedule
ompt_callback_implicit_task
ompt_callback_master
ompt_callback_reduction
ompt_callback_task_create
ompt_callback_cancel
ompt_callback_mutex_acquire, ompt_callback_mutex_acquired
ompt_callback_mutex_acquired, ompt_callback_mutex_released
ompt_callback_mutex_released
ompt_callback_work
ompt_callback_dispatch
ompt_callback_flush

Note

The raw OMPT events are used to generate ranges indicating the runtime of OpenMP operations and constructs.

Example screenshot:



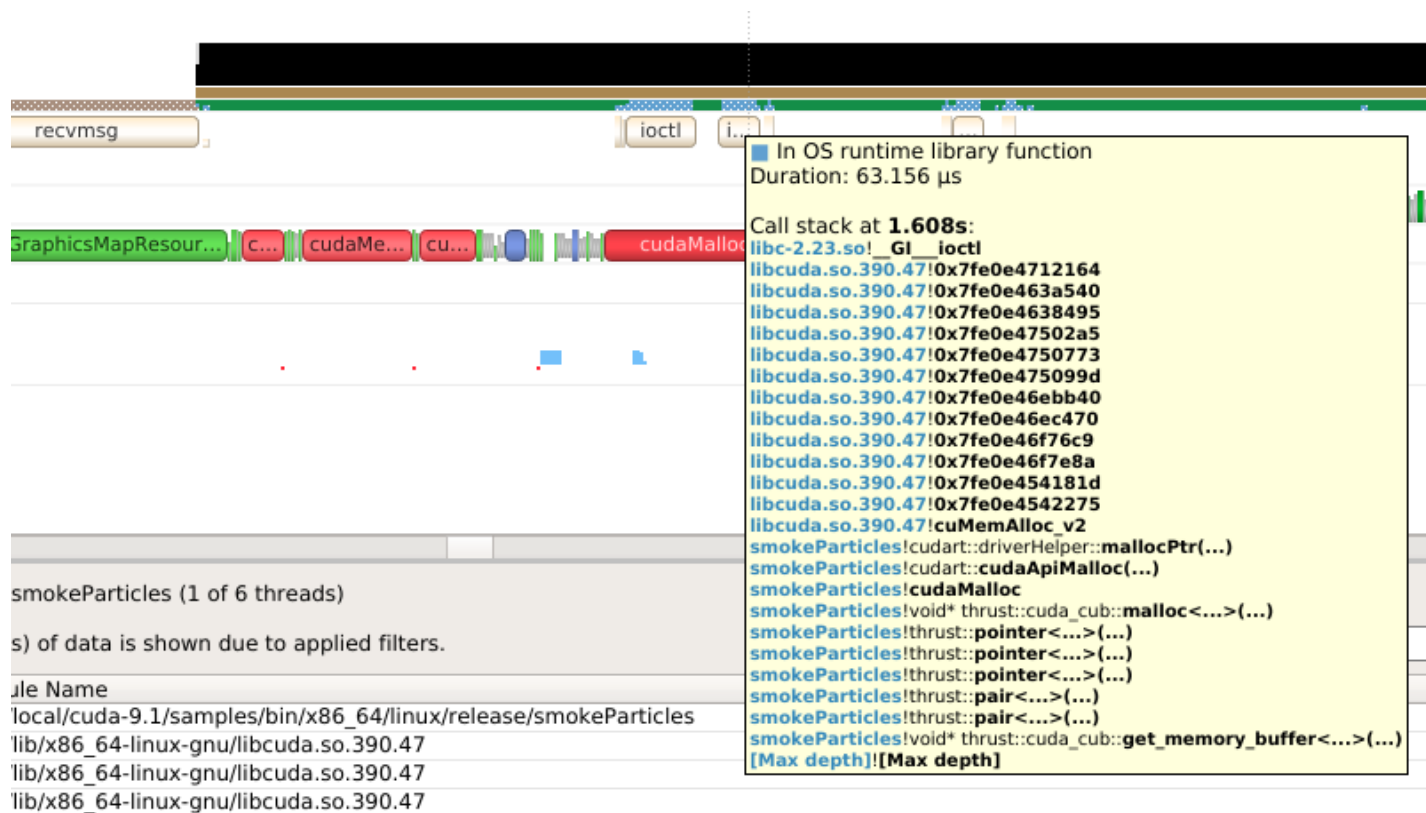
OS Runtime Libraries Trace#

On Linux, OS runtime libraries can be traced to gather information about low-level userspace APIs. This traces the system call wrappers and thread synchronization interfaces exposed by the C runtime and POSIX Threads (pthread) libraries. This does not perform a complete runtime library API trace, but instead focuses on the functions that can take a long time to execute, or could potentially cause your thread be unscheduled from the CPU while waiting for an event to complete. OS runtime trace is not available for Windows targets.

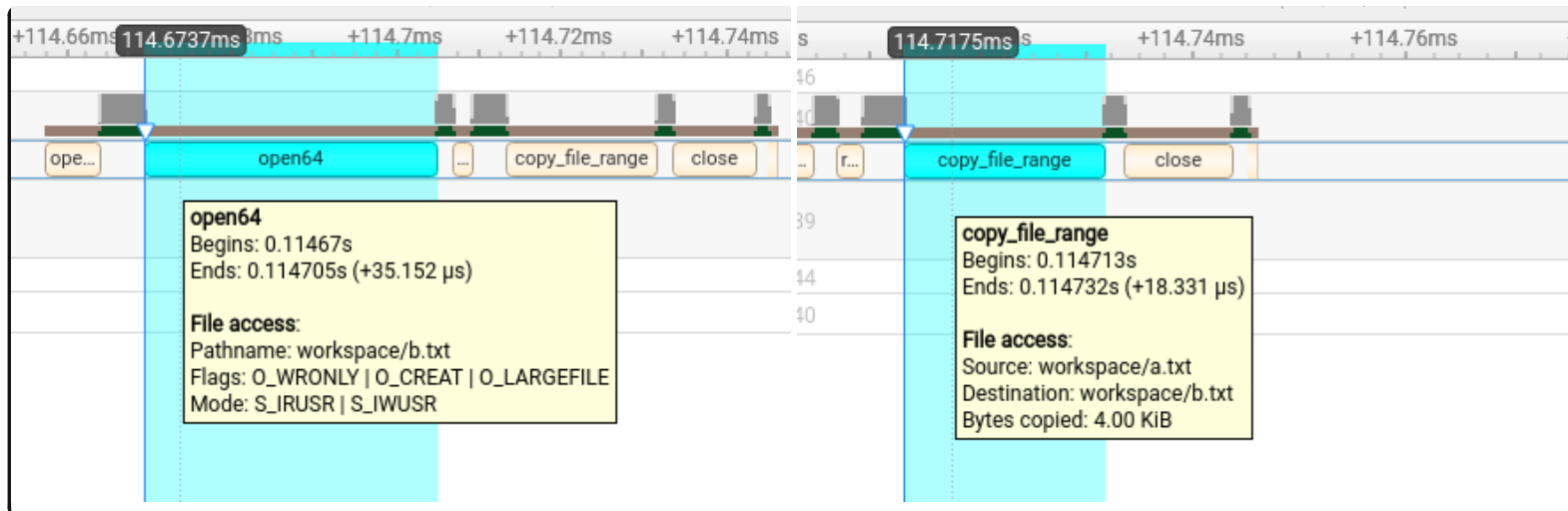
OS runtime tracing complements and enhances sampling information by:

1. Visualizing when the process is communicating with the hardware, controlling resources, performing multi-threading synchronization or interacting with the kernel scheduler.
2. Adding additional thread states by correlating how OS runtime libraries traces affect the thread scheduling:
 - **Waiting** — the thread is not scheduled on a CPU, it is inside of an OS runtime libraries trace and is believed to be waiting on the firmware to complete a request.
 - **In OS runtime library function** — the thread is scheduled on a CPU and inside of an OS runtime libraries trace. If the trace represents a system call, the process is likely running in kernel mode.

3. Collecting backtraces for long OS runtime libraries call. This provides a way to gather blocked-state backtraces, allowing you to gain more context about why the thread was blocked so long, yet avoiding unnecessary overhead for short events.



4. Collecting file access data for API calls that interact with files. This helps in identifying performance bottlenecks related to file I/O operations and provides insights into how file access patterns affect overall application performance.



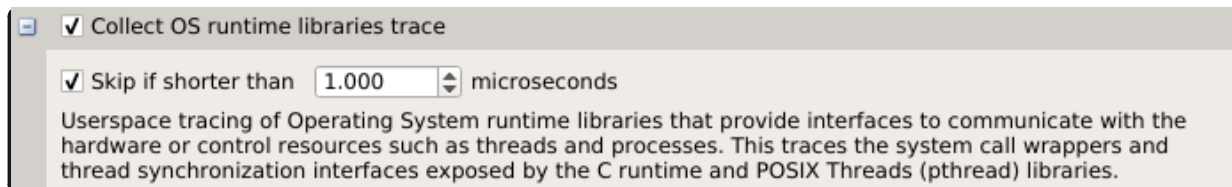
Note

File access data collection is not enabled by default.

To enable OS runtime libraries tracing from Nsight Systems:

CLI — Use the `-t`, `--trace` option with the `os rt` parameter. See [Command Line Options](#) for more information.

GUI — Select the **Collect OS runtime libraries trace** checkbox.



You can also use **Skip if shorter than**. This will skip calls shorter than the given threshold. Enabling this option will improve performances as well as reduce noise on the timeline. We strongly encourage you to skip OS runtime libraries call shorter than 1 μs.

Locking a Resource#

The functions listed below receive a special treatment. If the tool detects that the resource is already acquired by

another thread and will induce a blocking call, we always trace it. Otherwise, it will never be traced.

pthread_mutex_lock
pthread_rwlock_rdlock
pthread_rwlock_wrlock
pthread_spin_lock
sem_wait

Note that even if a call is determined as potentially blocking, there is a chance that it may not actually block after a few cycles have elapsed. The call will still be traced in this scenario.

Limitations#

- Nsight Systems only traces syscall wrappers exposed by the C runtime. It is not able to trace syscall invoked through assembly code.
- Additional thread states, as well as backtrace collection on long calls, are only enabled if sampling is turned on.
- It is not possible to configure the depth and duration threshold when collecting backtraces. Currently, only OS runtime libraries calls longer than 80 μ s will generate a backtrace with a maximum of 24 frames. This limitation will be removed in a future version of the product.
- It is required to compile your application and libraries with the `-funwind-tables` compiler flag in order for Nsight Systems to unwind the backtraces correctly.

OS Runtime Libraries Trace Filters#

The OS runtime libraries tracing is limited to a select list of functions. It also depends on the version of the C runtime linked to the application.

OS Runtime Default Function List#

Libc system call wrappers

accept
accept4
acct
alarm
arch_prctl
bind
bpf
brk
chroot
clock_nanosleep
connect
copy_file_range
creat
creat64
dup
dup2
dup3
epoll_ctl
epoll_pwait
epoll_wait
fallocate
fallocate64
fcntl
fdatasync
flock
fork
fsync
ftruncate

futex
ioctl
ioperm
iopl
kill
killpg
listen
membarrier
mlock
mlock2
mlockall
mmap
mmap64
mount
move_pages
mprotect
mq_notify
mq_open
mq_receive
mq_send
mq_timedreceive
mq_timedsend
mremap
msgctl
msgget
msgrcv
msgsnd
msync

munmap
nanosleep
nfsservctl
open
open64
openat
openat64
pause
pipe
pipe2
pivot_root
poll
ppoll
prctl
pread
pread64
preadv
preadv2
preadv64
process_vm_readv
process_vm_writev
pselect6
ptrace
pwrite
pwrite64
pwritev
pwritev2
pwritev64

read
readv
reboot
recv
recvfrom
recvmsg
recvmsg
rt_sigaction
rt_sigqueueinfo
rt_sigsuspend
rt_sigtimedwait
sched_yield
seccomp
select
semctl
semget
semop
semtimedop
send
sendfile
sendfile64
sendmmsg
sendmsg
sendto
shmat
shmctl
shmdt
shmget

shutdown
sigaction
sigsuspend
sigtimedwait
socket
socketpair
splice
swapoff
swapon
sync
sync_file_range
syncfs
tee
tgkill
tgsigqueueinfo
tkill
truncate
umount2
unshare
uselib
vfork
vhangup
vmsplice
wait
wait3
wait4
waitid
waitpid

write
writev
_sysctl

POSIX Threads

pthread_barrier_wait
pthread_cancel
pthread_cond_broadcast
pthread_cond_signal
pthread_cond_timedwait
pthread_cond_wait
pthread_create
pthread_join
pthread_kill
pthread_mutex_lock
pthread_mutex_timedlock
pthread_mutex_trylock
pthread_rwlock_rdlock
pthread_rwlock_timedrdlock
pthread_rwlock_timedwrlock
pthread_rwlock_tryrdlock
pthread_rwlock_trywrlock
pthread_rwlock_wrlock
pthread_spin_lock
pthread_spin_trylock
pthread_timedjoin_np
pthread_tryjoin_np
pthread_yield
sem_timedwait

sem_trywait

sem_wait

I/O

aio_fsync

aio_fsync64

aio_suspend

aio_suspend64

fclose

fcloseall

fflush

fflush_unlocked

fgetc

fgetc_unlocked

fgets

fgets_unlocked

fgetwc

fgetwc_unlocked

fgetws

fgetws_unlocked

flockfile

fopen

fopen64

fputc

fputc_unlocked

fputs

fputs_unlocked

fputwc

fputwc_unlocked

fputws
fputws_unlocked
fread
fread_unlocked
freopen
freopen64
ftrylockfile
fwrite
fwrite_unlocked
getc
getc_unlocked
getdelim
getline
getw
getwc
getwc_unlocked
lockf
lockf64
mkfifo
mkfifoat
posix_fallocate
posix_fallocate64
putc
putc_unlocked
putwc
putwc_unlocked

Miscellaneous

forkpty

popen
posix_spawn
posix_spawnnp
sigwait
sigwaitinfo
sleep
system
usleep

Syscall Trace#

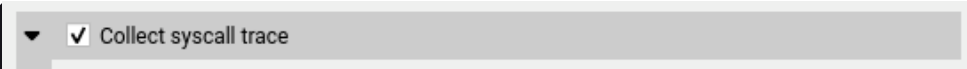
Nsight Systems for Linux and Nsight Systems Embedded Platforms Edition are capable of tracing Linux system calls in kernel space. This feature uses Linux's eBPF technology, and is supported on systems running Linux v5.11 or newer, specifically those that are built with `CONFIG_DEBUG_INFO_BTf` enabled, which is the default on most major Linux distros. This feature requires `CAP_BPF` and `CAP_PERFMON` capabilities (alternatively, `CAP_SYS_ADMIN` or root privileges) for the `nsys` process, see the capabilities [man page](#) for details.

To enable this feature:

CLI — Add the `--syscall` option to the `nsys start` or `nsys profile` commands (setting `syscall` in the `--trace` option is deprecated and will be ignored). The following values are supported:

- `none` — No syscall tracing [default].
- `process-tree` — Collect syscalls for the profiled application process and its child processes.
- `pid-namespace` — Collect syscalls made by all processes in the current PID namespace and its child namespaces. This is very close to how other features work in the system-wide mode, e.g. inside a container, tracing will be limited to this container.

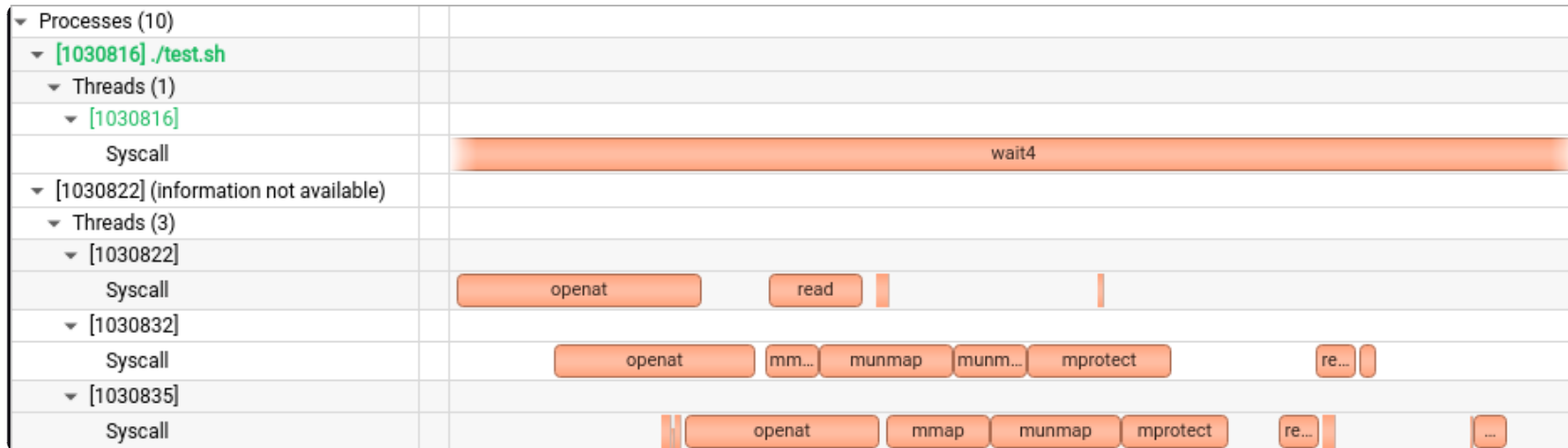
GUI — Select the **Collect syscall trace** checkbox. Currently, equivalent to the `--syscall=process-tree` option.



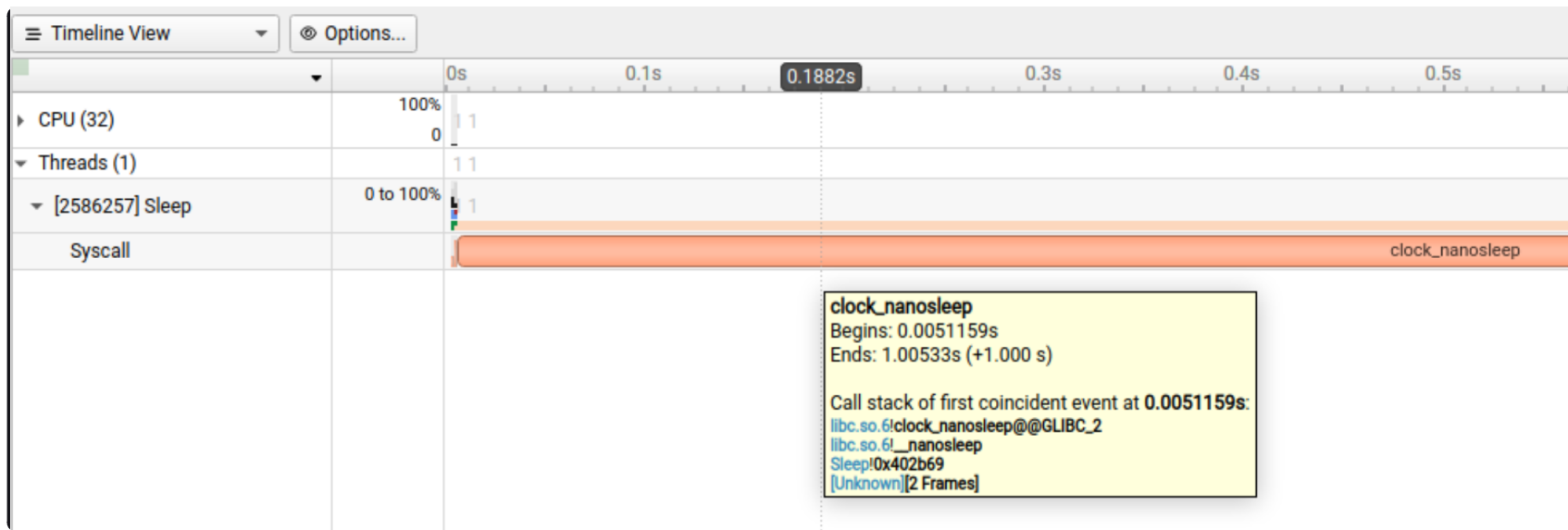
▼ ☒ Collect syscall trace

Please note that only syscalls running 1000ns and more are traced.

Example screenshot:



Long running (more than 80 microseconds) syscalls are also collected with their backtraces:



NVTX Trace#

The NVIDIA Tools Extension Library (NVTX) is a powerful mechanism that allows users to manually instrument their application. Nsight Systems can then collect the information and present it on the timeline.

Nsight Systems supports version 3.0 of the NVTX specification.

The following features are supported:

- Domains

`nvtxDomainCreate()`, `nvtxDomainDestroy()`

`nvtxDomainRegisterString()`

- Push-pop ranges (nested ranges that start and end in the same thread).

`nvtxRangePush()`, `nvtxRangePushEx()`

`nvtxRangePop()`

`nvtxDomainRangePushEx()`

`nvtxDomainRangePop()`

- Start-end ranges (ranges that are global to the process and are not restricted to a single thread)

`nvtxRangeStart()`, `nvtxRangeStartEx()`

`nvtxRangeEnd()`

`nvtxDomainRangeStartEx()`

`nvtxDomainRangeEnd()`

- Marks

`nvtxMark()`, `nvtxMarkEx()`

`nvtxDomainMarkEx()`

- Thread names

`nvtxNameOsThread()`

- Categories

`nvtxNameCategory()`

`nvtxDomainNameCategory()`

To learn more about specific features of NVTX, please refer to the NVTX header file: `nvToolsExt.h` or the [NVTX documentation](#).

To use NVTX in your application, follow these steps:

1. Add `#include "nvtx3/nvToolsExt.h"` in your source code. The `nvtx3` directory is located in the Nsight Systems package in the `Target-<architecture>/nvtx/include` directory and is available via github at [NVIDIA/NVTX](#).
2. Add the following compiler flag: `-ldl`
3. Add calls to the NVTX API functions. For example, try adding `nvtxRangePush("main")` in the beginning of the `main()` function, and `nvtxRangePop()` just before the return statement in the end.

For convenience in C++ code, consider adding a wrapper that implements RAI (resource acquisition is initialization) pattern, which would guarantee that every range gets closed.

4. In the project settings, select the **Collect NVTX trace** checkbox.

In addition, by enabling the “Insert NVTX Marker hotkey” option it is possible to add NVTX markers to a running non-console applications by pressing the F11 key. These will appear in the report under the NVTX Domain named “HotKey markers.”

Typically, calls to NVTX functions can be left in the source code even if the application is not being built for profiling purposes, since the overhead is very low when the profiler is not attached.

NVTX is not intended to annotate very small pieces of code that are being called very frequently. A good rule of thumb to use: if code being annotated usually takes less than 1 microsecond to execute, adding an NVTX range around this code should be done carefully.

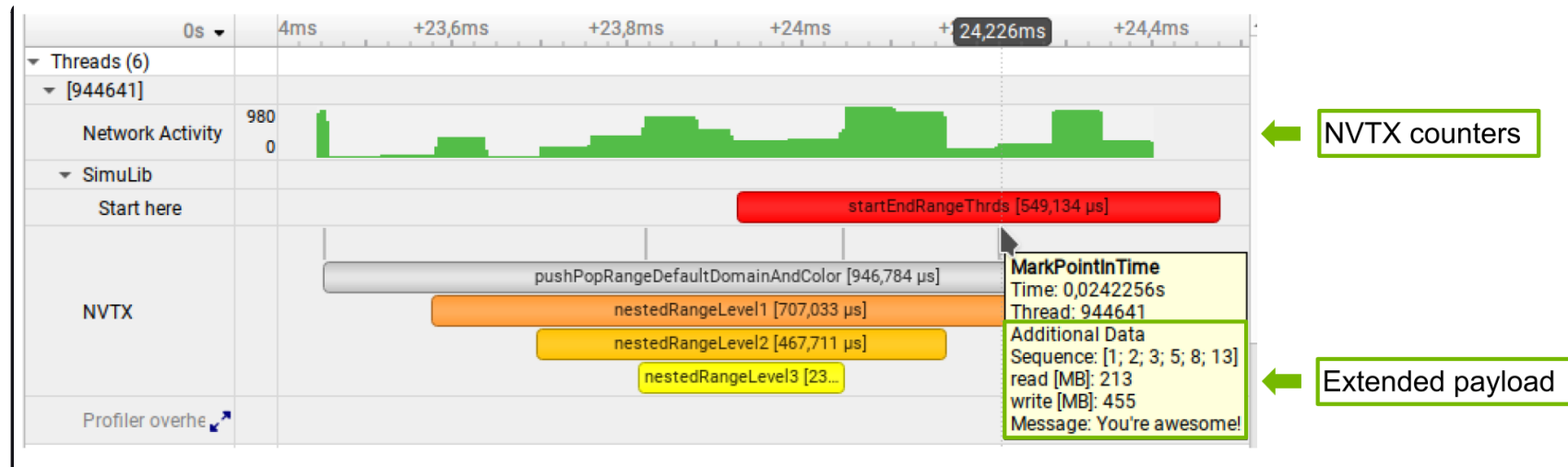
Note

Range annotations should be matched carefully. If many ranges are opened but not closed, Nsight Systems has no meaningful way to visualize it. A rule of thumb is to not have more than a couple dozen ranges open at any point in time. Nsight Systems does not support reports with many unclosed ranges.

NVTX Payloads and Counters (Preview)

NVTX Extended Payloads and NVTX Counters increase the flexibility of NVTX annotations by allowing users to pass arbitrary structured data to NVTX events. This then will allow users to define the layout of this data in the Nsight Systems UI without additional data conversion.

For more information, see [NVTX documentation](#).



NVTX Domains and Categories

NVTX domains enable scoping of annotations. Unless specified differently, all events and annotations are in the default domain. Additionally, categories can be used to group events.

Nsight Systems gives the user the ability to include or exclude NVTX events from a particular domain. This can be especially useful if you are profiling across multiple libraries and are only interested in nvtx events from some of them.

▼ ☐ Collect NVTX trace

The NVIDIA Tools Extension SDK (NVTX) is a C-based API for marking events and ranges in your applications. NVIDIA Nsight Systems supports collecting and visualizing of these events and ranges on the timeline.

☐ Insert NVTX Marker using hotkey

(not available in console apps)

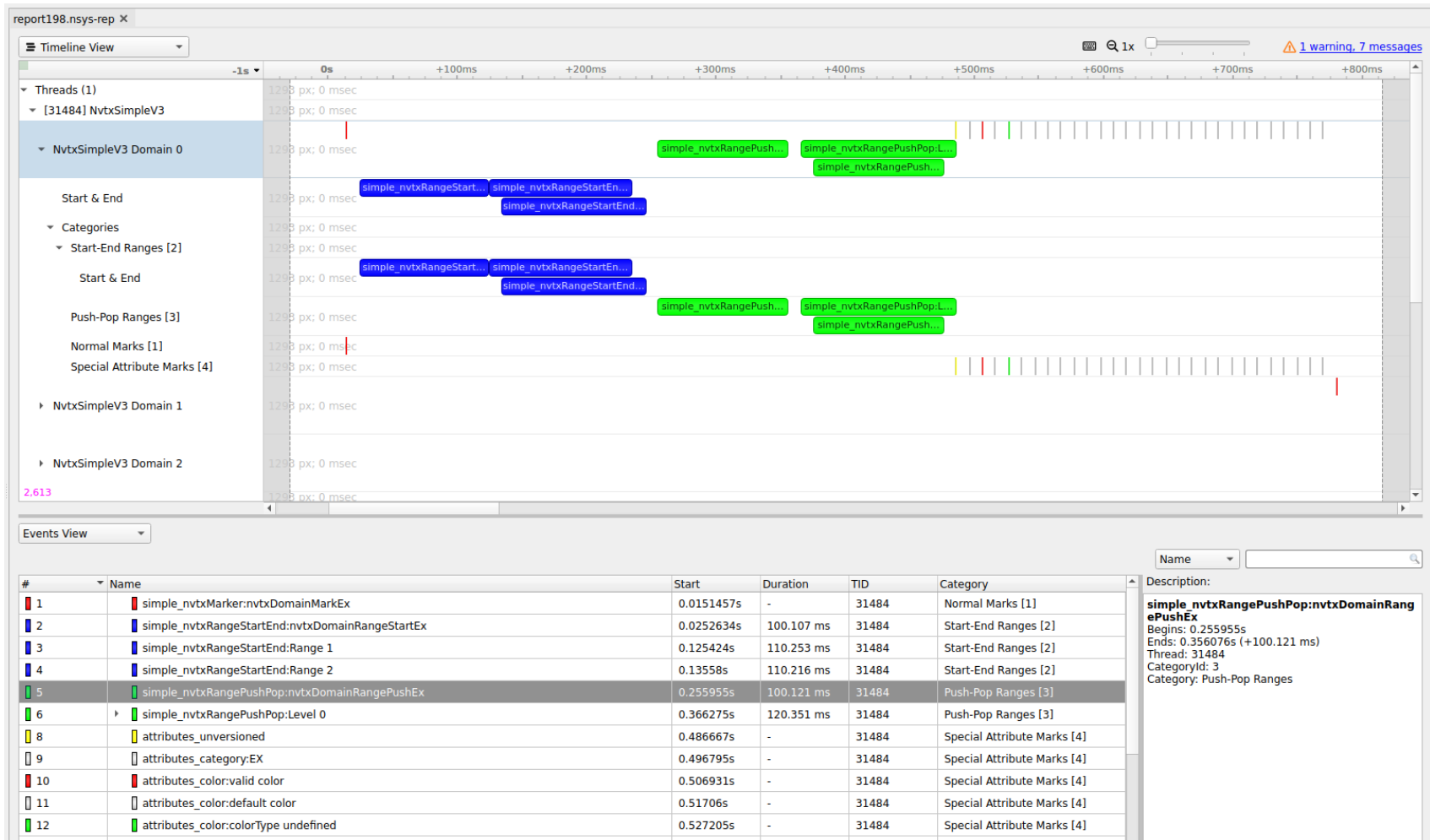
▼ ☐ NVTX domain filter

Select the filtering mode to (only) include or exclude the specified domains. Select the default domain and/or specify a comma-separated list of NVTX domains. Commas in a domain name have to be escaped with '\'.

☐ Include ☒ Exclude ☐ Default domain

This functionality is also available from the CLI. See the CLI documentation for `--nvtx-domain-include` and `--nvtx-domain-exclude` for more details.

Categories that are set in by the user will be recognized and displayed in the GUI.



CUDA Trace#

Basic CUDA trace#

Nsight Systems is capable of capturing information about CUDA execution in the profiled process.

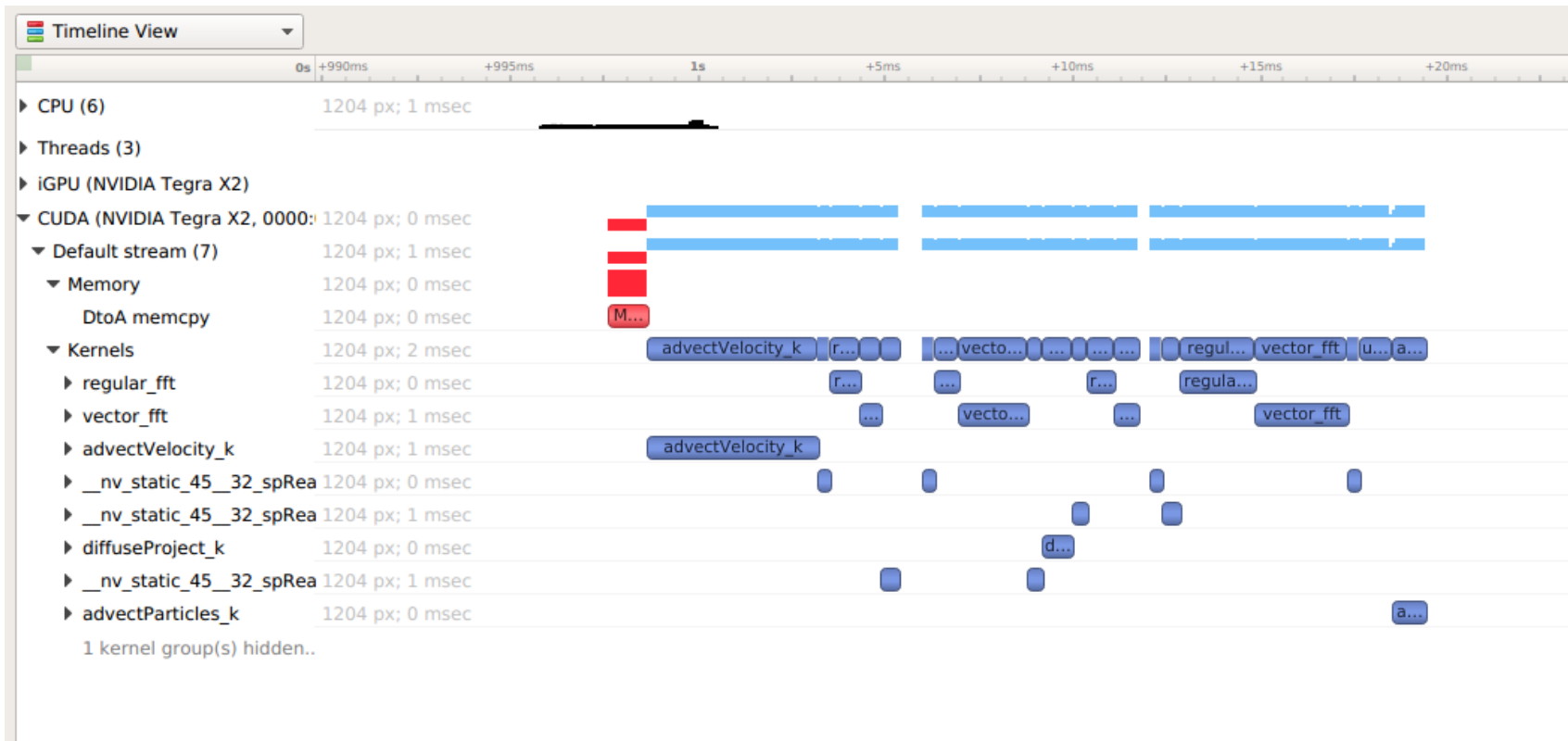
The following information can be collected and presented on the timeline in the report:

- CUDA API trace — trace of CUDA Runtime and CUDA Driver calls made by the application.

- CUDA Runtime calls typically start with cuda prefix (e.g. cudaLaunch).
- CUDA Driver calls typically start with cu prefix (e.g. cuDeviceGetCount).
- CUDA workload trace — trace of activity happening on the GPU, which includes memory operations (e.g., Host-to-Device memory copies) and kernel executions. Within the threads that use the CUDA API, additional child rows will appear in the timeline tree.
- On Nsight Systems Workstation Edition, cuDNN and cuBLAS API tracing and OpenACC tracing.

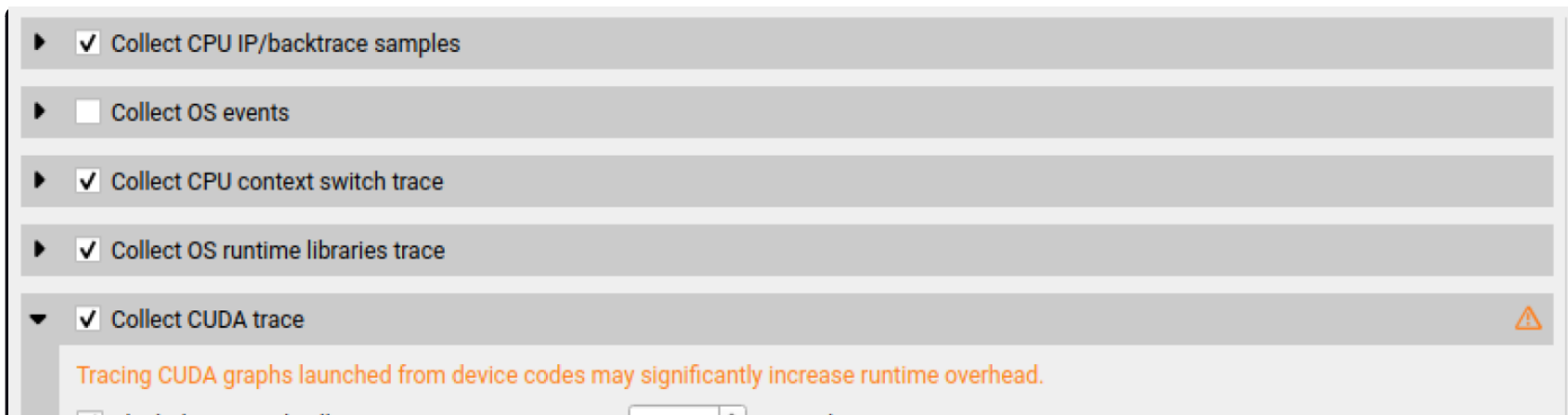


Near the bottom of the timeline row tree, the GPU node will appear and contain a CUDA node. Within the CUDA node, each CUDA context used within the process will be shown along with its corresponding CUDA streams. Streams will contain memory operations and kernel launches on the GPU. Kernel launches are represented by blue, while memory transfers are displayed in red.



The easiest way to capture CUDA information is to launch the process from Nsight Systems, and it will setup the environment for you. To do so, simply set up a normal launch and select the **Collect CUDA trace** checkbox.

For Nsight Systems Workstation Edition this looks like:



☒ Flush data periodically 3.00 seconds

☒ Flush CUDA trace buffers on cudaProfilerStop()

☒ Skip some API calls

CUDA Event trace mode Off

CUDA graph trace granularity Graph

☒ Trace CUDA graphs launched from device codes

☐ Collect GPU memory usage

☐ Collect hardware-based trace

☐ Collect UM CPU page faults

☐ Collect UM GPU page faults

☐ Collect cuDNN trace

☐ Collect cuBLAS trace

☐ Collect OpenACC trace

▶ ☐ Collect CUDA backtraces

▶ ☐ Collect OpenMP trace

▶ ☐ Collect GPU context switch trace

▶ ☐ Collect GPU Metrics

▶ ☐ Collect NV Video trace

▶ ☒ Collect NVTX trace

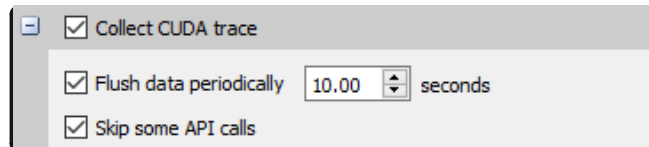
▶ ☐ Collect OpenGL trace

▶ ☐ Collect Vulkan trace

▶ Network profiling options

▶ Python profiling options

For Nsight Systems Embedded Platforms Edition this looks like:



Additional configuration parameters are available:

- **Collect hardware-based trace** - switches Nsight Systems from an instrumented CUDA collection to a hardware based collection. For workloads that launch many short duration kernels, the overhead of kernel timestamp collection can be significantly reduced. **Note:** Works on Blackwell based or later GPUs. Does not support CUDA tracing of MPS workloads. Does not work in MIG partitions or virtual GPU environments (vGPU).
- **Collect backtraces for API calls longer than X seconds** - turns on collection of CUDA API backtraces and sets the minimum time a CUDA API event must take before its backtraces are collected. Setting this value too low can cause high application overhead and seriously increase the size of your results file.
- **Flush data periodically** — specifies the period after which an attempt to flush CUDA trace data will be made. Normally, in order to collect full CUDA trace, the application needs to finalize the device used for CUDA work (call `cudaDeviceReset()`), and then let the application gracefully exit (as opposed to crashing).

This option allows flushing CUDA trace data even before the device is finalized. However, it might introduce additional overhead to a random CUDA Driver or CUDA Runtime API call.
- **Skip some API calls** — avoids tracing insignificant CUDA Runtime API calls (namely, `cudaConfigureCall()`, `cudaSetupArgument()`, `cudaHostGetDevicePointers()`). Not tracing these functions allows Nsight Systems to significantly reduce the profiling overhead, without losing any interesting data.
- **Collect GPU Memory Usage** - collects information used to generate a graph of CUDA allocated memory across time. Note that this will increase overhead. See [CUDA GPU Memory Allocation Graph](#).
- **Collect Unified Memory CPU page faults** - collects information on page faults that occur when CPU code tries to access a memory page that resides on the device. See [Unified Memory CPU Page Faults](#).

- **Collect Unified Memory GPU page faults** - collects information on page faults that occur when GPU code tries to access a memory page that resides on the CPU. See [Unified Memory GPU Page Faults](#).
- **Collect CUDA Graph trace** - by default, CUDA tracing will collect and expose information on a whole graph basis. The user can opt to collect on a node per node basis. See [CUDA Graph Trace](#).
- **Collect CUDA Event trace** - track device-side CUDA Event (the synchronization mechanism) completion, and get better correlation support among CUDA Event APIs. [CUDA Event Trace](#).
- For Nsight Systems Workstation Edition, **Collect cuDNN trace**, **Collect cuBLAS trace**, **Collect OpenACC trace** - selects which (if any) extra libraries that depend on CUDA to trace.

OpenACC versions 2.0, 2.5, and 2.6 are supported when using PGI runtime version 15.7 or greater and not compiling statically. In order to differentiate constructs, a PGI runtime of 16.1 or later is required. Note that Nsight Systems Workstation Edition does not support the GCC implementation of OpenACC at this time.

Note

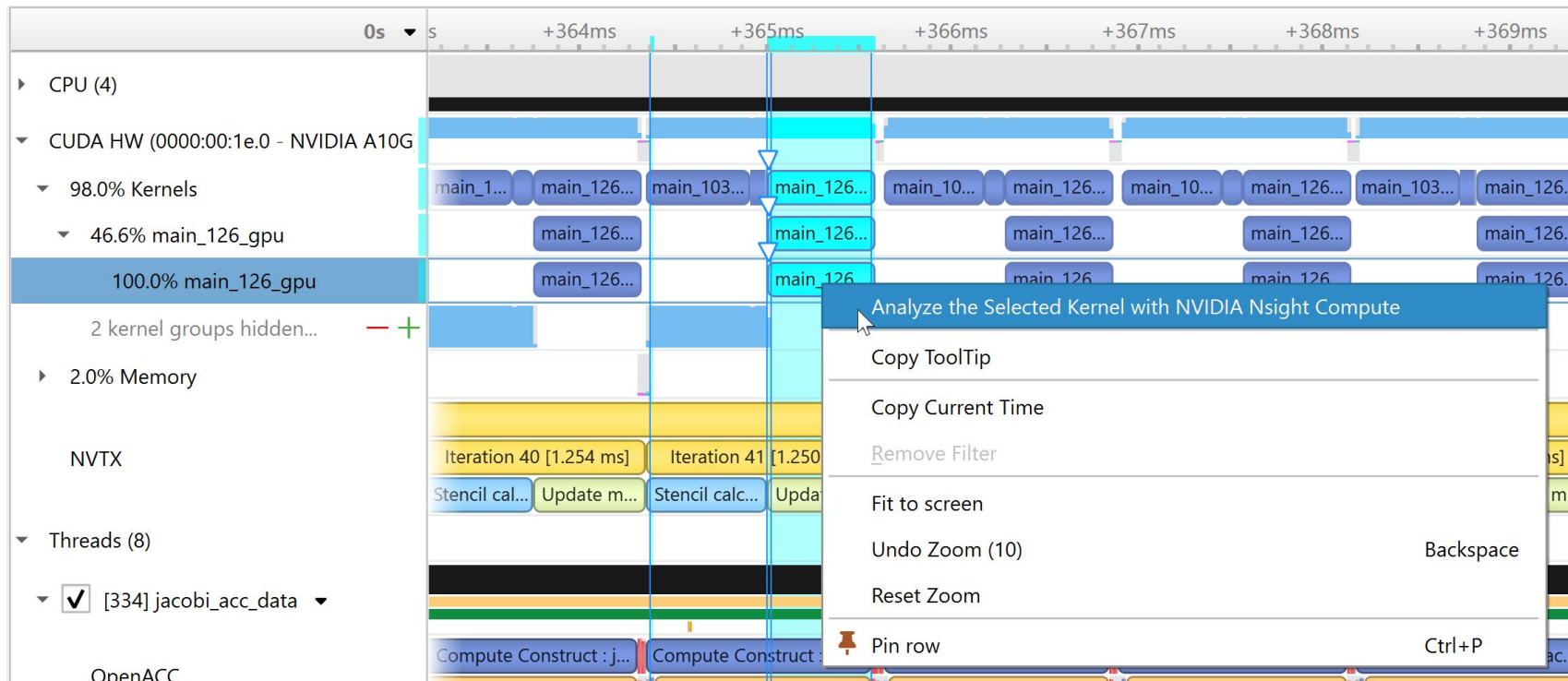
If your application crashes before all collected CUDA trace data has been copied out, some or all data might be lost and not present in the report.

Note

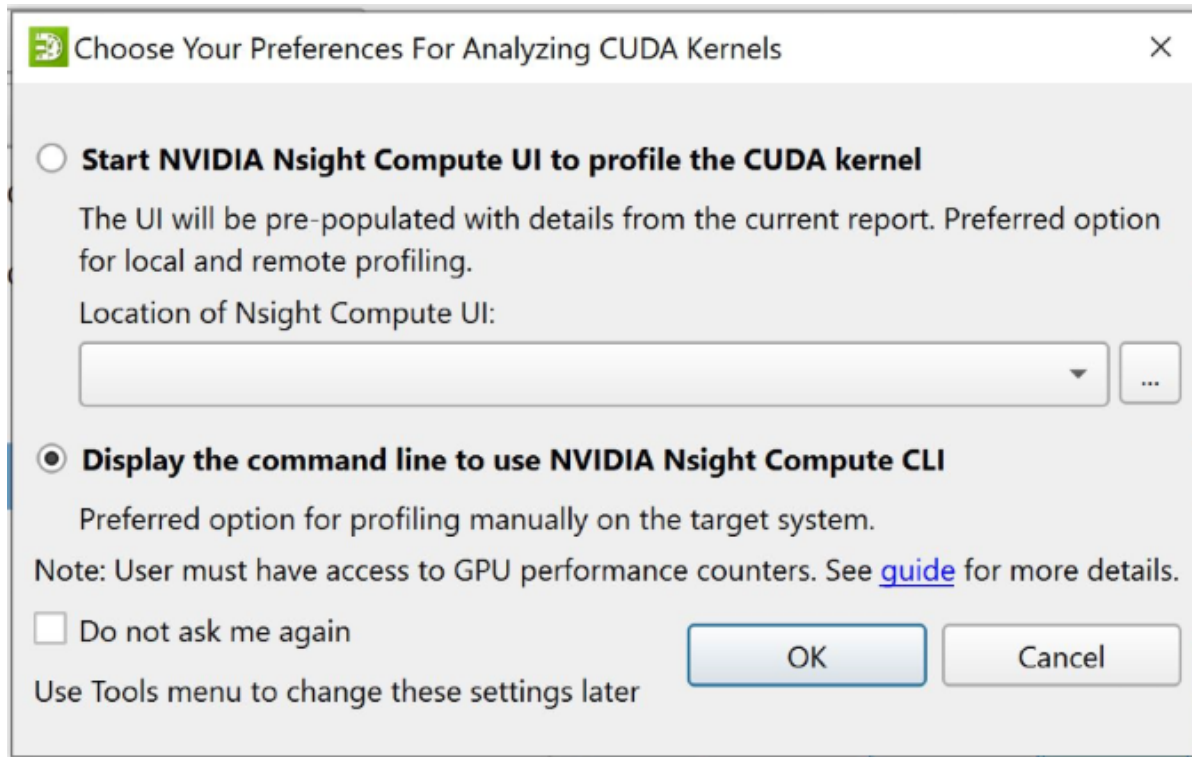
Nsight Systems will not have information about CUDA events that were still in device buffers when analysis terminated. It is a good idea, if using cudaProfilerAPI to control analysis to call cudaDeviceReset before ending analysis.

Launching NVIDIA Nsight Compute from a CUDA Kernel#

After you have used CUDA trace in Nsight Systems to locate a potential problem kernel, you may want to run NVIDIA Nsight Compute on that specific kernel. Right click on the kernel to bring up a menu.

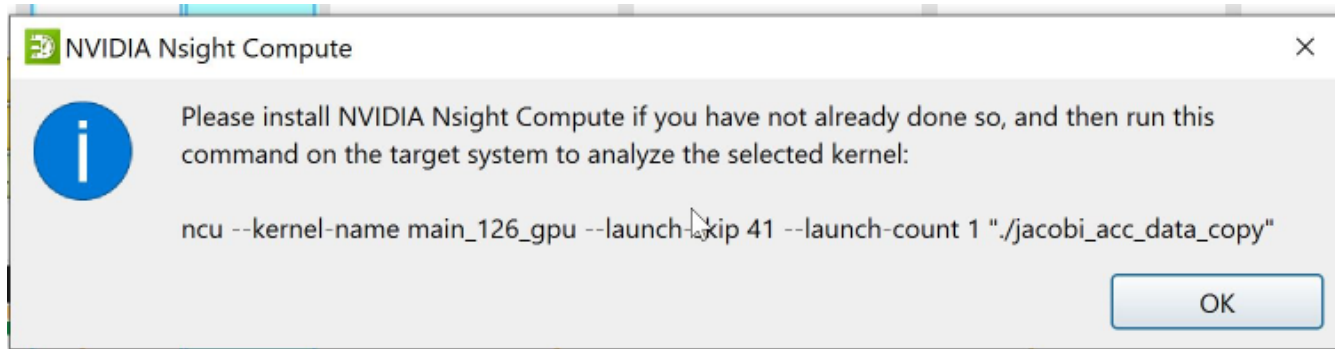


If this is the first time that the user has selected this feature, then we show the following dialog box to get their preferences:



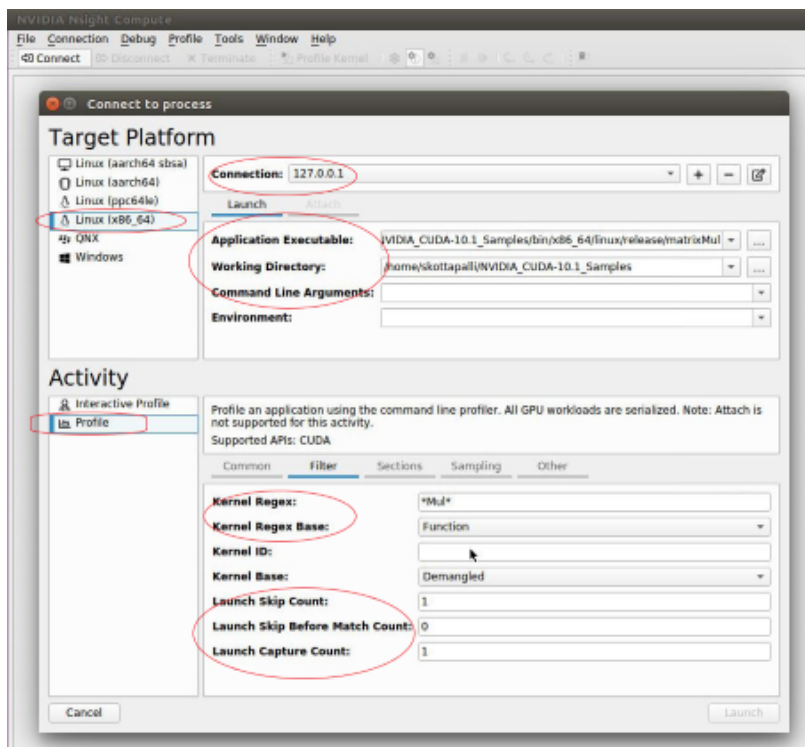
The first option invokes the NVIDIA Nsight Compute UI with known parameters. It is the preferred option for local or remote profiling. The user must provide the location of the ncu-ui executable. Nsight Systems will verify that the path and executable are valid.

The second option is provided for the convenience of users who do not have NVIDIA Nsight Compute installed on the host system, but simply want the command line they can use to run the Nsight Compute on the remote target by themselves without much automation.



If the user selects the option to start the NCU UI, Nsight Systems invokes it with any relevant parameters from the Nsight Systems run.

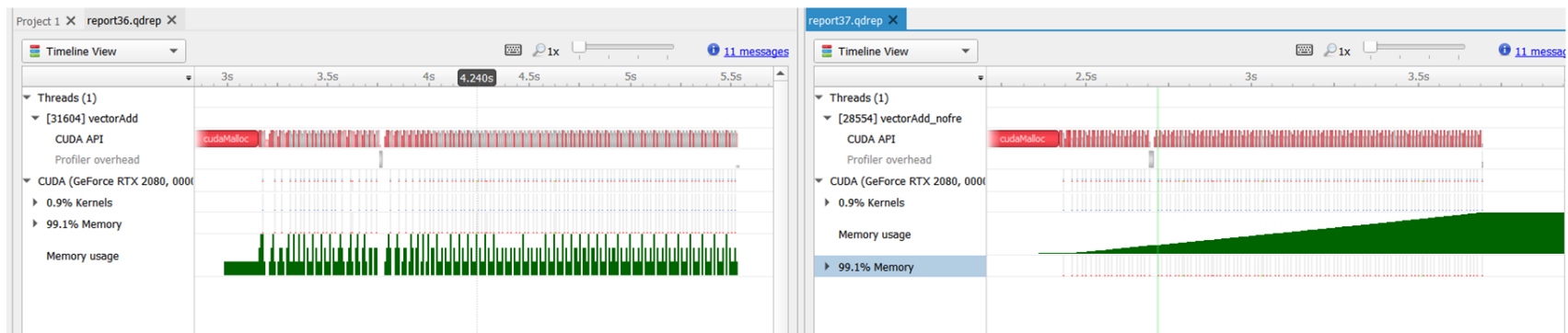
The screenshot below shows NCU UI invoked by Nsight Systems. The red circles indicate the parameters pre-populated by Nsight Systems. Users may modify any of these parameters before launching the application and profiling the selected kernel.



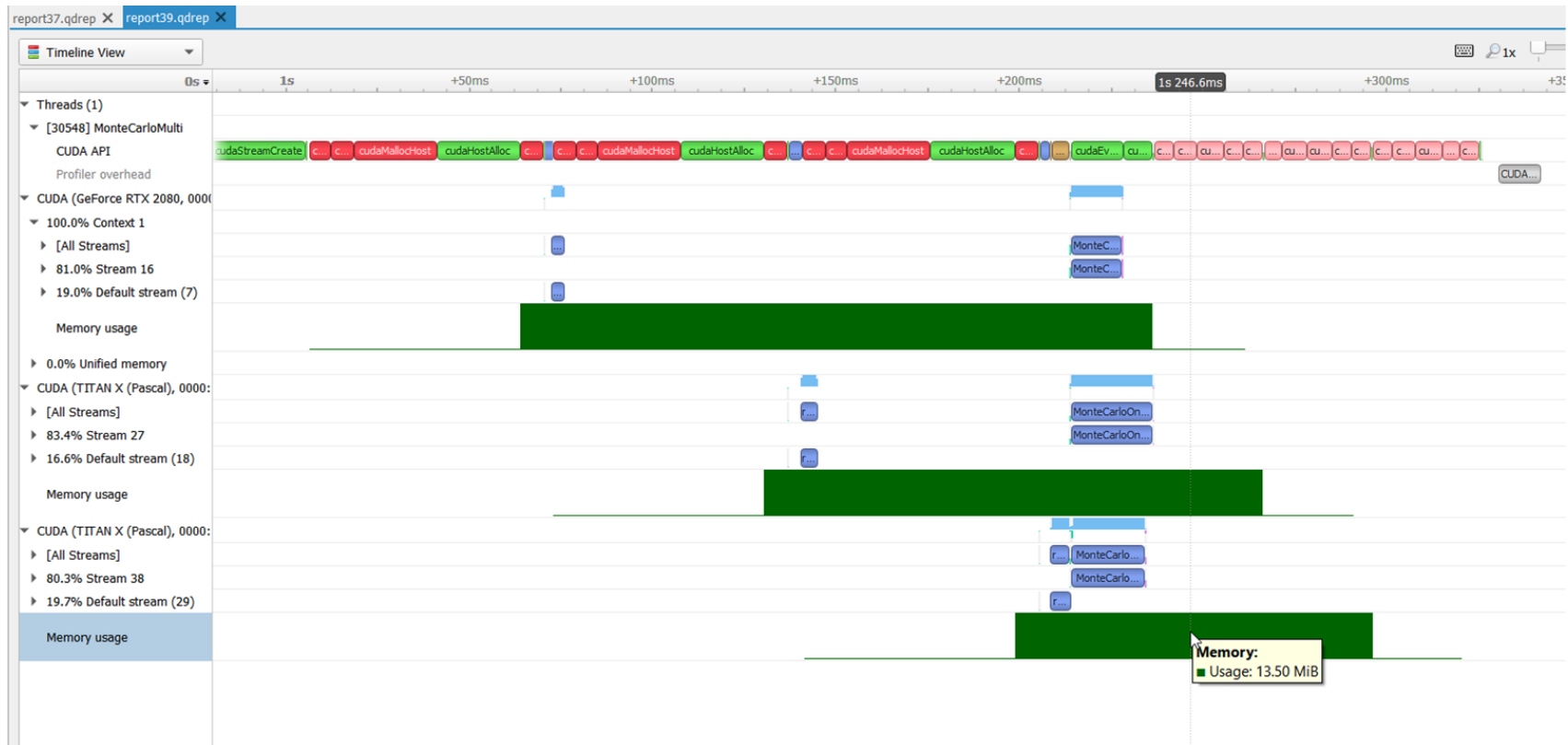
CUDA GPU Memory Allocation Graph#

When the **Collect GPU Memory Usage** option is selected from the **Collect CUDA trace** option set, Nsight Systems will track CUDA GPU memory allocations and deallocations and present a graph of this information in the timeline. This is not the same as the GPU memory graph generated during stutter analysis on the Windows target. See [Windows GPU Memory Utilization](#).

Below, in the report on the left, memory is allocated and freed during the collection. In the report on the right, memory is allocated, but not freed during the collection.



Here is another example, where allocations are happening on multiple GPUs.

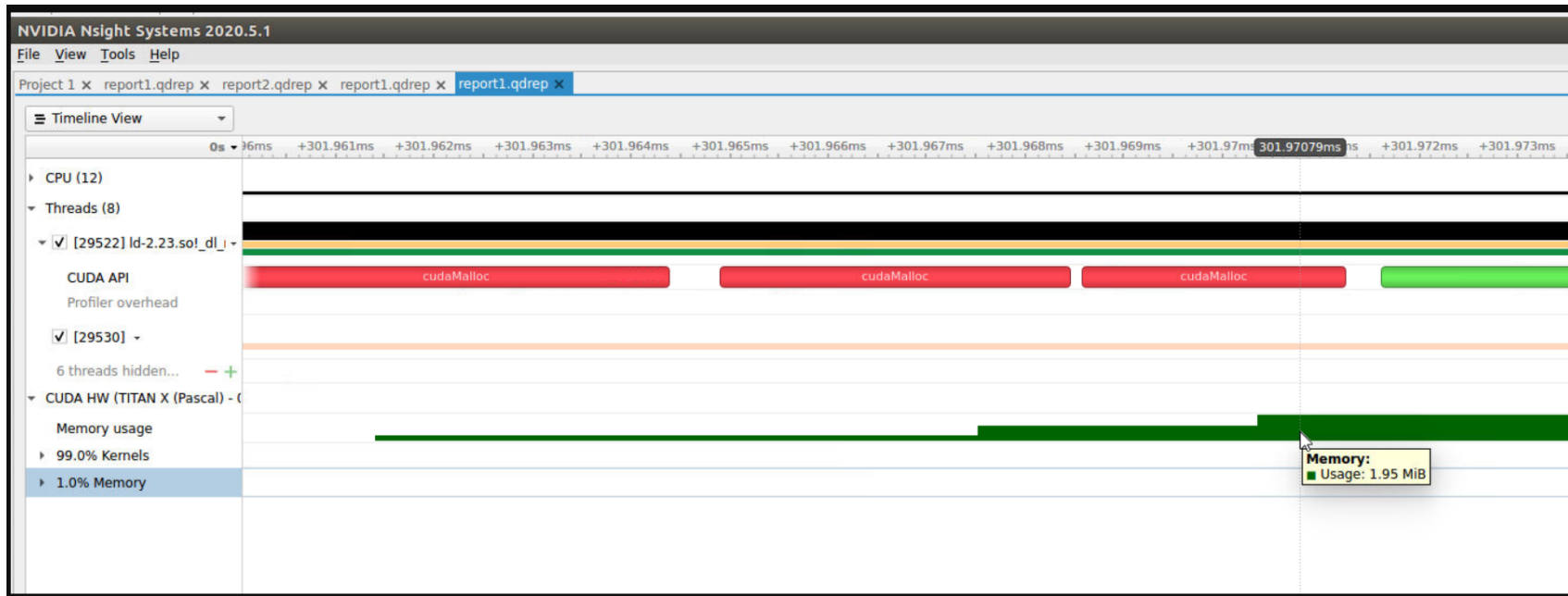


Nsight Systems uses CUPTI for CUDA profiling, including to collect the CUDA memory usage by the application processes. CUPTI tracks various kinds of memory allocations and deallocations done by the user application that is being profiled. See: [CUPTI documentation](#).

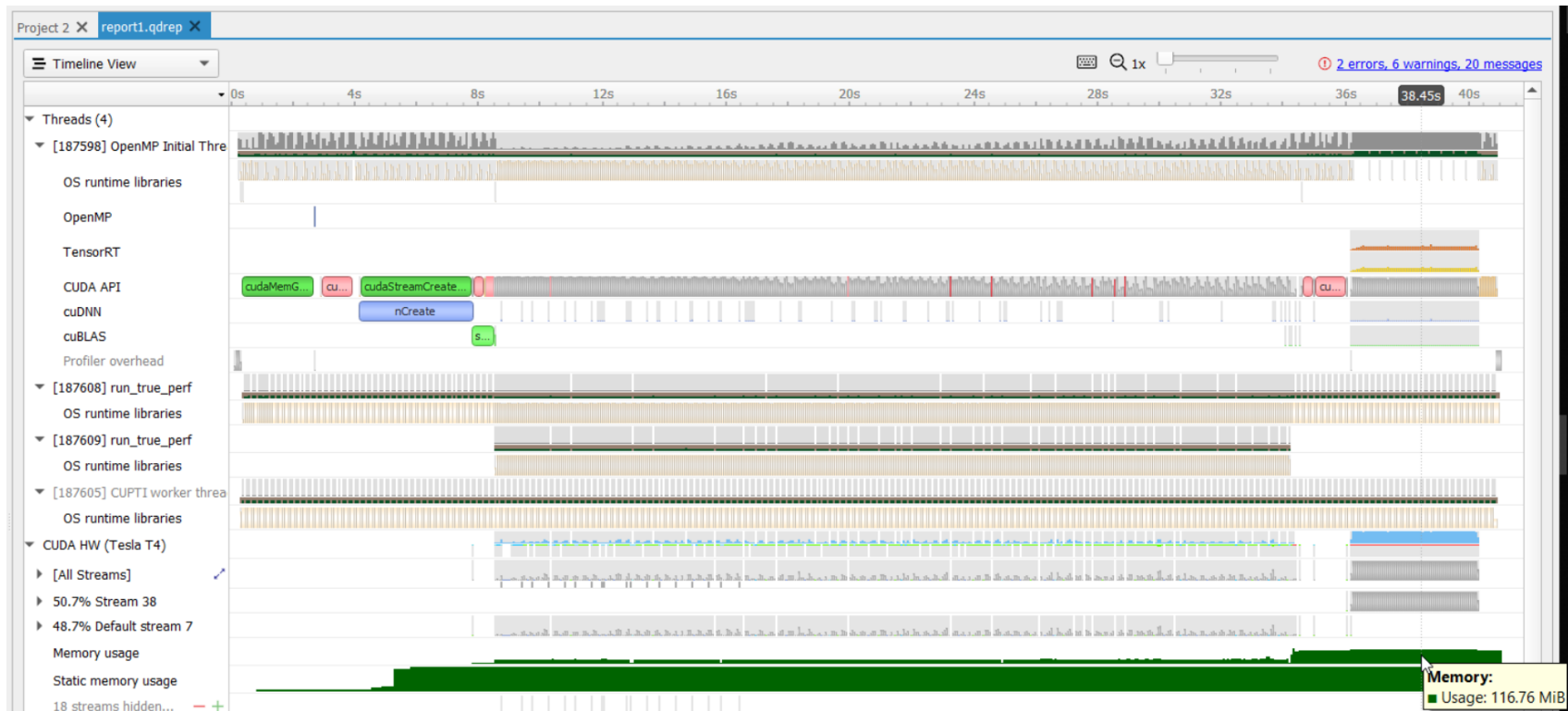
CUPTI_ACTIVITY_MEMORY_KIND_PAGEABLE = 1 The memory is pageable.
 CUPTI_ACTIVITY_MEMORY_KIND_PINNED = 2 The memory is pinned.
 CUPTI_ACTIVITY_MEMORY_KIND_DEVICE = 3 The memory is on the device.
 CUPTI_ACTIVITY_MEMORY_KIND_ARRAY = 4 The memory is an array.
 CUPTI_ACTIVITY_MEMORY_KIND_MANAGED = 5 The memory is managed
 CUPTI_ACTIVITY_MEMORY_KIND_DEVICE_STATIC = 6 The memory is device static
 CUPTI_ACTIVITY_MEMORY_KIND_MANAGED_STATIC = 7 The memory is managed static

There are three graphs shown in nsys GUI timeline. One graph is called the “Memory usage” under each GPU. It is

the sum of memory kinds device and array used by that process. The graph increases when memory allocation APIs (such as `cudaMalloc`, `cudaMallocManaged`) are called and the graph decreases when memory deallocation APIs (such as `cudaFree`) are called.

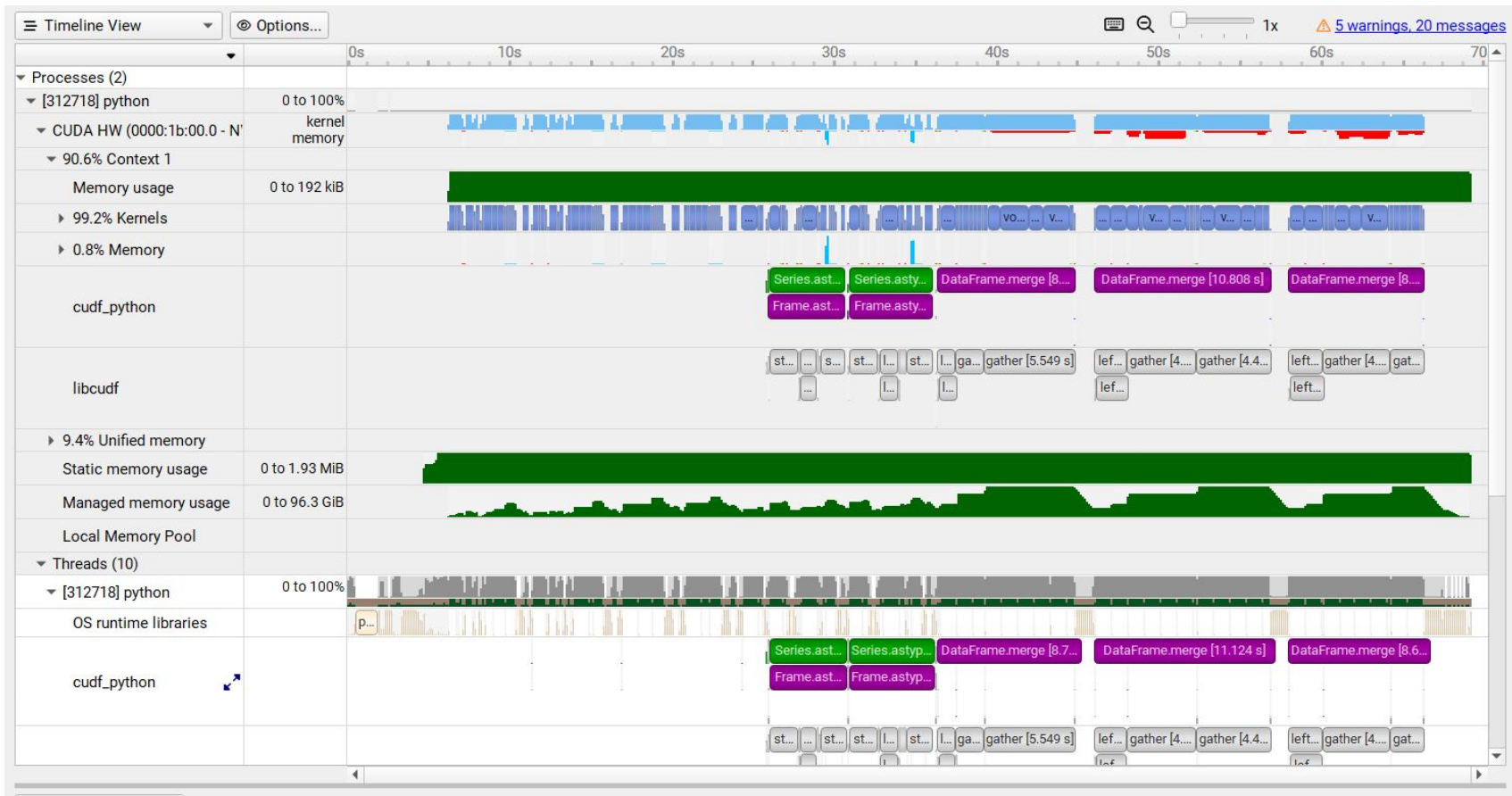


The second graph called “Managed Memory usage” under each GPU is the graph of memory kind `{{managed}}` used by that process.



The third graph called “Static Memory usage” is the sum of memory kind device static and managed static used by the process:

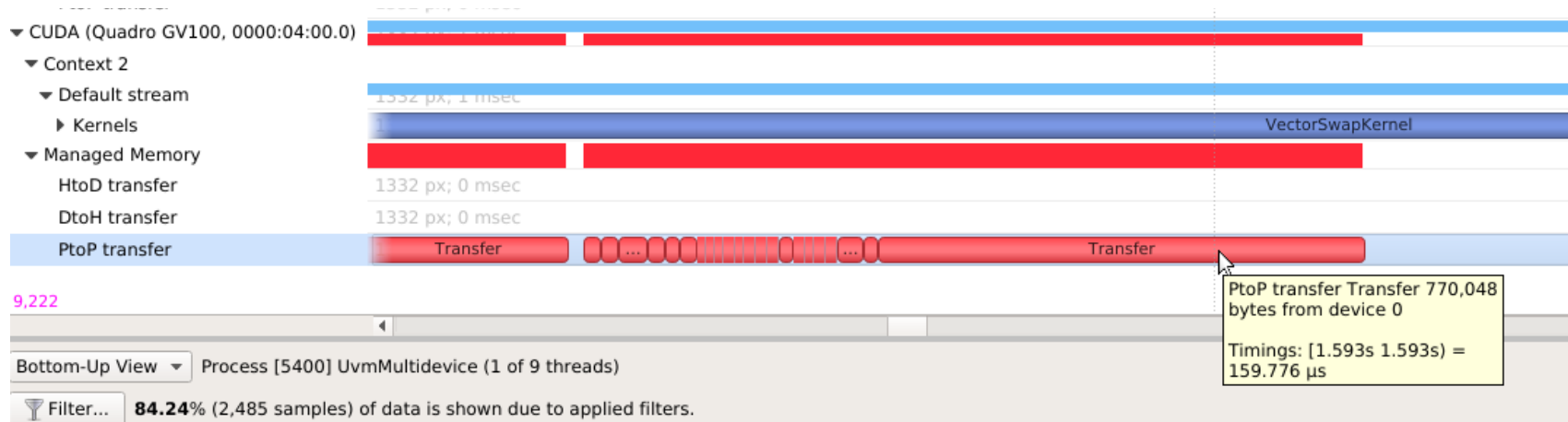
- CUPTI_ACTIVITY_MEMORY_KIND_DEVICE_STATIC is a static memory allocation. It does not have a context. Since it is static, it is allocated by variable declaration. For example, `__device__ int var;`
- CUPTI_ACTIVITY_MEMORY_KIND_MANAGED_STATIC s static managed allocation. For example, `__device__ __managed__ int var;` In other words, this is static equivalent of `cudaMallocManaged()` API.



The pageable and pinned are both host-side memory calls, so we don't show those on the GUI timeline.

Unified Memory Transfer Trace#

For Nsight Systems Workstation Edition, Unified Memory (also called Managed Memory) transfer trace is enabled automatically in Nsight Systems when CUDA trace is selected. It incurs no overhead in programs that do not perform any Unified Memory transfers. Data is displayed in the Managed Memory area of the timeline:

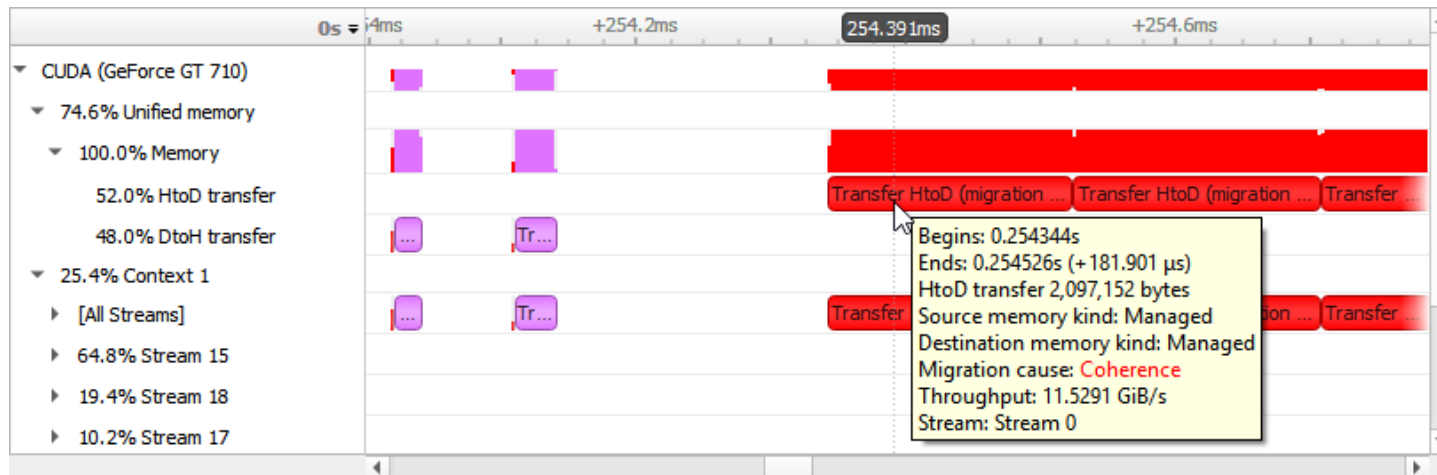


HtoD transfer indicates the CUDA kernel accessed managed memory that was residing on the host, so the kernel execution paused and transferred the data to the device. Heavy traffic here will incur performance penalties in CUDA kernels, so consider using manual `cudaMemcpy` operations from pinned host memory instead.

PtoP transfer indicates the CUDA kernel accessed managed memory that was residing on a different device, so the kernel execution paused and transferred the data to this device. Heavy traffic here will incur performance penalties, so consider using manual `cudaMemcpyPeer` operations to transfer from other devices' memory instead. The row showing these events is for the destination device - the source device is shown in the tooltip for each transfer event.

DtoH transfer indicates the CPU accessed managed memory that was residing on a CUDA device, so the CPU execution paused and transferred the data to system memory. Heavy traffic here will incur performance penalties in CPU code, so consider using manual `cudaMemcpy` operations from pinned host memory instead.

Some Unified Memory transfers are highlighted with red to indicate potential performance issues:



Transfers with the following migration causes are highlighted:

- **Coherence**

Unified Memory migration occurred to guarantee data coherence. SMs (streaming multiprocessors) stop until the migration completes.

- **Eviction**

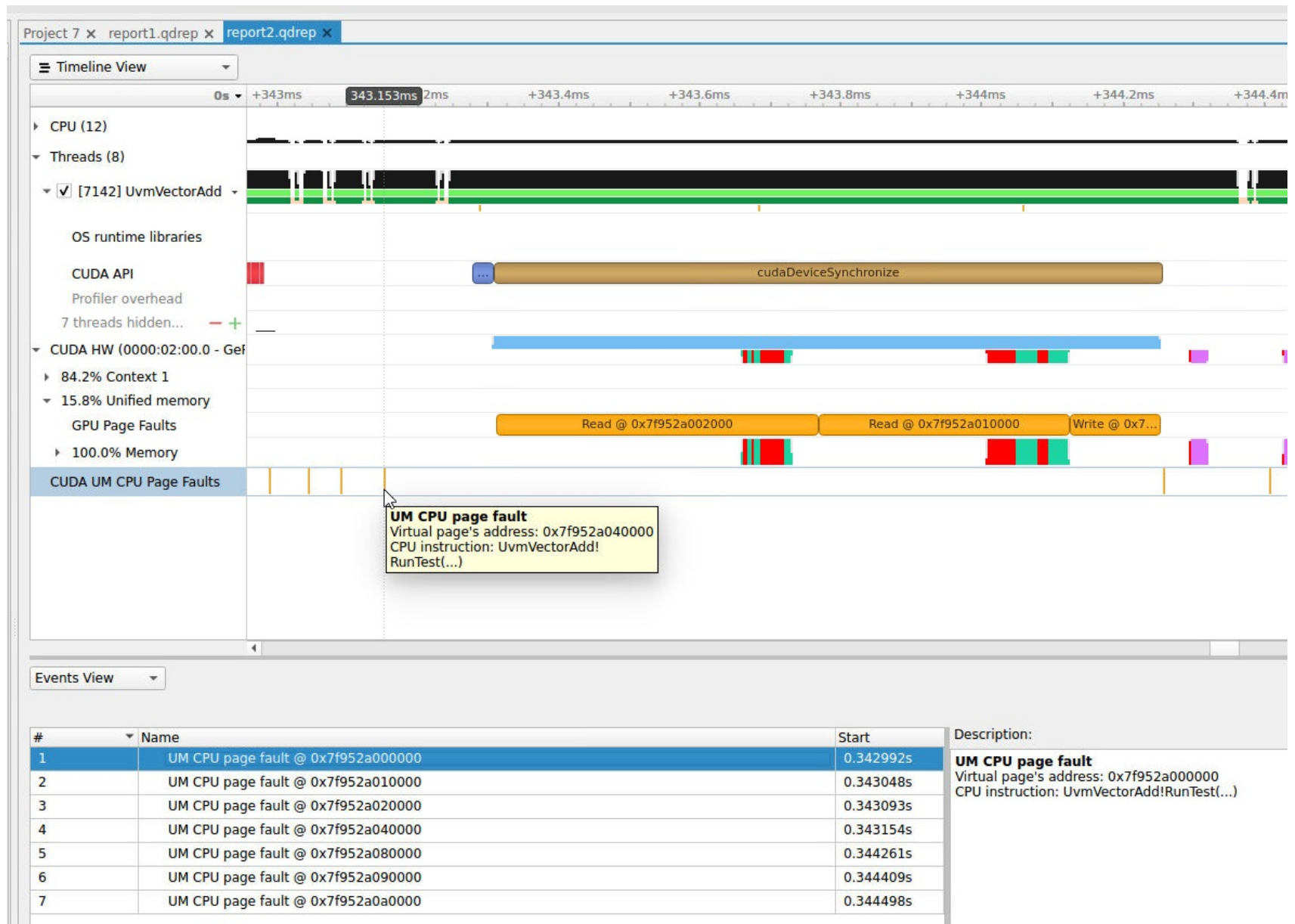
Unified Memory migrated to the CPU because it was evicted to make room for another block of memory on the GPU. This happens due to memory overcommitment which is available on Linux with Compute Capability ≥ 6 .

Unified Memory CPU Page Faults#

The Unified Memory CPU page faults feature in Nsight Systems tracks the page faults that occur when CPU code tries to access a memory page that resides on the device.

Note

Collecting Unified Memory CPU page faults can cause overhead of up to 70% in testing. Use this functionality only when needed.



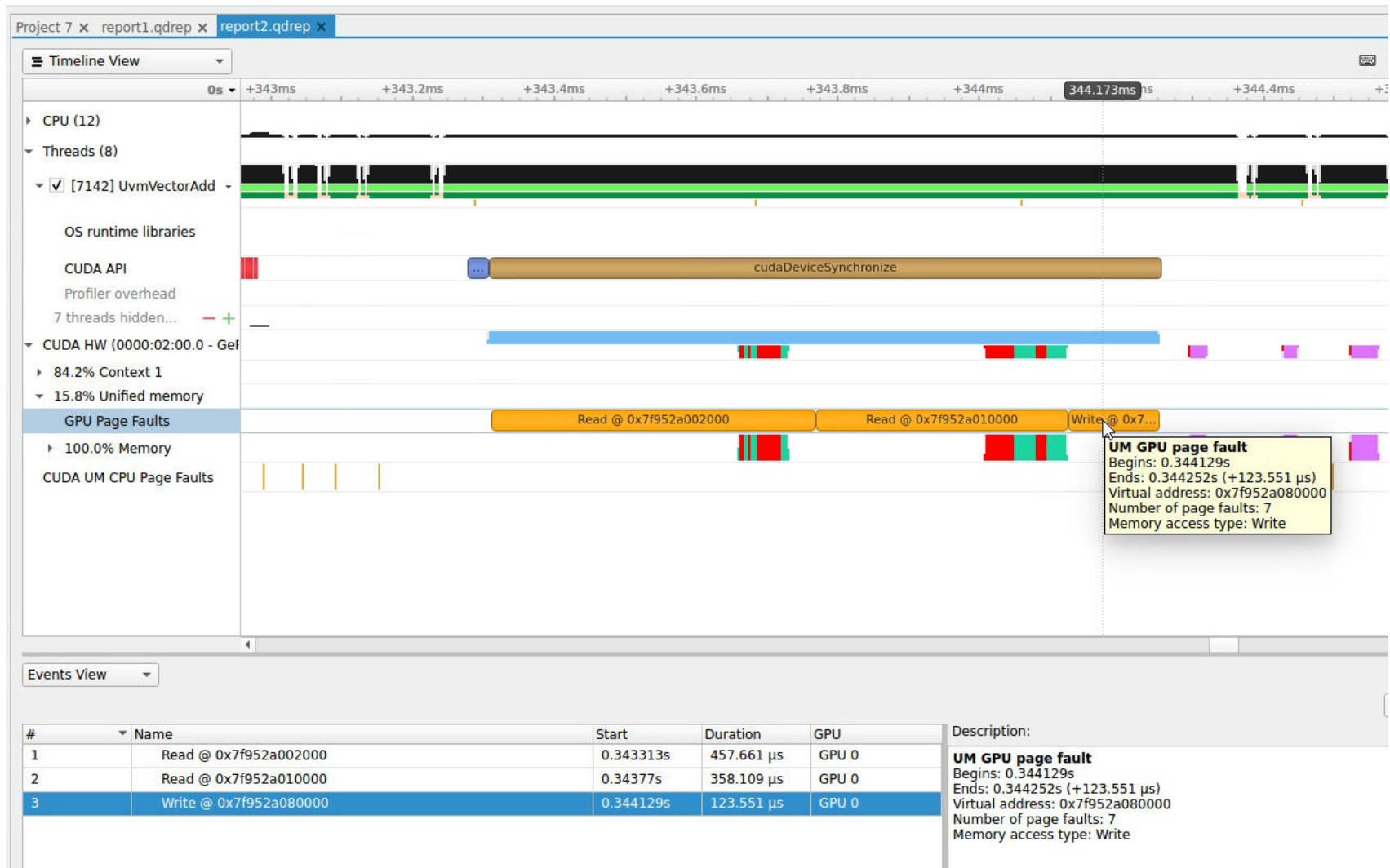
Unified Memory GPU Page Faults#

The Unified Memory GPU page faults feature in Nsight Systems tracks the page faults that occur when GPU code

tries to access a memory page that resides on the host.

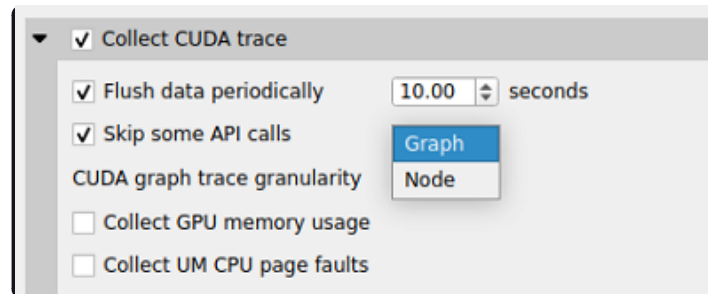
Note

Collecting Unified Memory GPU page faults can cause overhead of up to 70% in testing. Use this functionality only when needed.

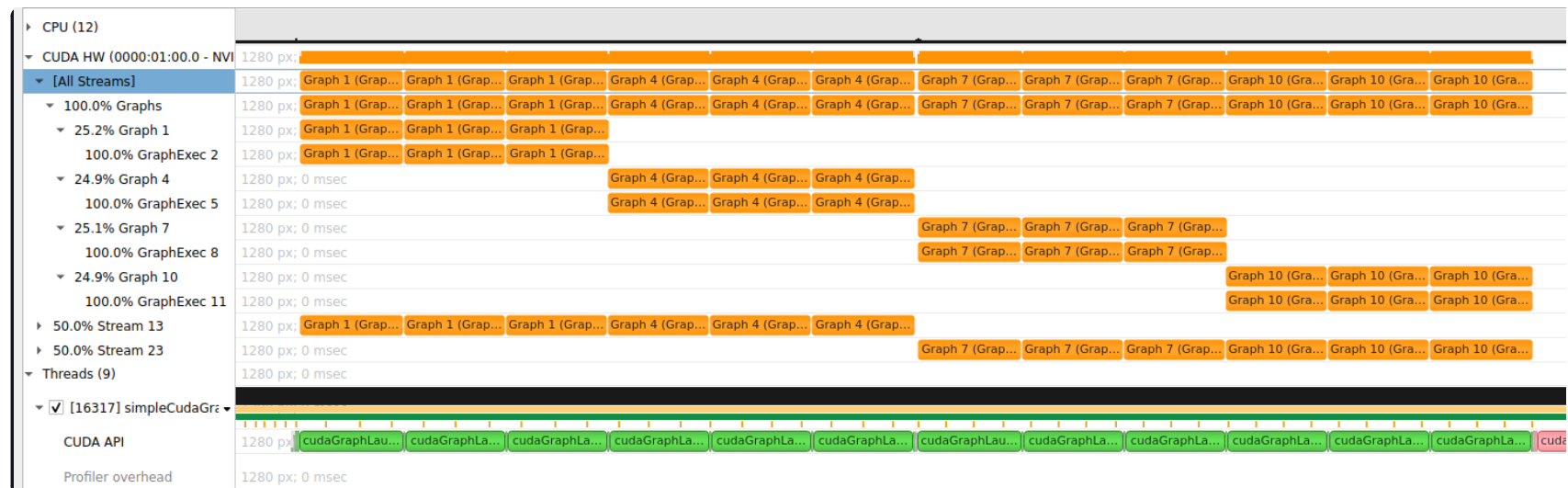


CUDA Graph Trace#

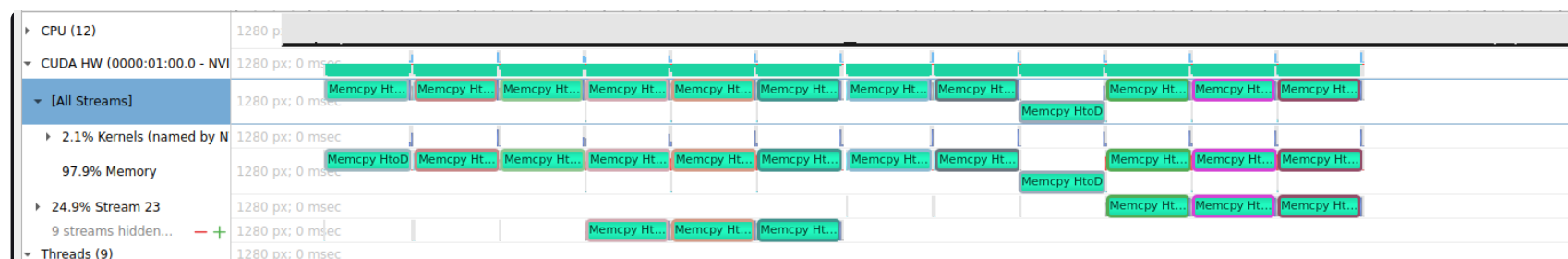
Nsight Systems is capable of capturing information about CUDA graphs in your application at either the graph or node granularity. This can be set in the CLI using the `--cuda-graph-trace` option, or in the GUI by setting the appropriate drop down.

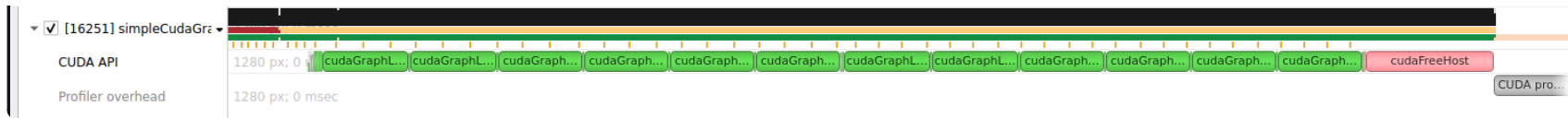


When CUDA graph trace is set to graph, the users sees each graph as one item on the timeline:



When CUDA graph trace is set to node, the users sees each graph as a set of nodes on the timeline:





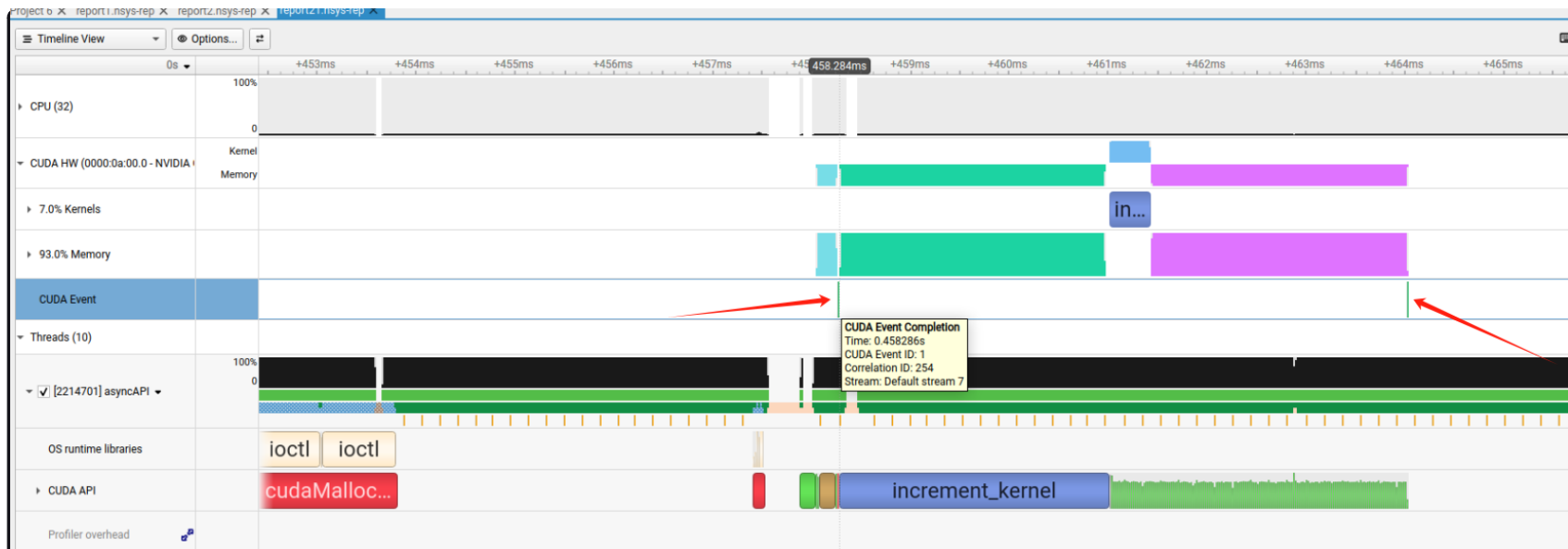
Tracing CUDA graphs at the graph level rather than the tracing the underlying nodes results in significantly less overhead. This option is only available with CUDA driver 515.43 or higher.

CUDA Event Trace#

Nsight Systems is capable of capturing information about CUDA Events (the synchronization mechanism, e.g. `cudaEventRecord()`, `cudaStreamWaitEvent()` etc.) in your application. This can be set in the CLI using the `--cuda-event-t-trace` option, or in the GUI by setting the appropriate drop down.

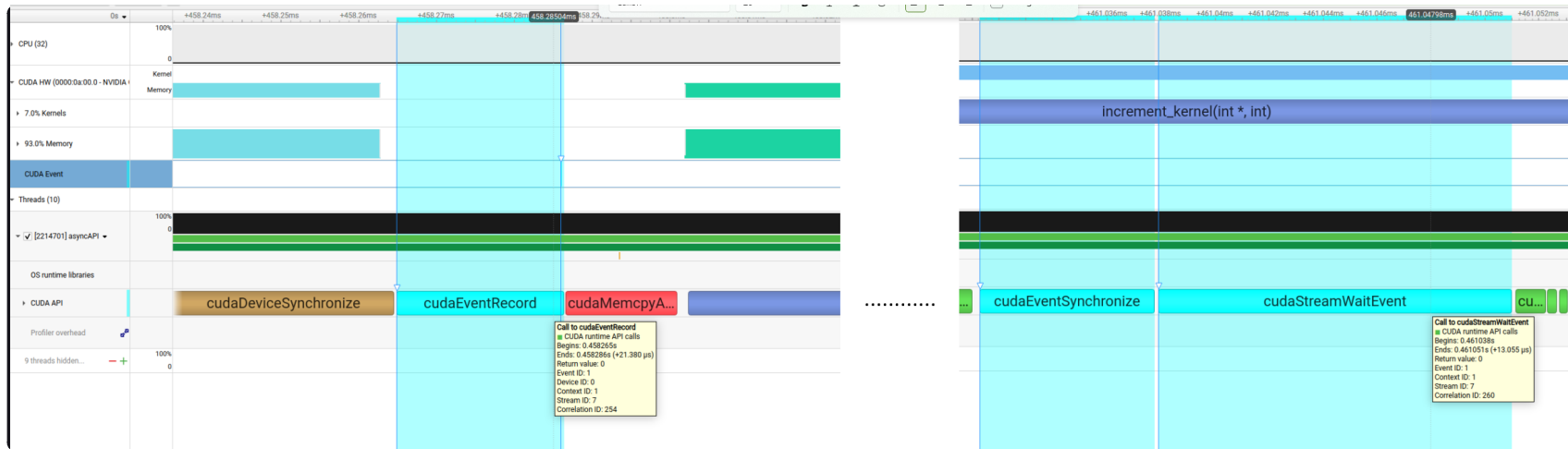


When CUDA event trace is set to `true`, users can see device-side CUDA event completion markers in CUDA HW timelines:





Additionally, there will be better correlation support among CUDA Event APIs, for example when clicking a `cudaEventRecord()`, the related calls such `cudaEventSynchronize()`, `cudaStreamWaitEvent()` that operate on the same CUDA event object will be highlighted:



However, there are also some limitations with this feature:

- Currently, CUDA Event object created with `cudaEventInterprocess` flag and/or used in CUDA Graphs are not supported. The support only works under non-IPC and non-CUDA-Graph scenarios.
- Currently, the underlying mechanism for tracing device-side CUDA Event completion is same as CUDA Event's own timing functionality (i.e. with a CUDA Event object is created without `cudaEventDisableTiming` flag). The mechanism is known to increase the possibility of false dependencies among seemingly unrelated CUDA Streams. Therefore, if the app's behavior changes with this feature, consider disabling it.
- This option is only available with CUDA user-mode driver 12.8 or higher.

CUDA Python Backtrace#

Nsight Systems for Arm server (SBSA) platforms and x86 Linux targets, is capable of capturing Python backtrace

information when CUDA backtrace is being captured.

To enable CUDA Python backtrace from Nsight Systems:

CLI — Set `--python-backtrace=cuda`.

GUI — Select the **Collect Python backtrace for selected API calls** checkbox.

The screenshot shows the Nsight Systems configuration window. The 'Collect CUDA trace' section is expanded, showing various options. The 'Collect CUDA backtraces' section is also expanded, showing options for collecting backtraces for different types of operations. The 'Collect Python backtrace for selected API calls' checkbox is highlighted with a red underline.

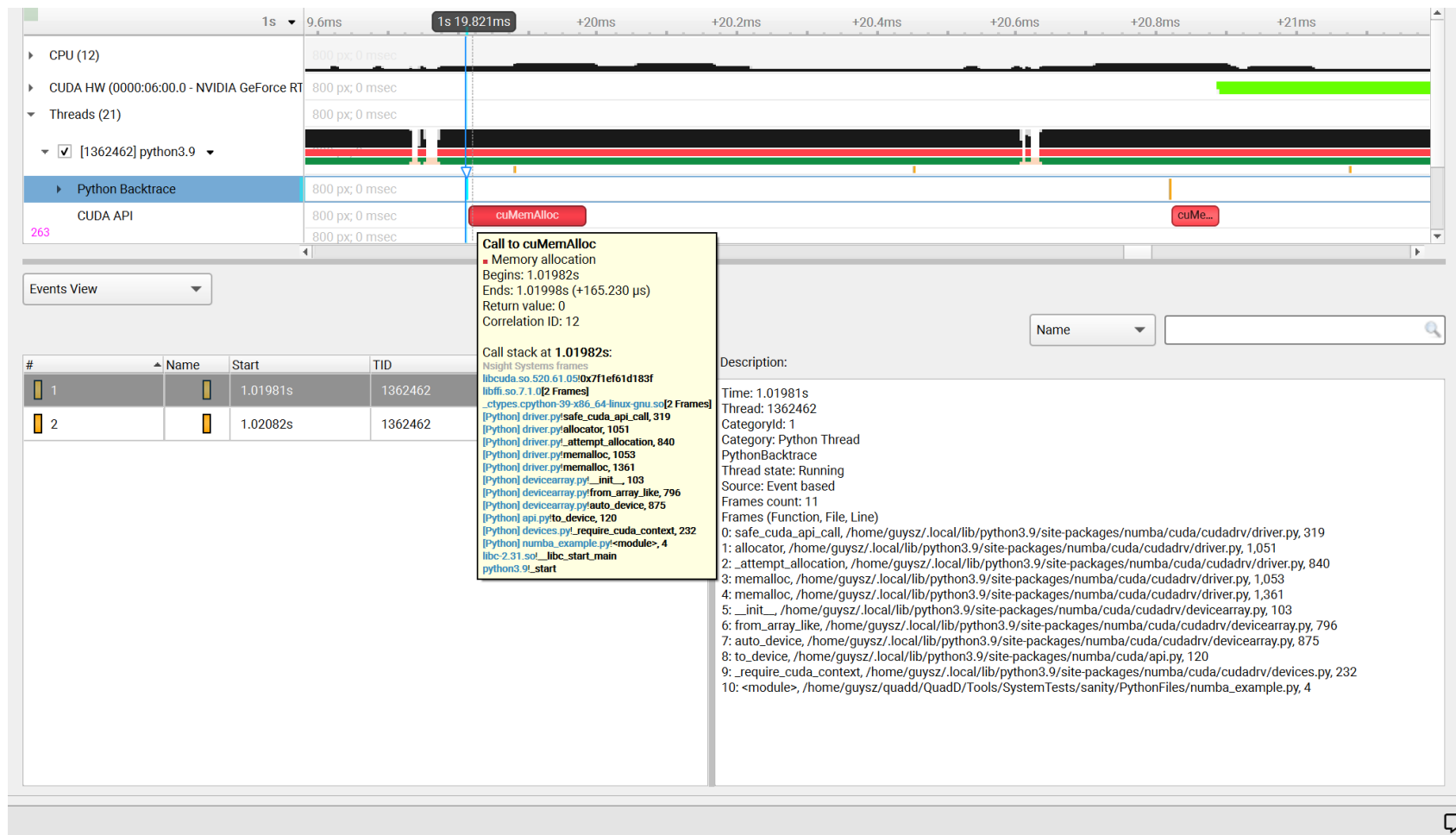
☒ Collect CUDA trace

- ☒ Flush data periodically 10.00 seconds
- ☒ Flush CUDA trace buffers on cudaProfilerStop()
- ☒ Skip some API calls
- CUDA graph trace granularity Graph
- ☐ Collect GPU memory usage
- ☐ Collect UM CPU page faults
- ☐ Collect UM GPU page faults
- ☐ Collect cuDNN trace
- ☐ Collect cuBLAS trace
- ☐ Collect OpenACC trace

☐ Collect CUDA backtraces

- ☒ Collect backtraces for kernel launches longer than 80.000 microseconds
- ☒ Collect backtraces for memory operations longer than 80.000 microseconds
- ☒ Collect backtraces for synchronizations longer than 80.000 microseconds
- ☐ Collect backtraces for other API calls longer than 80.000 microseconds
- ☐ Collect Python backtrace for selected API calls

Example screenshot:



CUDA Functions Skipped by Default#

In order to decrease the overhead of CUDA API trace, Nsight Systems does not trace short duration CUDA functions with little impact on performance.

You can turn on full CUDA API trace using the CLI or the GUI, but since CUDA trace overhead is on a per API call basis, this will dramatically impact your overhead and may lead to non-representative performance analysis.

CUDA Runtime API Calls Skipped by Default

cudaBindSurfaceToArray
cudaBindTexture
cudaBindTexture2D
cudaBindTextureToArray
cudaBindTextureToMipmappedArray
cudaConfigureCall
cudaCreateSurfaceObject
cudaCreateTextureObject
cudaD3D10MapResources
cudaD3D10RegisterResource
cudaD3D10UnmapResources
cudaD3D10UnregisterResource
cudaD3D9MapResources
cudaD3D9MapVertexBuffer
cudaD3D9RegisterResource
cudaD3D9RegisterVertexBuffer
cudaD3D9UnmapResources
cudaD3D9UnmapVertexBuffer
cudaD3D9UnregisterResource
cudaD3D9UnregisterVertexBuffer
cudaDestroySurfaceObject
cudaDestroyTextureObject
cudaDeviceReset
cudaDeviceSynchronize
cudaEGLStreamConsumerAcquireFrame
cudaEGLStreamConsumerConnect
cudaEGLStreamConsumerConnectWithFlags
cudaEGLStreamConsumerDisconnect

cudaEGLStreamConsumerReleaseFrame
cudaEGLStreamConsumerReleaseFrame
cudaEGLStreamProducerConnect
cudaEGLStreamProducerDisconnect
cudaEGLStreamProducerReturnFrame
cudaEventCreate
cudaEventCreateFromEGLSync
cudaEventCreateWithFlags
cudaEventDestroy
cudaEventQuery
cudaEventRecord
cudaEventRecord_ptsz
cudaEventSynchronize
cudaFree
cudaFreeArray
cudaFreeHost
cudaFreeMipmappedArray
cudaGLMapBufferObject
cudaGLMapBufferObjectAsync
cudaGLRegisterBufferObject
cudaGLUnmapBufferObject
cudaGLUnmapBufferObjectAsync
cudaGLUnregisterBufferObject
cudaGraphicsD3D10RegisterResource
cudaGraphicsD3D11RegisterResource
cudaGraphicsD3D9RegisterResource
cudaGraphicsEGLRegisterImage
cudaGraphicsGLRegisterBuffer

cudaGraphicsGLRegisterImage
cudaGraphicsMapResources
cudaGraphicsUnmapResources
cudaGraphicsUnregisterResource
cudaGraphicsVDPAURegisterOutputSurface
cudaGraphicsVDPAURegisterVideoSurface
cudaHostAlloc
cudaHostRegister
cudaHostUnregister
cudaLaunch
cudaLaunchCooperativeKernel
cudaLaunchCooperativeKernelMultiDevice
cudaLaunchCooperativeKernel_ptsz
cudaLaunchKernel
cudaLaunchKernel_ptsz
cudaLaunch_ptsz
cudaMalloc
cudaMalloc3D
cudaMalloc3DArray
cudaMallocArray
cudaMallocHost
cudaMallocManaged
cudaMallocMipmappedArray
cudaMallocPitch
cudaMemGetInfo
cudaMemPrefetchAsync
cudaMemPrefetchAsync_ptsz
cudaMemcpy

cudaMemcpy2D
cudaMemcpy2DArrayToArray
cudaMemcpy2DArrayToArray_ptds
cudaMemcpy2DAsync
cudaMemcpy2DAsync_ptsz
cudaMemcpy2DFromArray
cudaMemcpy2DFromArrayAsync
cudaMemcpy2DFromArrayAsync_ptsz
cudaMemcpy2DFromArray_ptds
cudaMemcpy2DToArray
cudaMemcpy2DToArrayAsync
cudaMemcpy2DToArrayAsync_ptsz
cudaMemcpy2DToArray_ptds
cudaMemcpy2D_ptds
cudaMemcpy3D
cudaMemcpy3DAsync
cudaMemcpy3DAsync_ptsz
cudaMemcpy3DPeer
cudaMemcpy3DPeerAsync
cudaMemcpy3DPeerAsync_ptsz
cudaMemcpy3DPeer_ptds
cudaMemcpy3D_ptds
cudaMemcpyArrayToArray
cudaMemcpyArrayToArray_ptds
cudaMemcpyAsync
cudaMemcpyAsync_ptsz
cudaMemcpyFromArray
cudaMemcpyFromArrayAsync

cudaMemcpyFromArrayAsync_ptsz
cudaMemcpyFromArray_ptds
cudaMemcpyFromSymbol
cudaMemcpyFromSymbolAsync
cudaMemcpyFromSymbolAsync_ptsz
cudaMemcpyFromSymbol_ptds
cudaMemcpyPeer
cudaMemcpyPeerAsync
cudaMemcpyToArray
cudaMemcpyToArrayAsync
cudaMemcpyToArrayAsync_ptsz
cudaMemcpyToArray_ptds
cudaMemcpyToSymbol
cudaMemcpyToSymbolAsync
cudaMemcpyToSymbolAsync_ptsz
cudaMemcpyToSymbol_ptds
cudaMemcpy_ptds
cudaMemset
cudaMemset2D
cudaMemset2DAsync
cudaMemset2DAsync_ptsz
cudaMemset2D_ptds
cudaMemset3D
cudaMemset3DAsync
cudaMemset3DAsync_ptsz
cudaMemset3D_ptds
cudaMemsetAsync
cudaMemsetAsync_ptsz

cudaMemset_ptds
cudaPeerRegister
cudaPeerUnregister
cudaStreamAddCallback
cudaStreamAddCallback_ptsz
cudaStreamAttachMemAsync
cudaStreamAttachMemAsync_ptsz
cudaStreamCreate
cudaStreamCreateWithFlags
cudaStreamCreateWithPriority
cudaStreamDestroy
cudaStreamQuery
cudaStreamQuery_ptsz
cudaStreamSynchronize
cudaStreamSynchronize_ptsz
cudaStreamWaitEvent
cudaStreamWaitEvent_ptsz
cudaThreadSynchronize
cudaUnbindTexture

CUDA Primary API Calls Skipped by Default

cu64Array3DCreate
cu64ArrayCreate
cu64D3D9MapVertexBuffer
cu64GLMapBufferObject
cu64GLMapBufferObjectAsync
cu64MemAlloc
cu64MemAllocPitch
cu64MemFree

cu64MemGetInfo
cu64MemHostAlloc
cu64Memcpy2D
cu64Memcpy2DAsync
cu64Memcpy2DUnaligned
cu64Memcpy3D
cu64Memcpy3DAsync
cu64MemcpyAtoD
cu64MemcpyDtoA
cu64MemcpyDtoD
cu64MemcpyDtoDAsync
cu64MemcpyDtoH
cu64MemcpyDtoHAsync
cu64MemcpyHtoD
cu64MemcpyHtoDAsync
cu64MemsetD16
cu64MemsetD16Async
cu64MemsetD2D16
cu64MemsetD2D16Async
cu64MemsetD2D32
cu64MemsetD2D32Async
cu64MemsetD2D8
cu64MemsetD2D8Async
cu64MemsetD32
cu64MemsetD32Async
cu64MemsetD8
cu64MemsetD8Async
cuArray3DCreate

cuArray3DCreate_v2
cuArrayCreate
cuArrayCreate_v2
cuArrayDestroy
cuBinaryFree
cuCompilePtx
cuCtxCreate
cuCtxCreate_v2
cuCtxDestroy
cuCtxDestroy_v2
cuCtxSynchronize
cuD3D10CtxCreate
cuD3D10CtxCreateOnDevice
cuD3D10CtxCreate_v2
cuD3D10MapResources
cuD3D10RegisterResource
cuD3D10UnmapResources
cuD3D10UnregisterResource
cuD3D11CtxCreate
cuD3D11CtxCreateOnDevice
cuD3D11CtxCreate_v2
cuD3D9CtxCreate
cuD3D9CtxCreateOnDevice
cuD3D9CtxCreate_v2
cuD3D9MapResources
cuD3D9MapVertexBuffer
cuD3D9MapVertexBuffer_v2
cuD3D9RegisterResource

cuD3D9RegisterVertexBuffer
cuD3D9UnmapResources
cuD3D9UnmapVertexBuffer
cuD3D9UnregisterResource
cuD3D9UnregisterVertexBuffer
cuEGLStreamConsumerAcquireFrame
cuEGLStreamConsumerConnect
cuEGLStreamConsumerConnectWithFlags
cuEGLStreamConsumerDisconnect
cuEGLStreamConsumerReleaseFrame
cuEGLStreamProducerConnect
cuEGLStreamProducerDisconnect
cuEGLStreamProducerPresentFrame
cuEGLStreamProducerReturnFrame
cuEventCreate
cuEventCreateFromEGLSync
cuEventCreateFromNVNSync
cuEventDestroy
cuEventDestroy_v2
cuEventQuery
cuEventRecord
cuEventRecord_ptsz
cuEventSynchronize
cuGLCtxCreate
cuGLCtxCreate_v2
cuGLInit
cuGLMapBufferObject
cuGLMapBufferObjectAsync

cuGLMapBufferObjectAsync_v2
cuGLMapBufferObjectAsync_v2_ptsz
cuGLMapBufferObject_v2
cuGLMapBufferObject_v2_ptds
cuGLRegisterBufferObject
cuGLUnmapBufferObject
cuGLUnmapBufferObjectAsync
cuGLUnregisterBufferObject
cuGraphicsD3D10RegisterResource
cuGraphicsD3D11RegisterResource
cuGraphicsD3D9RegisterResource
cuGraphicsEGLRegisterImage
cuGraphicsGLRegisterBuffer
cuGraphicsGLRegisterImage
cuGraphicsMapResources
cuGraphicsMapResources_ptsz
cuGraphicsUnmapResources
cuGraphicsUnmapResources_ptsz
cuGraphicsUnregisterResource
cuGraphicsVDPAURegisterOutputSurface
cuGraphicsVDPAURegisterVideoSurface
cuInit
cuLaunch
cuLaunchCooperativeKernel
cuLaunchCooperativeKernelMultiDevice
cuLaunchCooperativeKernel_ptsz
cuLaunchGrid
cuLaunchGridAsync

cuLaunchKernel
cuLaunchKernel_ptsz
cuLinkComplete
cuLinkCreate
cuLinkCreate_v2
cuLinkDestroy
cuMemAlloc
cuMemAllocHost
cuMemAllocHost_v2
cuMemAllocManaged
cuMemAllocPitch
cuMemAllocPitch_v2
cuMemAlloc_v2
cuMemFree
cuMemFreeHost
cuMemFree_v2
cuMemGetInfo
cuMemGetInfo_v2
cuMemHostAlloc
cuMemHostAlloc_v2
cuMemHostRegister
cuMemHostRegister_v2
cuMemHostUnregister
cuMemPeerRegister
cuMemPeerUnregister
cuMemPrefetchAsync
cuMemPrefetchAsync_ptsz
cuMemcpy

cuMemcpy2D
cuMemcpy2DAsync
cuMemcpy2DAsync_v2
cuMemcpy2DAsync_v2_ptsz
cuMemcpy2DUnaligned
cuMemcpy2DUnaligned_v2
cuMemcpy2DUnaligned_v2_ptds
cuMemcpy2D_v2
cuMemcpy2D_v2_ptds
cuMemcpy3D
cuMemcpy3DAsync
cuMemcpy3DAsync_v2
cuMemcpy3DAsync_v2_ptsz
cuMemcpy3DPeer
cuMemcpy3DPeerAsync
cuMemcpy3DPeerAsync_ptsz
cuMemcpy3DPeer_ptds
cuMemcpy3D_v2
cuMemcpy3D_v2_ptds
cuMemcpyAsync
cuMemcpyAsync_ptsz
cuMemcpyAtoA
cuMemcpyAtoA_v2
cuMemcpyAtoA_v2_ptds
cuMemcpyAtoD
cuMemcpyAtoD_v2
cuMemcpyAtoD_v2_ptds
cuMemcpyAtoH

cuMemcpyAtoHAsync
cuMemcpyAtoHAsync_v2
cuMemcpyAtoHAsync_v2_ptsz
cuMemcpyAtoH_v2
cuMemcpyAtoH_v2_ptds
cuMemcpyDtoA
cuMemcpyDtoA_v2
cuMemcpyDtoA_v2_ptds
cuMemcpyDtoD
cuMemcpyDtoDAsync
cuMemcpyDtoDAsync_v2
cuMemcpyDtoDAsync_v2_ptsz
cuMemcpyDtoD_v2
cuMemcpyDtoD_v2_ptds
cuMemcpyDtoH
cuMemcpyDtoHAsync
cuMemcpyDtoHAsync_v2
cuMemcpyDtoHAsync_v2_ptsz
cuMemcpyDtoH_v2
cuMemcpyDtoH_v2_ptds
cuMemcpyHtoA
cuMemcpyHtoAAsync
cuMemcpyHtoAAsync_v2
cuMemcpyHtoAAsync_v2_ptsz
cuMemcpyHtoA_v2
cuMemcpyHtoA_v2_ptds
cuMemcpyHtoD
cuMemcpyHtoDAsync

cuMemcpyHtoDAsync_v2
cuMemcpyHtoDAsync_v2_ptsz
cuMemcpyHtoD_v2
cuMemcpyHtoD_v2_ptds
cuMemcpyPeer
cuMemcpyPeerAsync
cuMemcpyPeerAsync_ptsz
cuMemcpyPeer_ptds
cuMemcpy_ptds
cuMemcpy_v2
cuMemsetD16
cuMemsetD16Async
cuMemsetD16Async_ptsz
cuMemsetD16_v2
cuMemsetD16_v2_ptds
cuMemsetD2D16
cuMemsetD2D16Async
cuMemsetD2D16Async_ptsz
cuMemsetD2D16_v2
cuMemsetD2D16_v2_ptds
cuMemsetD2D32
cuMemsetD2D32Async
cuMemsetD2D32Async_ptsz
cuMemsetD2D32_v2
cuMemsetD2D32_v2_ptds
cuMemsetD2D8
cuMemsetD2D8Async
cuMemsetD2D8Async_ptsz

cuMemsetD2D8_v2
cuMemsetD2D8_v2_ptds
cuMemsetD32
cuMemsetD32Async
cuMemsetD32Async_ptsz
cuMemsetD32_v2
cuMemsetD32_v2_ptds
cuMemsetD8
cuMemsetD8Async
cuMemsetD8Async_ptsz
cuMemsetD8_v2
cuMemsetD8_v2_ptds
cuMipmappedArrayCreate
cuMipmappedArrayDestroy
cuModuleLoad
cuModuleLoadData
cuModuleLoadDataEx
cuModuleLoadFatBinary
cuModuleUnload
cuStreamAddCallback
cuStreamAddCallback_ptsz
cuStreamAttachMemAsync
cuStreamAttachMemAsync_ptsz
cuStreamBatchMemOp
cuStreamBatchMemOp_ptsz
cuStreamCreate
cuStreamCreateWithPriority
cuStreamDestroy

cuStreamDestroy_v2
cuStreamSynchronize
cuStreamSynchronize_ptsz
cuStreamWaitEvent
cuStreamWaitEvent_ptsz
cuStreamWaitValue32
cuStreamWaitValue32_ptsz
cuStreamWaitValue64
cuStreamWaitValue64_ptsz
cuStreamWriteValue32
cuStreamWriteValue32_ptsz
cuStreamWriteValue64
cuStreamWriteValue64_ptsz
cuSurfObjectCreate
cuSurfObjectDestroy
cuSurfRefCreate
cuSurfRefDestroy
cuTexObjectCreate
cuTexObjectDestroy
cuTexRefCreate
cuTexRefDestroy
cuVDPAUCtxCreate
cuVDPAUCtxCreate_v2

cuDNN Function List for X86 CLI#

cuDNN API functions

cudaNNActivationBackward
cudaNNActivationBackward_v3

cudaActivationBackward_v4
cudaActivationForward
cudaActivationForward_v3
cudaActivationForward_v4
cudaAddTensor
cudaBatchNormalizationBackward
cudaBatchNormalizationBackwardEx
cudaBatchNormalizationForwardInference
cudaBatchNormalizationForwardTraining
cudaBatchNormalizationForwardTrainingEx
cudaCTCLoss
cudaConvolutionBackwardBias
cudaConvolutionBackwardData
cudaConvolutionBackwardFilter
cudaConvolutionBiasActivationForward
cudaConvolutionForward
cudaCreate
cudaCreateAlgorithmPerformance
cudaDestroy
cudaDestroyAlgorithmPerformance
cudaDestroyPersistentRNNPlan
cudaDivisiveNormalizationBackward
cudaDivisiveNormalizationForward
cudaDropoutBackward
cudaDropoutForward
cudaDropoutGetReserveSpaceSize
cudaDropoutGetStatesSize
cudaFindConvolutionBackwardDataAlgorithm

cudaFindConvolutionBackwardDataAlgorithmEx
cudaFindConvolutionBackwardFilterAlgorithm
cudaFindConvolutionBackwardFilterAlgorithmEx
cudaFindConvolutionForwardAlgorithm
cudaFindConvolutionForwardAlgorithmEx
cudaFindRNNBackwardDataAlgorithmEx
cudaFindRNNBackwardWeightsAlgorithmEx
cudaFindRNNForwardInferenceAlgorithmEx
cudaFindRNNForwardTrainingAlgorithmEx
cudaFusedOpsExecute
cudaIm2Col
cudaLRNCrossChannelBackward
cudaLRNCrossChannelForward
cudaMakeFusedOpsPlan
cudaMultiHeadAttnBackwardData
cudaMultiHeadAttnBackwardWeights
cudaMultiHeadAttnForward
cudaOpTensor
cudaPoolingBackward
cudaPoolingForward
cudaRNNBackwardData
cudaRNNBackwardDataEx
cudaRNNBackwardWeights
cudaRNNBackwardWeightsEx
cudaRNNForwardInference
cudaRNNForwardInferenceEx
cudaRNNForwardTraining
cudaRNNForwardTrainingEx

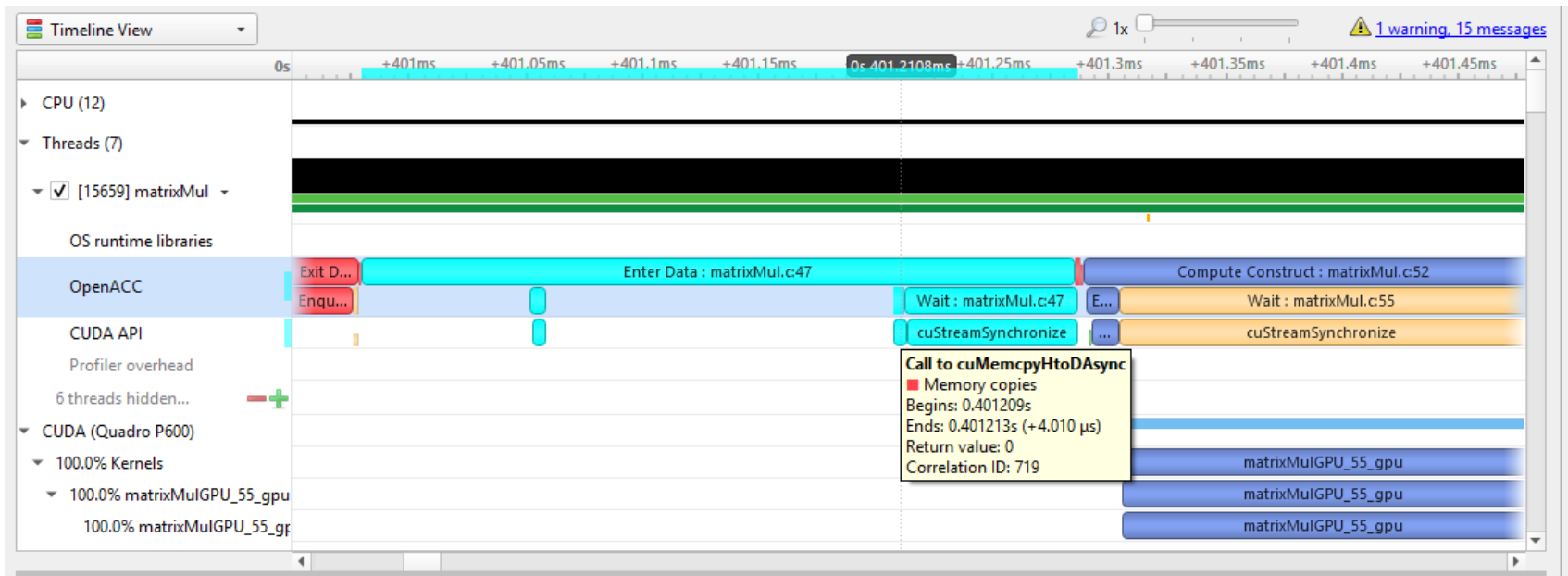
cudaReduceTensor
cudaReorderFilterAndBias
cudaRestoreAlgorithm
cudaRestoreDropoutDescriptor
cudaSaveAlgorithm
cudaScaleTensor
cudaSoftmaxBackward
cudaSoftmaxForward
cudaSpatialTfGridGeneratorBackward
cudaSpatialTfGridGeneratorForward
cudaSpatialTfSamplerBackward
cudaSpatialTfSamplerForward
cudaTransformFilter
cudaTransformTensor
cudaTransformTensorEx

OpenACC Trace#

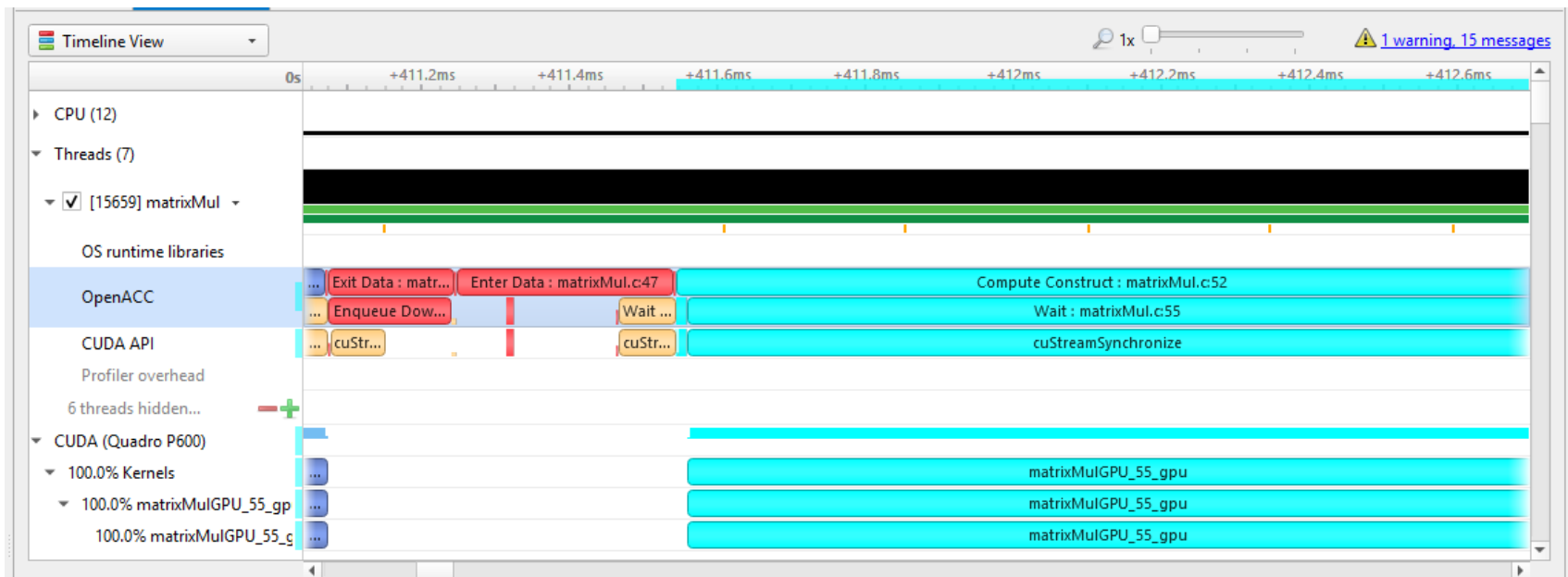
Nsight Systems for Linux x86_64 is capable of capturing information about OpenACC execution in the profiled process.

OpenACC versions 2.0, 2.5, and 2.6 are supported when using PGI runtime version 15.7 or later. In order to differentiate constructs (see tooltip below), a PGI runtime of 16.0 or later is required. Note that Nsight Systems does not support the GCC implementation of OpenACC at this time.

Under the CPU rows in the timeline tree, each thread that uses OpenACC will show OpenACC trace information. You can click on an OpenACC API call to see correlation with the underlying CUDA API calls (highlighted in teal):



If the OpenACC API results in GPU work, that will also be highlighted:

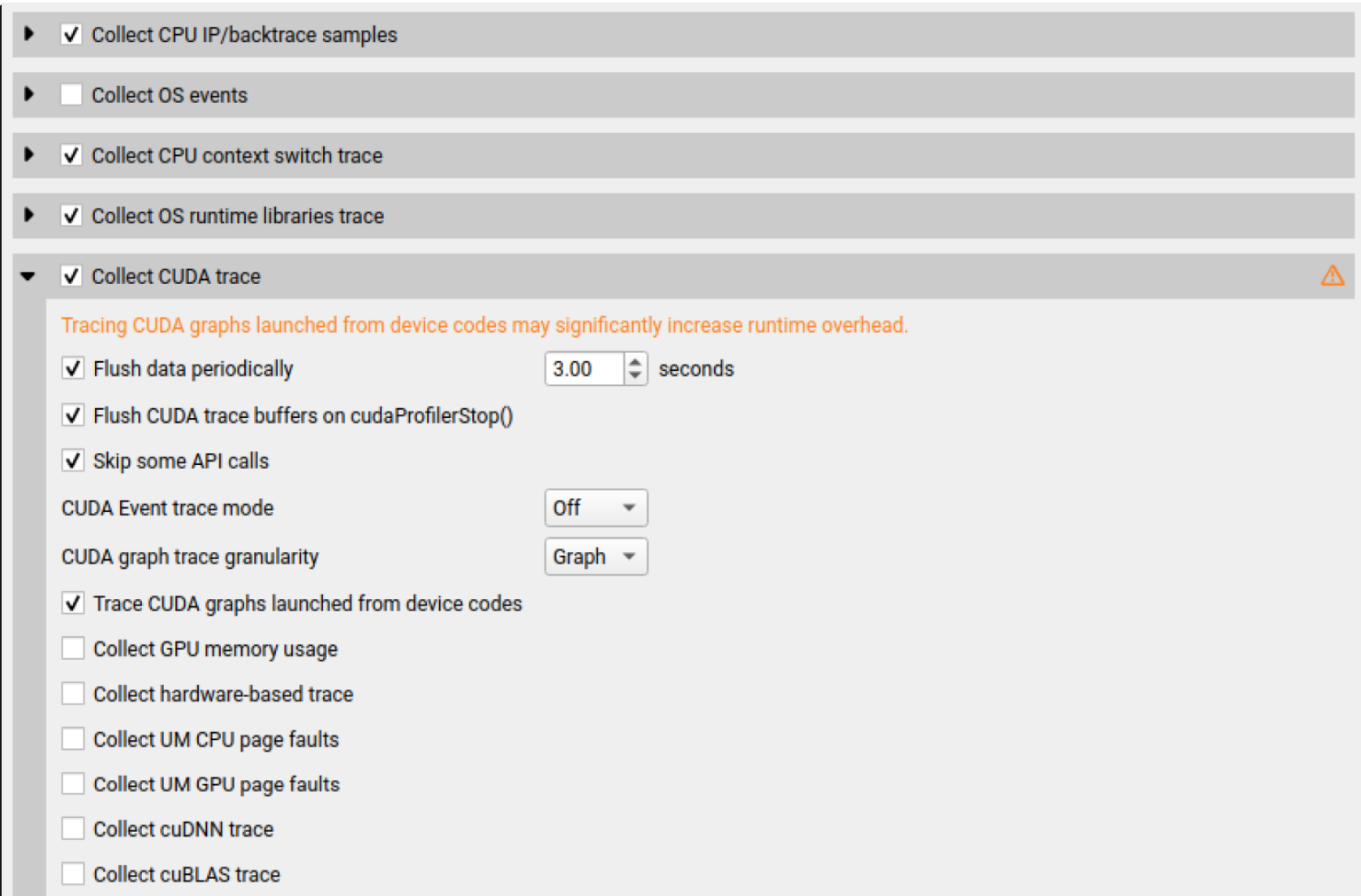


Hovering over a particular OpenACC construct will bring up a tooltip with details about that construct:

Enter Data : openacc_app.cpp:29

```
Timings: [0.355s 0.374s] = 18.626 ms
Construct Kind: Data Construct
Async: -1
Async Map: 16
Source File: openacc_app.cpp
Func Name: openaccKernel(int, float, float*, float*)
Variable Name: <Unknown>
```

To capture OpenACC information from the Nsight Systems GUI, select the **Collect OpenACC trace** checkbox under **Collect CUDA trace** configurations. Note that turning on OpenACC tracing will also turn on CUDA tracing.



Tracing CUDA graphs launched from device codes may significantly increase runtime overhead.

- ☒ Collect CPU IP/backtrace samples
- ☐ Collect OS events
- ☒ Collect CPU context switch trace
- ☒ Collect OS runtime libraries trace
- ☒ Collect CUDA trace 

☒ Flush data periodically 3.00 seconds

☒ Flush CUDA trace buffers on cudaProfilerStop()

☒ Skip some API calls

CUDA Event trace mode Off

CUDA graph trace granularity Graph

- ☒ Trace CUDA graphs launched from device codes
- ☐ Collect GPU memory usage
- ☐ Collect hardware-based trace
- ☐ Collect UM CPU page faults
- ☐ Collect UM GPU page faults
- ☐ Collect cuDNN trace
- ☐ Collect cuBLAS trace

The image shows a configuration window for profiling. It contains a list of checkboxes and expandable sections. The 'Collect NVTX trace' checkbox is checked, while all others are unchecked. The expandable sections for 'Network profiling options' and 'Python profiling options' are visible at the bottom.

☐	Collect OpenACC trace
▶ ☐	Collect CUDA backtraces
▶ ☐	Collect OpenMP trace
▶ ☐	Collect GPU context switch trace
▶ ☐	Collect GPU Metrics
▶ ☐	Collect NV Video trace
▶ ☒	Collect NVTX trace
▶ ☐	Collect OpenGL trace
▶ ☐	Collect Vulkan trace
▶	Network profiling options
▶	Python profiling options

Note

If your application crashes before all collected OpenACC trace data has been copied out, some or all data might be lost and not present in the report.

OpenGL Trace#

OpenGL and OpenGL ES APIs can be traced to assist in the analysis of CPU and GPU interactions.

A few usage examples are:

1. Visualize how long `eglSwapBuffers` (or similar) is taking.
2. API trace can easily show correlations between thread state and graphics driver's behavior, uncovering where the

CPU may be waiting on the GPU.

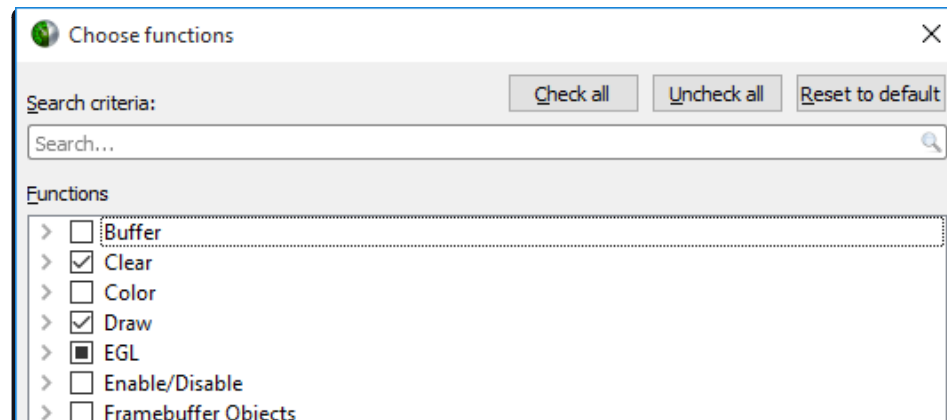
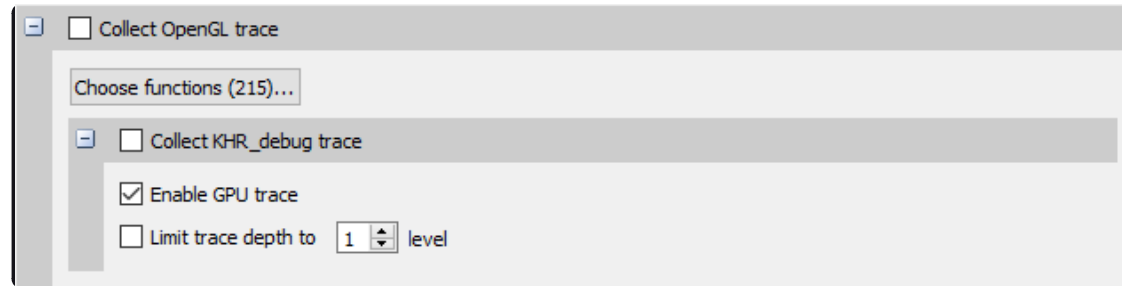
3. Spot bubbles of opportunity on the GPU, where more GPU workload could be created.

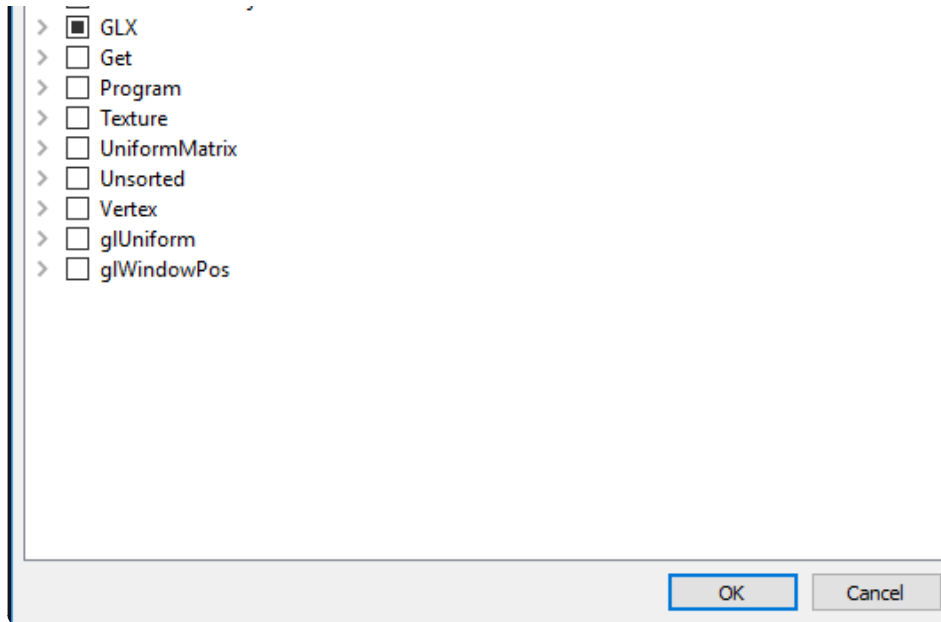
4. Use KHR_debug extension to trace GL events on both the CPU and GPU.

OpenGL trace feature in Nsight Systems consists of two different activities which will be shown in the CPU rows for those threads

- **CPU trace**: interception of API calls that an application does to APIs (such as OpenGL, OpenGL ES, EGL, GLX, WGL, etc.).
- **GPU trace** (or **workload trace**): trace of GPU workload (activity) triggered by use of OpenGL or OpenGL ES. Since draw calls are executed back-to-back, the GPU workload trace ranges include many OpenGL draw calls and operations in order to optimize performance overhead, rather than tracing each individual operation.

To collect GPU trace, the `glQueryCounter()` function is used to measure how much time batches of GPU workload take to complete.





Ranges defined by the KHR_debug calls are represented similarly to OpenGL API and OpenGL GPU workload trace. GPU ranges in this case represent *incremental draw cost*. They cannot fully account for GPUs that can execute multiple draw calls in parallel. In this case, Nsight Systems will not show overlapping GPU ranges.

OpenGL Trace Using Command Line#

For general information on using the target CLI, see [CLI Profiling on Linux](#). For the CLI, the functions that are traced are set to the following list:

glWaitSync
glReadPixels
glReadnPixelsKHR
glReadnPixelsEXT
glReadnPixelsARB
glReadnPixels
glFlush
glFinishFenceNV

glFinish
glClientWaitSync
glClearTexSubImage
glClearTexImage
glClearStencil
glClearNamedFramebufferiv
glClearNamedFramebufferiv
glClearNamedFramebufferfv
glClearNamedFramebufferfi
glClearNamedBufferSubDataEXT
glClearNamedBufferSubData
glClearNamedBufferDataEXT
glClearNamedBufferData
glClearIndex
glClearDepthx
glClearDepthf
glClearDepthdNV
glClearDepth
glClearColorx
glClearColorIuiEXT
glClearColorIiEXT
glClearColor
glClearBufferiv
glClearBufferSubData
glClearBufferiv
glClearBufferfv
glClearBufferfi
glClearBufferData

glClearAccum
glClear
glDispatchComputeIndirect
glDispatchComputeGroupSizeARB
glDispatchCompute
glComputeStreamNV
glNamedFramebufferDrawBuffers
glNamedFramebufferDrawBuffer
glMultiDrawElementsIndirectEXT
glMultiDrawElementsIndirectCountARB
glMultiDrawElementsIndirectBindlessNV
glMultiDrawElementsIndirectBindlessCountNV
glMultiDrawElementsIndirectAMD
glMultiDrawElementsIndirect
glMultiDrawElementsEXT
glMultiDrawElementsBaseVertex
glMultiDrawElements
glMultiDrawArraysIndirectEXT
glMultiDrawArraysIndirectCountARB
glMultiDrawArraysIndirectBindlessNV
glMultiDrawArraysIndirectBindlessCountNV
glMultiDrawArraysIndirectAMD
glMultiDrawArraysIndirect
glMultiDrawArraysEXT
glMultiDrawArrays
glListDrawCommandsStatesClientNV
glFramebufferDrawBuffersEXT
glFramebufferDrawBufferEXT

glDrawTransformFeedbackStreamInstanced
glDrawTransformFeedbackStream
glDrawTransformFeedbackNV
glDrawTransformFeedbackInstancedEXT
glDrawTransformFeedbackInstanced
glDrawTransformFeedbackEXT
glDrawTransformFeedback
glDrawTexxvOES
glDrawTexxOES
glDrawTextureNV
glDrawTexsvOES
glDrawTexsOES
glDrawTexivOES
glDrawTexiOES
glDrawTexfvOES
glDrawTexfOES
glDrawRangeElementsEXT
glDrawRangeElementsBaseVertexOES
glDrawRangeElementsBaseVertexEXT
glDrawRangeElementsBaseVertex
glDrawRangeElements
glDrawPixels
glDrawElementsInstancedNV
glDrawElementsInstancedEXT
glDrawElementsInstancedBaseVertexOES
glDrawElementsInstancedBaseVertexEXT
glDrawElementsInstancedBaseVertexBaseInstanceEXT
glDrawElementsInstancedBaseVertexBaseInstance

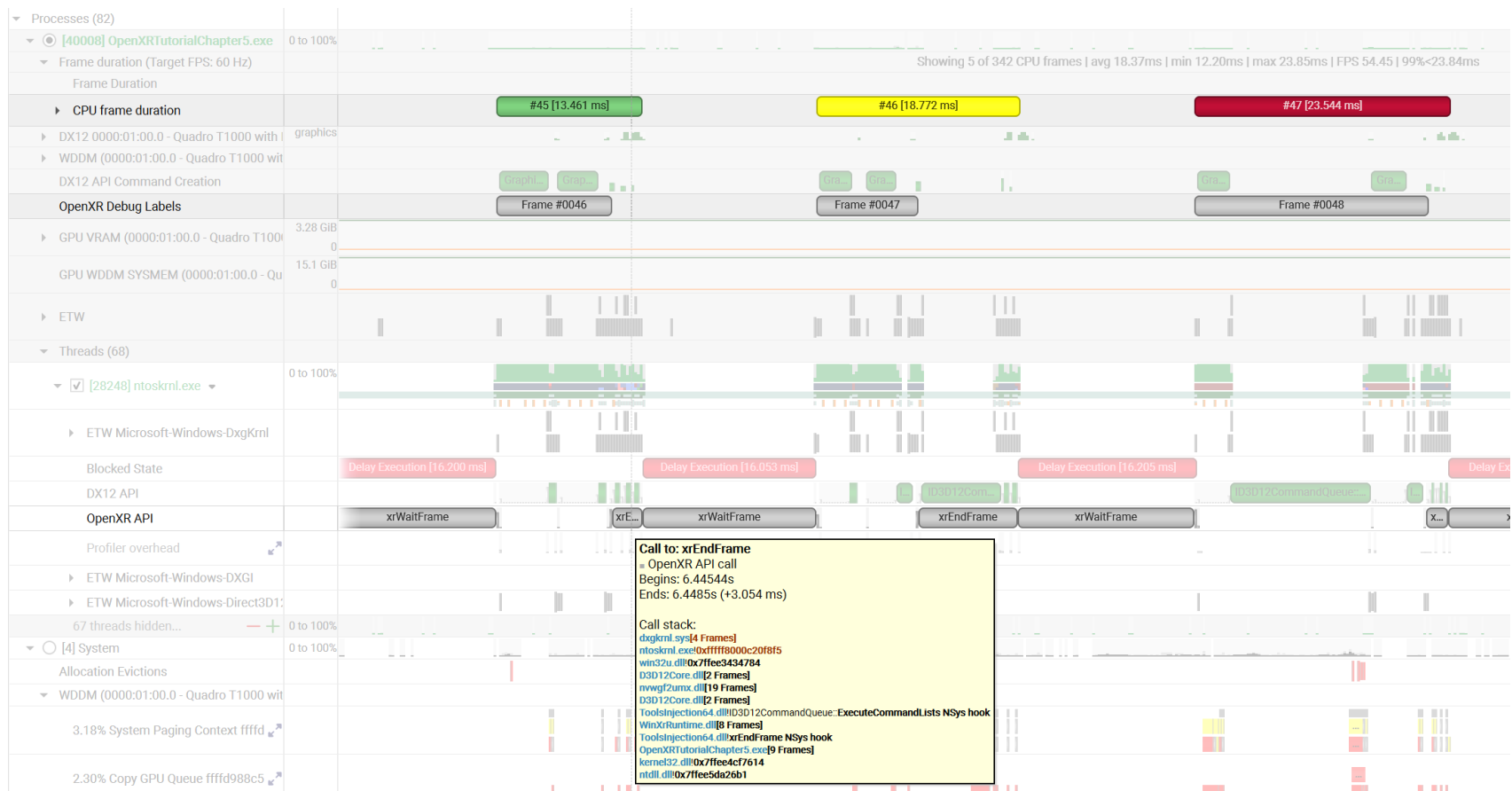
glDrawElementsInstancedBaseVertex
glDrawElementsInstancedBaseInstanceEXT
glDrawElementsInstancedBaseInstance
glDrawElementsInstancedARB
glDrawElementsInstanced
glDrawElementsIndirect
glDrawElementsBaseVertexOES
glDrawElementsBaseVertexEXT
glDrawElementsBaseVertex
glDrawElements
glDrawCommandsStatesNV
glDrawCommandsStatesAddressNV
glDrawCommandsNV
glDrawCommandsAddressNV
glDrawBuffersNV
glDrawBuffersATI
glDrawBuffersARB
glDrawBuffers
glDrawBuffer
glDrawArraysInstancedNV
glDrawArraysInstancedEXT
glDrawArraysInstancedBaseInstanceEXT
glDrawArraysInstancedBaseInstance
glDrawArraysInstancedARB
glDrawArraysInstanced
glDrawArraysIndirect
glDrawArraysEXT
glDrawArrays

eglSwapBuffersWithDamageKHR
eglSwapBuffers
glXSwapBuffers
glXQueryDrawable
glXGetCurrentReadDrawable
glXGetCurrentDrawable
glGetQueryObjectivEXT
glGetQueryObjectivARB
glGetQueryObjectiv
glGetQueryObjectivARB
glGetQueryObjectiv

OpenXR API Trace#

OpenXR is a royalty-free, open standard that provides high-performance access to Augmented Reality (AR) and Virtual Reality (VR)—collectively known as XR—platforms and devices. Information about OpenXR can be found at the [OpenXR Overview](#).

Nsight Systems can capture information about OpenXR usage by the profiled process. This includes capturing the execution time of OpenXR API functions, debug labels, and frame durations. OpenXR profiling is supported on Windows operating systems.



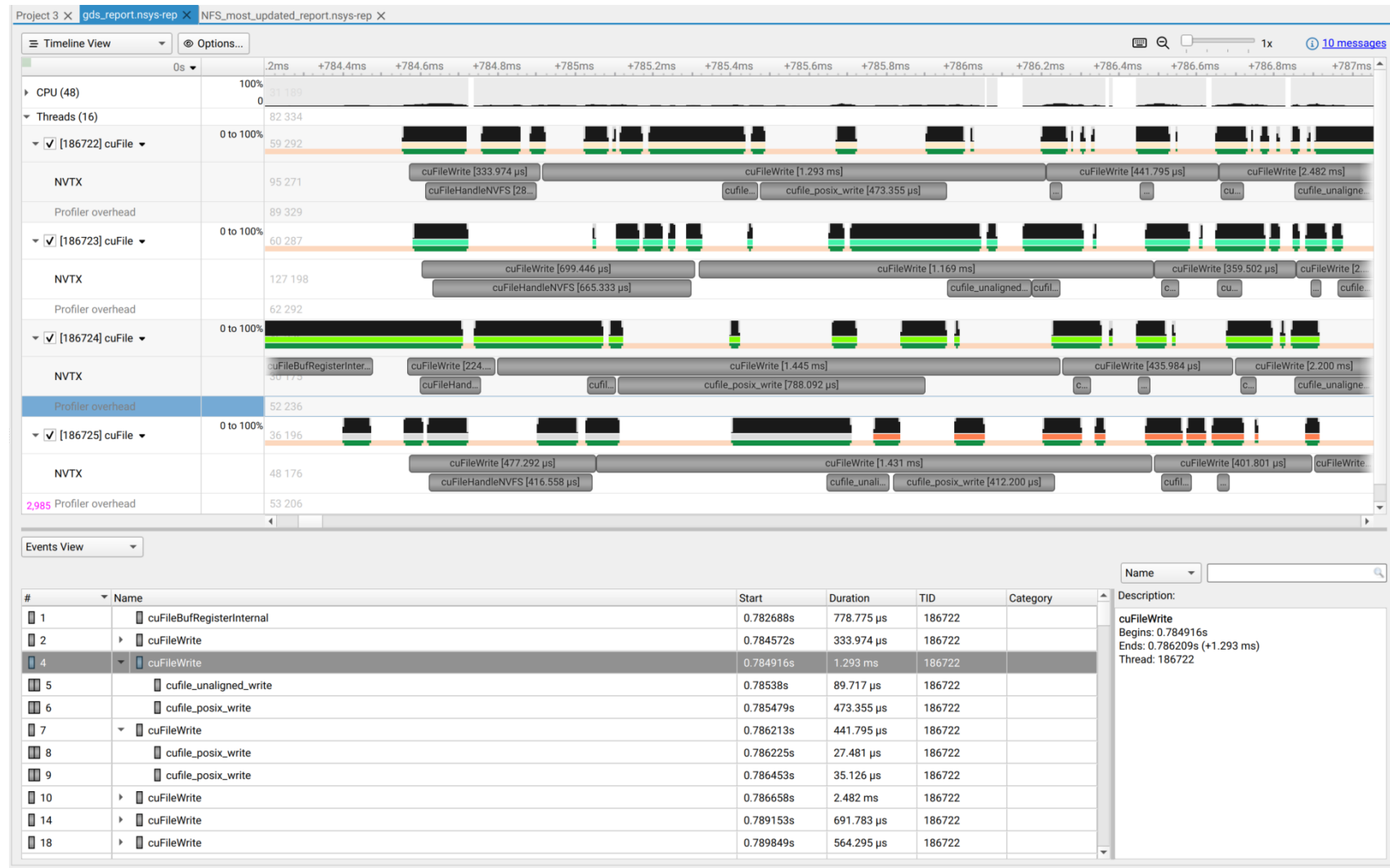
GPUDirect Storage Trace (Beta)#

NVIDIA GPUDirect Storage (GDS) enables direct memory access (DMA) between storage and GPU memory. This avoids a bounce buffer through the CPU, increasing storage access bandwidth and decreasing latency and utilization load on the CPU. Information about GDS can be found at [NVIDIA Magnum IO GPUDirect Storage](#).

Nsight Systems can capture information about GDS, specifically the various cuFile API calls made by the profiled process. GDS profiling is supported on Linux x64 and SBSA operating systems.

You can validate that GDS is installed correctly by running the gdscheck.py tool: `/usr/local/cuda/gds/tools/gdscheck.py -p` It should report that the target filesystem type is supported and that platform verification

succeeded.



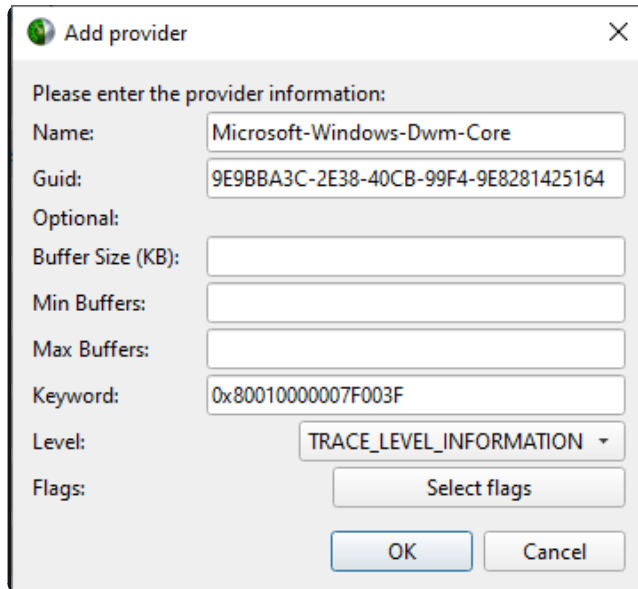
Note

Nsight Systems does not support GPUDirect Storage metrics collection on Ext4 and XFS file-systems with NVMe drives when using the “NVME P2PDMA” feature added in GPUDirect Storage v1.13.

Custom ETW Trace#

Use the custom ETW trace feature to enable and collect any manifest-based ETW log. The collected events are displayed on the timeline on dedicated rows for each event type.

Custom ETW is available on Windows target machines.



A dialog box titled "Add provider" with a close button (X) in the top right corner. The text "Please enter the provider information:" is displayed. Below this, there are several input fields and a dropdown menu. The "Name" field contains "Microsoft-Windows-Dwm-Core". The "Guid" field contains "9E9BBA3C-2E38-40CB-99F4-9E8281425164". The "Optional:" section has three empty input fields for "Buffer Size (KB)", "Min Buffers", and "Max Buffers". The "Keyword" field contains "0x80010000007F003F". The "Level" dropdown menu is set to "TRACE_LEVEL_INFORMATION". The "Flags" field has a "Select flags" button. At the bottom are "OK" and "Cancel" buttons.

Please enter the provider information:

Name: Microsoft-Windows-Dwm-Core

Guid: 9E9BBA3C-2E38-40CB-99F4-9E8281425164

Optional:

Buffer Size (KB):

Min Buffers:

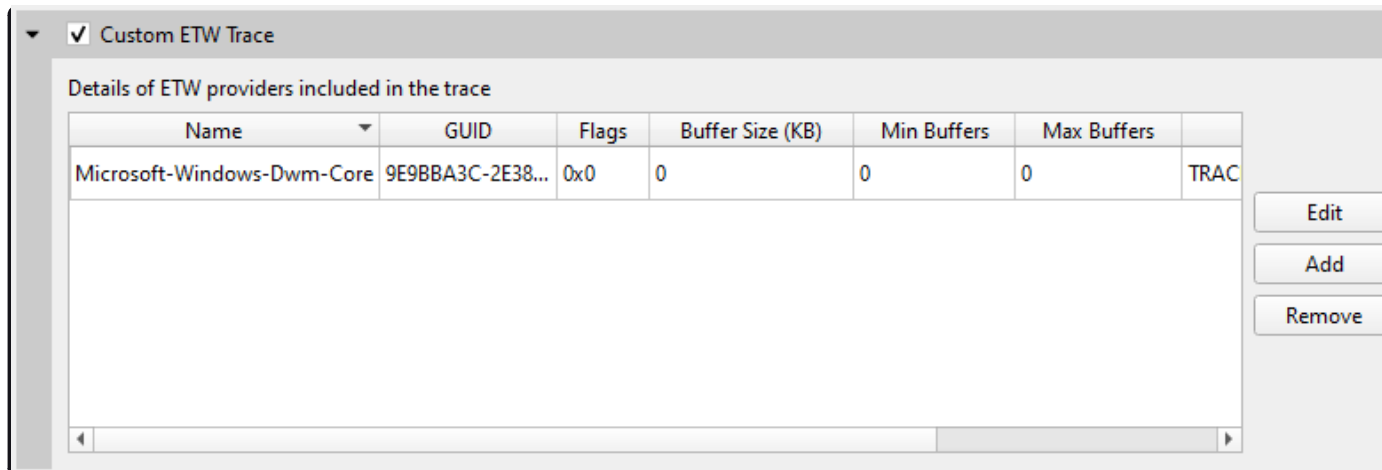
Max Buffers:

Keyword: 0x80010000007F003F

Level: TRACE_LEVEL_INFORMATION

Flags: Select flags

OK Cancel



A window titled "Custom ETW Trace" with a checked checkbox. Below the title bar is a section titled "Details of ETW providers included in the trace". It contains a table with 7 columns: Name, GUID, Flags, Buffer Size (KB), Min Buffers, Max Buffers, and a partially visible column. The first row of the table contains the following data: Name: Microsoft-Windows-Dwm-Core, GUID: 9E9BBA3C-2E38..., Flags: 0x0, Buffer Size (KB): 0, Min Buffers: 0, Max Buffers: 0, and the partially visible column contains "TRAC". To the right of the table are three buttons: "Edit", "Add", and "Remove".

Custom ETW Trace

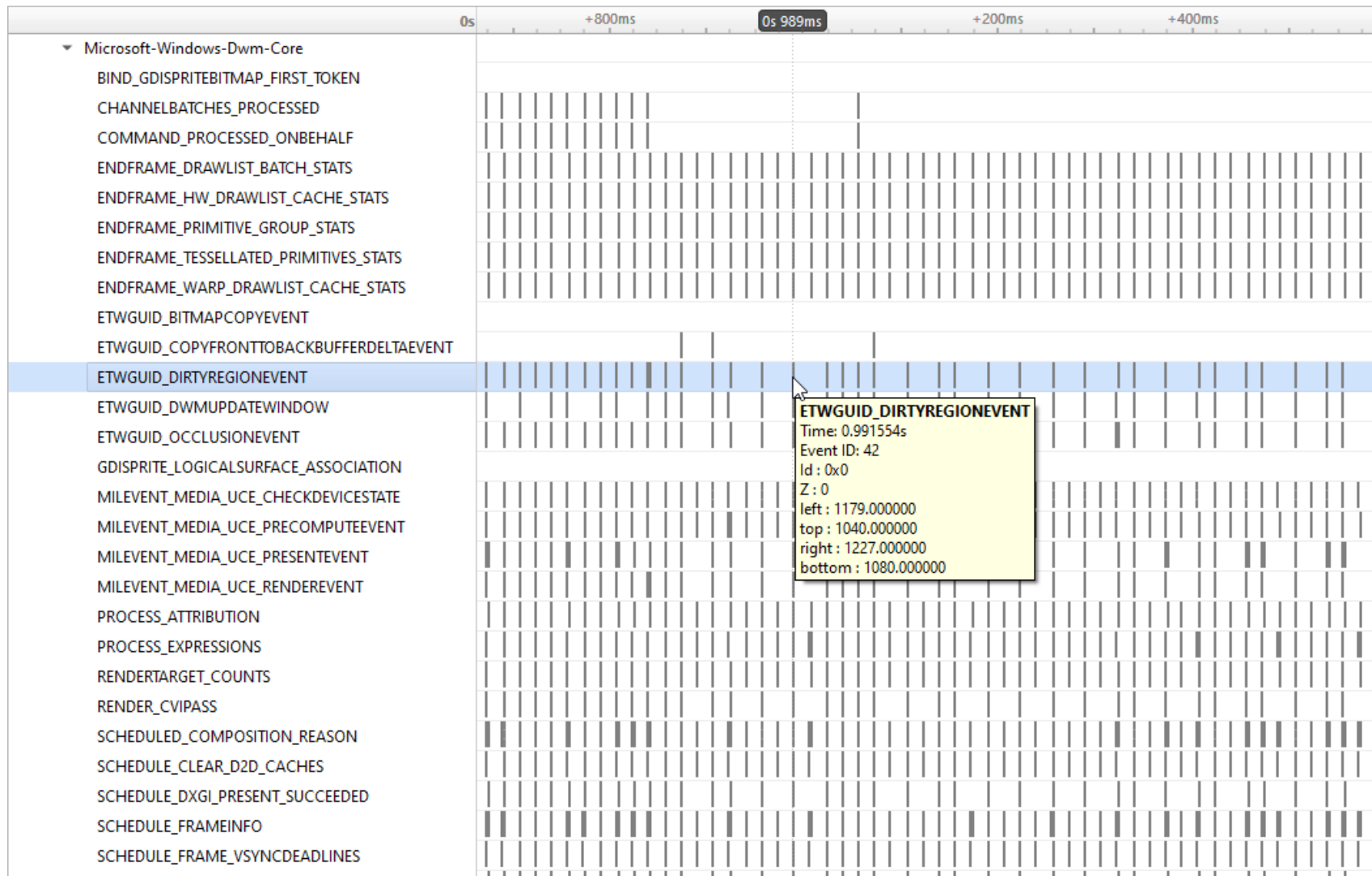
Details of ETW providers included in the trace

Name	GUID	Flags	Buffer Size (KB)	Min Buffers	Max Buffers	
Microsoft-Windows-Dwm-Core	9E9BBA3C-2E38...	0x0	0	0	0	TRAC

Edit

Add

Remove



To retain the .etl trace files captured, so that they can be viewed in other tools (e.g., GPUView), change the **Save ETW log files in project folder** option under **Profile Behavior** in Nsight Systems's global Options dialog. The .etl files will appear in the same folder as the .nsys-rep file, accessible by right-clicking the report in the Project Explorer and choosing **Show in Folder...**. Data collected from each ETW provider will appear in its own .etl file, and an additional .etl file named Report XX-Merged-*.etl, containing the events from all captured sources, will be created as well.

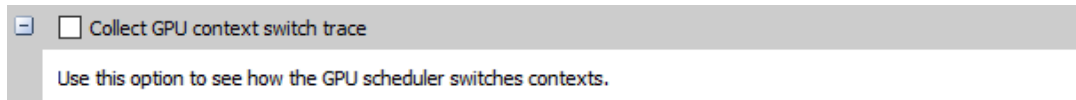
GPU Hardware Profiling[#]

GPU Context Switch[#]

Nsight Systems provides the ability to trace GPU context switches.

To enable trace, run from the CLI using the `--gpuctxsw` option

From the GUI:

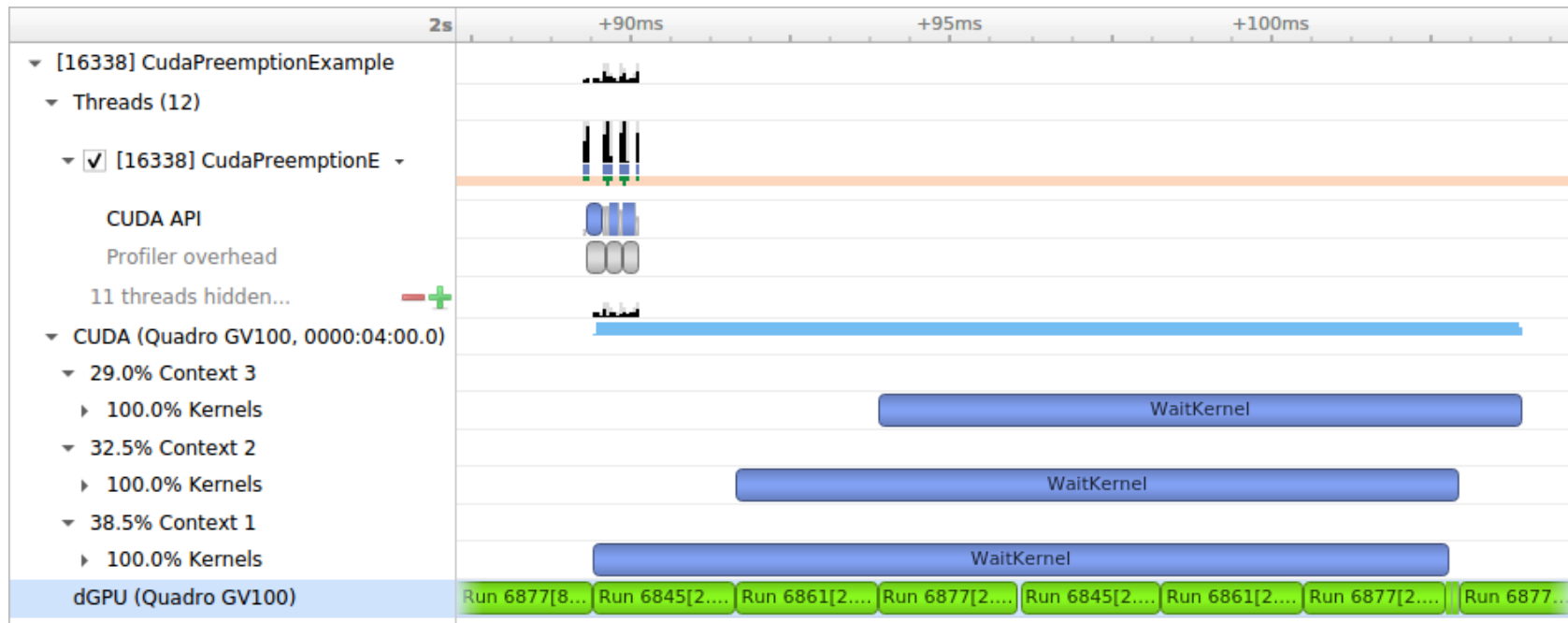


Specifically, the behavior is as follows:

When collecting GPU context switch data as root, you will get records about contexts from all processes. The records have valid context IDs and process IDs, and have full-precision timestamps.

When collecting GPU context switch data as a normal user, you will still get records about contexts from all processes. For processes running as your user, the records have valid context ID and process IDs, and full-precision timestamps. For processes running as a different user, the records have context ID = 0 and process ID = 0, and reduced-precision timestamps (which are still guaranteed to be in the correct order).

When collecting GPU context switch data in a virtual machine using vGPU, the above rules apply to records relating to your VM. No records are collected for contexts running on other VMs, so the timeline may show gaps when the vGPU is switched to another VM's context(s). We do not currently support collecting GPU context switch data on a host system where vGPUs are in use by VMs.



GPU Metrics#

Overview#

GPU Metrics feature is intended to identify performance limiters in applications using GPU for computations and graphics. It uses periodic sampling to gather performance metrics and detailed timing statistics associated with different GPU hardware units taking advantage of specialized hardware to capture this data in a single pass with minimal overhead.

Note

GPU Metrics will give you precise device level information, but it does not know which process or context is involved. GPU context switch trace provides less precise information, but will give you process and context information.



These metrics provide an overview of GPU efficiency over time within compute, graphics, and input/output (IO) activities such as:

- **IO throughputs:** PCIe, NVLink, and GPU memory bandwidth
- **SM utilization:** SMs activity, tensor core activity, instructions issued, warp occupancy, and unassigned warp slots

It is designed to help users answer the common questions:

- Is my GPU idle?
- Is my GPU full? Enough kernel grids size and streams? Are my SMs and warp slots full?
- Am I using TensorCores?
- Is my instruction rate high?
- Am I possibly blocked on IO, or number of warps, etc.?

Nsight Systems GPU Metrics is only available for Linux targets on x86-64 and aarch64, and for Windows targets. It requires NVIDIA Turing architecture or newer.

Minimum required driver versions:

- NVIDIA Turing architecture TU10x, TU11x - r440

- NVIDIA Ampere architecture GA100 - r450
- NVIDIA Ampere architecture GA100 MIG - r470 TRD1
- NVIDIA Ampere architecture GA10x - r455

Note

Permissions: Elevated permissions are required. On Linux use sudo to elevate privileges. On Windows the user must run from an admin command prompt or accept the UAC escalation dialog. See [Permissions Issues and Performance Counters](#) for more information.

Note

Tensor Core: If you run `nsys profile --gpu-metrics-devices all`, the Tensor Core utilization can be found in the GUI under the **SM instructions/Tensor Active** row.

Note that it is not practical to expect a CUDA kernel to reach 100% Tensor Core utilization since there are other overheads. In general, the more computation-intensive an operation is, the higher Tensor Core utilization rate the CUDA kernel can achieve.

Launching GPU Metrics from the CLI#

GPU Metrics feature is controlled with 3 CLI switches:

- `--gpu-metrics-devices=[all, cuda-visible, none, <index>]` selects GPUs to sample (default is none).
- `--gpu-metrics-set=[<alias>, file:<file name>]` selects metric set to use (default is the 1st suitable from the list).
- `--gpu-metrics-frequency=[10..200000]` selects sampling frequency in Hz (default is 10000).

To profile with default options and sample GPU Metrics on GPU 1:

Must have elevated permissions (see https://developer.nvidia.com/ERR_NVGPUCTRPERM) or be root (Linux) or

Administrator (Windows)

```
$ nsys profile --gpu-metrics-devices=1 ./my-app
```

To list available GPUs, use:

```
$ nsys profile --gpu-metrics-devices=help
```

Possible --gpu-metrics-devices values are:

- 1: Turing TU104 | GeForce RTX 2070 SUPER PCI[0000:65:00.0]

- all: Select all supported GPUs

- cuda-visible: Select GPUs that match CUDA_VISIBLE_DEVICES

- none: Disable GPU Metrics [Default]

Some GPUs are not supported:

- 0: Volta GV100 | Quadro GV100 PCI[0000:17:00.0]

See the user guide: <https://docs.nvidia.com/nsight-systems/UserGuide/index.html#gpu-metrics>

By default, the first **metric set** which supports all selected GPUs is used. You can manually select another metric set from the list. To see metrics sets available for the selected GPUs, use:

```
$ nsys profile --gpu-metrics-devices=all --gpu-metrics-set=help
```

Possible --gpu-metrics-set values are:

- tu10x : General Metrics for NVIDIA TU10x (any frequency)

- tu10x-gfxt : Graphics Throughput Metrics for NVIDIA TU10x (frequency >= 10kHz)

- file:<file name> : use metric set from a given file

By default, **sampling frequency** is set to 10 kHz. But you can manually set it from 10 Hz to 200 kHz using the following:

```
--gpu-metrics-frequency=<value>
```

Launching GPU Metrics from the GUI#

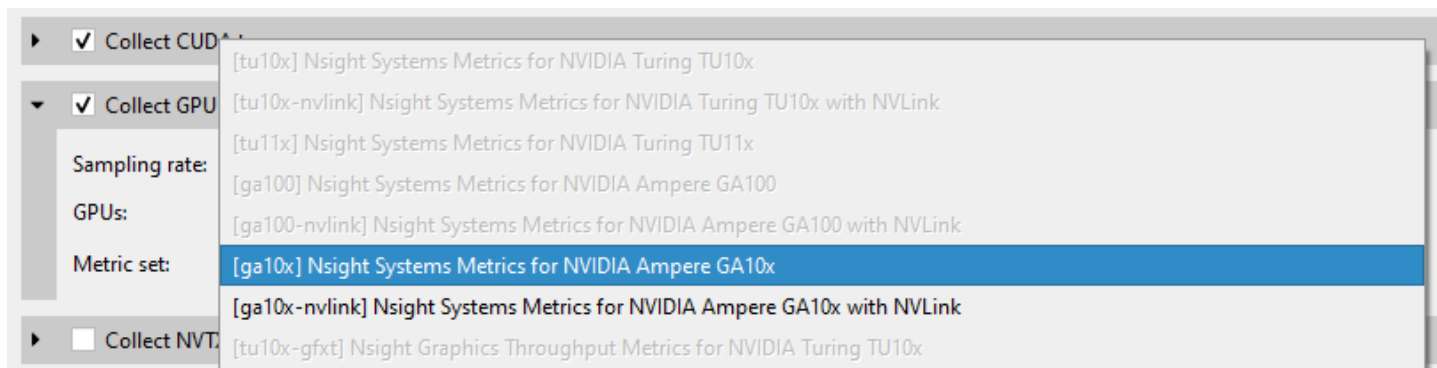
For commands to launch GPU Metrics from the CLI with examples, see [Profiling from the CLI](#).

When launching analysis in Nsight Systems, select **Collect GPU Metrics**.



Select the **GPUs** dropdown to pick which GPUs you wish to sample.

Select the **Metric set:** dropdown to choose which available metric set you would like to sample.



Note

Metric sets for GPUs that are not being sampled will be greyed out.

Sampling frequency#

Sampling frequency can be selected from the range of 10 Hz - 200 kHz. The default value is 10 kHz.

The maximum sampling frequency without buffer overflow events depends on GPU (SM count), GPU load intensity, and overall system load. The bigger the chip and the higher the load, the lower the maximum frequency. If you need higher frequency, you can increase it until you get “Buffer overflow” message in the Diagnostics Summary report page.

Each metric set has a recommended sampling frequency range in its description. These ranges are approximate. If

you observe Inconsistent Data or Missing Data ranges on timeline, please try closer to the recommended frequency.

Available metrics#

- **GPC Clock Frequency** - `gpc__cycles_elapsed.avg.per_second`

The average GPC clock frequency in hertz. In public documentation the GPC clock may be called the “Application” clock, “Graphic” clock, “Base” clock, or “Boost” clock.

Note

The collection mechanism for GPC can result in a small fluctuation between samples.

- **SYS Clock Frequency** - `sys__cycles_elapsed.avg.per_second`

The average SYS clock frequency in hertz. The GPU front end (command processor), copy engines, and the performance monitor run at the SYS clock. On Turing and NVIDIA GA100 GPUs, the sampling frequency is based upon a period of SYS clocks (not time) so samples per second will vary with SYS clock. On NVIDIA GA10x GPUs, the sampling frequency is based upon a fixed frequency clock. The maximum frequency scales linearly with the SYS clock.

- **GR Active** - `gr__cycles_active.sum.pct_of_peak_sustained_elapsed`

The percentage of cycles the graphics/compute engine is active. The graphics/compute engine is active if there is any work in the graphics pipe or if the compute pipe is processing work.

GA100 MIG - MIG is not yet supported. This counter will report the activity of the primary GR engine.

- **Sync Compute In Flight** -

`gr__dispatch_cycles_active_queue_sync.avg.pct_of_peak_sustained_elapsed`

The percentage of cycles with synchronous compute in flight.

CUDA: CUDA will only report synchronous queue in the case of MPS configured with 64 sub-context. Synchronous

refers to work submitted in VEID=0.

Graphics: This will be true if any compute work submitted from the direct queue is in flight.

- **Async Compute in Flight -**

`gr__dispatch_cycles_active_queue_async.avg.pct_of_peak_sustained_elapsed`

The percentage of cycles with asynchronous compute in flight.

CUDA: CUDA will only report all compute work as asynchronous. The one exception is if MPS is configured and all 64 sub-context are in use. 1 sub-context (VEID=0) will report as synchronous.

Graphics: This will be true if any compute work submitted from a compute queue is in flight.

- **Draw Started -** `fe__draw_count.avg.pct_of_peak_sustained_elapsed`

The ratio of draw calls issued to the graphics pipe to the maximum sustained rate of the graphics pipe.

Note

The percentage will always be very low as the front end can issue draw calls significantly faster than the pipe can execute the draw call. The rendering of this row will be changed to help indicate when draw calls are being issued.

- **Dispatch Started -** `gr__dispatch_count.avg.pct_of_peak_sustained_elapsed`

The ratio of compute grid launches (dispatches) to the compute pipe to the maximum sustained rate of the compute pipe.

Note

The percentage will always be very low as the front end can issue grid launches significantly faster than the pipe can execute the draw call. The rendering of this row will be changed to help indicate when grid launches are being issued.

- **Vertex/Tess/Geometry Warps in Flight -**

`tpc__warps_active_shader_vtg_realtime.avg.pct_of_peak_sustained_elapsed`

The ratio of active vertex, geometry, tessellation, and meshlet shader warps resident on the SMs to the maximum number of warps per SM as a percentage.

- **Pixel Warps in Flight -**

`tpc__warps_active_shader_ps_realtime.avg.pct_of_peak_sustained_elapsed`

The ratio of active pixel/fragment shader warps resident on the SMs to the maximum number of warps per SM as a percentage.

- **Compute Warps in Flight -**

`tpc__warps_active_shader_cs_realtime.avg.pct_of_peak_sustained_elapsed`

The ratio of active compute shader warps resident on the SMs to the maximum number of warps per SM as a percentage.

- **Active SM Unused Warp Slots -**

`tpc__warps_inactive_sm_active_realtime.avg.pct_of_peak_sustained_elapsed`

The ratio of inactive warp slots on the SMs to the maximum number of warps per SM as a percentage. This is an indication of how many more warps may fit on the SMs if occupancy is not limited by a resource such as max warps of a shader type, shared memory, registers per thread, or thread blocks per SM.

- **Idle SM Unused Warp Slots -**

`tpc__warps_inactive_sm_idle_realtime.avg.pct_of_peak_sustained_elapsed`

The ratio of inactive warp slots due to idle SMs to the the maximum number of warps per SM as a percentage.

This is an indicator that the current workload on the SM is not sufficient to put work on all SMs. This can be due to:

- CPU starving the GPU.
- Current work is too small to saturate the GPU.
- Current work is trailing off but blocking next work.

- **SMs Active -** `sm__cycles_active.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles SMs had at least 1 warp in flight (allocated on SM) to the number of cycles as a percentage. A value of 0 indicates all SMs were idle (no warps in flight). A value of 50% can indicate some gradient between all SMs active 50% of the sample period or 50% of SMs active 100% of the sample period.

- **SM Issue** - `sm__inst_executed_realtime.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles that SM sub-partitions (warp schedulers) issued an instruction to the number of cycles in the sample period as a percentage.

- **Tensor Active** -

`sm__pipe_tensor_cycles_active_realtime.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the SM tensor pipes were active issuing tensor instructions to the number of cycles in the sample period as a percentage.

TU102/4/6: This metric is not available on TU10x for periodic sampling. Please see Tensor Active/FP16 Active.

- **Tensor Active / FP16 Active** -

`sm__pipe_shared_cycles_active_realtime.avg.pct_of_peak_sustained_elapsed`

TU102/4/6 only.

The ratio of cycles the SM tensor pipes or FP16x2 pipes were active issuing tensor instructions to the number of cycles in the sample period as a percentage.

- **DRAM Read Bandwidth** - `dramc__read_throughput.avg.pct_of_peak_sustained_elapsed`,
`dram__read_throughput.avg.pct_of_peak_sustained_elapsed`

- **VRAM Read Bandwidth** -

`FBPA.TriageA.dramc__read_throughput.avg.pct_of_peak_sustained_elapsed`,
`FBSP.TriageSCG.dramc__read_throughput.avg.pct_of_peak_sustained_elapsed`,
`FBSP.TriageAC.dramc__read_throughput.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the DRAM interface was active reading data to the elapsed cycles in the same period as a percentage.

- **DRAM Write Bandwidth** - `dramc__write_throughput.avg.pct_of_peak_sustained_elapsed`,
`dram__write_throughput.avg.pct_of_peak_sustained_elapsed`

- **VRAM Write Bandwidth** -

FBPA.TriageA.dramc__write_throughput.avg.pct_of_peak_sustained_elapsed,
FBSP.TriageSCG.dramc__write_throughput.avg.pct_of_peak_sustained_elapsed,
FBSP.TriageAC.dramc__write_throughput.avg.pct_of_peak_sustained_elapsed

The ratio of cycles the DRAM interface was active writing data to the elapsed cycles in the same period as a percentage.

- **NVENC Active**

NVENC.TriageTop.nvenc__cycles_active.avg.pct_of_peak_sustained_elapsed

The ratio of cycles the NVENC unit was actively processing a command to the number of cycles in the same sample period as a percentage.

- **NVENC Read Throughput**

NVENC.TriageTop.nvenc__memif2nvenc_read_throughput.avg.pct_of_peak_sustained_elapsed

NVENC Write Throughput

NVENC.TriageTop.nvenc__nvenc2memif_write_throughput.avg.pct_of_peak_sustained_elapsed

The ratio of cycles the NVENC unit was actively processing read/write operations to the number of cycles in the same sample period as a percentage.

- **OFA Active**

OFA.TriageTop.ofa_cycles_active.avg.pct_of_peak_sustained_elapsed

The ratio of cycles the OFA (Optical Flow Accelerator) was actively processing a command to the number of cycles in the same sample period as a percentage.

- **OFA Read Throughput**

OFA.TriageTop.ofa__memif2ofa_read_throughput.avg.pct_of_peak_sustained_elapsed

OFA Write Throughput

OFA.TriageTop.ofa__ofa2memif_write_throughput.avg.pct_of_peak_sustained_elapsed

The ratio of cycles the OFA (Optical Flow Accelerator) was actively processing read/write operations to the number of

cycles in the same sample period as a percentage.

- **NVLink bytes received** - `nvlnrx__bytes.avg.pct_of_peak_sustained_elapsed`

The ratio of bytes received on the NVLink interface to the maximum number of bytes receivable in the sample period as a percentage. This value includes protocol overhead.

- **NVLink bytes transmitted** - `nvlnrx__bytes.avg.pct_of_peak_sustained_elapsed`

The ratio of bytes transmitted on the NVLink interface to the maximum number of bytes transmittable in the sample period as a percentage. This value includes protocol overhead.

- **PCIe Read Throughput** - `pcie__read_bytes.avg.pct_of_peak_sustained_elapsed`

The ratio of bytes received on the PCIe interface to the maximum number of bytes receivable in the sample period as a percentage. The theoretical value is calculated based upon the PCIe generation and number of lanes. This value includes protocol overhead.

- **PCIe Write Throughput** - `pcie__write_bytes.avg.pct_of_peak_sustained_elapsed`

The ratio of bytes transmitted on the PCIe interface to the maximum number of bytes receivable in the sample period as a percentage. The theoretical value is calculated based upon the PCIe generation and number of lanes. This value includes protocol overhead.

- **PCIe Read Requests to BAR1** - `pcie__rx_requests_aperture_bar1_op_read.sum`

- **PCIe Write Requests to BAR1** - `pcie__rx_requests_aperture_bar1_op_write.sum`

BAR1 is a PCI Express (PCIe) interface used to allow the CPU or other devices to directly access GPU memory. The GPU normally transfers memory with its copy engines, which would not show up as BAR1 activity. The GPU drivers on the CPU do a small amount of BAR1 accesses, but heavier traffic is typically coming from other technologies.

On Linux, technologies like GPU Direct, GPU Direct RDMA, and GPU Direct Storage transfer data across PCIe BAR1. In the case of GPU Direct RDMA, that would be an Ethernet or InfiniBand adapter directly writing to GPU memory.

On Windows, Direct3D12 resources can also be made accessible directly to the CPU via NVAPI functions to support

small writes or reads from GPU buffers, in this case too many BAR1 accesses can indicate a performance issue, like it has been demonstrated in the [Optimizing DX12 Resource Uploads to the GPU Using CPU-Visible VRAM](#) technical blog post.

Exporting and Querying Data[#]

It is possible to access metric values for automated processing using the Nsight Systems CLI export capabilities.

An example that extracts values of **SMs Active**:

```
$ nsys export -t sqlite report.nsys-rep
```

```
$ sqlite3 report.sqlite "SELECT timestamp, value FROM GPU_METRICS JOIN TARGET_INFO_GPU_METRICS  
USING (metricId) WHERE value != 0 and metricName == 'SMs Active' LIMIT 10;"
```

```
309277039|80
```

```
309301295|99
```

```
309325583|99
```

```
309349776|99
```

```
309373872|60
```

```
309397872|19
```

```
309421840|100
```

```
309446000|100
```

```
309470096|100
```

```
309494161|99
```

Values are integer percentages (0..100).

Limitations[#]

- If metric sets with NVLink are used but the links are not active, they may appear as fully utilized.

- Only one tool that subscribes to these counters can be used at a time, therefore, Nsight Systems GPU Metrics feature cannot be used at the same time as the following tools:

- Nsight Graphics
- Nsight Compute
- DCGM (Data Center GPU Manager)

Use the following command:

- `dcgmi profile --pause`
- `dcgmi profile --resume`

Or API:

- `dcgmProfPause`
- `dcgmProfResume`
- Non-NVIDIA products which use:
 - CUPTI sampling used directly in the application. CUPTI trace is okay (although it will block Nsight Systems CUDA trace)
- DCGM library
- Nsight Systems limits the amount of memory that can be used to store GPU Metrics samples. Analysis with higher sampling rates or on GPUs with more SMs has a risk of exceeding this limit. This will lead to gaps on timeline filled with Missing Data ranges. Future releases will reduce the frequency of this happening.

NVML power and temperature metrics (preview)<#>

Nsight Systems can now periodically sample power and temperature metrics from GPUs and plot them on the timeline in the GUI. These metrics are provided by the NVML API calls `nvmlDeviceGetPowerUsage` and `nvmlDeviceGetTemperature` respectively. The power metrics are provided in milliwatts (mW) and the

temperature in degrees Celcius (C).

To enable the power and temperature sampling add the following option to the `nsys profile` or `start` commands:

```
--enable nvmf_metrics[,arg1[=value1],arg2[=value2], ...]
```

There are no spaces following `nvmf_metrics` plugin name. It is followed by a comma separated list of arguments or argument=value pairs. Arguments with spaces should be enclosed in double quotes.

Supported arguments are:

:name: table_nvmlpowertemp_table :class: table-compact#

Short name	Long name	Possible Parameters	Default	Switch Description
-i	--interval	integer	100	Sampling interval in milliseconds
-h	--help			Print help message

Usage Examples

- `nsys profile --enable nvmf_metrics ...`
Sample power and temperature on all available GPUs every 100ms.
- `nsys profile --enable nvmf_metrics,-i10`
Sample power and temperature on all available GPUs every 10ms.

For general information on Nsight Systems plugins please refer to [Nsight Systems Plugins \(Preview\)](#) system.

SoC Metrics#

Overview#

SoC Metrics feature is intended to identify performance limiters in applications running on NVIDIA SoCs and is similar to GPU Metrics.

Nsight Systems SoC Metrics is only available for Linux and QNX targets on aarch64. It requires NVIDIA Orin architecture or newer.

Available metrics#

- **CPU Read Throughput**

`mcc__dram_throughput_srcnode_cpu_op_read.avg.pct_of_peak_sustained_elapsed`

CPU Write Throughput

`mcc__dram_throughput_srcnode_cpu_op_write.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the SoC memory controllers were actively processing read/write operations from the CPU to the number of cycles in the same sample period as a percentage.

- **GPU Read Throughput**

`mcc__dram_throughput_srcnode_gpu_op_read.avg.pct_of_peak_sustained_elapsed`

GPU Write Throughput

`mcc__dram_throughput_srcnode_gpu_op_write.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the SoC memory controllers were actively processing read/write operations from the GPU to the number of cycles in the same sample period as a percentage.

- **DBB Read Throughput**

`mcc__dram_throughput_srcnode_dbb_op_read.avg.pct_of_peak_sustained_elapsed`

DBB Write Throughput

`mcc__dram_throughput_srcnode_dbb_op_write.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the SoC memory controllers were actively processing read/write operations from not-CPU/not-GPU to the number of cycles in the same sample period as a percentage.

- **DRAM Read Throughput**

`mcc__dram_throughput_op_read.avg.pct_of_peak_sustained_elapsed`

DRAM Write Throughput

`mcc__dram_throughput_op_write.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the SoC memory controllers were actively processing read/write operations to the number of cycles in the same sample period as a percentage.

- **DLA0/DLA1 Active**

`nvdla__cycles_active.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the DLA (Deep Learning Accelerator) was actively processing a command to the number of cycles in the same sample period as a percentage.

- **DLA0/DLA1 Read Throughput**

`nvdla__dbb2nvdla_read_throughput.avg.pct_of_peak_sustained_elapsed`

DLA0/DLA1 Write Throughput

`nvdla__nvdla2dbb_write_throughput.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the DLA (Deep Learning Accelerator) was actively processing read/write operations to the number of cycles in the same sample period as a percentage.

- **NVENC Active**

`nvenc__cycles_active.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the NVENC unit was actively processing a command to the number of cycles in the same sample period as a percentage.

- **NVENC Read Throughput**

`nvenc__memif2nvenc_read_throughput.avg.pct_of_peak_sustained_elapsed`

NVENC Write Throughput

`nvenc__nvenc2memif_write_throughput.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the NVENC unit was actively processing read/write operations to the number of cycles in the same sample period as a percentage.

- **PVA VPU Active**

`pvavpu__vpu_cycles_active.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the PVA (Programmable Vision Accelerator) VPU (Vector Processing Unit) was actively processing a command to the number of cycles in the same sample period as a percentage.

- **PVA DMA Read Throughput**

`pva__dbb2pvadma_read_throughput.avg.pct_of_peak_sustained_elapsed`

PVA DMA Write Throughput

`pva__pvadma2dbb_write_throughput.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the PVA (Programmable Vision Accelerator) VPU (Vector Processing Unit) was actively processing read/write operations to the number of cycles in the same sample period as a percentage.

Note

To enable PVA trace on DRIVE 6.0.8.0, run these two commands before mounting any additional partitions:

`echo 1 >/dev/nvprvadebugfs/pva0/tracing echo 2 >/dev/nvprvadebugfs/pva0/trace_level`

- **OFA Active**

`ofa_cycles_active.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the OFA (Optical Flow Accelerator) was actively processing a command to the number of cycles in the same sample period as a percentage.

- **OFA Read Throughput**

`ofa__memif2ofa_read_throughput.avg.pct_of_peak_sustained_elapsed`

OFA Write Throughput

`ofa__ofa2memif_write_throughput.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the OFA (Optical Flow Accelerator) was actively processing read/write operations to the number of cycles in the same sample period as a percentage.

- **VIC Active**

`vic_cycles_active.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the VIC (Video Image Compositor) was actively processing a command to the number of cycles in the same sample period as a percentage.

- **VIC Read Throughput**

`vic__dbb2vic_read_throughput.avg.pct_of_peak_sustained_elapsed`

VIC Write Throughput

`vic__vic2dbb_write_throughput.avg.pct_of_peak_sustained_elapsed`

The ratio of cycles the VIC (Video Image Compositor) was actively processing read/write operations to the number of cycles in the same sample period as a percentage.

Launching SoC Metrics from the CLI#

SoC Metrics feature is controlled with 3 CLI switches:

- `--soc-metrics=[true, false]` enables SoC Metrics sampling (default is false)
- `--soc-metrics-set=[<alias>, file:<file name>]` selects metric set to use (default is the 1st suitable from the list)
- `--soc-metrics-frequency=[100..200000]` selects sampling frequency in Hz (default is 10000)

To profile with default options:

```
# Must be root or added to 'debug' group
$ nsys profile --soc-metrics=true ./my-app
```

Launching SoC Metrics from the GUI#

When launching analysis in Nsight Systems, select **Collect SoC Metrics**.

The settings are similar to [GPU Metrics](#).

For commands to launch SoC Metrics from the CLI with examples, see the [CLI documentation](#).

CPU Profiling on Linux#

Nsight Systems on Linux targets, utilizes the Linux OS' perf subsystem to sample CPU Instruction Pointers (IPs) and backtraces, trace CPU context switches, and sample CPU and OS event counts. The Linux perf tool utilizes the same perf subsystem.

Nsight Systems Embedded Platforms Edition on Linux kernel prior to v5.15 uses a custom kernel module to collect the same data. The Nsight Systems CLI command `nsys status --environment` indicates when the kernel module is used instead of the Linux OS' perf subsystem.

Features#

- **CPU Instruction Pointer / Backtrace Sampling**

Nsight Systems can sample CPU Instruction Pointers / backtraces periodically. The collection of a sample is triggered by a hardware event overflow - e.g., a sample is collected after every 1 million CPU reference cycles on a per thread basis. In the GUI, samples are shown on the individual thread timelines, in the Event Viewer, and in the Top Down, Bottom Up, or Flat views which provide histogram-like summaries of the data. IP / backtrace collections can be configured in process-tree or system-wide mode. In process-tree mode, Nsight Systems will sample the process, and any of its descendants, launched by the tool. In system-wide mode, Nsight Systems will sample all processes running on the system, including any processes launched by the tool.

- **CPU Context Switch Tracing**

Nsight Systems can trace every time the OS schedules a thread on a logical CPU and every time the OS thread gets unscheduled from a logical CPU. The data is used to show CPU utilization and OS thread utilization within the Nsight Systems GUI. Context switch collections can be configured in process-tree or system-wide mode. In process-tree mode, Nsight Systems will trace the process, and any of its descendants, launched by Nsight Systems. In system-wide mode, Nsight Systems will trace all processes running on the system, including any processes launched by the Nsight Systems.

- **CPU Event Sampling**

Nsight Systems can periodically sample CPU hardware event counts and OS event counts and show the event’s rate over time in the Nsight Systems GUI. Event sample collections can be configured in system-wide mode only. In system-wide mode, Nsight Systems will sample event counts of all CPUs and the OS event counts running on the system. Event counts are not directly associated with processes or threads.

- **CPU Core Metrics**

Nsight Systems can access and make available information about CPU core metrics. This functionality is available only on Linux and only for the NVIDIA Grace (TM) CPU. The `--cpu-core-metrics=help` command will list 39 different metrics, Those metrics are described in the [Grace Performance Tuning Guide](#). Then selected option IDs can be fed into the `--cpu-core-metrics` switch.

System Requirements#

- **Paranoid Level**

The [system’s paranoid level](#) must be 2 or lower.

Paranoid Level	CPU IP/backtrace Sampling process-tree mode	CPU IP/backtrace Sampling system-wide mode	CPU Context Switch Tracing process-tree mode	CPU Context Switch Tracing system-wide mode	Event Sampling system-wide mode
3 or greater	not available	not available	not available	not available	not available
2	User mode IP/ backtrace samples only	not available	available	not available	not available

Paranoid Level	CPU IP/backtrace Sampling process-tree mode	CPU IP/backtrace Sampling system-wide mode	CPU Context Switch Tracing process-tree mode	CPU Context Switch Tracing system-wide mode	Event Sampling system-wide mode
1	Kernel and user mode IP/backtrace samples	not available	available	not available	not available
0, -1	Kernel and user mode IP/backtrace samples	Kernel and user mode IP/backtrace samples	available	available	hardware and OS events

- **Kernel Version**

To support the CPU profiling features utilized by Nsight Systems, the kernel version must be greater than or equal to v4.3. RedHat has backported the required features to the v3.10.0-693 kernel. RedHat distros and their derivatives (e.g. CentOS) require a 3.10.0-693 or later kernel. Use the `uname -r` command to check the kernel's version.

- **perf_event_open syscall**

The `perf_event_open` syscall needs to be available. When running within a Docker container, the default seccomp settings will normally block the `perf_event_open` syscall. To workaround this issue, use the `Docker run --privileged` switch when launching the docker or modify the docker's seccomp settings. Some VMs (virtual machines), e.g. AWS, may also block the `perf_event_open` syscall.

- **Sampling Trigger**

In some rare case, a sampling trigger is not available. The sampling trigger is either a hardware or software event that causes a sample to be collected. Some VMs block hardware events from being accessed and therefore, prevent hardware events from being used as sampling triggers. In those cases, Nsight Systems will fall back to using a software trigger if possible.

- **Checking Your Target System**

Use the `nsys status --environment` command to check if a system meets the Nsight Systems CPU profiling requirements. Example output from this command is shown below. Note that this command does not check for Linux capability overrides - i.e., if the user or executable files have `CAP_SYS_ADMIN` or `CAP_PERFMON` capability. Also, note that this command does not indicate if system-wide mode can be used.

```
(base) rknight@RKNIGHT-U18:~/quadd2/quadd/Built/Bin/QuadD-Debug/target-linux-x64$ ./nsys status -e
Timestamp counter supported: Yes

CPU Profiling Environment Check
Root privilege: disabled
Linux Kernel Paranoid Level = 2
Linux Distribution = Ubuntu
Linux Kernel Version = 5.4.0-122-generic: OK
Linux perf_event_open syscall available: OK
Sampling trigger event available: OK
Intel(c) Last Branch Record support: Available
CPU Profiling Environment (process-tree): OK
CPU Profiling Environment (system-wide): Fail

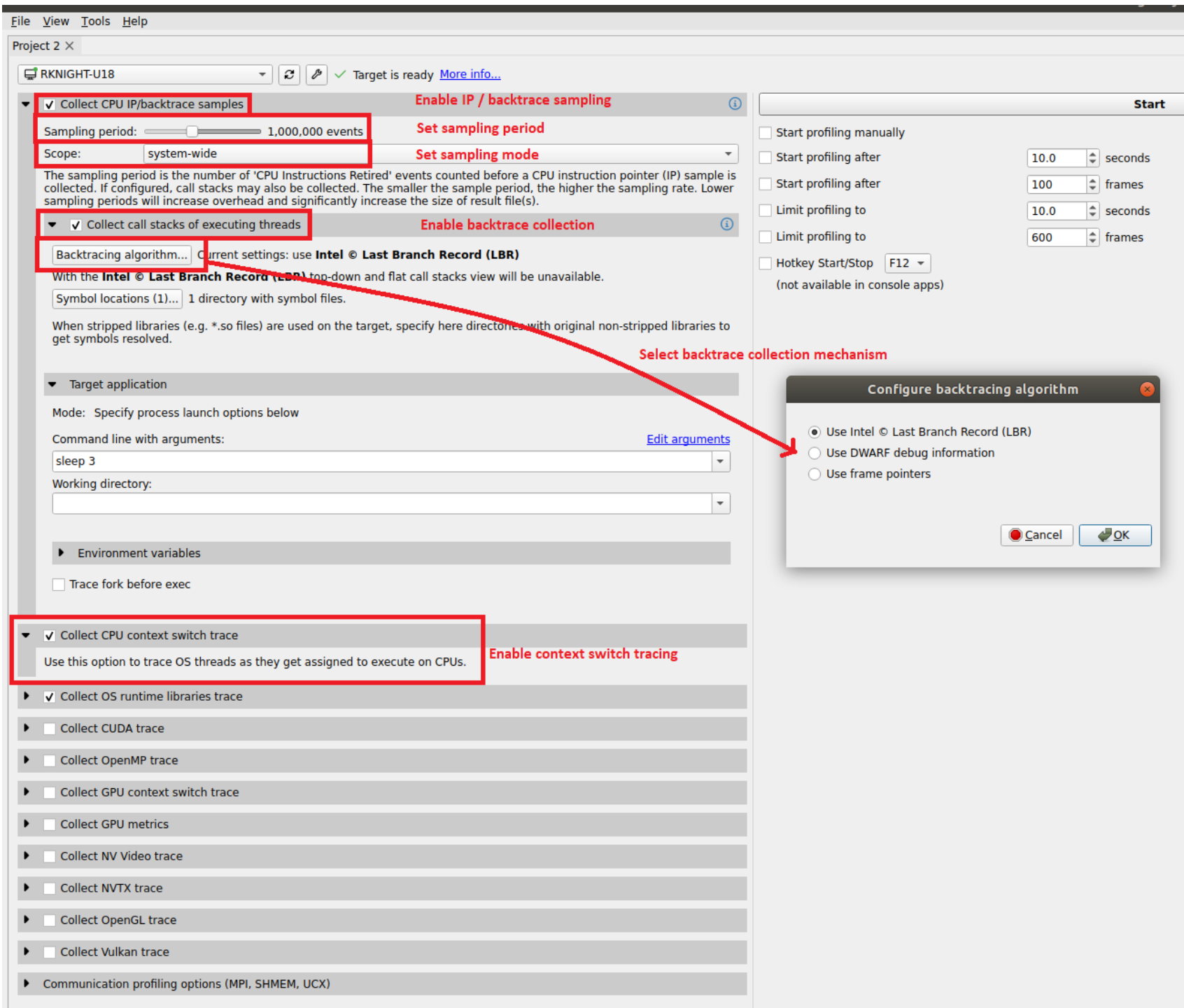
See the product documentation at https://docs.nvidia.com/nsight-systems for more information,
including information on how to set the Linux Kernel Paranoid Level.
```

Configuring a CPU Profiling Collection#

When configuring Nsight Systems for CPU Profiling from the CLI, use some or all of the following options: `--sample`, `--cpuctxsw`, `--event-sample`, `--backtrace`, `--cpu-core-events`, `--event-sampling-interval`, `--os-events`, `--samples-per-backtrace`, and `--sampling-period`.

Details about these options, including examples can be found at [Profiling from the CLI](#).

When configuring from the GUI, the following options are available:



The configuration used during CPU profiling is documented in the Analysis Summary:

Analysis options

Collect CPU IP samples	On
Sampling period	2,000,000 events/sample
CPU IP Sampling Trigger Event	Reference Cycles
CPU IP samples / backtrace	3
CPU profiling scope	process-tree
Collect CPU context switch trace	On
Collect backtraces	On
Backtracing algorithm	Use Intel © Last Branch Record (LBR)
Include child processes	On

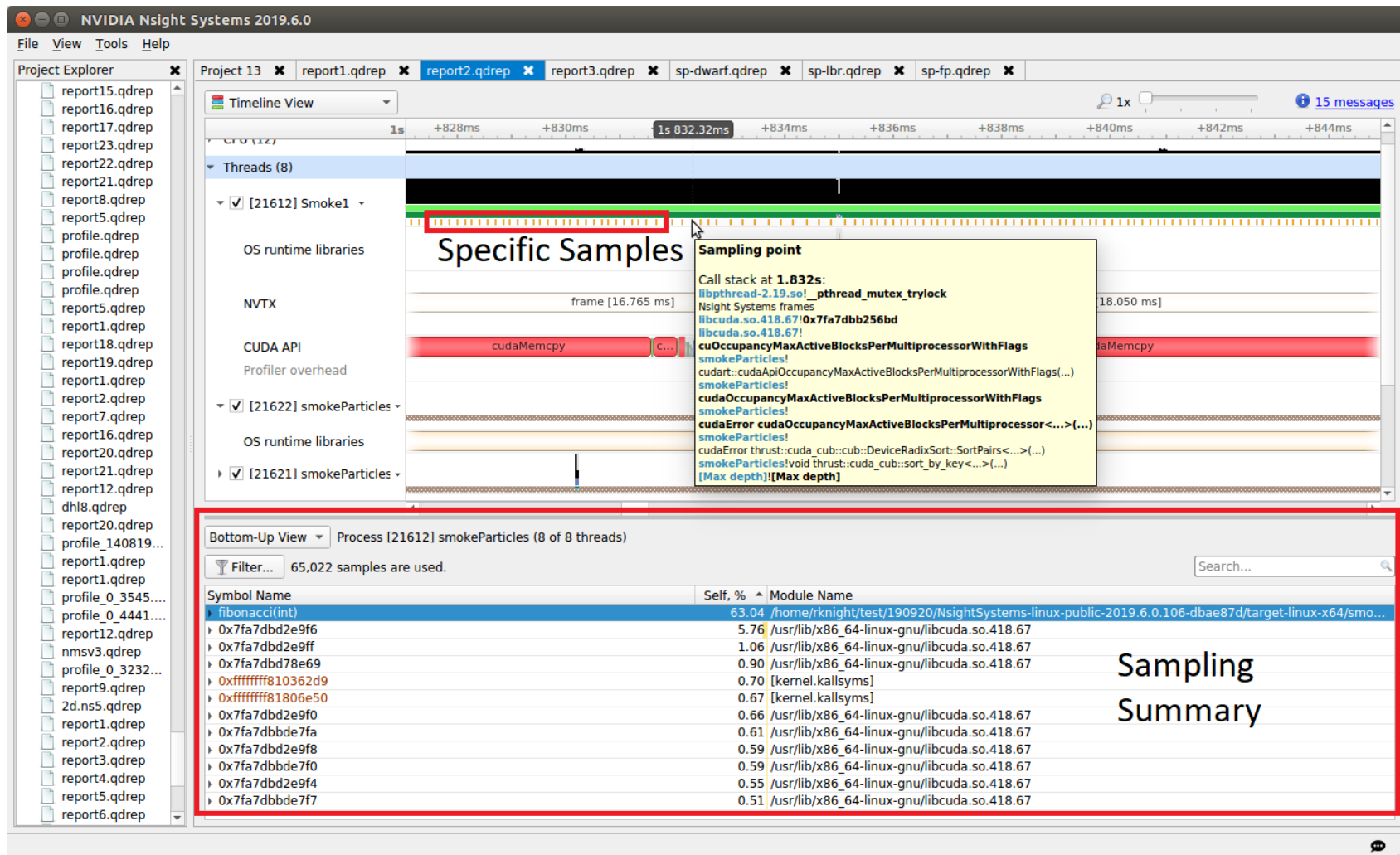
As well as in the Diagnostics Summary:

File View Tools Help			
Project 2 X report1 X report2 X report3 X			
Diagnostics Summary			
Messages			
Source	Process ID	Time	Description
Daemon		-00:00.493	Intel(c) Last Branch Record (LBR) backtraces collected.
Daemon		-00:00.493	Hardware event 'Reference Cycles', with sampling period 2000000, used to trigger system-wide CPU IP sample collection.
Daemon		-00:00.140	1 CPU IP samples collected for every CPU IP backtrace collected.
Daemon		-00:00.000	Vulkan runtime version 1.1.0 on target machine.
Analysis		00:00.000	Profiling has started.
Daemon	21128	00:00.000	Process was launched by the profiler, see /tmp/nvidia/nsight_systems/quadd_session_1021114/streams/pid_21128_stdout.log and stderr.log for program output
Injection	21128	00:00.218	Common injection library initialized successfully.
Injection	21128	00:00.253	OS runtime libraries injection initialized successfully.
Injection	21128	00:00.287	OpenGL injection initialized successfully.
Injection	21128	00:00.459	Buffers holding CUDA trace data will be flushed on CudaProfilerStop() call.
Injection	21128	00:00.800	CUDA injection initialized successfully.
Injection	21128	00:01.121	NVTX injection initialized successfully.
Injection	21128	00:05.504	Number of CUPTI events produced: 17,126, CUPTI buffers: 50.
Analysis		00:05.923	Profiling has stopped.
Daemon		00:07.184	Number of IP samples collected: 16,600.
Analysis	21128	02:00.149	OpenGL function MakeCurrent was called with wrong (duplicate) context argument 757 times.
Analysis	21128	02:00.150	Number of NVTX events collected: 753.
Analysis	21128	02:00.150	Number of OpenGL events collected: 201,455.
Analysis	21128	02:00.150	Number of CUDA events collected: 16,088.
Analysis	21128	02:00.150	Number of OS runtime libraries events collected: 4,547.

Visualizing CPU Profiling Results#

Here are example screenshots visualizing CPU profiling results. For details about navigating the Timeline View and the backtraces, see the section on [Timeline View in the Reading Your Report in the GUI section of the User Guide](#).

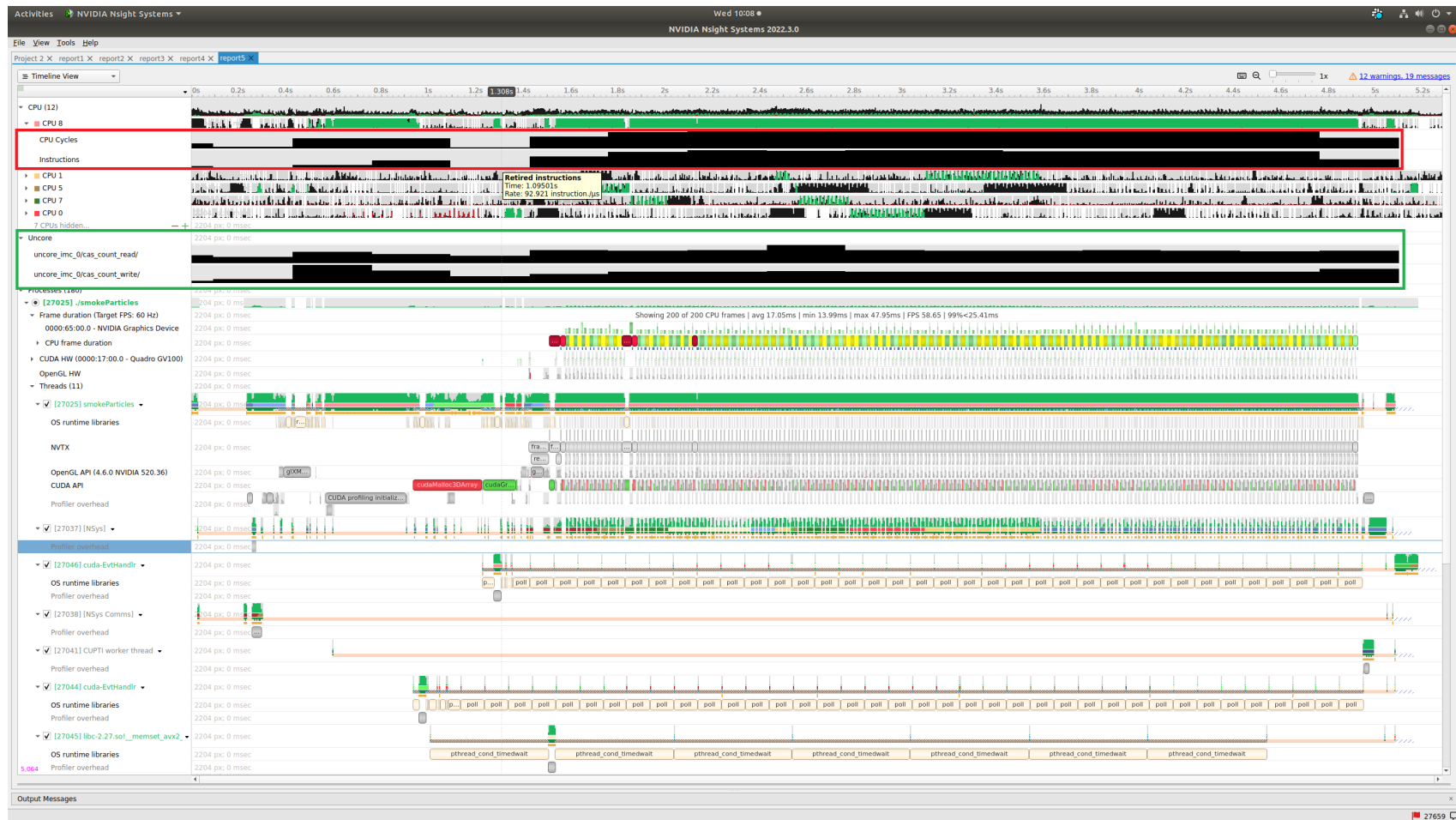
Example of CPU IP/Backtrace Data



In the timeline, yellow-orange marks can be found under each thread's timeline that indicate the moment an IP / backtrace sample was collected on that thread (e.g., see the yellow-orange marks in the Specific Samples box above). Hovering the cursor over a mark will cause a tooltip to display the backtrace for that sample.

Below the Timeline is a drop-down list with multiple options including Events View, Top-Down View, Bottom-Up View, and Flat View. All four of these views can be used to view CPU IP / back trace sampling data.

Example of Event Sampling



Event sampling samples hardware or software event counts during a collection and then graphs those events as rates on the Timeline. The above screenshot shows four hardware events. Core and cache events are graphed under the associated CPU row (see the red box in the screenshot) while uncore and OS events are graphed in their own row (see the green box in the screenshot). Hovering the cursor over an event sampling row in the timeline shows the event's rate at that moment.

Arm Topdown Analysis#

Arm Topdown methodology supports performance analysis, workload characterization, and microarchitecture

exploration. You can find details on the technique at [Arm Topdown Methodology](#).

Nsight Systems provides scripting to support running this analysis for the Grace CPU.

In your `target-linux-sbsa-armv8/cpu` directory, look for a script named `collect_grace_topdown.sh`. This script simplifies collecting all PMU core event and metric data needed to perform a traditional CPU Topdown analysis of the workload's CPU performance.

The script runs multiple system-wide `nsys profile` commands sequentially to collect the data. You can add additional Nsight Systems options to the command line as per usual, with the following exceptions:

- `--event-sample`, `--event-sampling-interval`, `--cpu-core-events`, and `--cpu-core-metrics` switches are set by the script for Topdown analysis.
- `-f`, `--force-overwrite` switch is set to true by the script
- `-o`, `--output` switch is set by the script to generate a list of predefined output `nsys-rep` files.
- `--kill` switch is set to the default value of `sigterm`

If an application is to be launched by the script, place a `--` between the `nsys` switches and the application command line.

Example command line:

```
collect_grace_topdown.sh --trace=osrt,nvtx,cuda -- myApp arg1 arg2
```

Output files will be written to the current working directory. The output consists of a collection of `.nsys-rep` files that contain the metric data required to do a Topdown analysis of the workload. These files can be opened in the Nsight Systems GUI to view the metric results on the timeline.

You can further use the NVTX CPU Topdown recipe (`nsys recipe nvtx_cpu_topdown --input .`) that will process the data from the `.nsys-rep` files and generate an output with CPU Topdown Methodology metrics computed for NVTX ranges. For details and use cases of this recipe, see [nvtx_cpu_topdown Recipe](#).

Note

Arm Topdown analysis requires multiple system-wide collections and may take a significantly long time to run and post-process.

Common Issues#

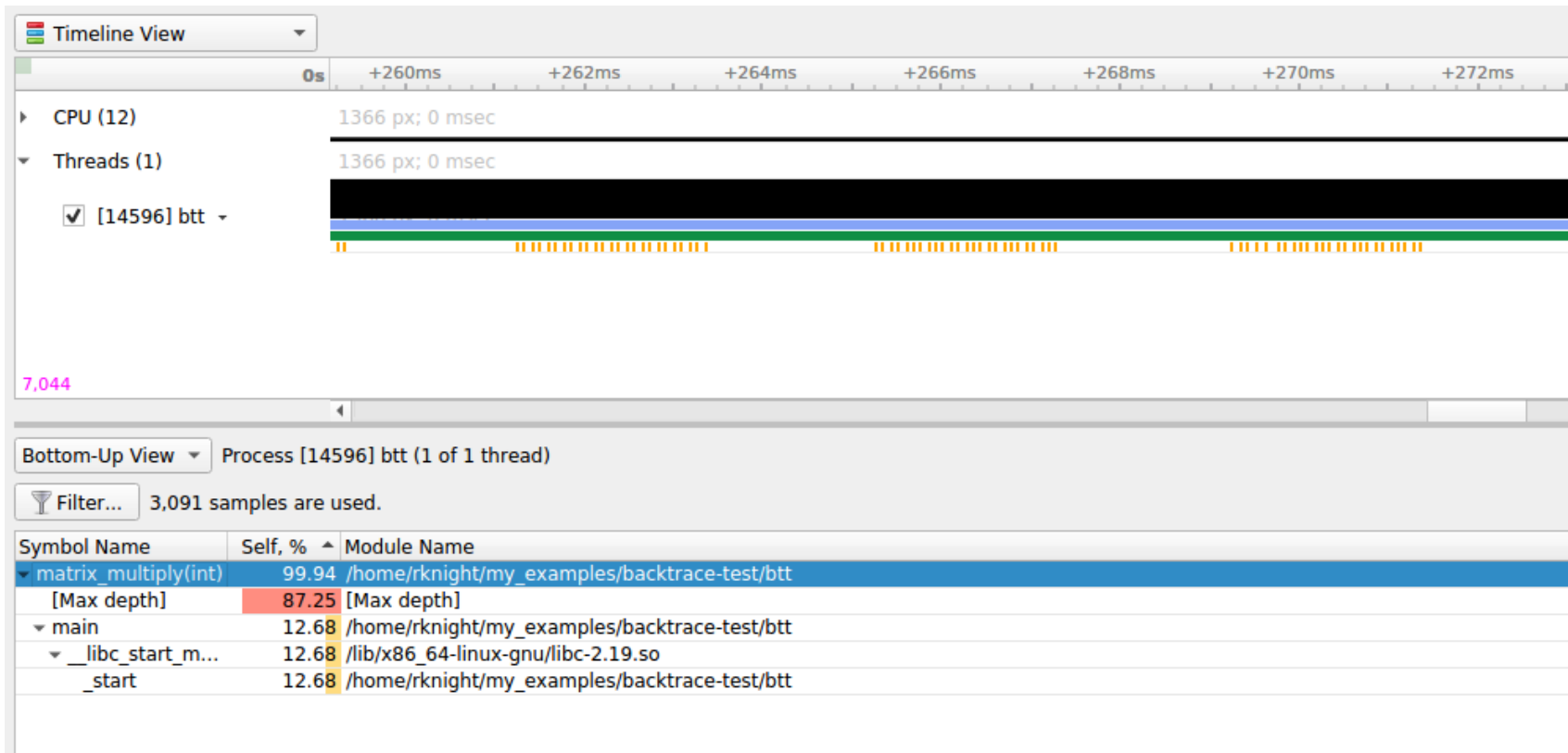
- **Reducing Overhead Caused By Sampling**

There are several ways to reduce overhead caused by sampling.

- Disable sampling (i.e., use the `--sampling=none` switch).
- Increase the sampling period (i.e., reduce the sampling rate) using the `--sampling-period` switch.
- Stop collecting backtraces (i.e., use the `--backtrace=none` switch) or collect more efficient backtraces - if available, use the `--backtrace=lbr` switch.
- reduce the number of backtraces collected per sample. See documentation for the `--samples-per-backtrace` switch.

- **Throttling**

The Linux operating system enforces a maximum time to handle sampling interrupts. This means that if collecting samples takes more than a specified amount of time, the OS will throttle (i.e., slow down) the sampling rate to prevent the perf subsystem from causing too much overhead. When this occurs, sampling data may become irregular even though the thread is very busy.



The above screenshot shows a case where CPU IP / backtrace sampling was throttled during a collection. Note the irregular intervals of sampling tickmarks on the thread timeline. The number of times a collection throttled is provided in the Nsight Systems GUI's Diagnostics messages. If a collection throttles frequently (e.g., 1000s of times), increasing the sampling period should help reduce throttling.

Note

When throttling occurs, the OS sets a new (lower) maximum sampling rate in the procs. This value must be reset before the sampling rate can be increased again. Use the following command to reset the OS' max sampling rate

```
echo '100000' | sudo tee /proc/sys/kernel/perf_event_max_sample_rate
```

- **Sample intervals are irregular**

My samples are not periodic - why? My samples are clumped up - why? There are gaps in between the samples -

why? Likely reasons:

- Throttling, as described above.
- The paranoid level is set to 2. If the paranoid level is set to 2, anytime the workload makes a system call and spends time executing kernel mode code, samples will not be collected and there will be gaps in the sampling data.
- The sampling trigger itself is not periodic. If the trigger event is not periodic, for example, the Instructions Retired event, sample collection will primarily occur when cache misses are occurring.
- **No CPU profiling data is collected**

There are a few common issues that cause CPU profiling data to not be collected:

- System requirements are not met. Check your system settings with the `nsys status --environment` command and see the System Requirements section above.
- I profiled my workload in a Docker container but no sampling data was collected. By default, Docker containers prevent the `perf_event_open` syscall from being utilized. To override this behavior, launch the Docker with the `--privileged` switch or modify the Docker's seccomp settings.
- I profiled my workload in a Docker container running Ubuntu 20+ running on top of a host system running CentOS with a kernel version `<3.10.0-693`. The `nsys status --environment` command indicated that CPU profiling was supported. The host OS kernel version determines if CPU profiling is allowed and a CentOS host with a version `<3.10.0-693` is too old. In this case, the `nsys status --environment` command is incorrect.

NVIDIA Video Profiling#

NVIDIA Video Hardware Profiling#

Limitations/Requirements#

NVIDIA Video Hardware profiling requires:

- Linux (x86_64 or Arm) and Windows (x86_64)
- Only covers desktop platforms
- Driver version ≥ 535
- GPU architecture Turing+

No NVIDIA Video Hardware profiling for:

- Mobile platforms
- Driver version < 535
- GPU architecture $< \text{Turing}$
- GSP is enabled and Driver < 545.31
- MIG is enabled
- Confidential computing is enabled
- vGPU software < 18.0

To learn more about GSP and on which GPUs it's enabled by default, see the following [link](#).

To turn off GSP permanently:

```
sudo su -c 'echo options nvidia NVreg_EnableGpuFirmware=0 > /etc/modprobe.d/nvidia-gsp.conf'
sudo update-initramfs -u # for Ubuntu-based systems
```

Then reboot.

Alternatively if you do not wish to reboot, this will disable until the next reboot:

```
sudo rmmod nvidia_uvm nvidia_drm nvidia_modeset nvidia && \
sudo insmod /lib/modules/$(uname -r)/updates/dkms/nvidia.ko NVreg_EnableGpuFirmware=0
for i in $(seq 0 7); do sudo nvidia-smi -i $i -pm ENABLED; done
```

Running from the CLI[#]

The feature is enabled through the `--gpu-video-device` option. It is available from the `nsys profile`, `nsys launch` and `nsys start` commands.

The option behaves exactly like `--gpu-metrics-device` and accepts the following arguments:

- `--gpu-video-device help` - List supported devices and their IDs, List unsupported devices (if any) and the reason.
- `--gpu-video-device none` - Turn the feature off.
- `--gpu-video-device all` - Turn the feature on on all supported devices. An error is returned if no devices support the feature.
- `--gpu-video-device <id1,id2,...>` - Turn the feature on the specified devices. The ID corresponds to what `help` returns. An error is returned if the ID is invalid.

Example:

```
$ nsys profile --gpu-video-device help
```

Possible `--gpu-video-device` values are:

0: NVIDIA GeForce RTX 3070 PCI[0000:65:00.0]

all: Select all supported GPUs

none: Disable GPU video accelerator tracing [Default]

Some GPUs don't support video accelerator tracing:

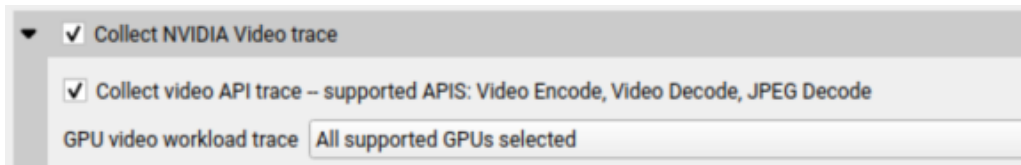
Quadro P620 PCI[0000:04:00.0] (reason = Arch Pascal < Turing)

See the user guide: <https://docs.nvidia.com/nsight-systems/UserGuide/index.html>

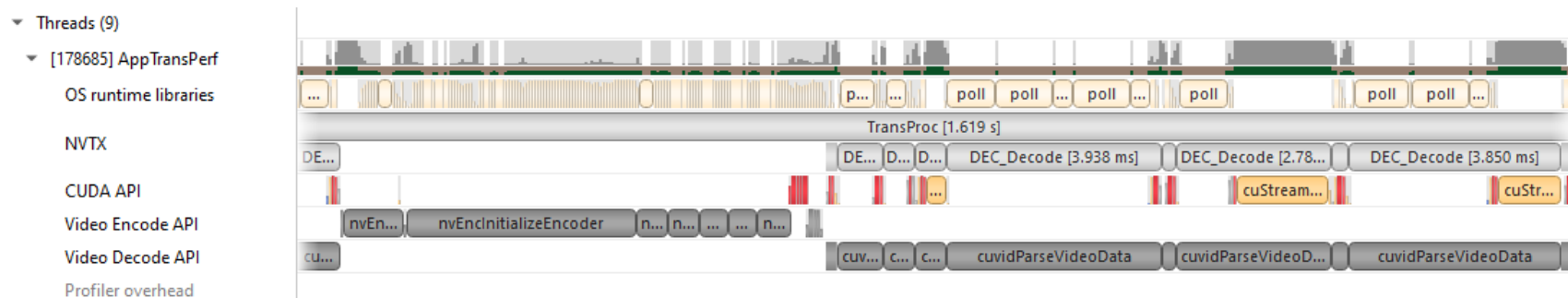
Note that this is a system-wide feature; i.e., it doesn't require a program to be launched.

NVIDIA Video Codec SDK Trace#

Nsight Systems for x86 Linux and Windows targets can trace calls from the NVIDIA Video Codec SDK. This software trace can be launched from the GUI or using the `--trace nvvideo` from the CLI



On the timeline, calls on the CPU to the NVIDIA Encoder API, NVIDIA Decoder API, and NVIDIA nvJPEG API will be shown.



NV Encoder API Functions Traced by Default#

NvEncodeAPICreateInstance
nvEncOpenEncodeSession
nvEncGetEncodeGUIDCount
nvEncGetEncodeGUIDs
nvEncGetEncodeProfileGUIDCount
nvEncGetEncodeProfileGUIDs
nvEncGetInputFormatCount
nvEncGetInputFormats
nvEncGetEncodeCaps

nvEncGetEncodePresetCount
nvEncGetEncodePresetGUIDs
nvEncGetEncodePresetConfig
nvEncGetEncodePresetConfigEx
nvEncInitializeEncoder
nvEncCreateInputBuffer
nvEncDestroyInputBuffer
nvEncCreateBitstreamBuffer
nvEncDestroyBitstreamBuffer
nvEncEncodePicture
nvEncLockBitstream
nvEncUnlockBitstream
nvEncLockInputBuffer
nvEncUnlockInputBuffer
nvEncGetEncodeStats
nvEncGetSequenceParams
nvEncRegisterAsyncEvent
nvEncUnregisterAsyncEvent
nvEncMapInputResource
nvEncUnmapInputResource
nvEncDestroyEncoder
nvEncInvalidateRefFrames
nvEncOpenEncodeSessionEx
nvEncRegisterResource
nvEncUnregisterResource
nvEncReconfigureEncoder
nvEncCreateMVBuffer
nvEncDestroyMVBuffer

nvEncRunMotionEstimationOnly
nvEncGetLastErrorString
nvEncSetIOCudaStreams
nvEncGetSequenceParamEx

NV Decoder API Functions Traced by Default#

cuidCreateVideoSource
cuidCreateVideoSourceW
cuidDestroyVideoSource
cuidSetVideoSourceState
cudaVideoState
cuidGetSourceVideoFormat
cuidGetSourceAudioFormat
cuidCreateVideoParser
cuidParseVideoData
cuidDestroyVideoParser
cuidCreateDecoder
cuidDestroyDecoder
cuidDecodePicture
cuidGetDecodeStatus
cuidReconfigureDecoder
cuidMapVideoFrame
cuidUnmapVideoFrame
cuidMapVideoFrame64
cuidUnmapVideoFrame64
cuidCtxLockCreate
cuidCtxLockDestroy
cuidCtxLock

cuvidCtxUnlock

NV JPEG API Functions Traced by Default<#>

nvjpegBufferDeviceCreate
nvjpegBufferDeviceDestroy
nvjpegBufferDeviceRetrieve
nvjpegBufferPinnedCreate
nvjpegBufferPinnedDestroy
nvjpegBufferPinnedRetrieve
nvjpegCreate
nvjpegCreateEx
nvjpegCreateSimple
nvjpegDecode
nvjpegDecodeBatched
nvjpegDecodeBatchedEx
nvjpegDecodeBatchedInitialize
nvjpegDecodeBatchedPreAllocate
nvjpegDecodeBatchedSupported
nvjpegDecodeBatchedSupportedEx
nvjpegDecodeJpeg
nvjpegDecodeJpegDevice
nvjpegDecodeJpegHost
nvjpegDecodeJpegTransferToDevice
nvjpegDecodeParamsCreate
nvjpegDecodeParamsDestroy
nvjpegDecodeParamsSetAllowCMYK
nvjpegDecodeParamsSetOutputFormat
nvjpegDecodeParamsSetROI

nvjpegDecodeParamsSetScaleFactor
nvjpegDecoderCreate
nvjpegDecoderDestroy
nvjpegDecoderJpegSupported
nvjpegDecoderStateCreate
nvjpegDestroy
nvjpegEncodeGetBufferSize
nvjpegEncodeImage
nvjpegEncodeRetrieveBitstream
nvjpegEncodeRetrieveBitstreamDevice
nvjpegEncoderParamsCopyHuffmanTables
nvjpegEncoderParamsCopyMetadata
nvjpegEncoderParamsCopyQuantizationTables
nvjpegEncoderParamsCreate
nvjpegEncoderParamsDestroy
nvjpegEncoderParamsSetEncoding
nvjpegEncoderParamsSetOptimizedHuffman
nvjpegEncoderParamsSetQuality
nvjpegEncoderParamsSetSamplingFactors
nvjpegEncoderStateCreate
nvjpegEncoderStateDestroy
nvjpegEncodeYUV,(nvjpegHandle_t handle
nvjpegGetCudartProperty
nvjpegGetDeviceMemoryPadding
nvjpegGetImageInfo
nvjpegGetPinnedMemoryPadding
nvjpegGetProperty
nvjpegJpegStateCreate

nvjpegJpegStateDestroy
nvjpegJpegStreamCreate
nvjpegJpegStreamDestroy
nvjpegJpegStreamGetChromaSubsampling
nvjpegJpegStreamGetComponentDimensions
nvjpegJpegStreamGetComponentNum
nvjpegJpegStreamGetFrameDimensions
nvjpegJpegStreamGetJpegEncoding
nvjpegJpegStreamParse
nvjpegJpegStreamParseHeader
nvjpegSetDeviceMemoryPadding
nvjpegSetPinnedMemoryPadding
nvjpegStateAttachDeviceBuffer
nvjpegStateAttachPinnedBuffer

Network Communication Profiling[#]

Nsight Systems can be used to profile several popular network communication protocols and many network hardware components. To enable this, please select the **Communication profiling options** dropdown.

Note

Network hardware profiling uses statistical sampling of counters on the various appliances. Network communication API profiling uses direct trace of relevant function calls. Nsight Systems correlates the data as well as we can, however the inherent profiling differences make correlation somewhat inexact.

▶ ☒ Collect CPU IP/backtrace samples

▶ ☒ Collect CPU context switch trace

▶ ☒ Collect OS runtime libraries trace

▶ ☒ Collect CUDA trace

▶ ☐ Collect OpenMP trace

▶ ☐ Collect GPU context switch trace

▶ ☐ Collect GPU metrics

▶ ☐ Collect NV Video trace

▶ ☒ Collect NVTX trace

▶ ☐ Collect OpenGL trace

▶ ☐ Collect Vulkan trace

▶ Network profiling options

▶ Python profiling options

Then select the libraries you would like to trace:

▼ Network profiling options

Trace API calls into communication libraries.

▼ ☐ MPI

Select the MPI implementation used by the target application to trace a default set of MPI calls. If no MPI implementation is selected, NVIDIA Nsight Systems tries to automatically detect it based on the dynamic linker's search path. If this fails, Open MPI is used. If the application uses another MPI implementation, see the documentation for additional setup required to trace MPI.

☐ Open MPI

☐ MPICH and its derivatives

▼ ☐ OpenSHMEM

OpenSHMEM is a library interface specification for parallel programming in the Partitioned Global Address Space (PGAS). NVIDIA Nsight Systems supports collecting and visualizing a default set of OpenSHMEM API calls.

▼ ☐ UCX

Unified Communication X (UCX) is an open-source communication framework used by several higher level communication libraries, e.g. Open MPI, MPICH, OpenSHMEM and NVSHMEM. The Unified Communication Protocol (UCP) is its high-level API.

☐ Trace UCP communication submit calls

☐ Trace UCP communication progress

☐ Collect UCP communication parameters

Profile Network Interface Cards (NICs).

☐ Collect NIC metrics

Profile InfiniBand network switches.

☐ Collect IB Switch performance metrics

The corresponding Nsight Systems CLI `--trace | -t` options are `mpi`, `oshmem` and `ucx`. For multi-node runs, please refer to section on **Handling Application Launchers** in the **Profiling From the CLI** topic.

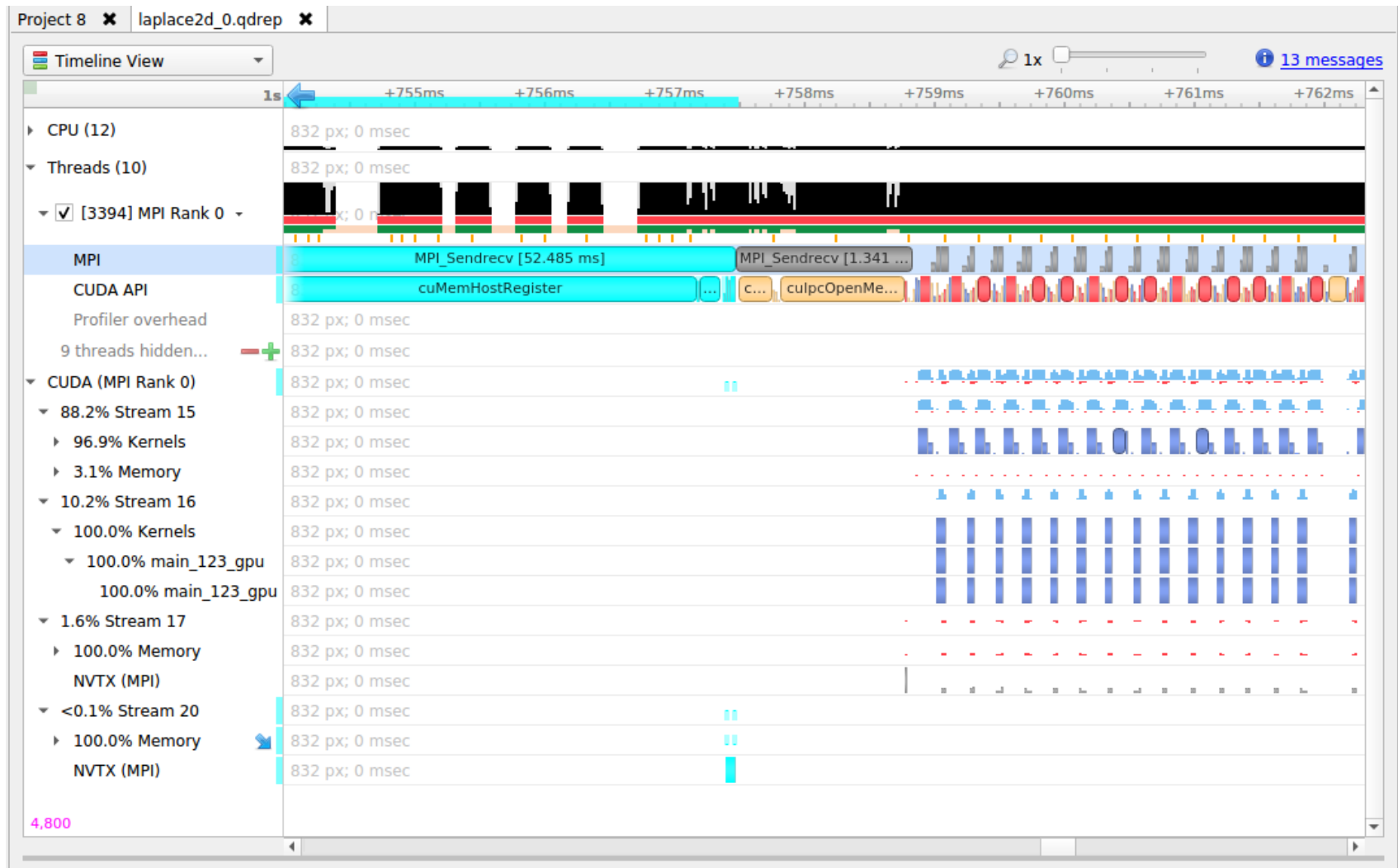
MPI API Trace#

Nsight Systems has built-in API trace support for Open MPI and MPICH based MPI implementations via `--trace=mpi` or by selecting the *MPI* checkbox under *Network profiling options*. If the auto-detection of the MPI implementation fails, it is possible to specify it via `--mpi-impl=[openmpi|mpich]` or the respective checkbox in the GUI.

Nsight Systems will trace a subset of the MPI API, including blocking and non-blocking point-to-point and collective communications as well as MPI one-sided communications, file I/O, and pack operations (see [MPI functions traced](#)).

If you require more control over the list of traced APIs or if you are using a different MPI implementation, you can use the [NVTX wrappers for MPI](#) on GitHub. Choose an NVTX domain name other than “MPI,” since it is filtered out by Nsight Systems when MPI tracing is not enabled. Use the NVTX-instrumented MPI wrapper library as follows:

```
nsys profile -e LD_PRELOAD=${PATH_TO_YOUR_NVTX_MPI_LIB} --trace=nvtx
```



Note

If not all ranks are traced, `NSYS_MPI_STORE_TEAMS_PER_RANK` has to be set to 1. If communicator tracking is still causing issues, it can be disabled by setting `NSYS_MPI_DISABLE_COMMUNICATOR_TRACKING=1`.

MPI Communication Parameters#

Nsight Systems can get additional information about MPI communication parameters. Currently, the parameters are only visible in the mouseover tooltips or in the event log. This means that the data is only available via the GUI. Future versions of the tool will export this information into the SQLite data files for postrun analysis.

In order to fully interpret MPI communications, data for all ranks associated with a communication operation must be loaded into Nsight Systems.

Here is an example of `MPI_COMM_WORLD` data. This does not require any additional team data, since local rank is the same as global rank.

(Screenshot shows communication parameters for an `MPI_Bcast` call on rank 3.)

The screenshot displays the Nsight Systems GUI. At the top left, there is a dropdown menu labeled "Events View". Below it, a table lists MPI events. The third row, "MPI_Bcast", is highlighted. To the right of the table, a "Description:" panel provides details for the selected event.

Name	Description:
MPI_Init	
MPI_Recv	
MPI_Bcast	MPI_Bcast Begins: 0,164935s Ends: 0,165045s (+109,210 μ s) Thread: 1342434 Bytes sent: 0 Bytes received: 4 Root: 0 MPI_COMM_WORLD
MPI_Bcast	
MPI_Recv	
MPI_Finalize	

When not all processes that are involved in an MPI communication are loaded into Nsight Systems the following information is available.

- Right-hand screenshot shows a reused communicator handle (last number increased).
- Encoding: `MPI_COMM[\*team size\*]*global-group-root-rank\*.*group-ID\*`

Events View

#	Name	Start
3	MPI_Bcast	0,06235
4	MPI_Send	0,06262
5	MPI_Bcast	0,06262
6	MPI_Send	0,06272
7	MPI_Bcast	0,06272
8	MPI_Finalize	0,06272

Name

Description:

MPI_Send

Begin: 0,062627s

End: 0,0626288s (+1,771 μs)

Thread: 978418

Tag: 42

Bytes sent: 4

Destination: 1

MPI_COMM[2]1.0

Events View

#	Name	Start
3	MPI_Bcast	0,06235
4	MPI_Send	0,06262
5	MPI_Bcast	0,06262
6	MPI_Send	0,06272
7	MPI_Bcast	0,06272
8	MPI_Finalize	0,06272

Name

Description:

MPI_Send

Begin: 0,0627267s

End: 0,062728s (+1,314 μs)

Thread: 978418

Tag: 42

Bytes sent: 4

Destination: 1

MPI_COMM[2]1.1

When all reports are loaded into Nsight Systems:

- World rank is shown in addition to group-local rank “(world rank X).”
- Encoding: `MPI_COMM[\*team size*\]{rank0, rank1, ...}`.
- At most 8 ranks are shown (the numbers represent world ranks, the position in the list is the group-local rank).

Events View

Name

Name

MPI_Init

Name

MPI_Recv

Name

MPI_Bcast

Name

MPI_Bcast

Name

MPI_Recv

Name

MPI_Send

Name

MPI_Recv

Events View

Name

Description:

MPI_Recv

Begins: 0,16549s

Ends: 0,165492s (+1,837 μs)

Thread: 1047429

Tag: 42

Bytes received: 4

Source: 0 (world rank 2)

MPI_COMM[2]{2, 3}

#

1

2

3

4

5

6

7

Name

MPI_Init

MPI_Recv

MPI_Bcast

MPI_Bcast

MPI_Recv

MPI_Send

MPI_Recv

Events View

Name

Description:

MPI_Send

Begins: 0,165609s

Ends: 0,165612s (+2,577 μs)

Thread: 1047429

Tag: 48

Bytes sent: 4

Destination: 7 (world rank 2)

MPI_COMM[10]{9, 8, 7, 6, 5, 4, 3, 2, ...}

#

1

2

3

4

5

6

7

Name

MPI_Init

MPI_Recv

MPI_Bcast

MPI_Bcast

MPI_Recv

MPI_Send

MPI_Recv

MPI functions traced#

MPI_Init[_thread], MPI_Finalize
MPI_Send, MPI_{B,S,R}send, MPI_Recv, MPI_Mrecv
MPI_Sendrecv[_replace]

MPI_Barrier, MPI_Bcast
MPI_Scatter[v], MPI_Gather[v]
MPI_Allgather[v], MPI_Alltoall[{v,w}]
MPI_Allreduce, MPI_Reduce[_{scatter,scatter_block,local}]
MPI_Scan, MPI_Exscan

MPI_Isend, MPI_{b,s,r}send, MPI_I[m]recv
MPI_{Send,Bsend,Ssend,Rsend,Recv}_init
MPI_Start[all]
MPI_Ibarrier, MPI_Ibcast
MPI_Iscatter[v], MPI_Igather[v]
MPI_Iallgather[v], MPI_Ialltoall[{v,w}]
MPI_Iallreduce, MPI_Ireduce[{scatter,scatter_block}]
MPI_I[ex]scan
MPI_Wait[{all,any,some}]

MPI_Put, MPI_Rput, MPI_Get, MPI_Rget
MPI_Accumulate, MPI_Raccumulate
MPI_Get_accumulate, MPI_Rget_accumulate
MPI_Fetch_and_op, MPI_Compare_and_swap

MPI_Win_allocate[_shared]
MPI_Win_create[_dynamic]
MPI_Win_{attach,detach}

MPI_Win_free
MPI_Win_fence
MPI_Win_{start, complete, post, wait}
MPI_Win_[un]lock[_all]
MPI_Win_flush[_local][_all]
MPI_Win_sync

MPI_File_{open,close,delete,sync}
MPI_File_{read,write}[_{all,all_begin,all_end}]
MPI_File_{read,write}_at[_{all,all_begin,all_end}]
MPI_File_{read,write}_shared
MPI_File_{read,write}_ordered[_{begin,end}]
MPI_File_{read,write}[_{all,at,at_all,shared}]
MPI_File_set_{size,view,info}
MPI_File_get_{size,view,info,group,amode}
MPI_File_preallocate

MPI_Pack[_external]
MPI_Unpack[_external]

OpenSHMEM Library Trace#

If OpenSHMEM library trace is selected Nsight Systems will trace the subset of OpenSHMEM API functions that are most likely be involved in performance bottlenecks. To keep overhead low Nsight Systems does not trace all functions.

OpenSHMEM 1.5 Functions Not Traced

shmem_my_pe
shmem_n_pes

shmem_global_exit
shmem_pe_accessible
shmem_addr_accessible
shmem_ctx_{create,destroy,get_team}
shmem_global_exit
shmem_info_get_{version,name}
shmem_{my_pe,n_pes,pe_accessible,ptr}
shmem_query_thread
shmem_team_{create_ctx,destroy}
shmem_team_get_config
shmem_team_{my_pe,n_pes,translate_pe}
shmem_team_split_{2d,strided}
shmem_test*

UCX API Trace#

If UCX API trace is selected Nsight Systems will trace the subset of functions of the UCX protocol layer UCP that are most likely be involved in performance bottlenecks. To keep overhead low Nsight Systems does not trace all functions.

The following environment variables control what is recorded:

- **NSYS_UCP_COMM_SUBMIT**: (enabled by default) If set to 0, UCP communication submission calls are not recorded any more. These calls are usually short, because the communication itself is handled in a worker thread.
- **NSYS_UCP_COMM_PROGRESS**: (enabled by default) If set to 0, tracking of (process-local) UCP communication progress is disabled. The progress tracking uses UCP completion callbacks.
- **NSYS_UCP_COMM_PARAMS**: (enabled by default) If set to 0, UCP communication parameters (tag, remote worker UID, packed message size, buffer address) will not be recorded. Recording the remote worker UID requires UCX >= 1.12.0. Recording the packed message size requires UCX >= 1.14.0.

UCX functions traced#

ucp_am_send_nb[x]
ucp_am_recv_data_nbx
ucp_am_data_release
ucp_atomic_{add{32,64},cswap{32,64},fadd{32,64},swap{32,64}}
ucp_atomic_{post,fetch_nb,op_nbx}
ucp_cleanup
ucp_config_{modify,read,release}
ucp_disconnect_nb
ucp_dt_{create_generic,destroy}
ucp_ep_{create,destroy,modify_nb,close_nbx}
ucp_ep_flush[{_nb,_nbx}]
ucp_listener_{create,destroy,query,reject}
ucp_mem_{advise,map,unmap,query}
ucp_{put,get}[_nbi]
ucp_{put,get}_nb[x]
ucp_request_{alloc,cancel,is_completed}
ucp_rkey_{buffer_release,destroy,pack,ptr}
ucp_stream_data_release
ucp_stream_recv_data_nb
ucp_stream_{send,recv}_nb[x]
ucp_stream_worker_poll
ucp_tag_msg_recv_nb[x]
ucp_tag_{send,recv}_nbr
ucp_tag_{send,recv}_nb[x]
ucp_tag_send_sync_nb[x]
ucp_worker_{create,destroy,get_address,get_efd,arm,fence,wait,signal,wait_mem}
ucp_worker_flush[{_nb,_nbx}]

ucp_worker_set_am_{handler,recv_handler}

UCX Functions Not Traced:

ucp_config_print

ucp_conn_request_query

ucp_context_{query,print_info}

ucp_get_version[_string]

ucp_ep_{close_nb,print_info,query,rkey_unpack}

ucp_mem_print_info

ucp_request_{check_status,free,query,release,test}

ucp_stream_recv_request_test

ucp_tag_probe_nb

ucp_tag_recv_request_test

ucp_worker_{address_query,print_info,progress,query,release_address}

Additional API functions from other UCX layers may be added in a future version of the product.

NVIDIA NVSHMEM and NCCL Trace<#>

The NVIDIA network communication libraries NVSHMEM and NCCL have been instrumented using NVTX annotations. To enable tracing these libraries in Nsight Systems, turn on NVTX tracing in the GUI or CLI. To enable the NVTX instrumentation of the NVSHMEM library, make sure that the environment variable NVSHMEM_NVTX is set properly; e.g., NVSHMEM_NVTX=common.

NIC Metric Sampling<#>

Overview

NVIDIA ConnectX smart network interface cards (smart NICs) offer advanced hardware offloads and accelerations for network operations. Viewing smart NICs metrics, on Nsight Systems timeline, enables developers to better understand their application's network usage. Developers can use this information to optimize the application's

performance.

Limitations/Requirements

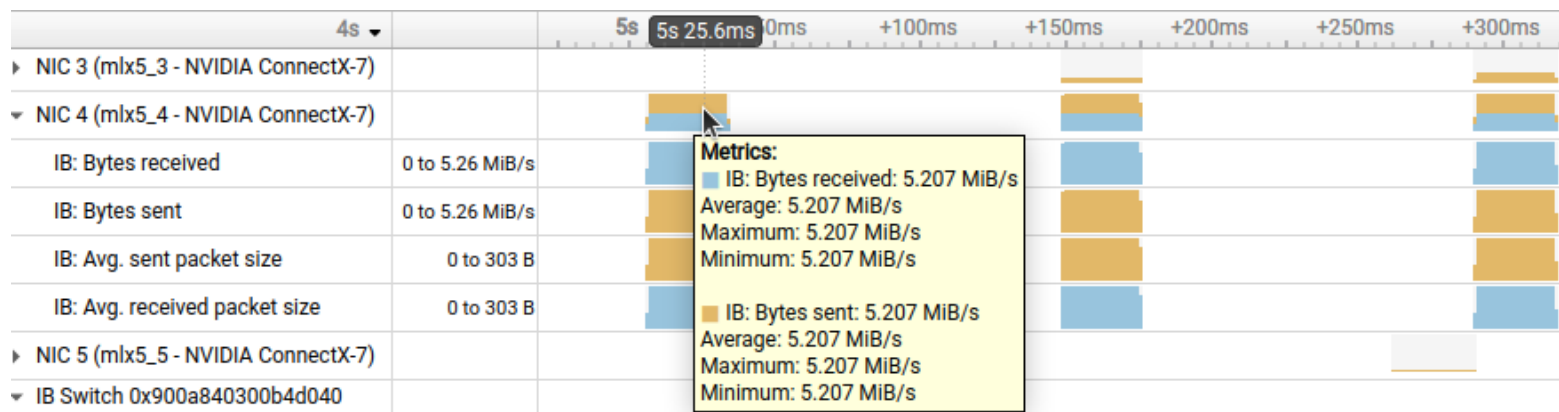
- NIC metric sampling supports NVIDIA ConnectX boards starting with ConnectX 5
- NIC metric sampling is supported on Linux x86_64 and Arm Server (SBSA) machines only, having minimum Linux kernel 4.12 and minimum MLNX_OFED 4.1. You can download the latest and archived versions of the MLX_OFED driver from the [MLNX_OFED Download Center](#). If collecting NIC metrics within a container, make sure that the container has access to the driver on the host machine. To check manually if OFED is installed and get its version you can run:
 - `/usr/bin/ofed_info`
 - `cat /sys/module/"$(cat /proc/modules | grep -o -E "^mlx._core")"/version`

To check if the target system meets the requirements for NIC metrics collection you can run `nsys status --network`.

Collecting NIC Metrics Using the Command Line

To collect NIC performance metrics, using Nsight Systems CLI, add the `--nic-metrics` command line switch:

`nsys profile --nic-metrics=true my_app`



Available Metrics

- **Bytes sent** - Number of bytes sent through all NIC ports.
- **Bytes received** - Number of bytes received by all NIC ports.
- **Average sent packet size** - Average byte size of packets sent through all NIC ports.
- **Average received packet size** - Average byte size of packets received by all NIC ports.
- **CNPs sent** - Number of congestion notification packets sent by the NIC.
- **CNPs received** - Number of congestion notification packets received and handled by the NIC.
- **Send waits** - The number of ticks during which ports had data to transmit but no data was sent during the entire tick (either because of insufficient credits or because of lack of arbitration)

Note

Each one of the mentioned metrics is shown only if it has non-zero value during profiling.

Usage Examples

- The Bytes sent/sec and the Bytes received/sec metrics enables identifying idle and busy NIC times.
- Developers may shift network operations from busy to idle times to reduce network congestion and latency.
- Developers can use idle NIC times to send additional data without reducing application performance.
- CNPs (congestion notification packets) received/sent and Send waits metrics may explain network latencies. A developer seeing the time periods when the network was congested may rewrite his algorithm to avoid the observed congestions.

InfiniBand Switch Metric Sampling#

NVIDIA Quantum InfiniBand switches offer high-bandwidth, low-latency communication. Viewing switch metrics, on Nsight Systems timeline, enables developers to better understand their application's network usage. Developers can use this information to optimize the application's performance.

Limitations/Requirements

IB switch metric sampling supports all NVIDIA Quantum switches. The user needs to have permission to query the InfiniBand switch metrics.

To check if the current user has permissions to query the InfiniBand switch metrics, check that the user have permission to access `/dev/infiniband/umad*`

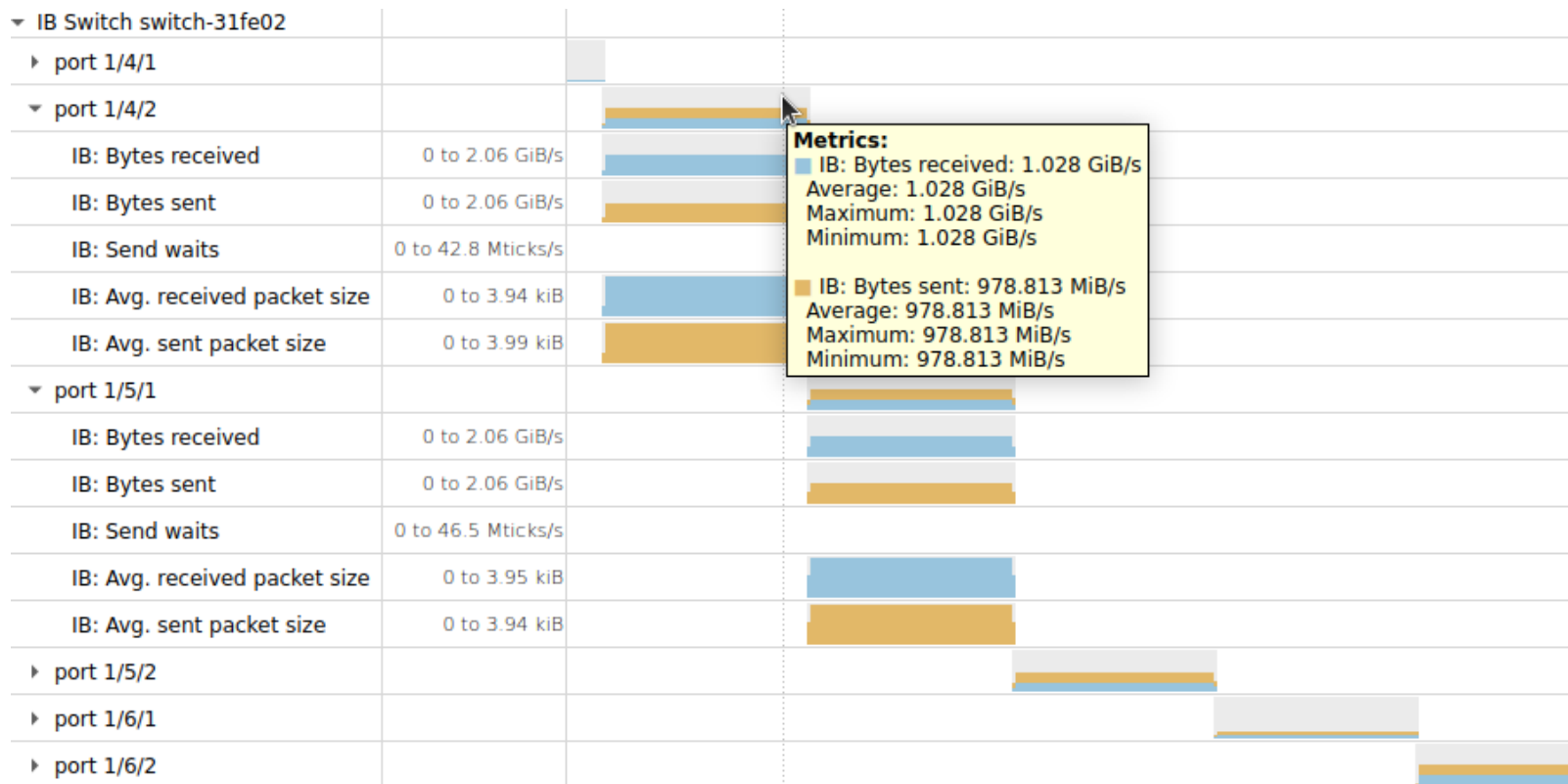
To give user permissions to query InfiniBand switch metrics on RedHat systems, follow the directions at [RedHat Solutions](#).

To collect InfiniBand switch performance metric, using Nsight Systems CLI, add the `--ib-switch-metrics-device` command line switch, followed by a comma separated list of InfiniBand switch GUIDs. For example:

```
nsys profile --ib-switch-metrics-device=<IB switch GUID> my_app
```

To get a list of InfiniBand switches, reachable by a given NIC, use:

```
sudo ibswitches -C <nic name>
```

Available Metrics

- **Bytes sent** - Number of bytes sent through all switch ports
- **Bytes received** - Number of bytes received by all switch ports
- **Send waits** - The number of ticks during which switch ports, selected by PortSelect, had data to transmit but no data was sent during the entire tick (either because of insufficient credits or of lack of arbitration)
- **Average sent packet size** - Average sent InfiniBand packet size
- **Average received packet size** - Average received InfiniBand packet size

InfiniBand Switch Congestion Events#

Overview#

NVIDIA Quantum InfiniBand switches offer high-bandwidth, low-latency communication.

When a switch egress port is congested, packets wait in the egress port queue before being sent out of the switch. This increases the latency of these packets.

Nsight Systems Workstation Edition gives you the ability to view when switch egress ports are congested on the Nsight Systems timeline. This enables developers to better understand latencies that are caused by the application's network usage. Developers can use this information to optimize the application's performance.

Limitations/Requirements#

IB switch congestion events support requires:

- Quantum 2 switch or newer
- Having firmware version 31.2012.1068 or higher
- User need to have permission to send management datagrams

To get a list of InfiniBand switches, reachable by a given NIC, use: `sudo ibswitches -C <nic name>`

To check if the current user has permissions to send management datagrams, check that the user have permission to access `/dev/infiniband/umad*`

To give user permissions to query InfiniBand switch congestion events on RedHat systems, follow the directions at [RedHat Solutions](#).

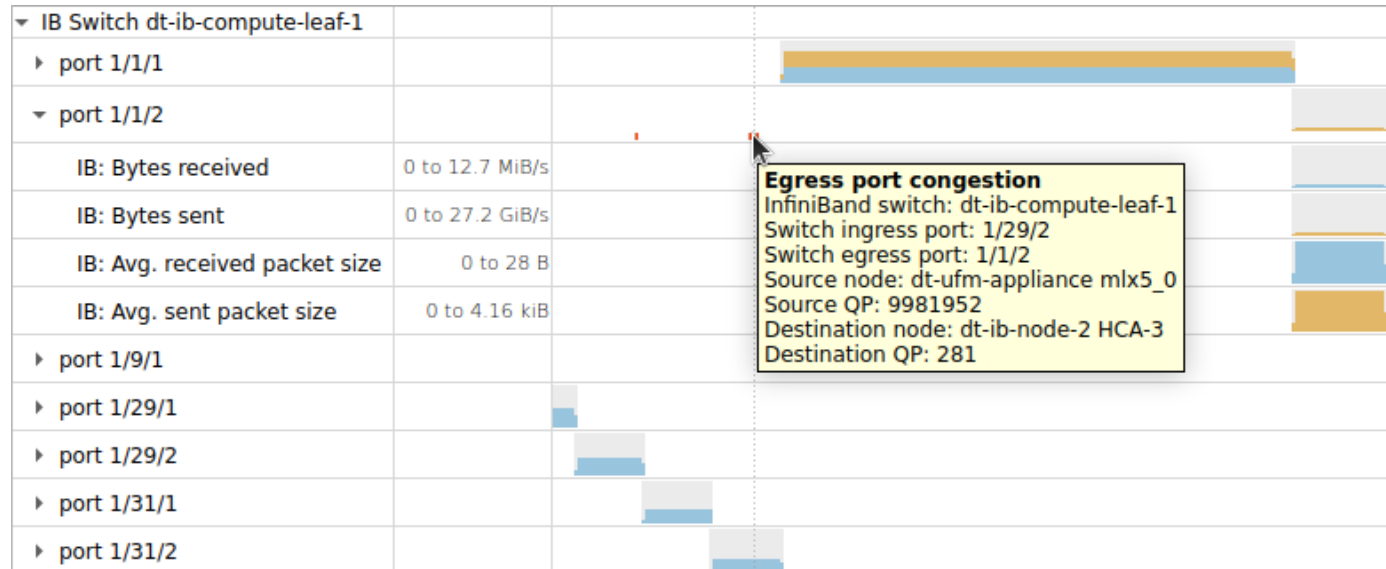
Using the Command Line#

To collect InfiniBand switch congestion events, using Nsight Systems CLI, add the following command line switches:

- `ib-switch-congestion-device` This should be followed by a comma separated list of InfiniBand switch GUIDs,

from which congestion events will be collected.

- `ib-switch-congestion-nic-device` This should be followed by the name of the NIC (HCA) through which InfiniBand switches will be accessed. The profiled InfiniBand switches should be reachable by this NIC.
- `ib-switch-congestion-percent` This defines the percent of InfiniBand switch congestion events to be collected. This option enables reducing the network bandwidth consumed by reporting congestion events. Values are in the [1,100] range.
- `ib-switch-congestion-threshold-high` This defines the high threshold for InfiniBand switch egress port queue size. When a packet enters an InfiniBand switch, its data is stored at an ingress port buffer. A pointer to the packet's data is inserted into the egress port's queue, from which the packet will be exiting the switch. At that point, the threshold given by this command switch is compared to the egress queue data size. If the queue data size exceeds the threshold, a congestion event is reported. The threshold is given in percent of the ingress port size. An egress port queue can point to data coming from multiple ingress port buffers, therefore the threshold can be bigger than 100%. Values are in the (1,1023] range



Overview

By default, Nsight Systems displays low-level identifiers like LIDs (Local Identifiers) and GUIDs (Globally Unique Identifiers). Instead, Nsight Systems can leverage InfiniBand network information to display the actual names of nodes and switches. This makes the Nsight Systems reports much more intuitive and easier to understand at a glance.

InfiniBand network information discovery is done using the `ibdiagnet` utility. Either:

- Run `ibdiagnet` and store the generated network information files to be later used by Nsight Systems.
- This method is useful for large networks, where `ibdiagnet`'s network discovery time may be long, and for networks where only administrators have permissions to query the network information.
- A user can ask Nsight Systems to run `ibdiagnet` to collect the network information during the profiling session.
- This method is useful for small networks.

Limitations/Requirements

The user needs to have permission to send MADs (management datagrams). To check if you have permission to send MADs, check if you can access the `/dev/infiniband/umad*` files. To give user permissions to send MADs on RedHat systems, follow the directions at [RedHat Solutions](#).

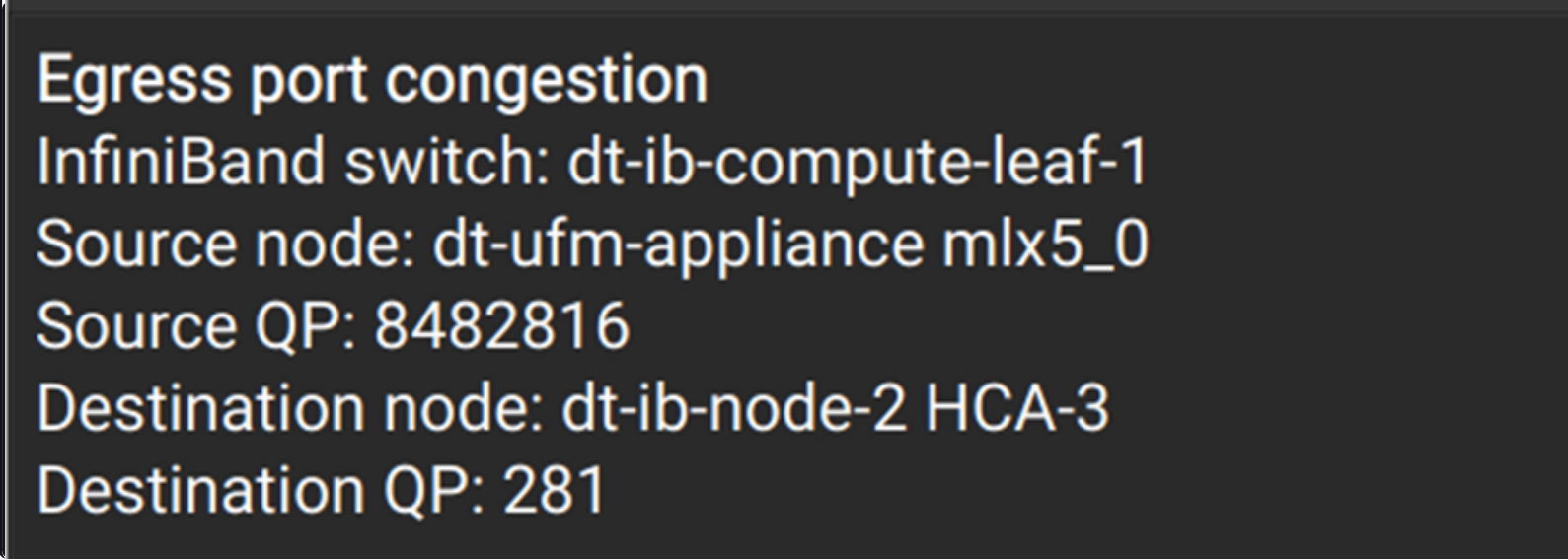
Relevant Switches

The following Nsight Systems command line switches enable collecting InfiniBand network information:

- `ib-net-info-devices`
This should be followed by a comma separated list of NIC names, from which `ibdiagnet` will run network discovery. The results of the network discovery will be automatically loaded into Nsight Systems.
- `ib-net-info-files`
This should be followed by a comma separated list of pre-generated `ibdiagnet db_csv` file paths, which Nsight Systems will read.

- `ib-net-info-output`

This should be followed by a path of a directory into which Nsight Systems will store the ibdiagnet network discovery data. These files will be used by the `ib-net-info-devices` command line switch. This command line switch can only be used together with the `ib-net-info-devices` command line switch.



```
Egress port congestion
InfiniBand switch: dt-ib-compute-leaf-1
Source node: dt-ufm-appliance mlx5_0
Source QP: 8482816
Destination node: dt-ib-node-2 HCA-3
Destination QP: 281
```

The above image displays a congestion event. InfiniBand network information is used for displaying node and switch names instead of LIDs.

Amazon AWS EFA Metrics#

Nsight Systems can now periodically sample performance counters for AWS Elastic Fabric Adapters (EFAs) and plot it on the timeline in the GUI. This enables developers to analyze how network communications may be involved with the critical path of their multi-node application. Created in collaboration with AWS, this plugin will work on [AWS EC2 NVIDIA GPU accelerated compute instances](#) .

To enable the AWS EFA metrics add the following option to the `nsys profile` or `start` commands:

```
--enable efa_metrics[,arg1[=value1],arg2[=value2], ...]
```

There are no spaces following `efa_metrics` plugin name. It is followed by a comma separated list of arguments or

argument=value pairs. Arguments with spaces should be enclosed in double quotes.

Supported arguments are:

Name	Possible Parameters	Default	Switch Description
-efa-non-rdma	true, false	false	Sample Infiniband non-RDMA counters
-efa-sysfs	<path>	/sys/class/infiniband	Root directory for EFA counters sysfs
-efa-work-requests	true, false	false	Sample Infiniband WorkRequest counters
-errors	true, false	false	Sample error counters
-freq	integer, a negative value means 1/F frequency	10	Target sample frequency in hertz
-mode	throughput, delta, total	throughput	Report sampled counters as a value per second, delta since previous sample, or an accumulated sum.
-packets	true, false	false	Sample packet counters

Usage Examples

- `nsys profile --enable efa_metrics ...`
Sample all EFA adapters, display as bytes per second.
- `nsys profile --enable efa_metrics,-packets,-errors,-efa-non-rdma ...`

Sample all available EFA adapter counters.

- `nsys profile --enable efa_metrics,-mode=total ...`

Sample all EFA adapters, display as total value sum since profiling start.

- `nsys profile --enable efa_metrics,-efa-counters-sysfs="/mnt/nv/sys", ...`

Look for EFA counters in a different sysfs directory. Useful in some k8s environments.

This collector is the first use case for the [Nsight Systems Plugins \(Preview\)](#) system.

Network Interface Metrics#

Nsight Systems can now periodically sample performance counters for network interface devices and plot them on the timeline in the GUI.

To enable the network devices metrics add the following option to the `nsys profile` or `start` commands:

`--enable network_interface[,arg1[=value1],arg2[=value2], ...]`

There are no spaces following `network_interface` plugin name. It is followed by a comma separated list of arguments or argument=value pairs. Arguments with spaces should be enclosed in double quotes.

Supported arguments are:

:name: table_netinterface_table :class: table-compact#				
Short name	Long name	Possible Parameters	Default	Switch Description
-i	--interval	integer	100000	Sampling interval in microseconds
-d	--device	regular expression	".+" (and filtering for physical devices)	Device(s) to sample

Short name	Long name	Possible Parameters	Default	Switch Description
-m	--metricid	regular expression	".*_bytes"	Metric(s) to sample
-h	--help			Print help message

Usage Examples

- `nsys profile --enable network_interface ...`
Sample bytes metrics for all physical network devices every 100ms.
- `nsys profile --enable network_interface,-dall ...`
Sample bytes metrics for all network devices every 100ms.
- `nsys profile --enable network_interface,-i10000,-dall,-m".+"`
Sample all metrics, for all network devices, every 10ms.

For general information on Nsight Systems plugins please refer to [Nsight Systems Plugins \(Preview\)](#) system.

Storage Metrics Profiling#

Nsight Systems can profile several major storage / remote storage protocols.

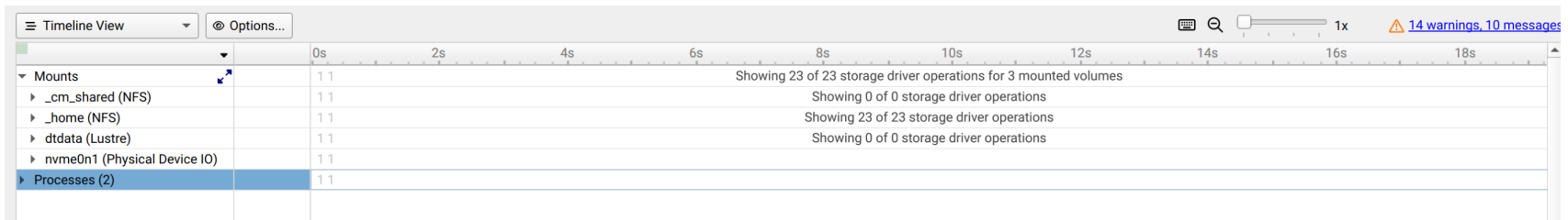
To activate this feature, use the Nsight Systems CLI `--storage-metrics` option, followed by a comma-separated list of the desired arguments.

Available arguments:

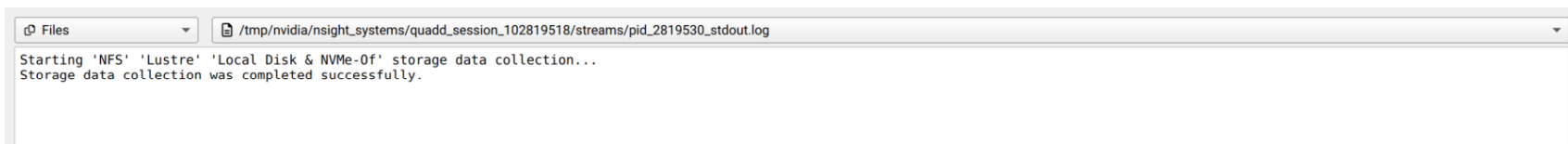
- `--nfs-volumes={all | volume1[,volume2][,volume3...]}`: enable NFS storage profiling for the specified volume(s) (specify `all` to profile all volumes).
- `--lustre-volumes={all | volume1[,volume2][,volume3...]}`: enable Lustre storage profiling for the specified volume(s) (specify `all` to profile all volumes).

- `--lustre-llite-dir=<path>`: specifies the path of the llite directory mount. This is the `/sys/kernel/debug/lustre/llite` directory mount point (mandatory if Lustre profiling is enabled).
- `--storage-devices={all | device1[,device2][,device3...]}`: enable storage profiling of the specified local storage or NVMeOF device(s) (specify `all` to profile all devices).
- `--gds-metrics={driver}`: enable GPUDirect Storage Kernel-Space metrics profiling.

Usage Examples



In the report file, under ‘Timeline view’, the storage metrics can be viewed in the **Mounts** section. Each row contains metrics for one volume or device, with the storage type next to the volume / device name. Expanding each row will show the collected metrics for that volume / device.



The `stdout` and `stderr` log files for the storage metrics collection process can be viewed under the ‘Files’ section, which may assist in debugging.

Exposing Lustre driver counters to non-privileged users

The Lustre driver exposes performance counters via virtual files residing under `/sys/kernel/debug/lustre`. However, this path is not accessible to non-privileged users.

To expose the Lustre counters to non-privileged users, a superuser should create a mount point to `/sys/kernel/debug/lustre`. For example:

```
su - root
```

```
mkdir /mnt/lustre-stats
```

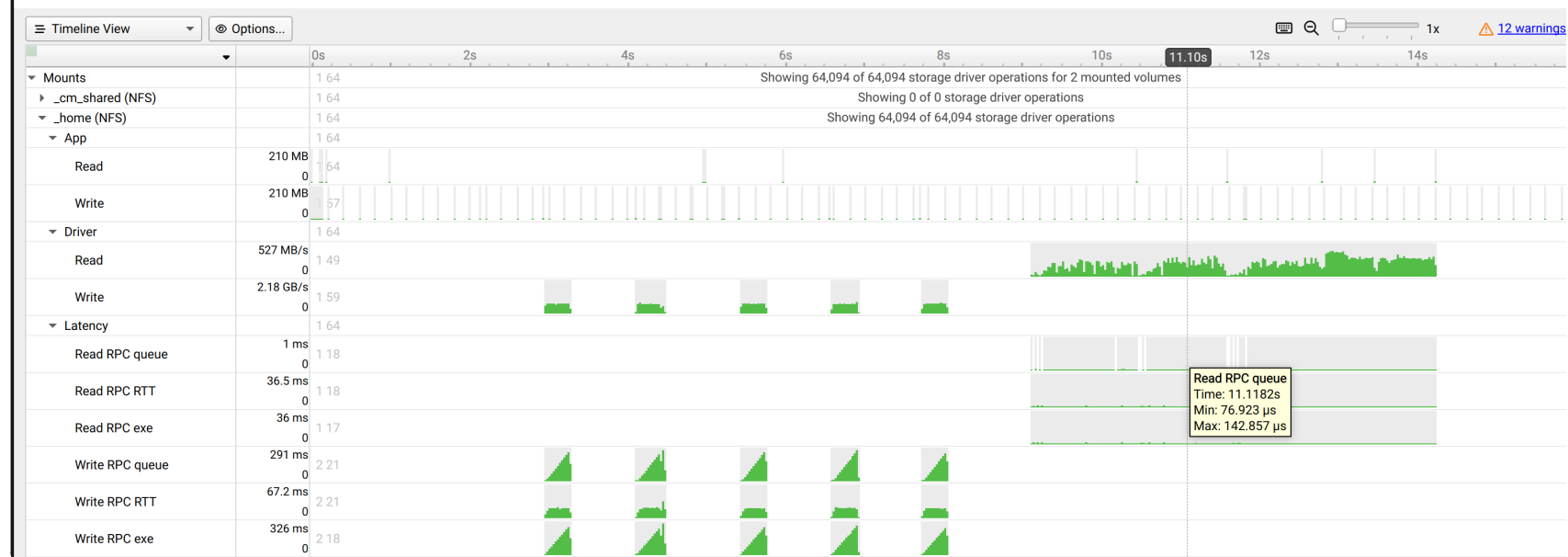
```
mount --bind /sys/kernel/debug/lustre /mnt/lustre-stats``
```

The `--lustre-llite-dir=` command line argument should point the `llite` directory under this mount point; this will enable Nsight Systems to read the Lustre counters. For example: `--lustre-llite-dir=/mnt/lustre-stats/llite`

1. NFS Storage example:

Example Nsight Systems command line for NFS storage profiling:

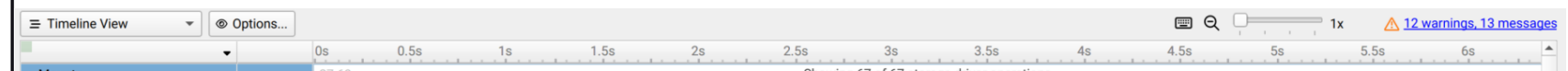
```
./nsys profile --storage-metrics --nfs-volumes=all <target-application>
```



2. Lustre Storage example:

Example Nsight Systems command line for Lustre storage profiling:

```
./nsys profile --storage-metrics --lustre-volumes=dtdata,--lustre-llite-dir=/mnt/lustre-stats/llite <target-application>
```



mounts		Showing 67 of 67 storage driver operations	
▼ dtdata (Lustre)	28 71		
▼ App	28 76		
Read	31.5 MB 0	25 65	
Write	31.5 MB 0	23 64	
▼ Operations	29 77		
read	1 Ops 0	23 65	
write	1 Ops 0	25 68	
close	1 Ops 0	24 67	
fsync	0 Ops	25 66	
getattr	2 Ops 0	23 66	
getxattr	1 Ops 0	22 63	
ioctl	1 Ops 0	23 64	

3. Local / NVMeOF Storage example:

Example Nsight Systems command line for local storage and NVMeOF device profiling:

```
./nsys profile --storage-metrics --storage-devices=all <target-application>
```



It is also possible to use combinations of these arguments to profile multiple storage protocols at once. For example:

```
./nsys profile --storage-metrics --nfs-volumes=all,--lustre-volumes=all,--storage-  
devices=<device_name1>,<device_name2>,--lustre-llite-dir=<path_to_llite_directory>  
<target-application>
```

Note

There are two types of Read/Write metrics

Application-level Read/Write - Displays **quantities** of data read/written to the storage device **by applications** (in Bytes).

Driver-level Read/Write - Displays **throughput** of data read/written to the storage device **by the driver** (in Bytes/sec).

For example, when an application uses the “write” POSIX function to write 10 MB of data into a file, the entire 10 MB will appear, in a single sampling point, at the Application-level Write counter. The same 10 MB of data may be spread across multiple Driver-level Write counter sampling points, since it may take a bit of time for the NFS driver to write 10 MB of data into the NFS storage server.

GPUDirect Storage installation and metrics collection

Note: Nsight Systems does not currently support GDS metrics collection on Ext4 and XFS file-systems with NVMe drives when using the “NVME P2PDMA” feature added in GPUDirect Storage v1.13.

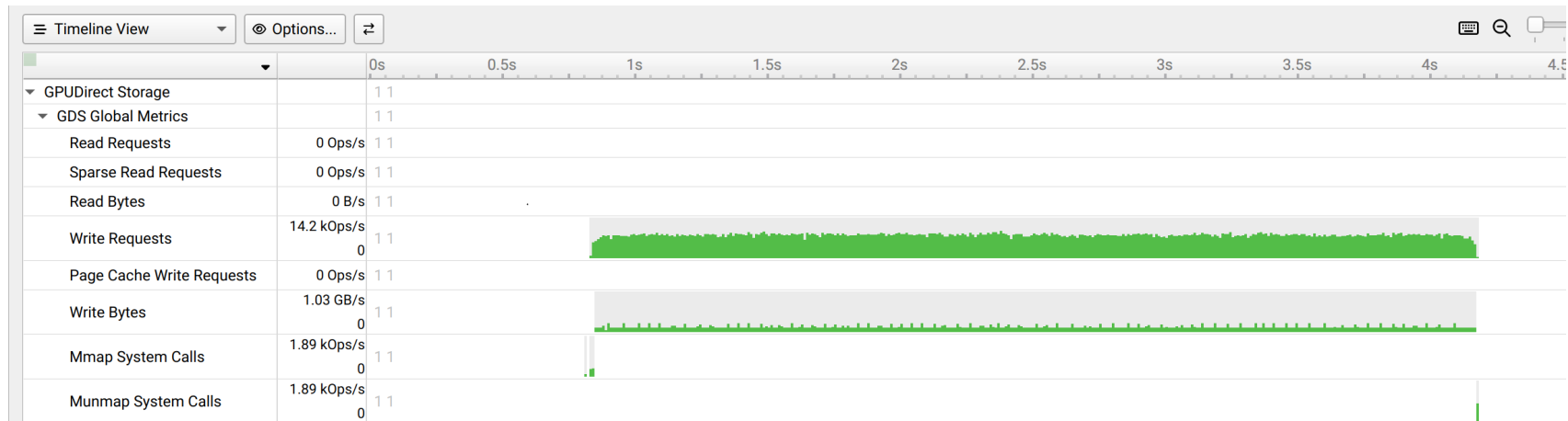
Before collecting GDS (GPUDirect Storage) metrics, ensure that **NVIDIA GPUDirect Storage** is installed on your system. For installation instructions, refer to the [NVIDIA GPUDirect Storage Installation and Troubleshooting Guide](#).

You can validate that GDS is installed correctly by running the gdscheck.py tool: `/usr/local/cuda/gds/tools/gdscheck.py -p` It should report that the target filesystem type is supported and that platform verification succeeded.

Once GDS is installed, you can enable GDS Kernel-Space metrics profiling by using the `--gds-metrics=driver` command line argument. The GDS metrics can be viewed in the **GPUDirect Storage** section of the report file, under ‘Timeline view’.

Example Nsight Systems command line for GDS Kernel-Space profiling:

```
./nsys profile --storage-metrics --gds-metrics=driver <target-application>
```



Python Profiling#

Nsight Systems has several features that have been added in the last few years to enhance users optimizing their python code.

Note

You may find that all of your python application output comes at the end of the run instead of as events happen.

Python will change the buffering of stdout depending on whether it points to a tty or something else. Nsight Systems redirects the application stdout to a pipe to demultiplex stdout to both a file and the terminal. As a side effect, it makes Python change stdout buffering from line-buffered to page-buffered. You can use `python -u` option or the `PYTHONUNBUFFERED` environment variable to override this behavior.

Python Backtrace Sampling#

Nsight Systems for Arm server (SBSA) platforms, x86 Linux and Windows targets, is capable of periodically capturing Python backtrace information. This functionality is available when tracing Python interpreters of version 3.9 or later. Capturing Python backtrace is done in periodic samples, in a selected frequency ranging from 1Hz - 2KHz with a default value of 1KHz. Note that this feature provides meaningful backtraces for Python processes, when profiling Python-only workflows, consider disabling the CPU sampling option to reduce overhead.

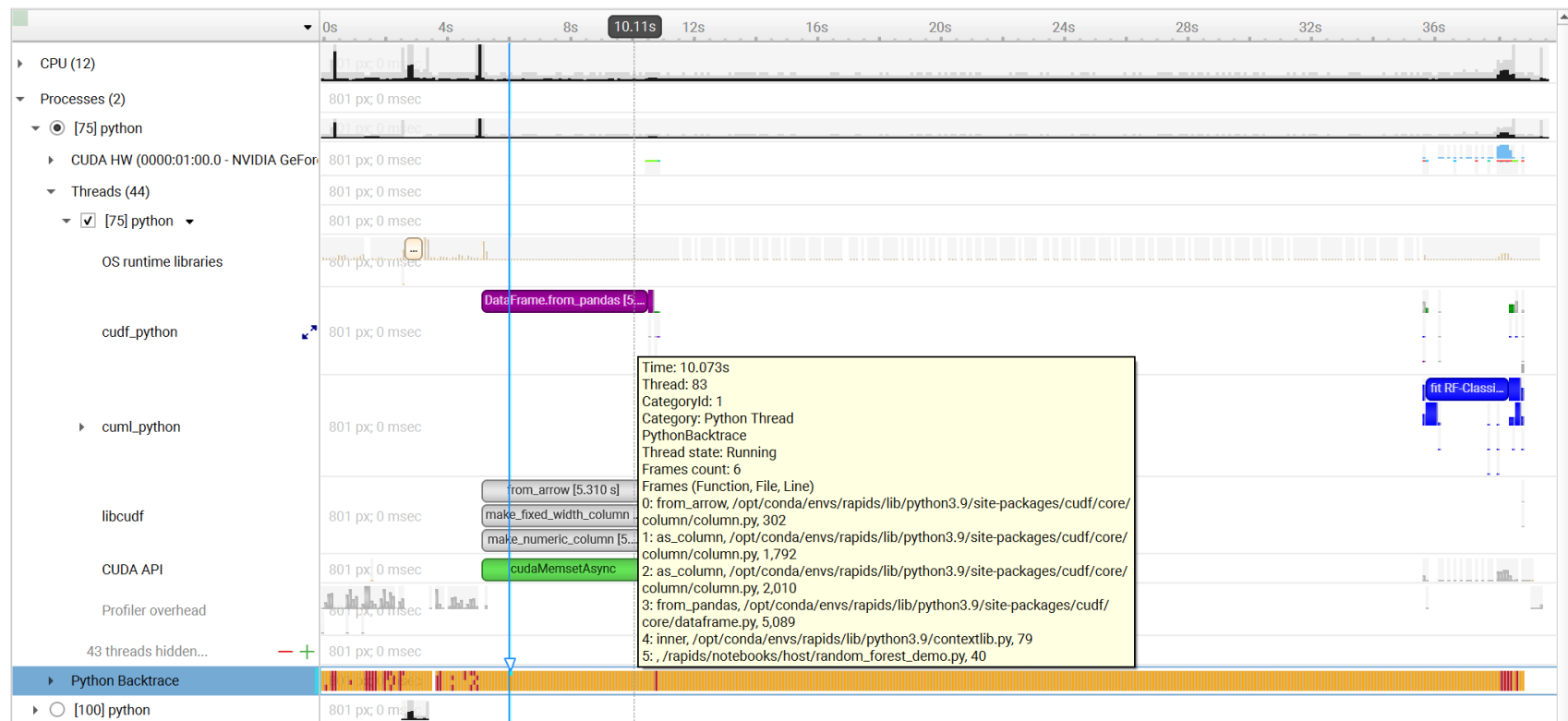
To enable Python backtrace sampling from Nsight Systems:

CLI — Set `--python-sampling=true` and use the `--python-sampling-frequency` option to set the sampling rate.

GUI — Select the **Collect Python backtrace samples** checkbox.



Example screenshot:



Python Functions Trace#

Nsight Systems for Arm server (SBSA) platforms, x86 Linux and Windows targets, is capable of using NVTX to annotate Python functions.

The Python source code does not require any changes. This feature requires CPython interpreter, release 3.8 or later.

The annotations are configured in a JSON file. An example file is located in Nsight Systems installation folder in `<target-platform-folder>/PythonFunctionsTrace/annotations.json`.

For PyTorch applications, Nsight Systems provides a predefined annotations file located in `<target-platform-folder>/PythonFunctionsTrace/pytorch.json`.

For Dask applications, Nsight Systems provides a predefined annotations file located in `<target-platform-folder>/PythonFunctionsTrace/dask.json`.

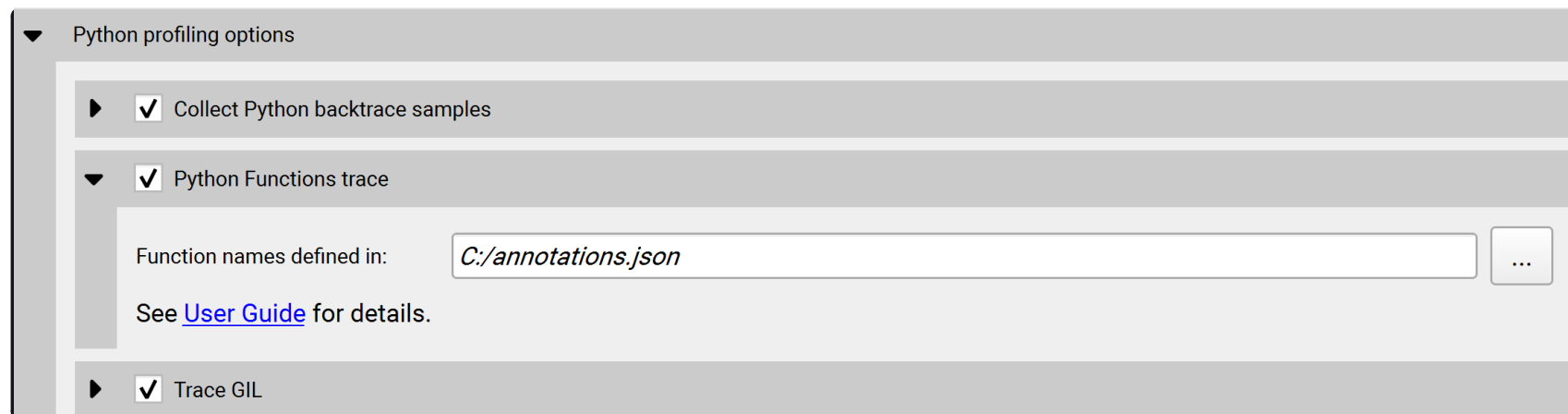
Note

Annotating a function from the module `__main__` is not supported.

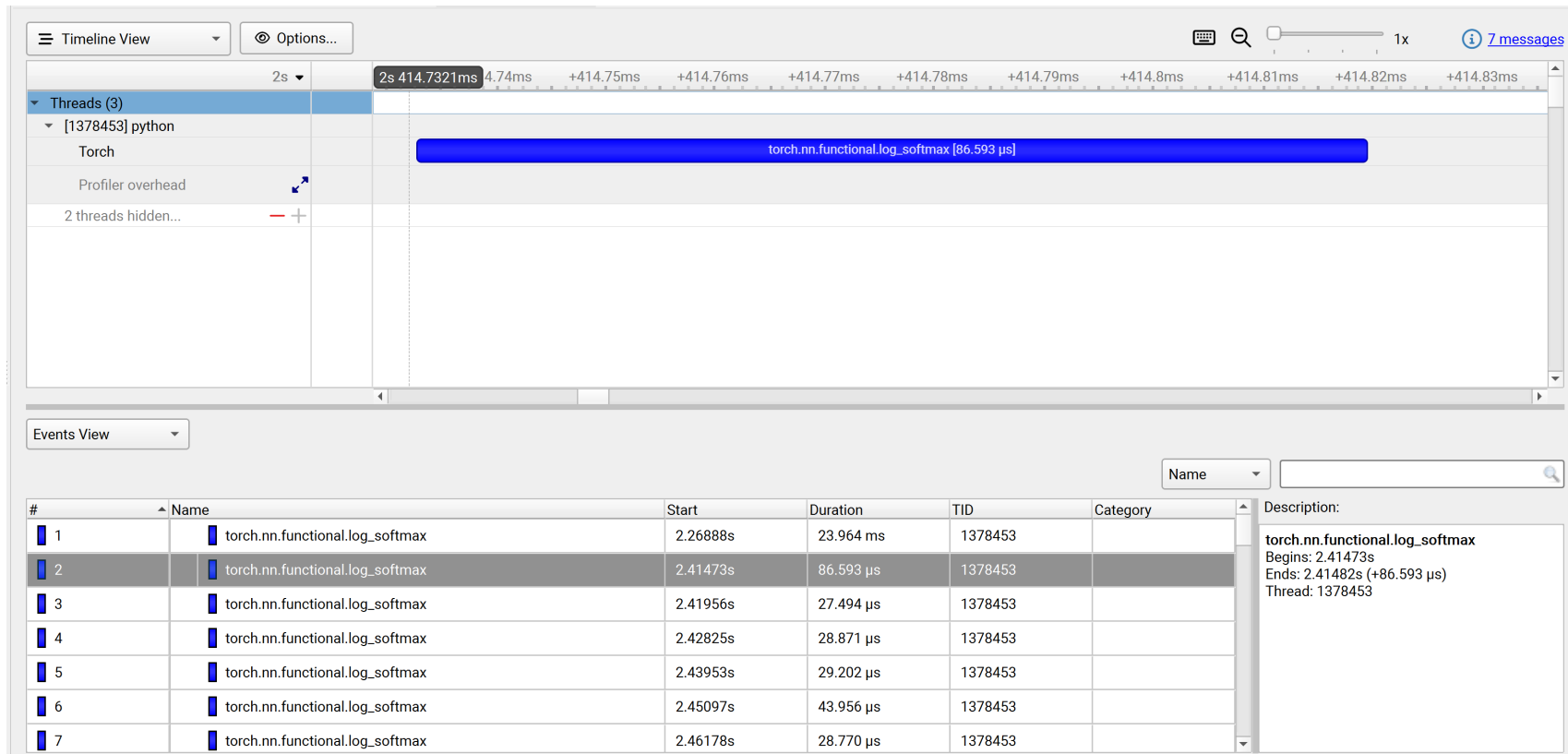
To enable Python functions trace from Nsight Systems:

CLI — Set `--python-functions-trace=<json_file>`.

GUI — Select the **Python Functions trace** checkbox and specify the JSON file.



Example screenshot:



Python GIL Tracing#

Nsight Systems for Arm server (SBSA) platforms, x86 Linux and Windows targets, is capable of tracing when Python threads are waiting to hold and holding the GIL (Global Interpreter Lock).

The Python source code does not require any changes. This feature requires CPython interpreter, release 3.9 or later.

This feature is not supported on Python that was compiled with `Py_GIL_DISABLED=1` (See [Python documentation](#) for details).

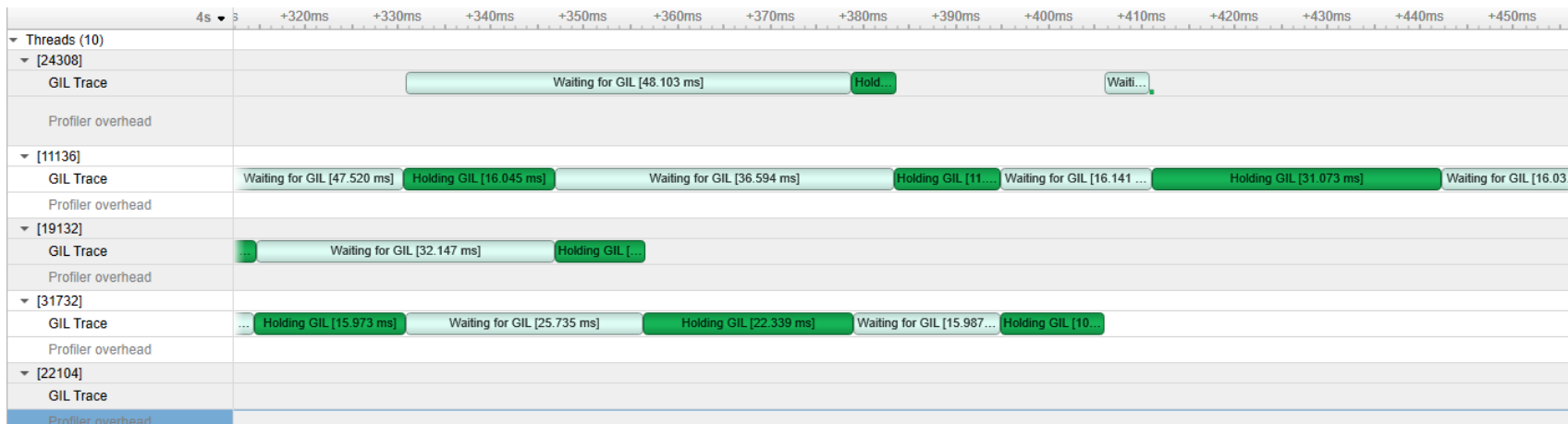
CLI — Set `--trace=python-gil`.

GUI — Select the **Trace GIL** checkbox under **Python profiling options**.

▼ Python profiling options



Example screenshot:



PyTorch Profiling#

Nsight Systems for Arm server (SBASA) platforms, x86 Linux and Windows targets, is capable of automatically annotating common PyTorch operations with execution time ranges.

The Python source code does not require any changes. This feature requires CPython interpreter, release 3.8 or later.

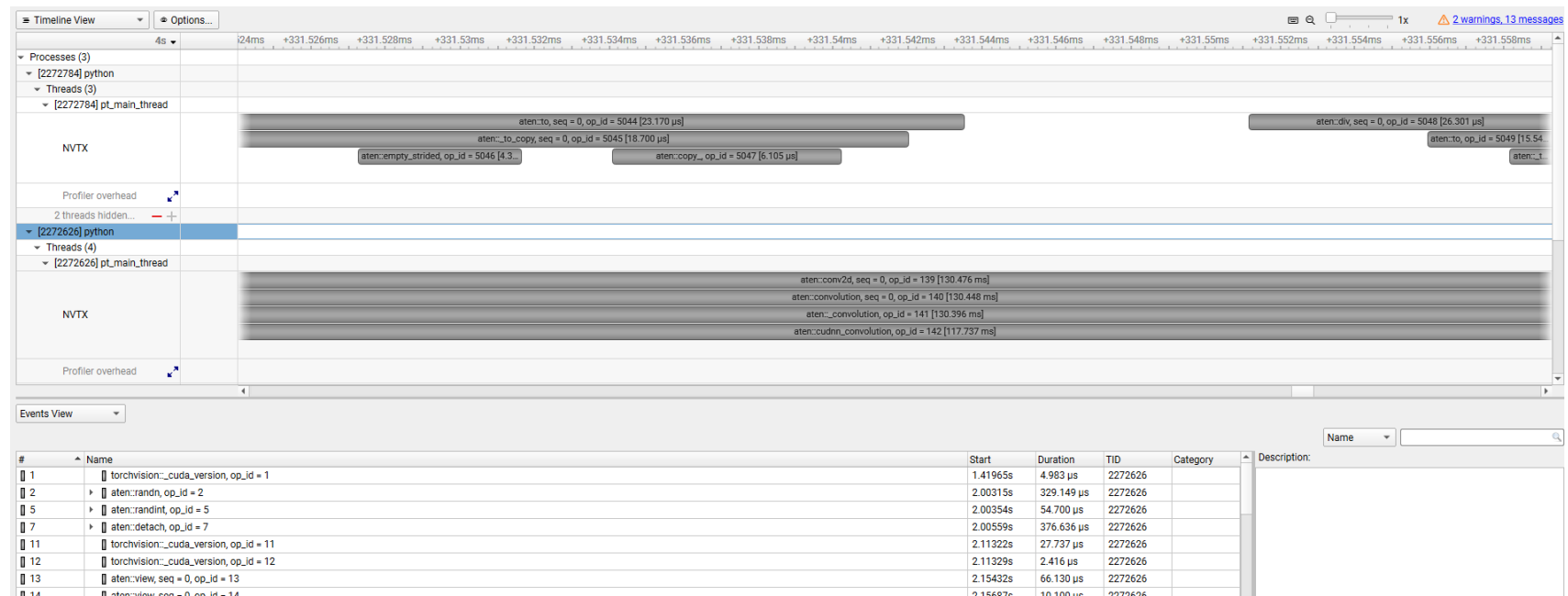
To enable PyTorch [autograd nvtx](#), run Nsight Systems from the CLI using the `--pytorch` option:

Set `--pytorch=autograd-nvtx` for enabling `torch.autograd.profiler.emit_nvtx(record_shapes=False)` or `--pytorch=autograd-shapes-nvtx` for enabling `torch.autograd.profiler.emit_nvtx(record_shapes=True)` (implies `--trace=nvtx`).

`--pytorch=functions-trace` is an alias to `--python=functions-trace=<nsys_install_dir>/<target-arch>/PythonFunctionsTrace/pytorch.json` and provides additional annotations set for PyTorch functions.

`autograd-nvtx` and `autograd-shapes-nvtx` options can be combined with the `functions-trace` option by adding them separated by a comma.

Example screenshot:



Dask Profiling#

Nsight Systems for Arm server (SBASA) platforms, x86 Linux and Windows targets, is capable of automatically annotating common Dask functions with execution time ranges.

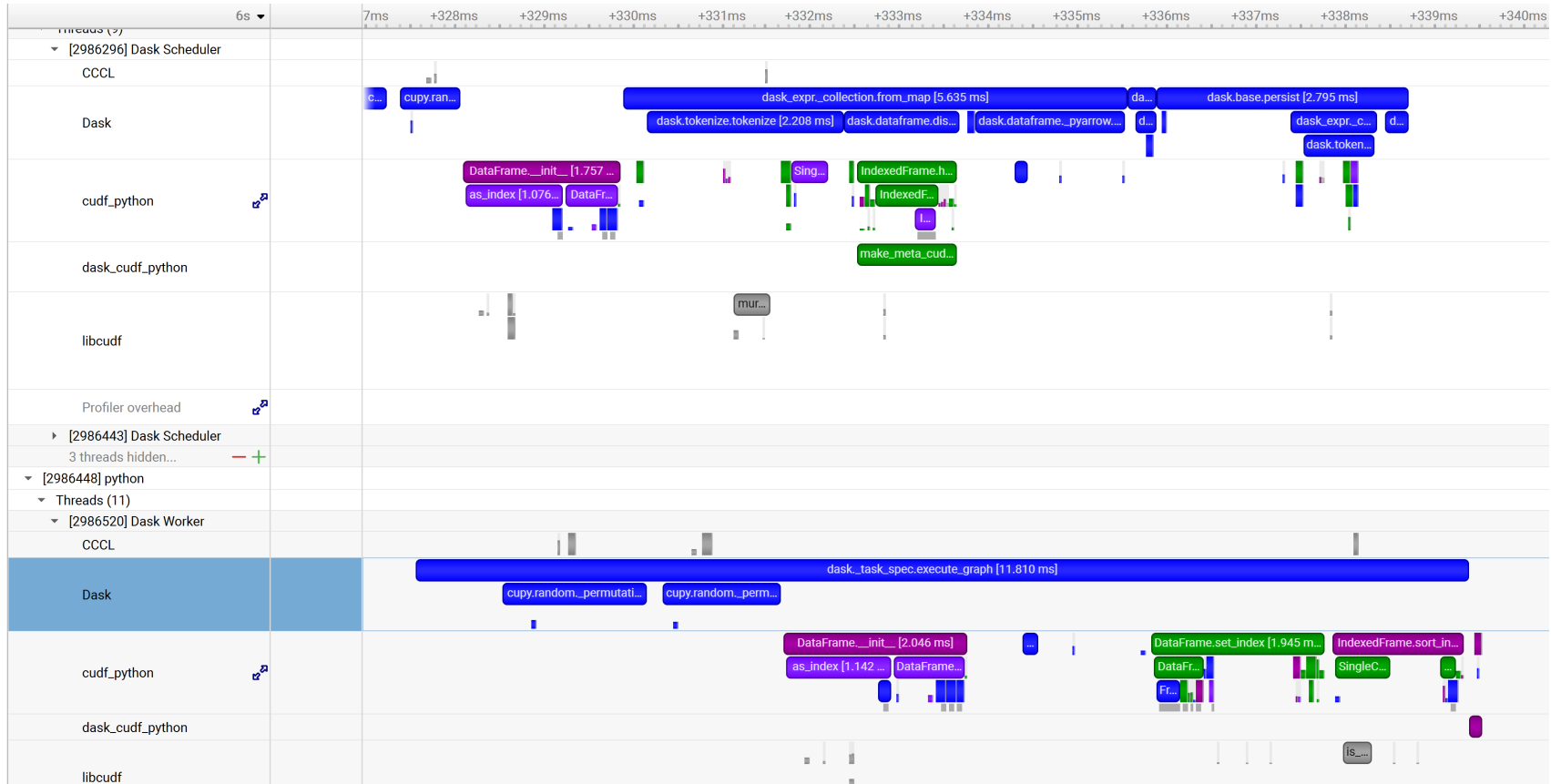
The Python source code does not require any changes. This feature requires CPython interpreter, release 3.8 or later.

Set `--dask=functions-trace` for enabling Dask functions trace. This option sets `--python=functions-trace=<nsys_install_dir>/<target-arch>/PythonFunctionsTrace/dask.json` and will rename

relevant threads to 'Dask Worker' and 'Dask Scheduler'.

dask.json can be modified to include additional functions to be traced from any Python module.

Example screenshot:



Profiling Embedded Virtual Machines#

Nsight Systems and DRIVE Hypervisor support periodic CPU sampling with call stacks. It works both on DRIVE Linux and QNX.

The call stacks are collected using frame pointers. The Linux kernel, QNX kernel, and user space libraries provided by NVIDIA are compiled with frame pointers. To ensure correct call stacks, we recommend compiling all application

code with frame pointer support, using `-fno-omit-frame-pointer` with GCC, Clang, and QCC.

This is an experimental feature and is expected to change in the future.

The symbols can be resolved both for user space code, and for kernel space code:

- In the user space, the Cross-Hypervisor (XHV) sampling events are matched with the CPU thread state trace coming from Linux Perf and QNX Tracelogger. After that, Nsight Systems can know the module filename, and can resolve symbols directly from these files if they are unstripped, or by looking up additional files with symbols. See more details below.
- In the kernel space (Linux kernel, QNX kernel, and additional service VMs), the symbols are resolved using the ELF file with symbols specified. `kernel_symbols.json` input file specifies the location of this ELF file.

Please follow the steps below to learn how to:

- Flash the devkit (these steps are given just as an example, the exact steps might differ in your case).
- Copy the necessary files: `pct.json`, eventlib schema files, and `kernel_symbols.json`.
- Compose `kernel_symbols.json` to allow resolving symbols in the Linux kernel, QNX kernel, and additional service VMs.
- See example CLI commands to collect data.

Known issues:

- At the moment, this feature is not compatible with standard CPU sampling on Linux and QNX.
- When enabled together, hypervisor trace plus XHV sampling can write too much data into the same eventlib buffers, and the Nsight Systems agent might not be able to keep up with the rate, losing events. If that happens, please disable hypervisor trace events with `--xhv-trace-events=none`.

Flashing DRIVE OS QNX/Linux

Log into the NVIDIA GPU Cloud (NGC):

```
sudo docker login nvcr.io
```

Username: ``\$oauthtoken``

Password: <NGC API key>

Docker command:

```
sudo docker run --rm --privileged --net host \  
-v /dev/bus/usb:/dev/bus/usb \  
-v /tmp:/drive_flashing \  
-it <docker image>
```

<docker image> - docker image link.

Examples:

6.0.8.0 QNX:

```
sudo docker run --rm --privileged --net host \  
-v /dev/bus/usb:/dev/bus/usb \  
-v /tmp:/drive_flashing \  
-it nvcr.io/{MY_NGC_ORG}/driveos-pdk/drive-agx-orin-qnx-aarch64-pdk-build-x86:6.0.8.0-0003
```

6.0.9.1 QNX:

```
sudo docker run --rm --privileged --net host \  
-v /dev/bus/usb:/dev/bus/usb \  
-v /tmp:/drive_flashing \  
-it nvcr.io/{MY_NGC_ORG}/driveos-pdk/drive-agx-orin-qnx-aarch64-pdk-build-x86:6.0.9.1-latest
```

6.0.8.0 Linux:

```
sudo docker run --rm --privileged --net host \  
-v /dev/bus/usb:/dev/bus/usb \  
-v /tmp:/drive_flashing \  
-it nvcr.io/{MY_NGC_ORG}/driveos-pdk/drive-agx-orin-linux-aarch64-pdk-build-x86:6.0.8.0-0003
```

Inside of container, flash with flash.py:

```
cd /drive
```

```
./flash.py <aurix> <board>
```

- <board> - target board base name: 'p3710' or 'p3663'.
- <aurix> - Aurix serial port, for example: */dev/ttyACM1*, */dev/ttyUSB1*.

Examples:

Firespray p3710:

```
./flash.py /dev/ttyACM1 p3710
```

Drive Orin p3663:

```
./flash.py /dev/ttyUSB1 p3663
```

List the available EMMC and UFS partitions:

```
df -h
```

Format a power-safe file system partition, and mount it, example for *vblk_ufs40*:

```
mkqnx6fs /dev/vblk_ufs40 -q
```

```
mount -o rw /dev/vblk_ufs40 /
```

```
# df -h
```

```
/dev/vblk_ufs40      116G   7.5G   108G    7% /
```

```
ifs                 16M   16M    0 100% /
```

```
ifs                 52M   52M    0 100% /
```

```
...
```

Note

For more information about DRIVE OS installation, see the following link: [NVIDIA DRIVE OS Documentation](#) (useful pages: **DRIVE OS Linux Installation Guide**, **DRIVE OS QNX Installation Guide**).

Create XHV Directory

Inside of container, examples for p3710, QNX/Linux:

QNX:

```
cd /drive_flashing
mkdir -p xhv/hypervisor/configs/t234ref-release/pct/qnx xhv/schemas
cp -rv /drive/drive-foundation/virtualization/hypervisor/t23x/configs/t234ref-release/pct/p3710-10-a03/qnx/pct.json ./
xhv/hypervisor/configs/t234ref-release/pct/qnx/
cp -rv /drive/drive-foundation/schemas/event ./xhv/schemas/
```

Linux:

```
cd /drive_flashing
mkdir -p xhv/hypervisor/configs/t234ref-release/pct/linux xhv/schemas
cp -rv /drive/drive-foundation/virtualization/hypervisor/t23x/configs/t234ref-release/pct/p3710-10-a03/linux/pct.json ./
xhv/hypervisor/configs/t234ref-release/pct/linux/
cp -rv /drive/drive-foundation/schemas/event ./xhv/schemas/
```

Example of XHV directory (Linux):

```
xhv/
├── hypervisor
│   ├── configs
│   │   ├── t234ref-release
│   │   │   ├── pct
│   │   │   │   ├── linux
│   │   │   │   │   └── pct.json
│   └── schemas
│       └── event
│           ├── audioserver_events.json
│           └── bpmp_events.json
```

```

├── cem_events.json
├── hv_events.json
├── i2c_events.json
├── Makefile.gen-event-headers.tmk
├── monitor_events.json
├── se_events.json
├── sysmgr_events.json
└── vsc_events.json

```

Copy XHV directory to target:

```
scp -r xhv <user>@<target-IP>
```

eventlib_dump tool (QNX/Linux):

```
cp -rv /drive/drive-qnx/nvidia-bsp/aarch64le/sbin/eventlib_dump /drive_flashing/
```

```
cp -rv /drive/drive-linux/filesystem/contents/bin/eventlib_dump /drive_flashing/
```

Specific Command Line Options

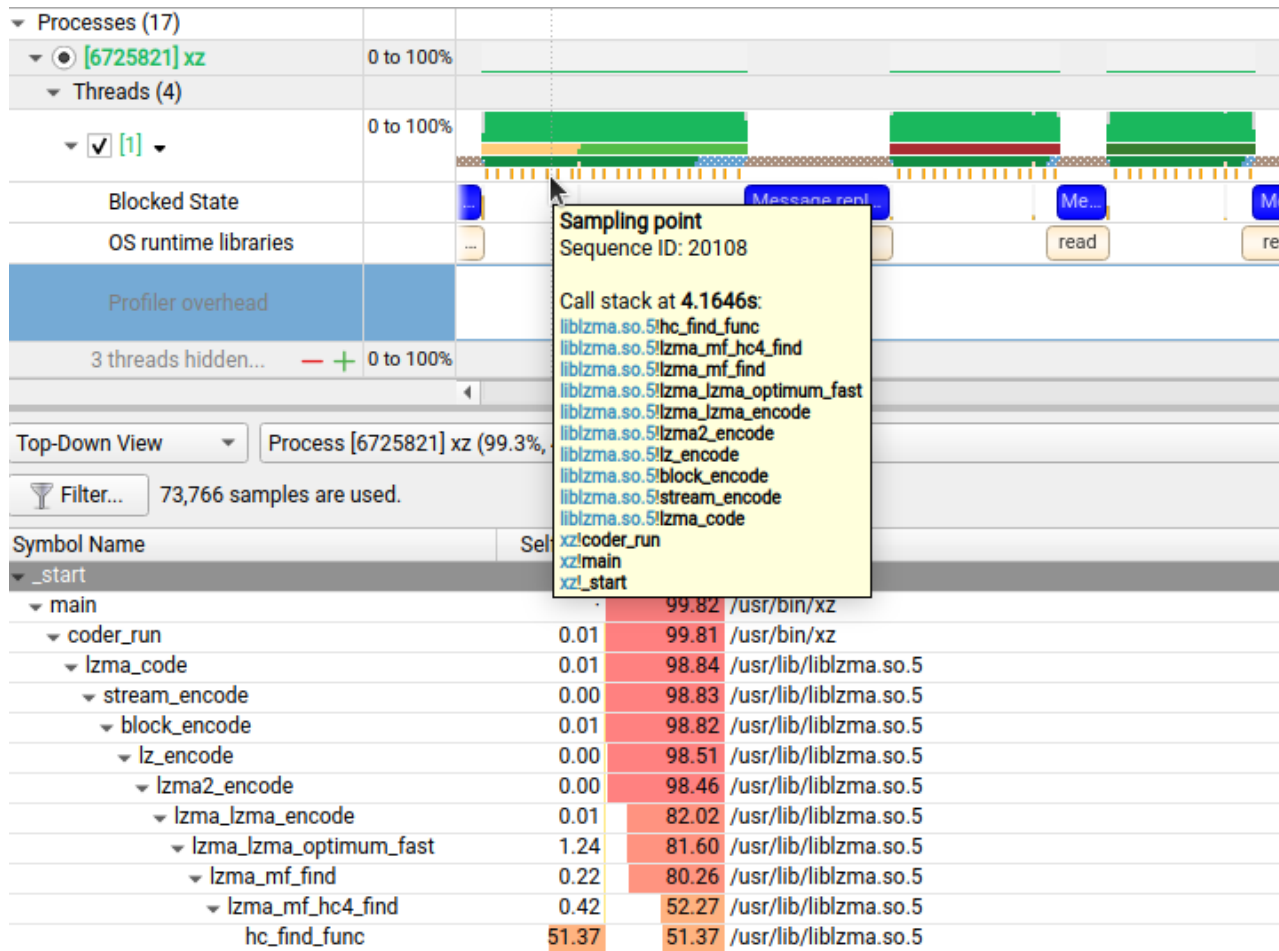
Option	Possible Parameters	Default	Switch Description
<code>--sample</code>	process-tree, system-wide, xhv, xhv-system-wide, none	process-tree	Select 'xhv' or 'xhv-system-wide' to enable Cross-Hypervisor (XHV) sampling, requires root privileges.
<code>--xhv-vm-symbols</code>	< filepath kernel_symbols.json >	none	XHV sampling config (optional, for kernel symbols).
<code>--xhv-trace</code>	< filepath pct.json >	none	Collect hypervisor trace.

Option	Possible Parameters	Default	Switch Description
--xhv-trace-events	all, none, core, sched, irq, trap	all	HV trace events.

Examples:

```
nsys profile --sample=xhv --trace=nvtx,osrt,cuda --xhv-vm-symbols=/root/kernel_symbols.json --xhv-trace=/root/xhv/hypervisor/configs/p3710-10-a01/pct/qnx/pct.json --xhv-trace-events=none sleep 5
nsys profile --sample=xhv-system-wide --xhv-vm-symbols=/root/kernel_symbols.json --xhv-trace=/root/xhv/hypervisor/configs/p3710-10-a01/pct/qnx/pct.json --xhv-trace-events=none sleep 5
```

Example screenshot:



Config File (for kernel symbols)

Examples:

QNX, kernel_symbols.json file:

```
{
  "guest_cfg": [
    {
      "guest_id": 0,
      "guest_name": "Guest VM 0",
```

```
    "symbols": "/root/symbols/procnto-smp-instr-safety.guest_vm.bin.sym"
  },
  {
    "guest_id": 1,
    "guest_name": "Update service",
    "symbols": "/root/symbols/procnto-smp-instr-safety.update_vm.bin.sym"
  },
  {
    "guest_id": 2,
    "guest_name": "Resource Manager Server"
  },
  {
    "guest_id": 3,
    "guest_name": "Storage Server"
  },
  {
    "guest_id": 4,
    "guest_name": "Ethernet Server"
  },
  {
    "guest_id": 5,
    "guest_name": "Debug Server"
  }
],
"symbol_files": {
  "Sidekick": "/root/symbols/sidekick.unstripped"
}
}
```

Linux, kernel_symbols.json file:

```
{
  "guest_cfg": [
    {
      "guest_id": 0,
      "guest_name": "Guest VM 0",
      "symbols": "/home/nvidia/vmlinux"
    },
    {
      "guest_id": 1,
      "guest_name": "Update service"
    }
  ],
  "symbol_files": {
  }
}
```

Symbol Files

The list of directories with symbol files:

- CLI: DbgFileSearchPath config option, for example: DbgFileSearchPath="/lib:/root/symbols" - list of directories with symbol/debug files. On Linux, the default path is /usr/lib/debug. On QNX, there is no default path.

Example:

```
NSYS_CONFIG_DIRECTIVES='DbgFileSearchPath="/lib:/root/symbols"' nsys profile --sample=xhv --xhv-vm-  
symbols=/root/kernel_symbols.json --xhv-trace=/root/xhv/hypervisor/configs/p3710-10-a01/pct/qnx/pct.json --xhv-  
trace-events=none sleep 5
```

- GUI: Symbol location button.

The search is non-recursive.

There are several ways of searching for symbol files - Nsight Systems tries them sequentially for each target file:

- Build-id debug files (CLI only)

<symbol directory>/<.build-id/>... - directories with debug files (or links to debug files).

Example:

```
.build-id/  
├── 00  
│   └── 6627b119cc2aee77e10e0535fc243fce8fe66e.debug  
├── 01  
│   ├── 3e4007e3cb24359203fc02b63bb90f16db5b23.debug  
│   └── fb938bc0f029c41a8e1e88f01f88f75cf3a0d3.debug  
...
```

- Debuglink files (CLI only)

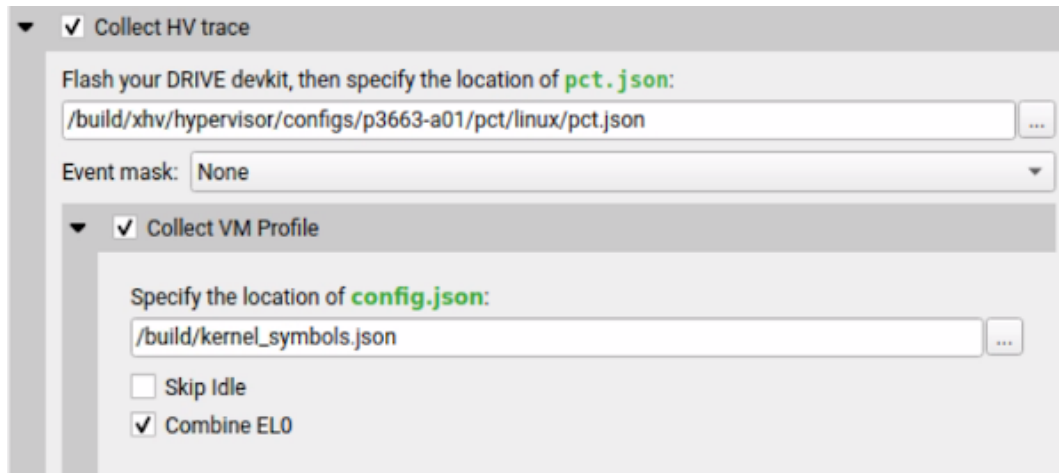
<symbol directory>/<symbol file> - both filename and CRC from debuglink section must be matched for the symbol file.

- File name and build-id (CLI/GUI)

<symbol directory>/<symbol file> - by filename and build-id.

XHV profiling from the GUI

XHV options:



▼ ☒ Collect HV trace

Flash your DRIVE devkit, then specify the location of **pct.json**:

/build/xhv/hypervisor/configs/p3663-a01/pct/linux/pct.json

Event mask: None

▼ ☒ Collect VM Profile

Specify the location of **config.json**:

/build/kernel_symbols.json

☐ Skip Idle

☒ Combine EL0

Use this dialog to specify XHV parameters:

- Collect HV Trace - Enable XHV tracing.
- The location of `pct.json` file on the host. There is predefined hierarchy of XHV JSON files, for example:

```
xhv/
├── hypervisor
│   ├── configs
│   │   ├── t234ref-release
│   │   │   ├── pct
│   │   │   │   ├── linux
│   │   │   │   └── pct.json
│   └── schemas
│       └── event
│           └── ...
│               ├── hv_events.json
│               └── ...
```

- Collect VM Profile - Enable XHV sampling, depends on Collect HV Trace.
- Event mask - Select XHV trace events, this option can be specified as None.

- The location of `kernel_symbols.json` file on the host. Note that this file contains target paths to the kernel symbol files (see examples above).
- Skip `idle` and `Combine EL0` checkboxes are deprecated.

Adding Your Own Collection to a Report<#>

Nsight Systems allows the user to add additional information to a report file for display with other Nsight Systems options.

Nsight Systems Plugins (Preview)<#>

What is a plugin?<#>

Nsight Systems plugins are standalone applications that can be profiled along with the main application or without one in a system-wide profiling. The NVTX events emitted by a plugin are displayed in the same timeline as the main application events. Additionally, any stdout and stderr streams are captured the same way as for a target application.

To make a plugin available for profiling, create a directory with a manifest file, `nsys-plugin.yaml`, then place it in a “plugins” directory next to the Nsight Systems target CLI binary. The manifest file describes the plugin and its configuration.

Manifest file contents<#>

The manifest file is a YAML file with the following required fields:

PluginName: SamplePlugin

ExecutablePath: PluginExecutableRelativeToManifest

Description: This is a sample plugin.

How to launch a plugin<#>

Plugins are supported in `nsys profile` and `nsys start` commands. Plugin processes are launched by Nsight Systems as if it was a target application and terminated at the end of profiling. It's possible to launch multiple instances of the same plugin by using multiple `—enable` options.

How to pass arguments to a plugin#

To pass arguments to a plugin, specify them as a part of `—enable` option after plugin name when launching the target application. The arguments should be separated by commas only (no spaces). Commas can be escaped with a backslash `\\`. The backslash itself can be escaped by another backslash `\\\\`. To include spaces in an argument, enclose the argument in double quotes `"`.

See the section on the [Amazon AWS Elastic Fabric Adapter \(EFA\) Network Counters](#) for an example.

Supported platforms#

Currently plugins are supported on `x86_64` and `arm64` Linux.

Sample plugin#

You can look at the Nsight Systems installation path, for example `/opt/nvidia/nsight-systems/2024.5.1/target-linux-x64`, under the directory `samples` for the source code of a sample plugin.

The `NetworkPlugin.cpp` source file is the exact source code for the `network_interface` plugin that ships in binary form with Nsight Systems. Users can modify this plugin or use it as a guide to create their own plugin for profiling their intended source of metrics.

Import NVTXT#

ImportNvtxt is an utility which allows conversion of a [NVTXT](#) file to a Nsight Systems report file (`*.nsys-rep`) or to merge it with an existing report file.

Note

NvtxImport supports custom **TimeBase** values. Only the following values are supported:

- **Manual** — timestamps are set using absolute values.
- **Relative** — timestamps are set using relative values with regards to report file which is being merged with the NVTXT file.
- **ClockMonotonicRaw** — timestamps values in the NVTXT file are considered to be gathered on the same target as the report file which is to be merged with NVTXT using `clock_gettime(CLOCK_MONOTONIC_RAW, ...)` call.
- **CNTVCT** — timestamps values in the NVTXT file are considered to be gathered on the same target as the report file which is to be merged with NVTXT using CNTVCT values.

You can get usage info via the help message.

Print the help message:

```
-h [ --help ]
```

Show information about the report file:

```
--cmd info -i [--input] arg
```

Create the report file from an existing NVTXT file:

```
--cmd create -n [--nvtxt] arg -o [--output] arg [-m [--mode] mode_name mode_args] [--target <Hw:Vm>] [--update_report_time]
```

Merge the NVTXT file to an existing report file:

```
--cmd merge -i [--input] arg -n [--nvtxt] arg -o [--output] arg [-m [--mode] mode_name mode_args] [--target <Hw:Vm>] [--update_report_time]
```

Modes' descriptions:

- **lerp** - Insert with linear interpolation

--mode lerp --ns_a arg --ns_b arg [--nvtxt_a arg --nvtxt_b arg]

- lin - insert with linear equation

--mode lin --ns_a arg --freq arg [--nvtxt_a arg]

Modes' parameters:

- ns_a - a nanoseconds value
- ns_b - a nanoseconds value (greater than ns_a)
- nvtxt_a - an nvtxt file's time unit value corresponding to ns_a nanoseconds
- nvtxt_b - an nvtxt file's time unit value corresponding to ns_b nanoseconds
- freq - the nvtxt file's timer frequency
- --target <Hw:Vm> - specify target id, e.g. --target 0:1
- --update_report_time - prolong report's profiling session time while merging if needed. Without this option all events outside the profiling session time window will be skipped during merging.

Commands#

Info

To find out report's start and end time use **info** command.

Usage:

ImportNvtxt --cmd info -i [--input] arg

Example:

ImportNvtxt info Report.nsys-rep

Analysis start (ns) 83501026500000

Analysis end (ns) 83506375000000

Create

You can create a report file using existing NVTXT with **create** command.

Usage:

```
ImportNvtxt --cmd create -n [--nvtxt] arg -o [--output] arg [-m [--mode] mode_name mode_args]
```

Available modes are:

- **lerp** — insert with linear interpolation.
- **lin** — insert with linear equation.

Usage for **lerp** mode is:

```
--mode lerp --ns_a arg --ns_b arg [--nvtxt_a arg --nvtxt_b arg]
```

with:

- **ns_a** — a nanoseconds value.
- **ns_b** — a nanoseconds value (greater than **ns_a**).
- **nvtxt_a** — an nvtxt file's time unit value corresponding to **ns_a** nanoseconds.
- **nvtxt_b** — an nvtxt file's time unit value corresponding to **ns_b** nanoseconds.

If **nvtxt_a** and **nvtxt_b** are not specified, they are respectively set to nvtxt file's minimum and maximum time value.

Usage for **lin** mode is:

```
--mode lin --ns_a arg --freq arg [--nvtxt_a arg]
```

with:

- **ns_a** — a nanoseconds value.

- `freq` — the nvtxt file's timer frequency.
- `nvtxt_a` — an nvtxt file's time unit value corresponding to `ns_a` nanoseconds.

If `nvtxt_a` is not specified, it is set to nvtxt file's minimum time value.

Examples:

```
ImportNvtxt --cmd create -n Sample.nvtxt -o Report.nsys-rep
```

The output will be a new generated report file which can be opened and viewed by Nsight Systems.

Merge

To merge NVTXT file with an existing report file use **merge** command.

Usage:

```
ImportNvtxt --cmd merge -i [--input] arg -n [--nvtxt] arg -o [--output] arg [-m [--mode] mode_name mode_args]
```

Available modes are:

- **lerp** — insert with linear interpolation.
- **lin** — insert with linear equation.

Usage for **lerp** mode is:

```
--mode lerp --ns_a arg --ns_b arg [--nvtxt_a arg --nvtxt_b arg]
```

with:

- `ns_a` — a nanoseconds value.
- `ns_b` — a nanoseconds value (greater than `ns_a`).
- `nvtxt_a` — an nvtxt file's time unit value corresponding to `ns_a` nanoseconds.
- `nvtxt_b` — an nvtxt file's time unit value corresponding to `ns_b` nanoseconds.

If `nvtxt_a` and `nvtxt_b` are not specified, they are respectively set to nvtxt file's minimum and maximum time

value.

Usage for **lin** mode is:

```
--mode lin --ns_a arg --freq arg [--nvtxt_a arg]
```

with:

- **ns_a** — a nanoseconds value.
- **freq** — the nvtxt file's timer frequency.
- **nvtxt_a** — an nvtxt file's time unit value corresponding to **ns_a** nanoseconds.

If **nvtxt_a** is not specified, it is set to nvtxt file's minimum time value.

Time values in `<filename.nvtxt>` are assumed to be nanoseconds if no mode specified.

Example

```
ImportNvtxt --cmd merge -i Report.nsys-rep -n Sample.nvtxt -o NewReport.nsys-rep
```

Reading Your Report in GUI#

Generating a New Report#

Users can generate a new report by stopping a profiling session. If a profiling session has been canceled, a report will not be generated, and all collected data will be discarded.

A new `.nsys-rep` file will be created and put into the same directory as the project file (`.qdproj`).

Opening an Existing Report#

An existing `.nsys-rep` file can be opened using **File > Open....**

Sharing a Report File#

Report files (.nsys-rep) are self-contained and can be shared with other users of Nsight Systems. The only requirement is that the same or newer version of Nsight Systems is always used to open report files.

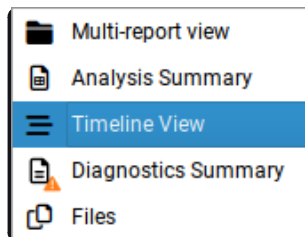
Project files (.qdproj) are currently not shareable, since they contain full paths to the report files.

To quickly navigate to the directory containing the report file, right click on it in the Project Explorer, and choose **Show in folder...** in the context menu.

Report Tab#

While generating a new report or loading an existing one, a new tab will be created. The most important parts of the report tab are:

- **View selector** — Allows switching between *Multi-report view* (absent for single reports), *Analysis Summary*, *Timeline View*, *Diagnostics Summary*, and *Symbol Resolution Logs* views.



- **Timeline** — This is where all charts are displayed.
- **Function table** — Located below the timeline, it displays statistical information about functions in the target application in multiple ways.

Additionally, the following controls are available:

- **Zoom slider** — Allows you to vertically zoom the charts on the timeline.

Analysis Summary View#

This view shows a summary of the profiling session. It can be used to review the various configurations used to generate this report. Depending on the features used, different subsections may be shown, but they may include

- **Profiling session information** - including information about the capture time, duration, report file, and host information.
- **Target information** - including OS and driver versions.
- **Process summary** - processes run during analysis, arguments, and CPU utilization.
- **Module summary** - modules used including name and CPU time (overall and per-process). **Note** - Module percentage in Analysis Summary page is calculated based on the cpu cycles logged in an IP sample for that module. When cpu cycles is inaccurate or absent (as is the case on x86_64 target), the module percentage is inaccurate.
- **Thread summary** - including process ID, name, and CPU utilization
- **CPU information** - information about all CPUs on the system
- **GPU information** - information about all GPUs on the system
- **Network hardware info** - information about NICs/Switches/Storage devices analyzed in the run.
- **Analysis options** - information about which Nsight Systems options were used when generating this report.
- **Nsight Systems** - information about the version used to collect and display the results.

Information from this view can be selected and copied using the mouse cursor.

Diagnostics Summary View<#>

This view shows important messages. Some of them were generated during the profiling session, while some were added while processing and analyzing data in the report. Messages can be one of the following types:

- Informational messages
- Warnings
- Errors

To draw attention to important diagnostics messages, a summary line is displayed on the timeline view in the top right corner:

|  [11 warnings, 8 messages](#)

Information from this view can be selected and copied using the mouse cursor.

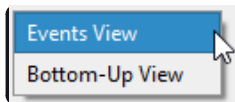
Symbol Resolution Logs View#

This view shows all messages related to the process of resolving symbols. It might be useful to debug issues when some of the symbol names in the symbols table of the timeline view are unresolved.

Timeline View#

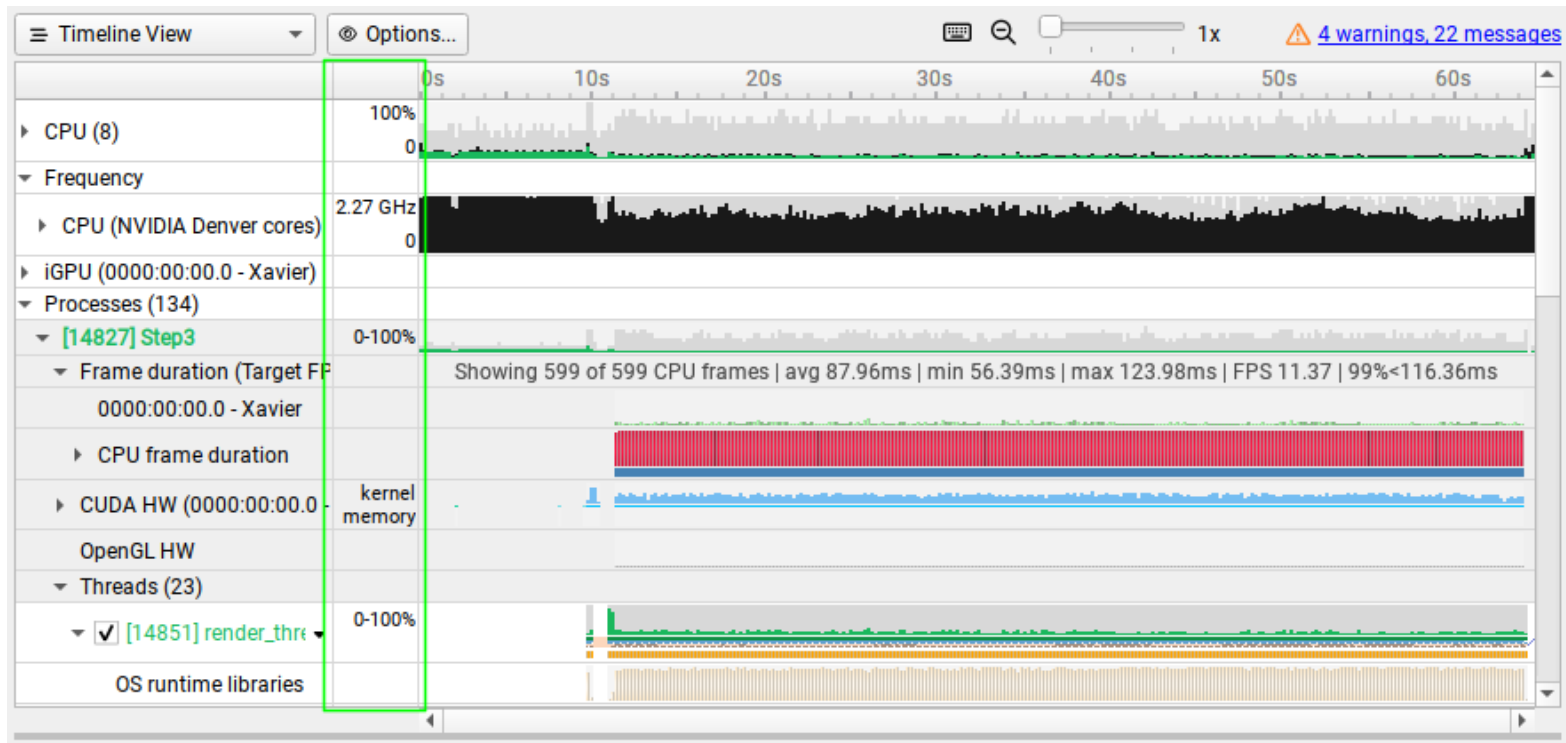
The timeline view consists of two main controls: the timeline at the top, and a bottom pane that contains the events view and the function table. In some cases, when sampling of a process has not been enabled, the function table might be empty and hidden.

The bottom view selector sets the view that is displayed in the bottom pane.



Timeline#

Timeline is a versatile control that contains a tree-like **hierarchy** on the left, a *line labels* column in the center, and the corresponding *charts* on the right. The line labels column can be hidden by using the timeline options.



Contents of the hierarchy depend on the project settings used to collect the report. For example, if a certain feature has not been enabled, corresponding rows will not be shown on the timeline.

Process Coloring

The CPU utilization timelines are colored based on the CPU operating mode:

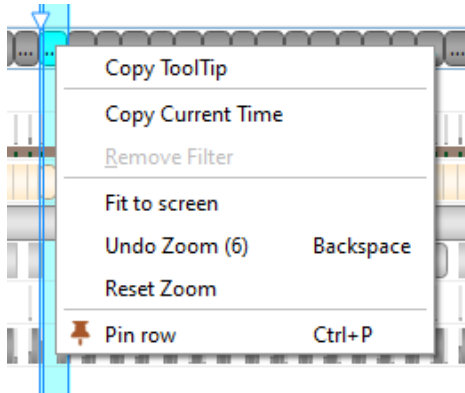
- User mode - Green
- Kernel mode - Red
- Other (for example system-wide processes) - Black

Exporting from Timeline

To generate a timeline screenshot without opening the full GUI, use the command:

```
nsys-ui.exe --screenshot filename.nsys-rep
```

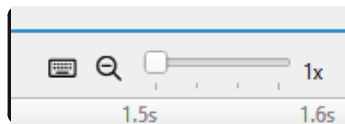
Hovering over elements in the GUI will cause a tooltip to pop open as appropriate to give additional information, such as the parameters of that function call or the call stack. Tooltips can be copied by hovering and right clicking to bring up the Copy ToolTip option in the context menu:



Timeline Navigation#

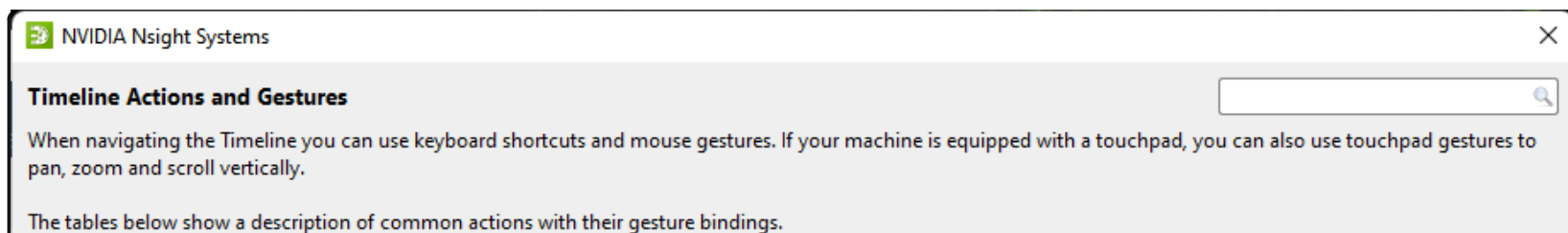
Zoom and Scroll#

At the upper right portion of your Nsight Systems GUI, you will see this section:



The slider sets the vertical size of screen rows, and the magnifying glass resets it to the original settings.

There are many ways to zoom and scroll horizontally through the timeline. Clicking on the keyboard icon seen above, opens the below dialog that explains them.



Show This Dialog

Press the '?' key when Timeline is in focus.

Navigation with Keyboard and Mouse

Here are combinations to use keyboard and mouse to scroll through the Timeline and the tree.

Action	Key or Mouse Gesture
Pan Left	LeftArrow
Pan Right	RightArrow
Zoom in X-axis	Keyboard +, or Keyboard =, or CTRL + MouseWheel up
Zoom out X-axis	Keyboard -, or CTRL + MouseWheel down
Next Row	ArrowDown
Previous Row	ArrowUp
Undo a Navigation Step	Backspace

Navigation with Touchpad

Navigating through the Timeline and the tree with touchpad.

Action	Key or Mouse Gesture
Pan Left	Swipe right with two fingers
Pan Right	Swipe left with two fingers
Zoom in X-axis	Pinch in with touchpad
Zoom out X-axis	Pinch out with touchpad
Scroll Tree Up	Swipe down with two fingers
Scroll Tree Down	Swipe up with two fingers

Selection of Items

You can select an item on the Timeline. Selecting items with double-click synchronizes the Timeline and the Events View

Action	Key or Mouse Gesture
Select an Item in the Timeline	MouseLeftClick on an item
Select an Item in the current Events View	MouseLeftDoubleClick on an item
Open the corresponding Events View and select an Item there	Shift + MouseLeftDoubleClick on an item
Select and Fit an Item to Screen	CTRL + MouseLeftDoubleClick on an item

Selection of a Time Range

You can select an time range on the Timeline. The selected area and its borders can be dragged with left mouse button. Also it has several shortcuts modifying zoom level and time filter.

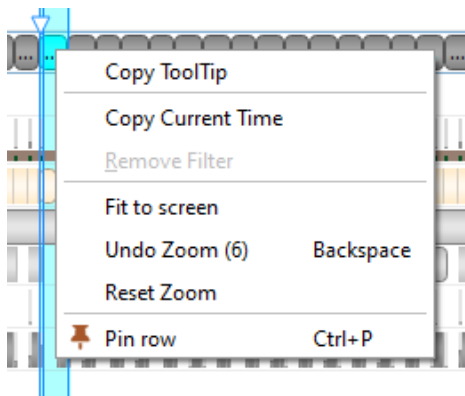
Action	Key or Mouse Gesture
Select a Time Range	Press and drag left mouse button
Drag Selection or Its Border	Press and drag left mouse button on a selection

Drag Selection or its border	Press and drag left mouse button on a selection
Deselect a Time Range	Escape, or Click outside a selection
Zoom into a Selection	Z
Zoom into a Selection and Deselect	Shift + Z, or MouseLeftDoubleClick on a selection
Apply Time Filter	F
Apply Time Filter and Deselect	Shift + F

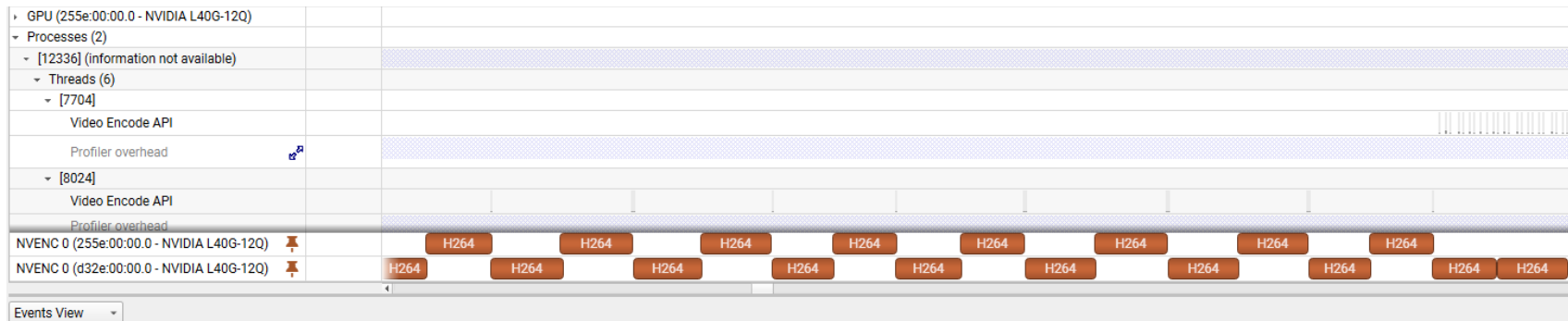
OK

Pinning Rows#

In order to better allow users to compare rows from different sections of the timeline, Nsight Systems gives the user the ability to select rows and “pin” them in the visual range. To select a row to pin, use **Ctrl+P** or **Pin** row from the right click menu.



Once a row has been pinned, it remain at the top or bottom of the window, rather than scrolling off.



Timeline Correlation#

The Nsight Systems GUI can correlate between calls on the CPU and GPU to help you understand the workflow.

Selecting an item will highlight that item in teal as well as:

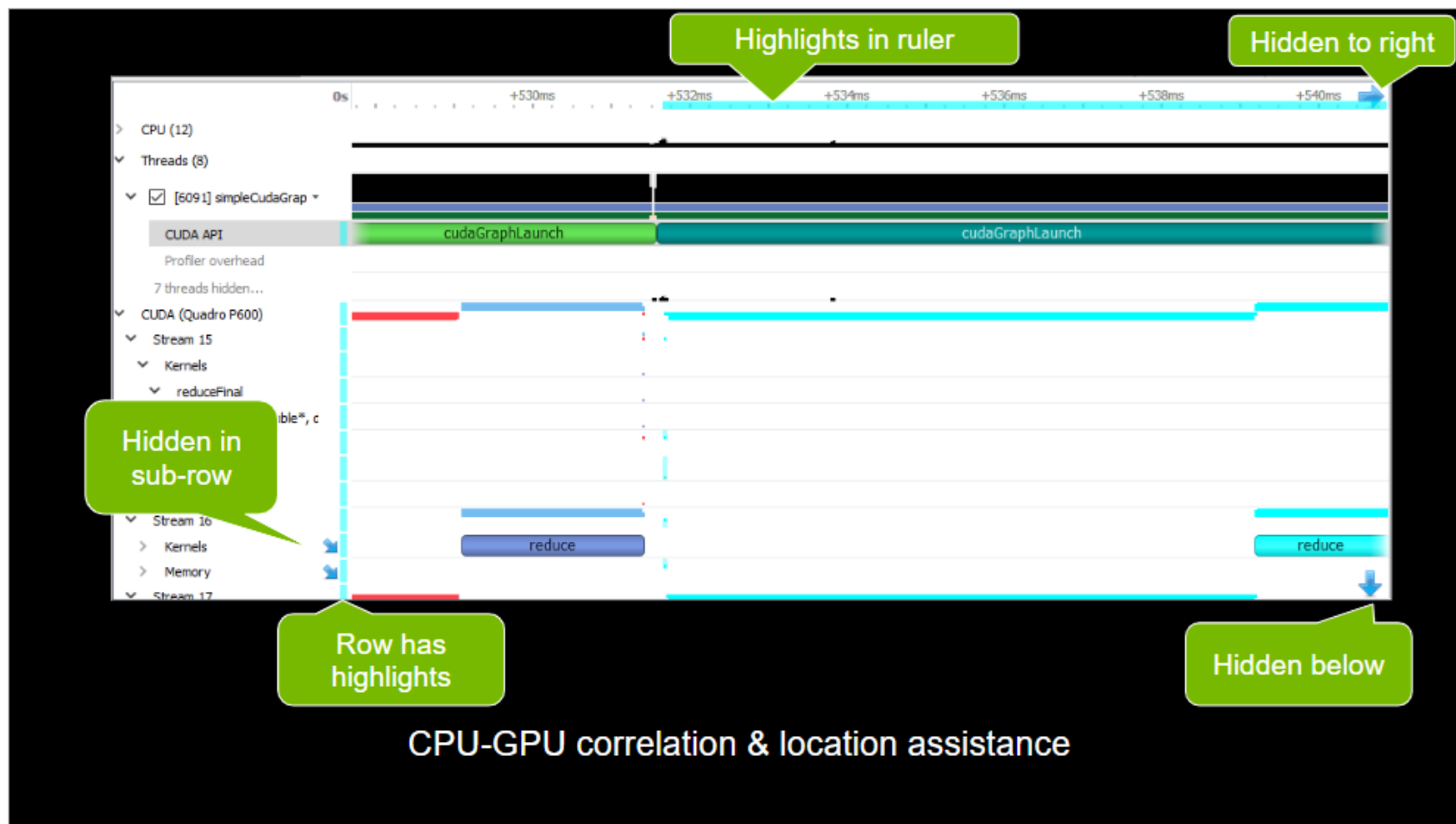
- Any copy of that same item in other rows. This means that if there is a summary row that includes this item it will also have the appropriate section highlighted.
- All correlated items. For example, if a CUDA kernel was launched by a CPU function both are highlighted.
- All things in that thread or stream that falls into the time range since they are of part of that larger range. For example, if you clicked an NVTX range it would select all NVTX ranges and CUDA launches inside and then extend to its correlations.

The highlighting also includes lines to each event to better distinguish when highlighted events or ranges are overlapping.

In addition, Nsight Systems also provides indicators to help you find correlated items not currently on your screen, including:

- Highlights in the row headers when there is something highlighted in that row.
- Diagonal arrows in row headers if something is in a child row.
- Highlights in the timeline rule.

- Arrows in corners when something highlighted is off-screen. You can click those and the timeline will pan or zoom to get them into view.



Correlation exists bidirectionally for:

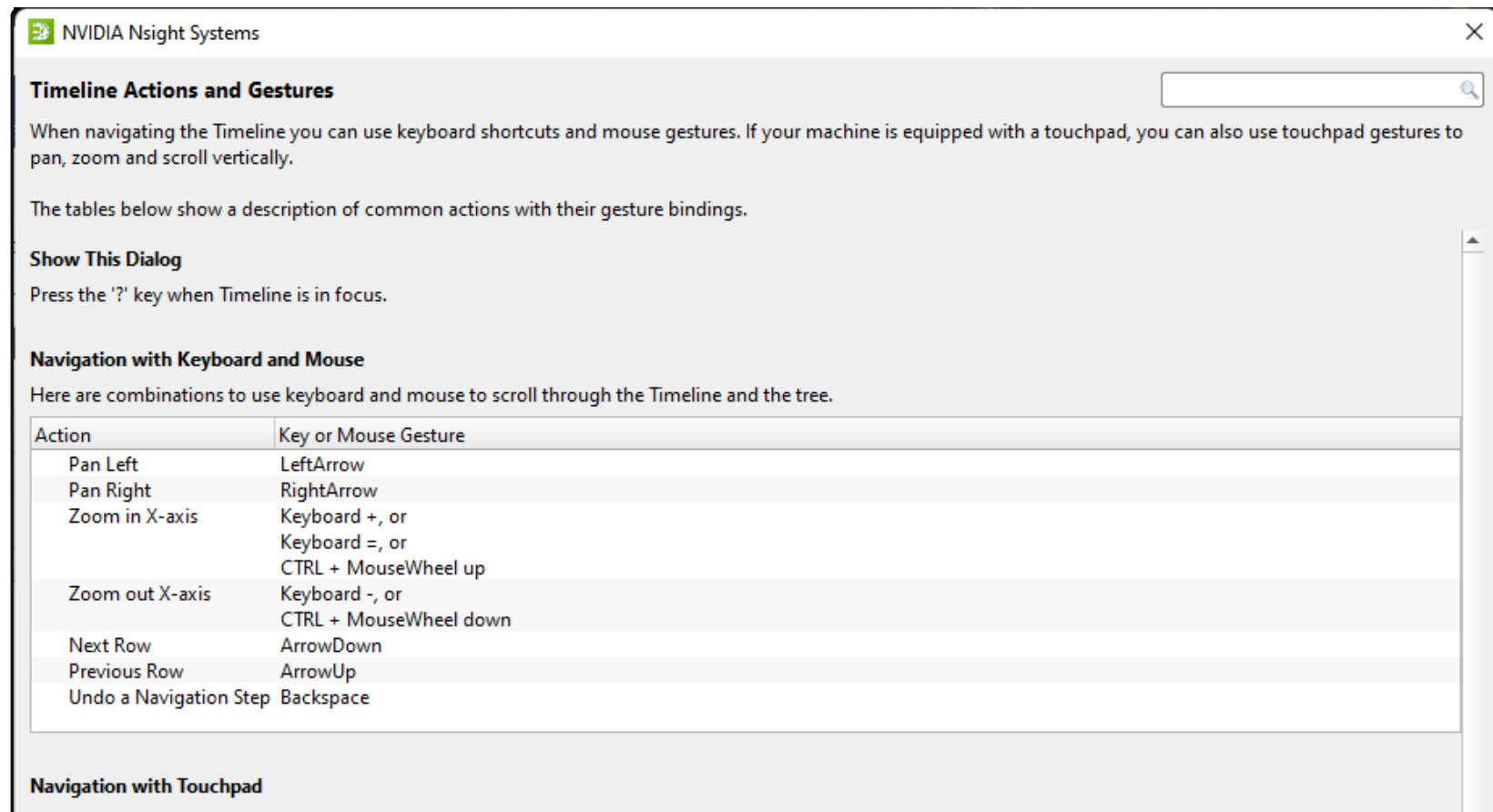
- CUDA kernels, CUDA graphs, GPU memcopies, and OptiX.
- GPU memsets.
- Vulkan QueueSubmits API and CommandBuffers on GPU.
- D3D and GL just like Vulkan.

Timeline/Events Correlation#

To display trace events in the Events View right-click a timeline row and select the **Show in Events View** command. The events of the selected row and all of its sub-rows will be displayed in the Events View. Note that the events displayed will correspond to the current zoom in the timeline, zooming in or out will reset the event pane filter.

If a timeline row has been selected for display in the Events View, then double-clicking a timeline item on that row will automatically scroll the content of the Events View to make the corresponding events view item visible and select it. If that event has tool tip information, it will be displayed in the right hand pane.

Likewise, double-clicking on a particular instance in the Events View will highlight the corresponding event in the timeline.



The screenshot shows a dialog box titled "Timeline Actions and Gestures" from NVIDIA Nsight Systems. It contains a search bar, introductory text about navigation shortcuts, a "Show This Dialog" section with a '?' key hint, and a table of keyboard and mouse gestures. The table lists actions like "Pan Left", "Zoom in X-axis", and "Next Row" with their corresponding key or mouse gestures. Below the table, the "Navigation with Touchpad" section is partially visible.

Timeline Actions and Gestures

When navigating the Timeline you can use keyboard shortcuts and mouse gestures. If your machine is equipped with a touchpad, you can also use touchpad gestures to pan, zoom and scroll vertically.

The tables below show a description of common actions with their gesture bindings.

Show This Dialog
Press the '?' key when Timeline is in focus.

Navigation with Keyboard and Mouse
Here are combinations to use keyboard and mouse to scroll through the Timeline and the tree.

Action	Key or Mouse Gesture
Pan Left	LeftArrow
Pan Right	RightArrow
Zoom in X-axis	Keyboard +, or Keyboard =, or CTRL + MouseWheel up
Zoom out X-axis	Keyboard -, or CTRL + MouseWheel down
Next Row	ArrowDown
Previous Row	ArrowUp
Undo a Navigation Step	Backspace

Navigation with Touchpad

navigating through the timeline and the tree with touchpad.

Action	Key or Mouse Gesture
Pan Left	Swipe right with two fingers
Pan Right	Swipe left with two fingers
Zoom in X-axis	Pinch in with touchpad
Zoom out X-axis	Pinch out with touchpad
Scroll Tree Up	Swipe down with two fingers
Scroll Tree Down	Swipe up with two fingers

Selection of Items

You can select an item on the Timeline. Selecting items with double-click synchronizes the Timeline and the Events View

Action	Key or Mouse Gesture
Select an Item in the Timeline	MouseLeftClick on an item
Select an Item in the current Events View	MouseLeftDoubleClick on an item
Open the corresponding Events View and select an Item there	Shift + MouseLeftDoubleClick on an item
Select and Fit an Item to Screen	CTRL + MouseLeftDoubleClick on an item

Selection of a Time Range

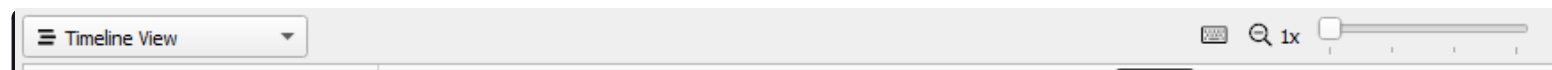
You can select a time range on the Timeline. The selected area and its borders can be dragged with left mouse button. Also it has several shortcuts modifying zoom level and time filter.

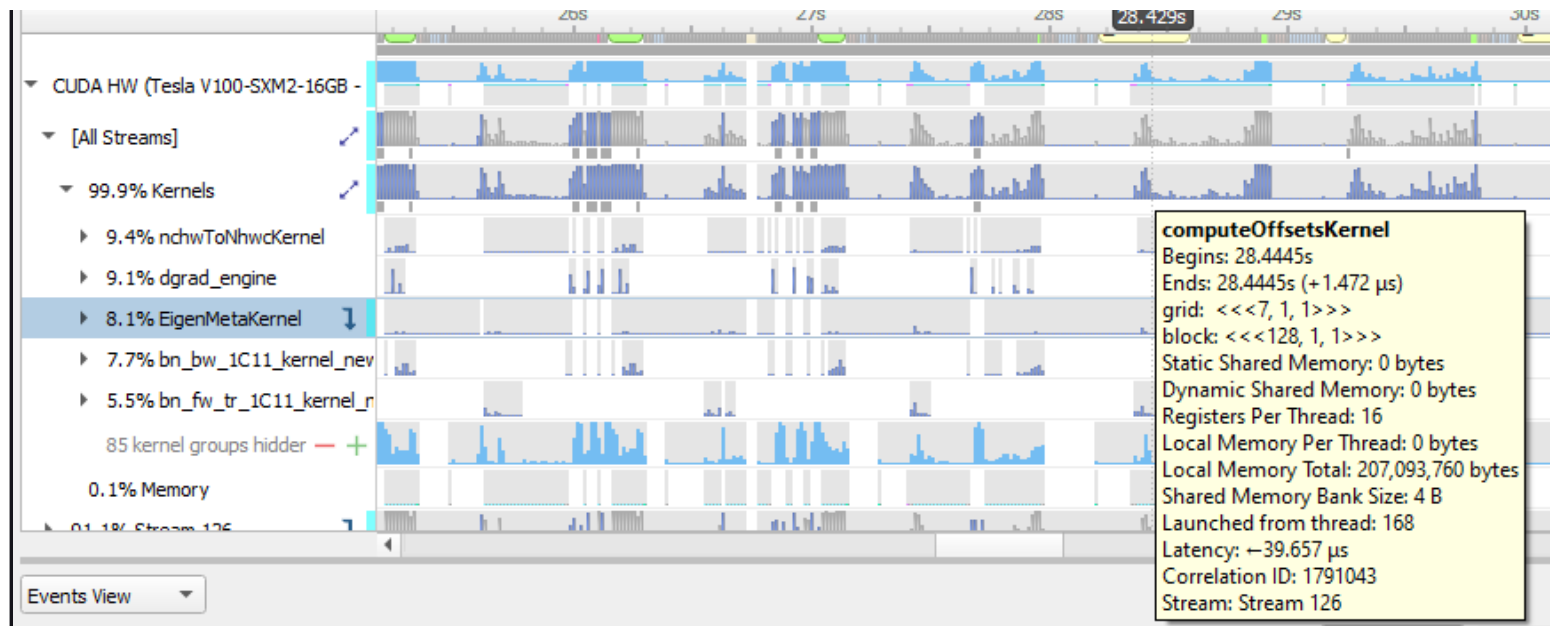
Action	Key or Mouse Gesture
Select a Time Range	Press and drag left mouse button
Drag Selection or Its Border	Press and drag left mouse button on a selection
Deselect a Time Range	Escape, or Click outside a selection
Zoom into a Selection	Z
Zoom into a Selection and Deselect	Shift + Z, or MouseLeftDoubleClick on a selection
Apply Time Filter	F
Apply Time Filter and Deselect	Shift + F

OK

Row Height#

Several of the rows in the timeline use height as a way to model the percent utilization of resources. This gives the user insight into what is going on even when the timeline is zoomed all the way out.





In this picture, you see that for kernel occupation there is a colored bar of variable height.

Nsight Systems calculates the average occupancy for the period of time represented by particular pixel width of screen. It then uses that average to set the top of the colored section. So, for instance, if 25% of that timeslice the kernel is active, the bar goes 25% of the distance to the top of the row.

In order to make the difference clear, if the percentage of the row height is non-zero, but would be represented by less than one vertical pixel, Nsight Systems displays it as one pixel high. The gray height represents the maximum usage in that time range.

This row height coding is used in the CPU utilization, thread and process occupancy, kernel occupancy, and memory transfer activity rows.

Row Percentage#

In the previous image, you also see that there are percentages prefixing the stream rows in the GPU.

The percentage shown in front of the stream indicates the proportion of context running time this particular stream

takes.

$\% \text{ stream} = 100.0 \times \text{streamUsage} / \text{contextUsage}$

streamUsage = total amount of time this stream is active on GPU

contextUsage = total amount of time all streams for this context are active on GPU

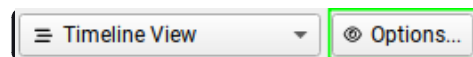
So “26% Stream 1” means that Stream 1 takes 26% of its context’s total running time.

Total running time = sum of durations of all kernels and memory ops that run in this context

Timeline Options#

We strongly recommend using the OS/Desktop defaults for size and color, but if you would like to set them for yourself, they are available using the **Tools > Options** dialog.

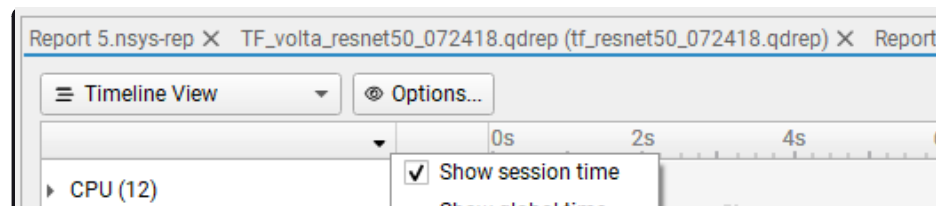
The above will change the options globally for this GUI. It’s also possible to change some options for a particular open report. There is an “Options...” button near the View Selector:

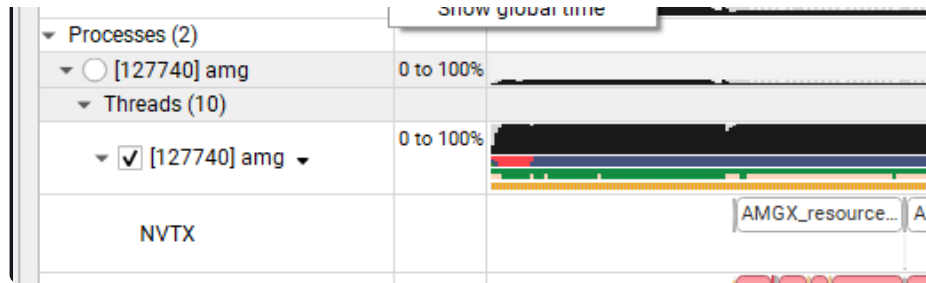


This button will show a dialog that allows showing/hiding the following:

- correlation arrows;
- line labels;
- CPU occupancy chart.

By default, the timeline will be based on session time. If you would like to switch to global time, click on the small arrow at the top of the leftmost column to reveal the dropdown shown below:





Events View#

The Events View provides a tabular display of the trace events. The view contents can be searched and sorted.

Double-clicking an item in the Events View automatically focuses the Timeline View on the corresponding timeline item.

API calls, GPU executions, and debug markers that occurred within the boundaries of a debug marker are displayed nested to that debug marker. Multiple levels of nesting are supported.

Events view recognizes these types of debug markers:

- NVTX
- Vulkan VK_EXT_debug_marker markers, VK_EXT_debug_utils labels
- PIX events and markers
- OpenGL KHR_debug markers

#	Name	Duration	TID	GPU	Context	Start	
39	ID3D12GraphicsCommandList::Reset	13.300 μ s	2092	-	-	0.0016661s	
40	Scene Render	352.100 μ s	2092	-	-	0.0017093s	
41	RenderLightShadows	1.900 μ s	2092	-	-	0.0017207s	
43	Z PrePass	80.300 μ s	2092	-	-	0.0017286s	
49	Generate SSAO	121.700 μ s	2092	-	-	0.0018155s	
57	Render Shadow Map	39.100 μ s	2092	-	-	0.0019445s	
59	Raytrace	68.600 μ s	2092	-	-	0.0019903s	
64	Marker End	-	2092	-	-	0.0020614s	
65	ID3D12GraphicsCommandList::Close	12.400 μ s	2092	-	-	0.0020753s	
66	ID3D12CommandQueue::ExecuteCommandLists	69.800 μ s	2092	-	-	0.0020892s	
67	ntdll.dll!0x7ff9a47ff3b4	-	2092	-	-	0.0021694s	
68	ID3D12GraphicsCommandList::Reset	10.300 μ s	2092	-	-	0.0021988s	
69	Post Effects	61.300 μ s	2092	-	-	0.0022258s	
75	ID3D12GraphicsCommandList::Close	7.100 μ s	2092	-	-	0.0022964s	
76	ID3D12CommandQueue::ExecuteCommandLists	33.300 μ s	2092	-	-	0.0023048s	
77	ntdll.dll!0x7ff9a47ff3b4	-	2092	-	-	0.0023465s	

Call to: ID3D12CommandQueue::ExecuteCommandLists

■ DX12 API calls

Begins: 0.0020892s

Ends: 0.002159s (+69.800 μ s)

Correlation IDs: [30507, 30507]

You can copy and paste from the events view by highlighting rows, using **Shift** or **Ctrl** to enable multi-select. Right clicking on the selection will give you a copy option.

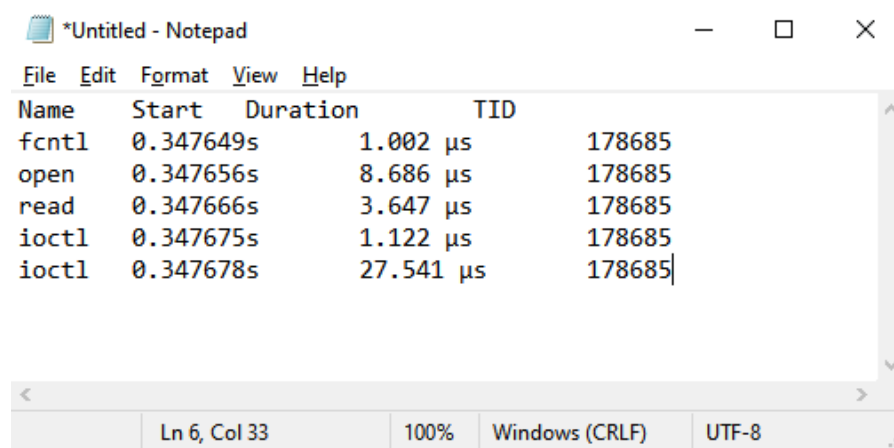
Events View					Name
#	Name	Start	Duration	TID	
4	fopen	0.347611s	5.921 μ s	178685	
5	fclose	0.347628s	1.724 μ s	178685	
6	open64	0.347636s	11.501 μ s	178685	
7	fcntl	0.347649s	1.002 μ s	178685	
8	ioctl	0.347652s	3.176 μ s	178685	
9	open	0.347656s	8.686 μ s	178685	
10	read	0.347666s	3.647 μ s	178685	
11	ioctl	0.347675s	1.122 μ s	178685	
12	ioctl	0.347678s	27.541 μ s	178685	
13	ioctl	0.347707s	6.141 μ s	178685	
14	ioctl	0.347713s	16.040 μ s	178685	

Highlight Selected on Timeline

Show Current on Timeline

Copy Selected

Pasting into text gives you a tab separated view:

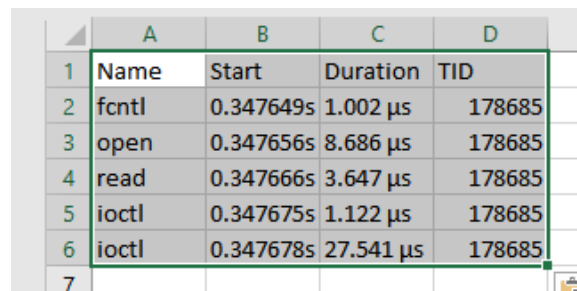


The screenshot shows a Notepad window titled '*Untitled - Notepad'. The menu bar includes File, Edit, Format, View, and Help. The text content is a tab-separated table with four columns: Name, Start, Duration, and TID. The data rows are as follows:

Name	Start	Duration	TID
fcntl	0.347649s	1.002 μs	178685
open	0.347656s	8.686 μs	178685
read	0.347666s	3.647 μs	178685
ioctl	0.347675s	1.122 μs	178685
ioctl	0.347678s	27.541 μs	178685

The status bar at the bottom indicates 'Ln 6, Col 33', '100%', 'Windows (CRLF)', and 'UTF-8'.

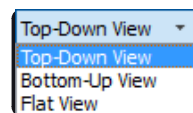
Pasting into spreadsheet properly copies into rows and columns:



The screenshot shows a spreadsheet with the same data as the Notepad window, pasted into rows and columns. The columns are labeled A, B, C, and D. The data rows are as follows:

	A	B	C	D
1	Name	Start	Duration	TID
2	fcntl	0.347649s	1.002 μs	178685
3	open	0.347656s	8.686 μs	178685
4	read	0.347666s	3.647 μs	178685
5	ioctl	0.347675s	1.122 μs	178685
6	ioctl	0.347678s	27.541 μs	178685

Function Table Modes#



The function table can work in three modes:

- **Top-Down View** — In this mode, expanding top-level functions provides information about the *callee* functions. One of the top-level functions is typically the main function of your application, or another entry point defined by the runtime libraries.

- **Bottom-Up View** — This is a reverse of the Top-Down view. On the top level, there are functions directly hit by the sampling profiler. To explore all possible call chains leading to these functions, you need to expand the subtrees of the top-level functions.
- **Flat View** — This view enumerates all functions ever observed by the profiler, even if they have never been directly hit, but just appeared somewhere on the call stack. This view typically provides a high-level overview of which parts of the code are CPU-intensive.

Each of the views helps understand particular performance issues of the application being profiled. For example:

- When trying to find specific bottleneck functions that can be optimized, the Bottom-Up view should be used. Typically, the top few functions should be examined. Expand them to understand in which contexts they are being used.
- To navigate the call tree of the application and while generally searching for algorithms and parts of the code that consume unexpectedly large amount of CPU time, the Top-Down view should be used.
- To quickly assess which parts of the application, or high level parts of an algorithm, consume significant amount of CPU time, use the Flat view.

The Top-Down and Bottom-Up views have *Self* and *Total* columns, while the Flat view has a *Flat* column. It is important to understand the meaning of each of the columns:

- Top-Down view
- **Self** column denotes the relative amount of time spent executing instructions of this particular function.
- **Total** column shows how much time has been spent executing this function, including all other functions called from this one. Total values of sibling rows sum up to the Total value of the parent row, or 100% for the top-level rows.
- Bottom-Up view
- **Self** column for *top-level rows*, as in the Top-Down view, shows how much time has been spent directly in this function. Self times of all top-level rows add up to 100%.
- **Self** column for *children rows* breaks down the value of the parent row based on the various call chains leading to that function. Self times of sibling rows add up to the value of the parent row.

- Flat view
- **Flat** column shows how much time this function has been anywhere on the call stack. Values in this column do not add up or have other significant relationships.

Note

If low-impact functions have been filtered out, values may not add up correctly to 100%, or to the value of the parent row. This filtering can be disabled.

Contents of the symbols table is tightly related to the timeline. Users can apply and modify filters on the timeline, and they will affect which information is displayed in the symbols table:

- **Per-thread filtering** — Each thread that has sampling information associated with it has a checkbox next to it on the timeline. Only threads with selected checkboxes are represented in the symbols table.
- **Time filtering** — A time filter can be setup on the timeline by pressing the left mouse button, dragging over a region of interest on the timeline, and then choosing **Filter by selection** in the dropdown menu. In this case, only sampling information collected during the selected time range will be used to build the symbols table.

Note

If too little sampling data is being used to build the symbols table (for example, when the sampling rate is configured to be low, and a short period of time is used for time-based filtering), the numbers in the symbols table might not be representative or accurate in some cases.

Function Table Notes#

Last Branch Records vs. Frame Pointers

Two of the mechanisms available for collecting backtraces are Intel Last Branch Records (LBRs) and frame pointers. LBRs are used to trace every branch instruction via a limited set of hardware registers. They can be configured to generate backtraces but have finite depth based on the CPU's microarchitecture. LBRs are effectively free to collect but may not be as deep as you need in order to fully understand how the workload arrived a specific Instruction

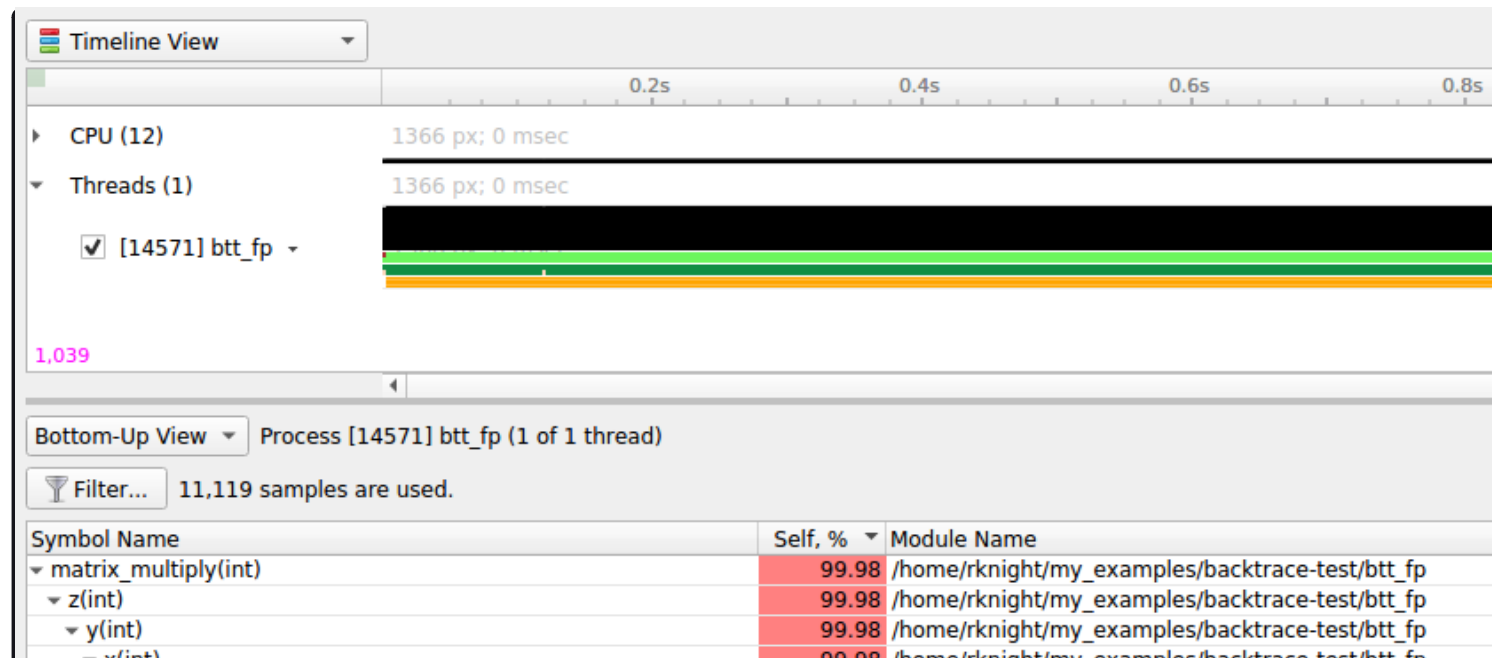
Pointer (IP).

Frame pointers only work when a binary is compiled with the `-fno-omit-frame-pointer` compiler switch. To determine if frame pointers are enabled on an x86_64 binary running on Linux, dump a binary's assembly code using the `objdump -d [binary_file]` command and look for this pattern at the beginning of all functions;

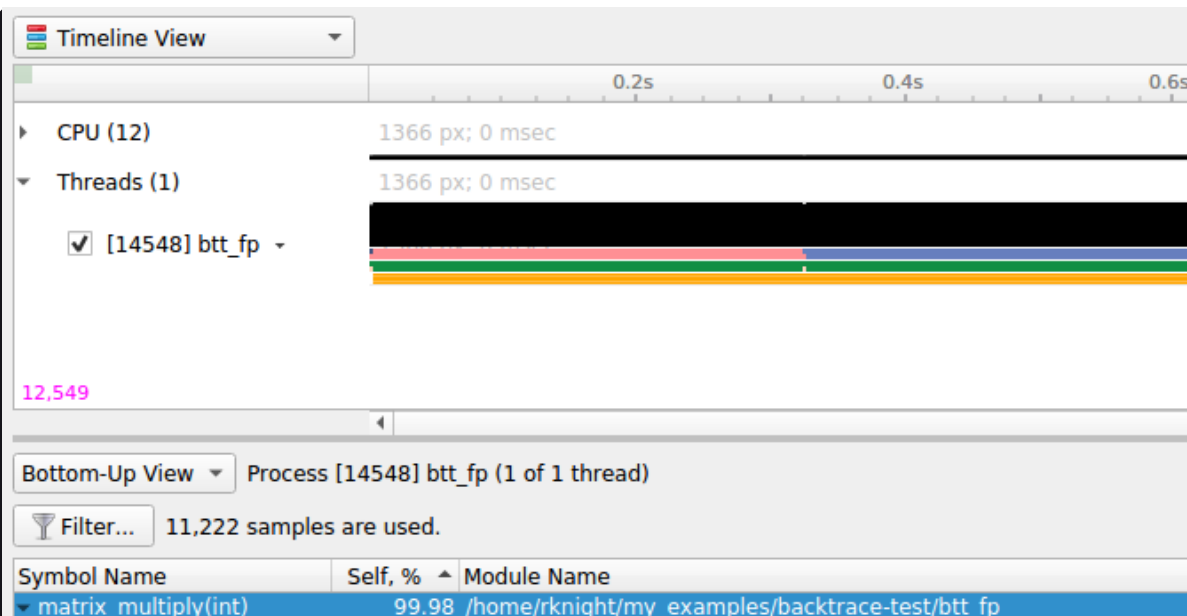
```
push %rbp
mov %rsp,%rbp
```

When frame pointers are available in a binary, full stack traces will be captured. Note that libraries that are frequently used by applications and ship with the operating system, such as `libc`, are generated in release mode and therefore do not include frame pointers. Frequently, when a backtrace includes an address from a system library, the backtrace will fail to resolve further as the frame pointer trail goes cold due to a missing frame pointer.

A simple application was developed to show the difference. The application calls function `a()`, which calls `b()`, which calls `c()`, etc. Function `z()` calls a heavy compute function called `matrix_multiply()`. Almost all of the IP samples are collected while `matrix_multiply` is executing. The next two screen shots show one of the main differences between frame pointers and LBRs.



x(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
w(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
v(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
u(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
t(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
s(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
r(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
q(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
p(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
o(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
n(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
m(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
l(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
k(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
j(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
i(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
h(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
g(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
f(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
e(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
d(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
c(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
b(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
a(int)	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
main	99.98	/home/rknight/my_examples/backtrace-test/btt_fp
_libc_start_main	99.98	/lib/x86_64-linux-gnu/libc-2.19.so

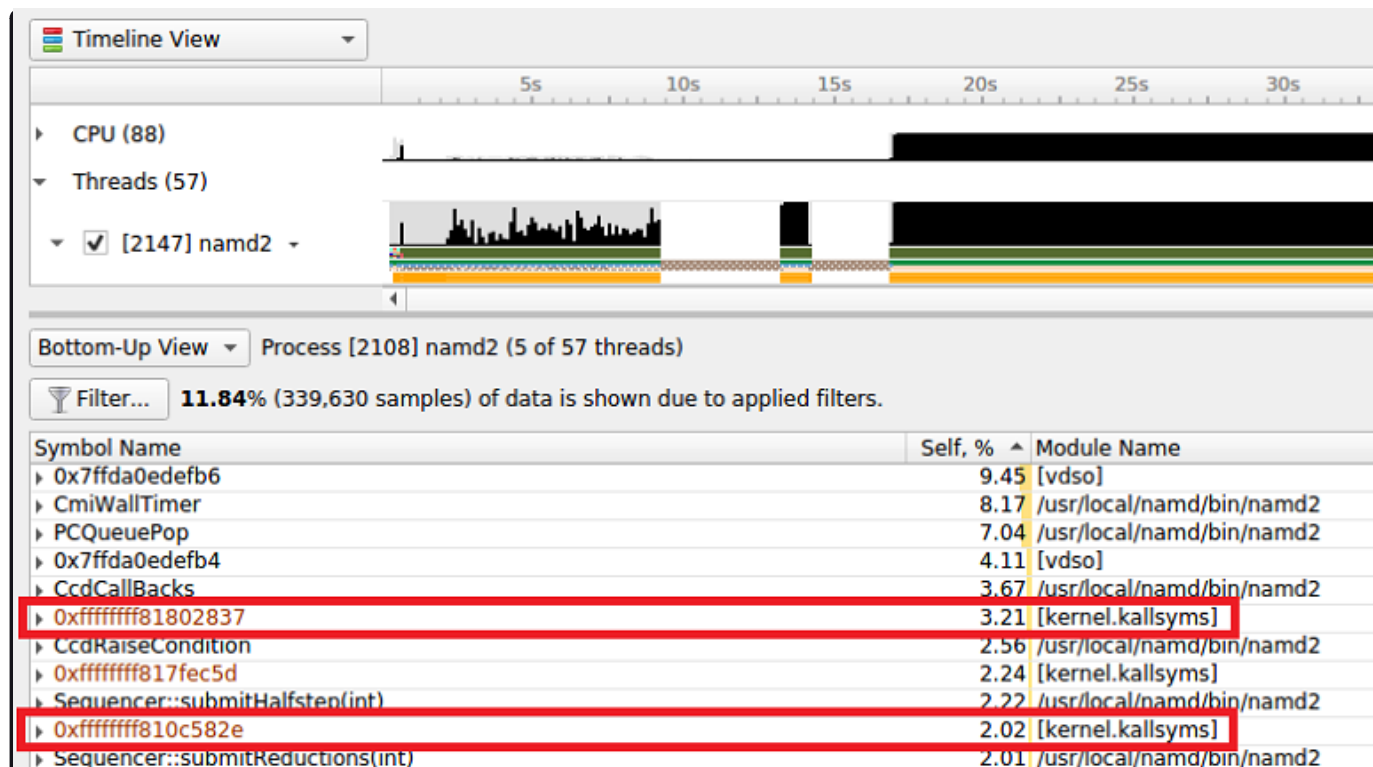


[Max depth]	97.07	[Max depth]
▼ z(int)	2.91	/home/rknight/my_examples/backtrace-test/btt_fp
▼ y(int)	2.91	/home/rknight/my_examples/backtrace-test/btt_fp
▼ x(int)	2.91	/home/rknight/my_examples/backtrace-test/btt_fp
▼ w(int)	2.91	/home/rknight/my_examples/backtrace-test/btt_fp
▼ v(int)	2.91	/home/rknight/my_examples/backtrace-test/btt_fp
▼ u(int)	2.91	/home/rknight/my_examples/backtrace-test/btt_fp
▼ t(int)	2.91	/home/rknight/my_examples/backtrace-test/btt_fp
▼ s(int)	2.91	/home/rknight/my_examples/backtrace-test/btt_fp
[Max depth]	2.91	[Max depth]

Note that the frame pointer example shows the full stack trace, while the LBR example only shows part of the stack due to the limited number of LBR registers in the CPU.

Kernel Samples

When an IP sample is captured while a kernel mode (i.e. operating system) function is executing, the sample will be shown with an address that starts with 0xffffffff and map to the [kernel.kallsyms] module.



[vdso]

Samples may be collected while a CPU is executing functions in the Virtual Dynamic Shared Object. In this case, the sample will be resolved (i.e., mapped) to the [vdso] module. The vdso [man page](#) provides the following description of the vdso:

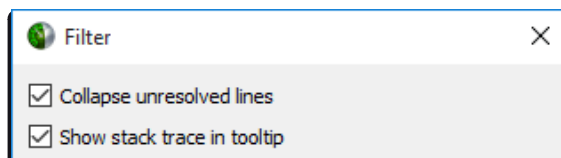
The “vDSO” (virtual dynamic shared object) is a small shared library that the kernel automatically maps into the address space of all user-space applications. Applications usually do not need to concern themselves with these details as the vDSO is most commonly called by the C library. This way you can code in the normal way using standard functions and the C library will take care of using any functionality that is available via the vDSO.

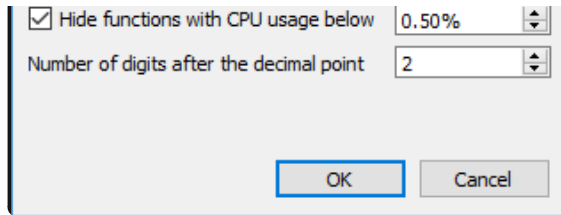
Why does the vDSO exist at all? There are some system calls the kernel provides that user-space code ends up using frequently, to the point that such calls can dominate overall performance. This is due both to the frequency of the call as well as the context-switch overhead that results from exiting user space and entering the kernel.

[Unknown]

When an address can not be resolved (i.e., mapped to a module), its address within the process' address space will be shown and its module will be marked as [Unknown].

Filter Dialog#





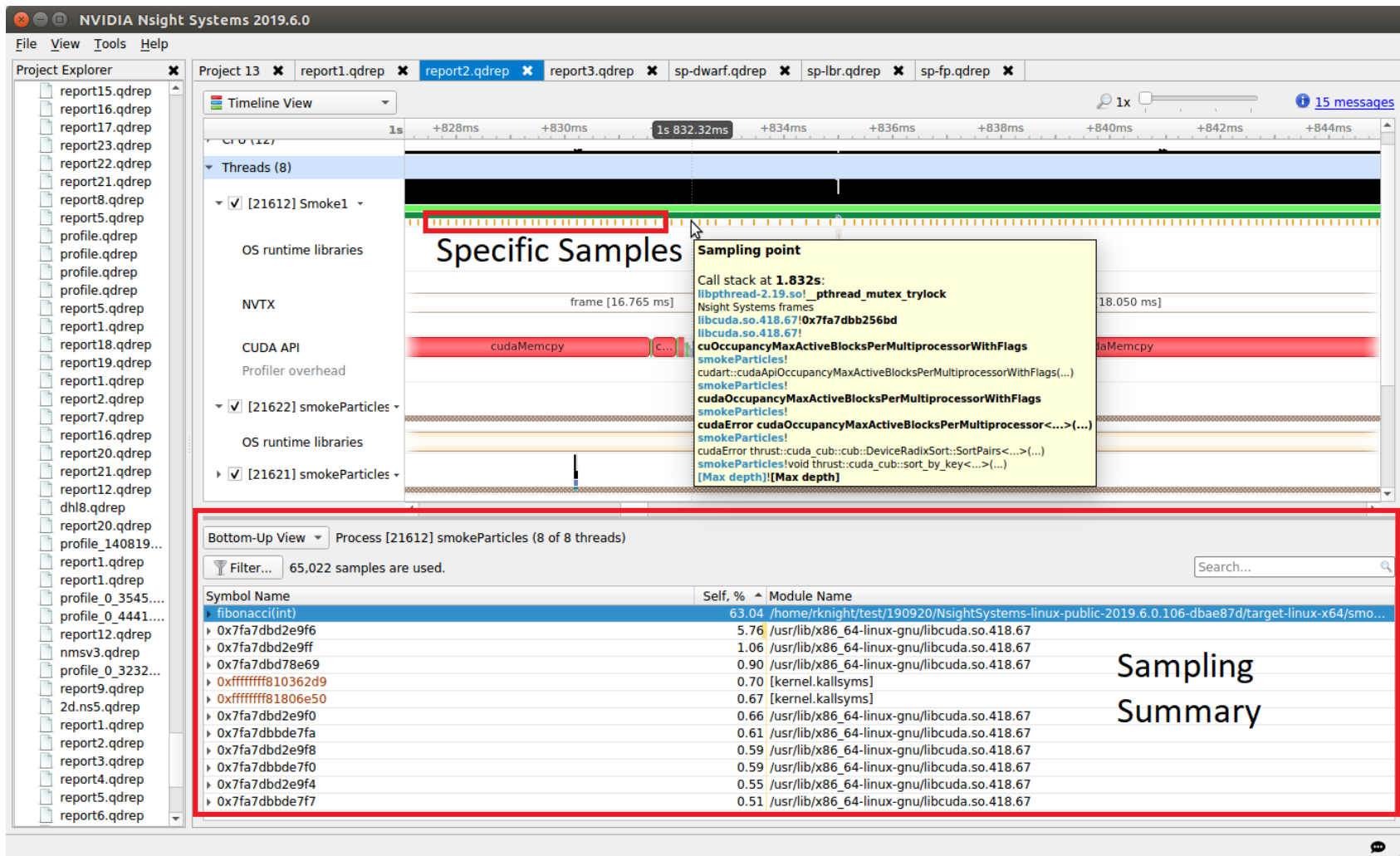
- **Collapse unresolved lines** is useful if some of the binary code does not have symbols. In this case, subtrees that consist of only unresolved symbols get collapsed in the Top-Down view, since they provide very little useful information.
- **Hide functions with CPU usage below X%** is useful for large applications, where the sampling profiler hits lots of function just a few times. To filter out the “long tail,” which is typically not important for CPU performance bottleneck analysis, this checkbox should be selected.

Example of Using Timeline with Function Table#

Here is an example walkthrough of using the timeline and function table with Instruction Pointer (IP)/backtrace Sampling Data

Timeline

When a collection result is opened in the Nsight Systems GUI, there are multiple ways to view the CPU profiling data - especially the CPU IP / backtrace data.

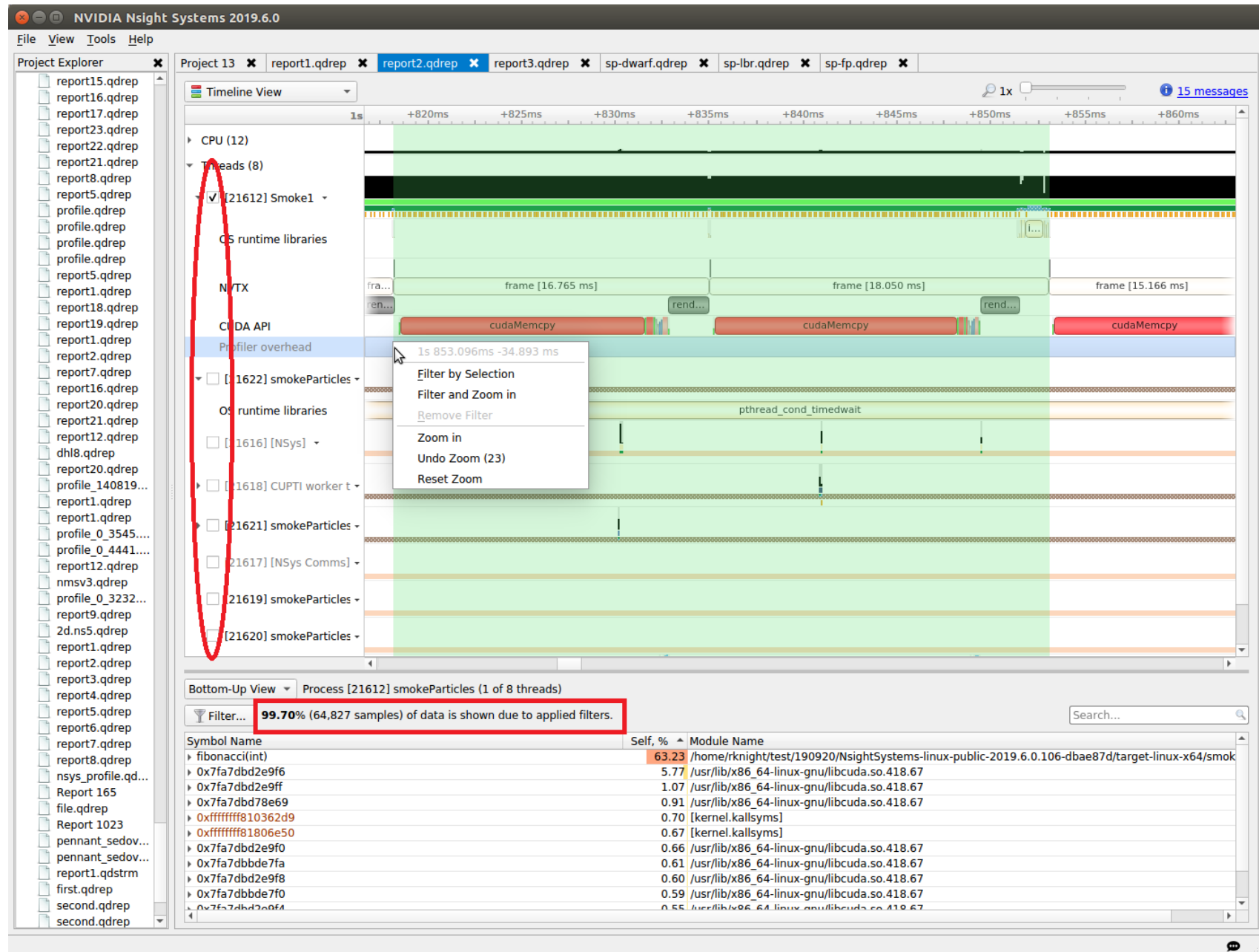


In the timeline, yellow-orange marks can be found under each thread's timeline that indicate the moment an IP / backtrace sample was collected on that thread (e.g., see the yellow-orange marks in the Specific Samples box above). Hovering the cursor over a mark will cause a tooltip to display the backtrace for that sample.

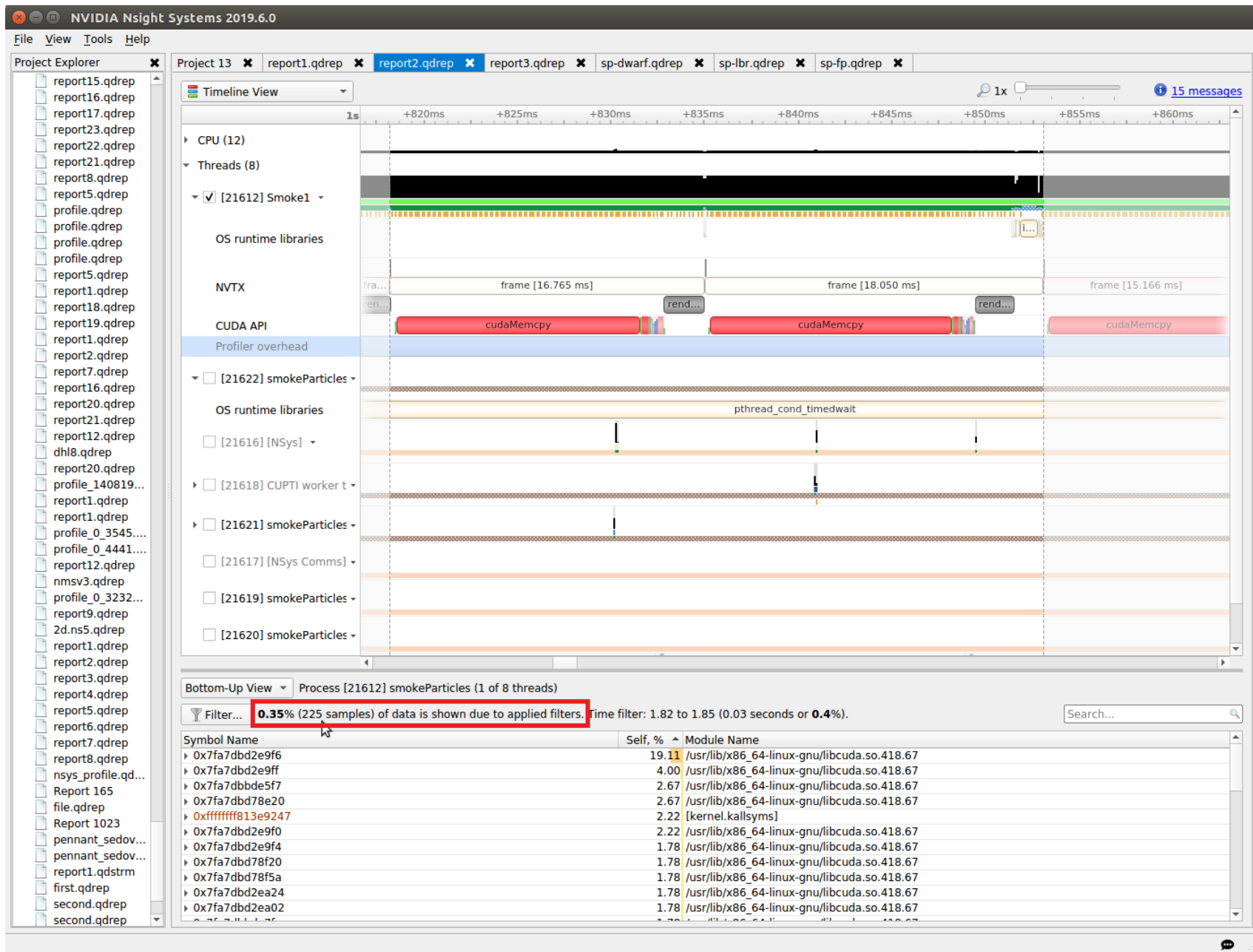
Below the Timeline is a drop-down list with multiple options including Events View, Top-Down View, Bottom-Up View, and Flat View. All four of these views can be used to view CPU IP / backtrace sampling data.

If the Bottom-Up View is selected, here is the sampling summary shown in the bottom half of the Timeline View screen. Notice that the summary includes the phrase "65,022 samples are used," indicating how many samples are

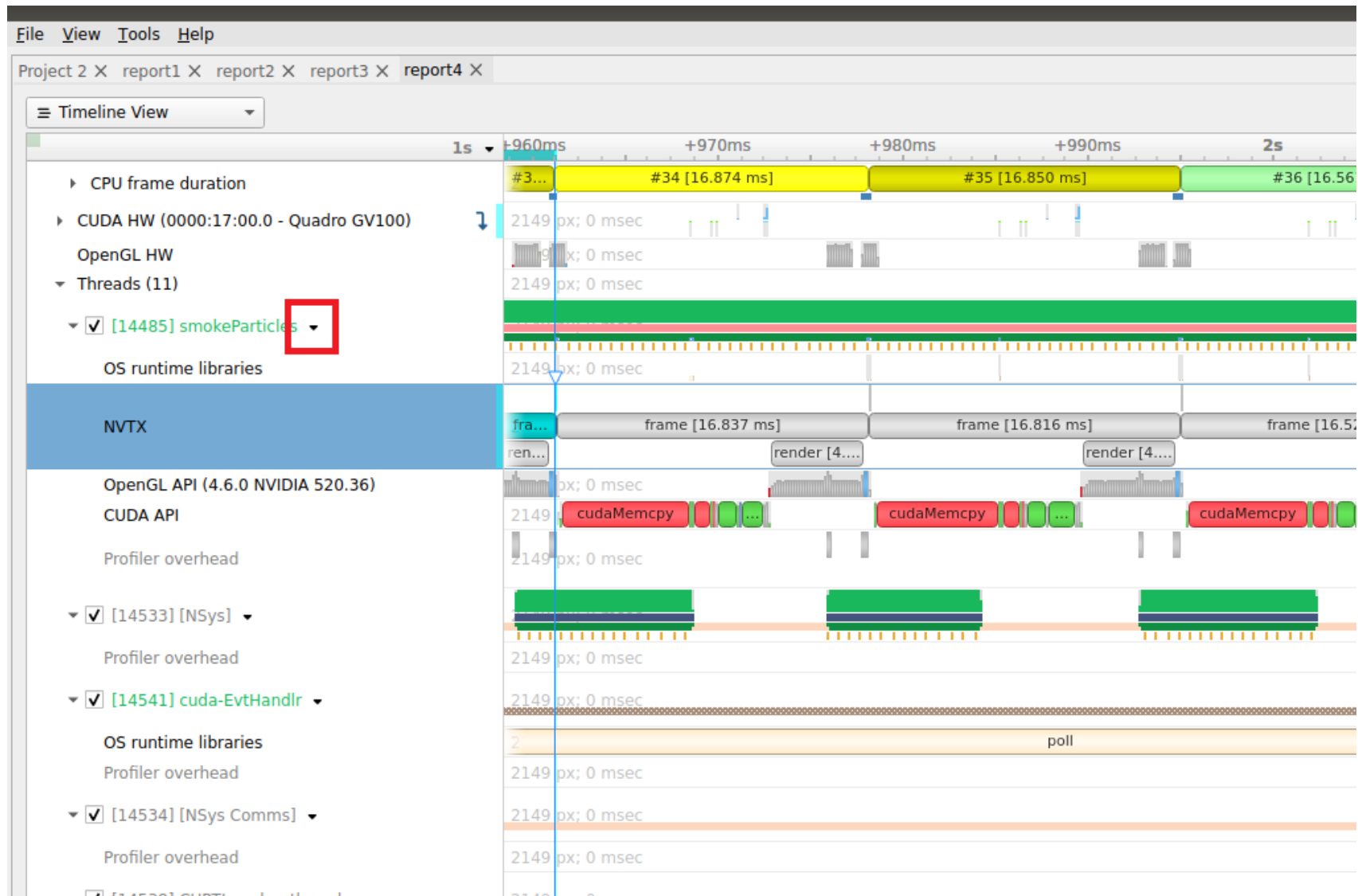
summarized. By default, functions that were found in less than 0.5% of the samples are not shown. Use the filter button to modify that setting.



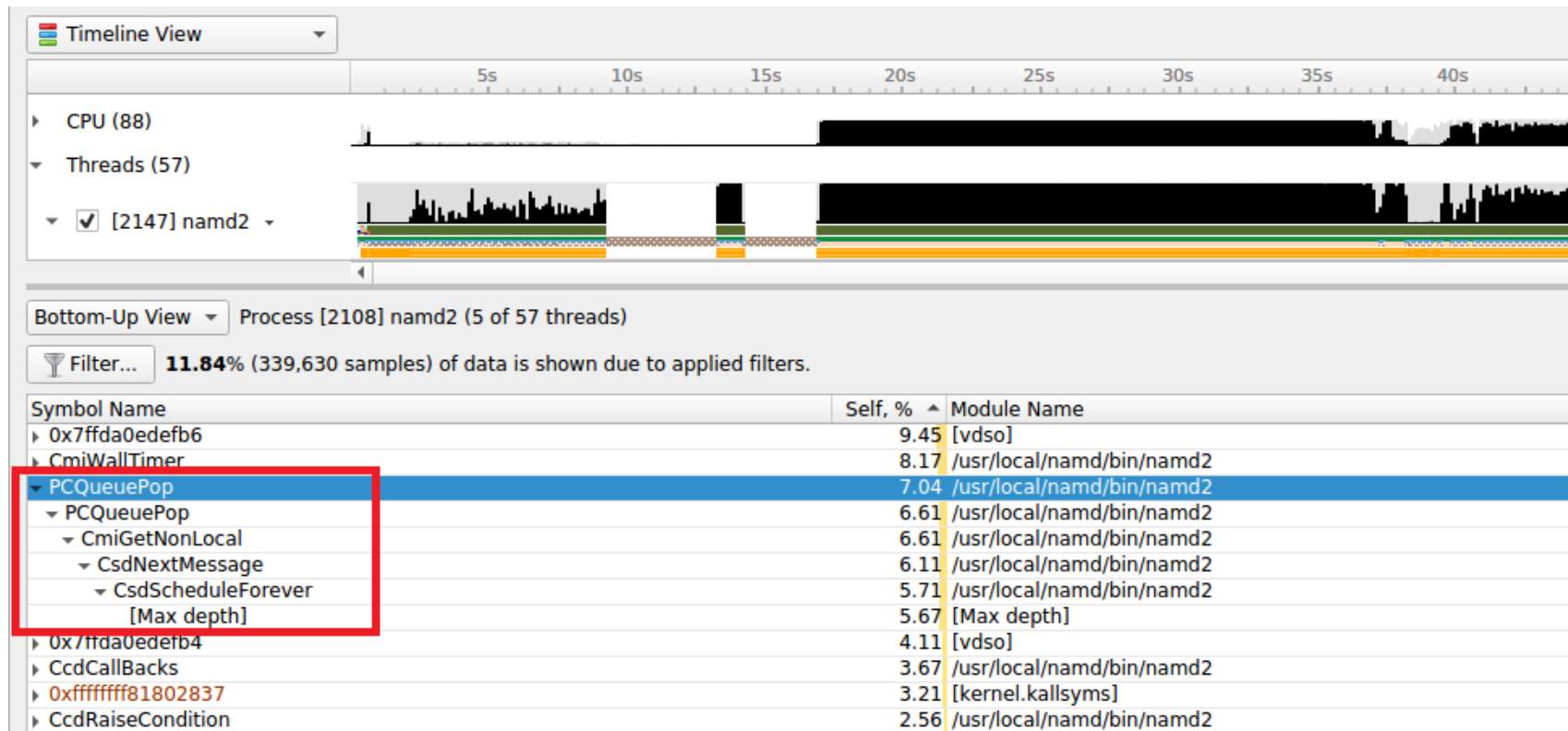
When sampling data is filtered, the Sampling Summary will summarize the selected samples. Samples can be filtered on an OS thread basis, on a time basis, or both. Above, deselecting a checkbox next to a thread removes its samples from the sampling summary. Dragging the cursor over the timeline and selecting “Filter and Zoom In” chooses the samples during the time selected, as seen below. The sample summary includes the phrase “0.35% (225 samples) of data is shown due to applied filters” indicating that only 225 samples are included in the summary results.



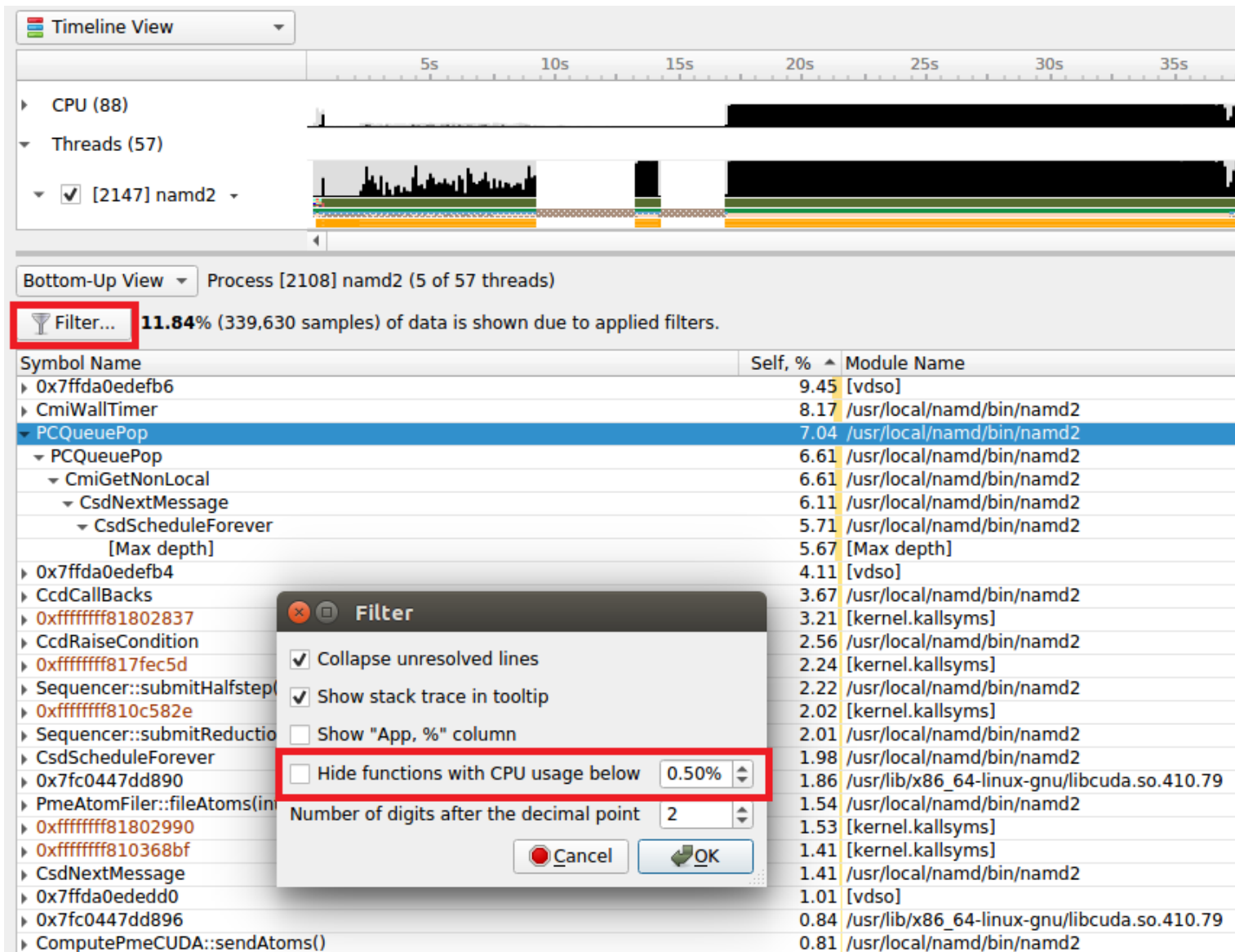
Deselecting threads one at a time by deselecting their checkbox can be tedious. Click on the down arrow next to a thread and choose Show Only This Thread to deselect all threads except that thread.



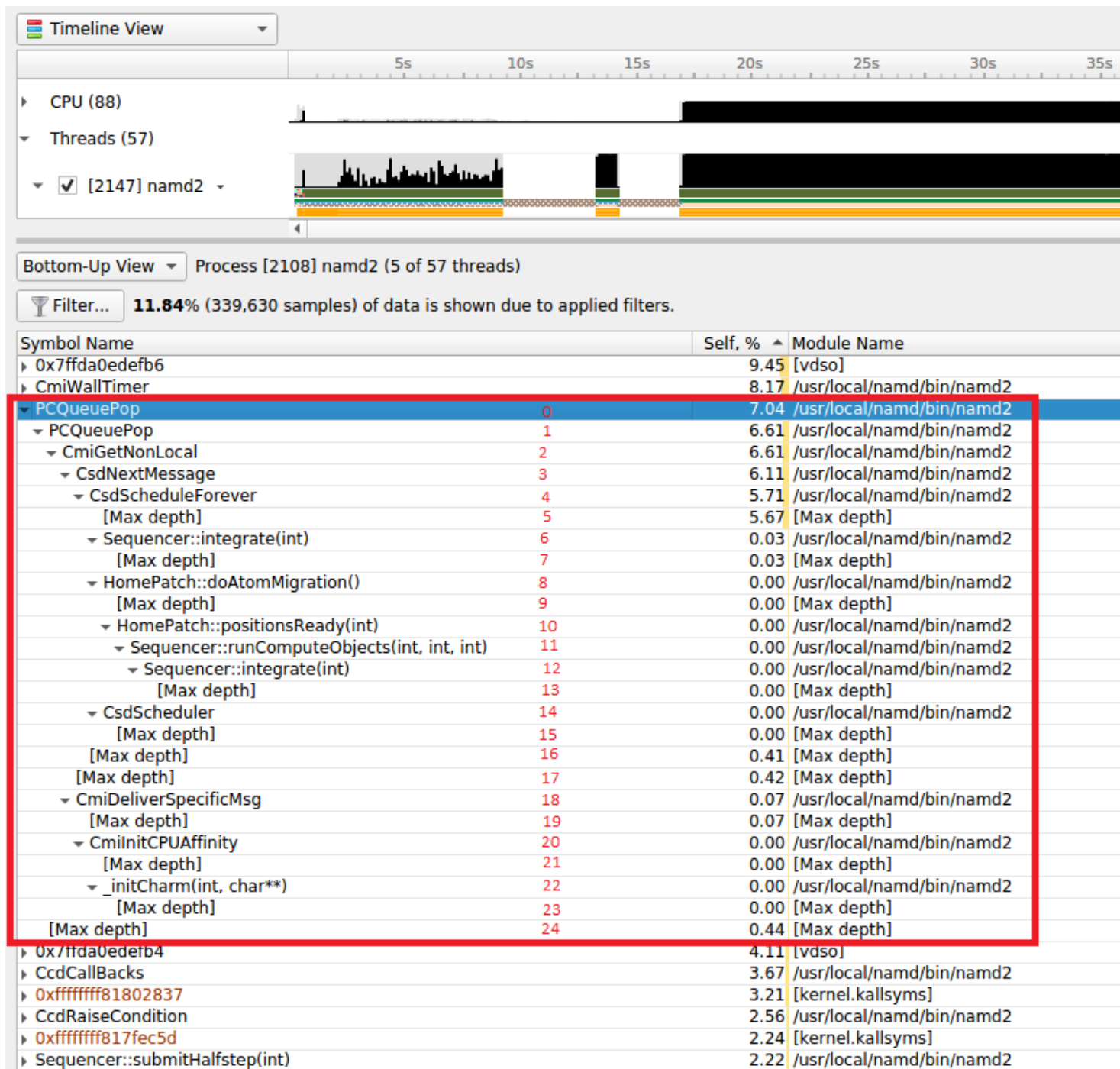
If the Events View is selected in the Timeline View's drop-down list, right click on a specific thread and choose Show in Events View. The samples collected while that thread executed will be shown in the Events View. Double-clicking on a specific sample in the Events view causes the timeline to show when that sample was collected; see the green boxes below. The backtrace for that sample is also shown in the Events View.



Note that, by default, backtraces with less than 0.5% of the total backtraces are hidden. This behavior can make the percentage results hard to understand. If all backtraces are shown (i.e., the filter is disabled), the results look very different and the numbers add up as expected. To disable the filter, click on the Filter... button and uncheck the **Hide functions with CPU usage below X%** checkbox.



When the filter is disabled, the backtraces are recalculated. Note that you may need to right-click on the function and select **Expand** again to get all of the backtraces to be shown.



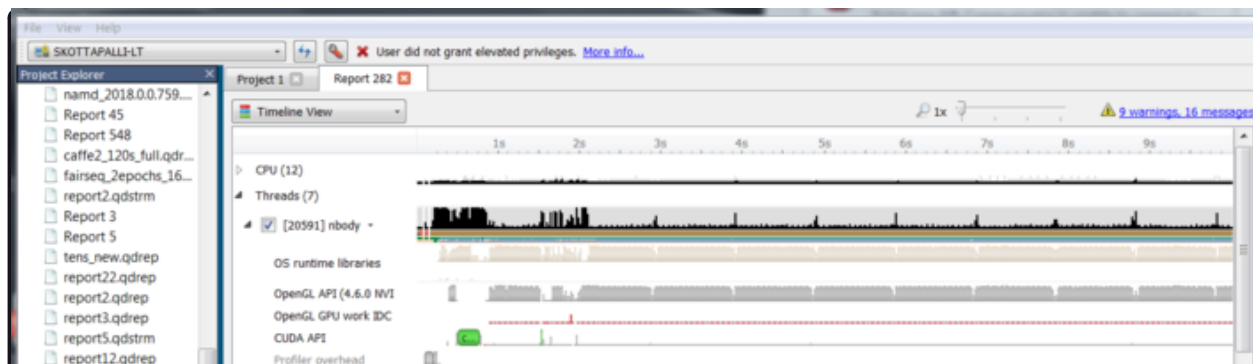
When backtraces are collected, the whole sample (IP and backtrace) is handled as a single sample. If two samples have the exact same IP and backtrace, they are summed in the final results. If two samples have the same IP but a different backtrace, they will be shown as having the same leaf (i.e., IP) but a different backtrace. As mentioned earlier, when backtraces end, they are marked with the [Max depth] string (unless the backtrace can be traced back to its origin; e.g., `__libc_start_main`) or the backtrace breaks because an IP cannot be resolved.

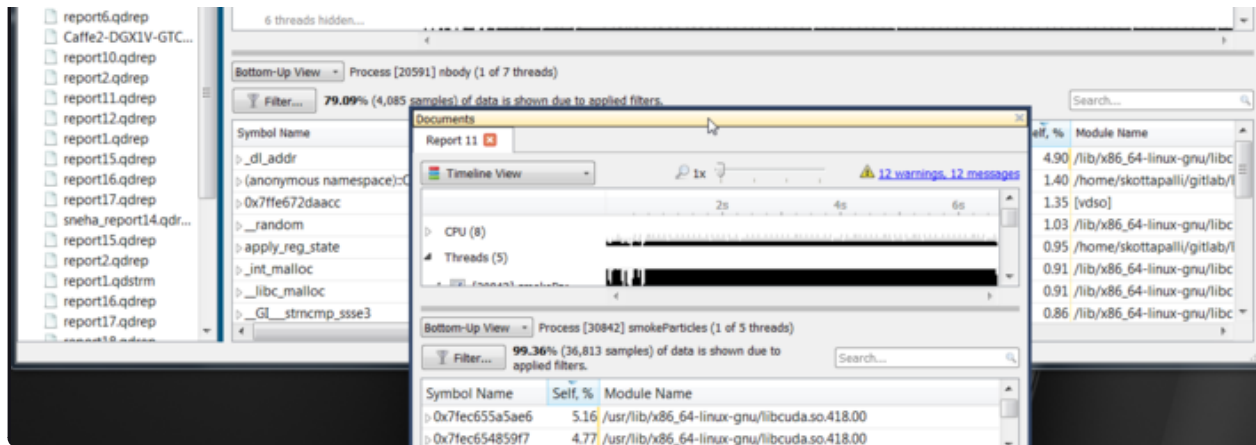
Above, the leaf function is `PCQueuePop`. In this case, there are 11 different backtraces that lead to `PCQueuePop` — all of them end with [Max depth]. For example, the dominant path is `PCQueuePop<-CmiGetNonLocal<-CsdNextmessage<-CsdScheduleForever<- [Max depth]`. This path accounts for 5.67% of all samples as shown in line 5 (red numbers). The second most dominant path is `PCQueuePop<-CmiGetNonLocal<- [Max depth]` which accounts for 0.44% of all samples as shown in line 24 (red numbers). The path `PCQueuePop<-CmiGetNonLocal<-CsdNextmessage<-CsdScheduleForever<-Sequencer::integrate(int)<- [Max depth]` accounts for 0.03% of the samples as shown in line 7 (red numbers). Adding up percentages shown in the [Max depth] lines (lines 5, 7, 9, 13, 15, 16, 17, 19, 21, 23, and 24) generates 7.04% which equals the percentage of samples associated with the `PCQueuePop` function shown in line 0 (red numbers).

Multi-Report Timeline Views#

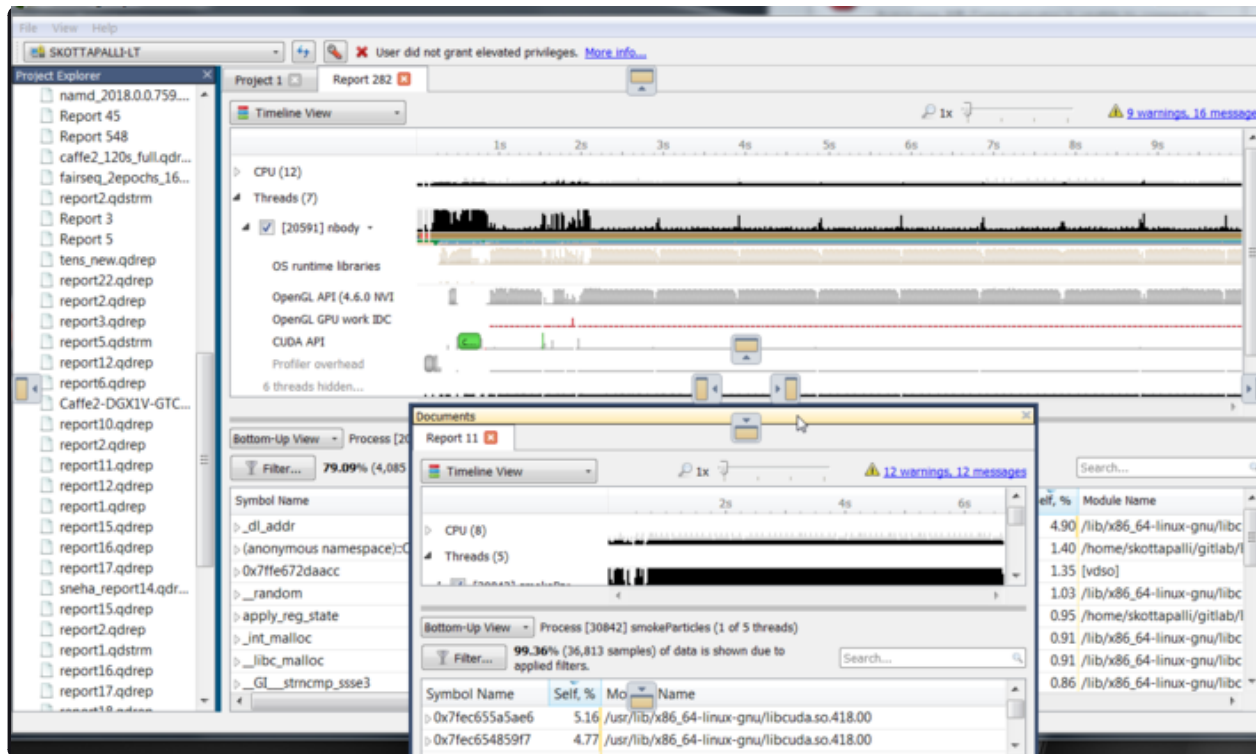
Viewing Multiple Reports in Separate Panes#

You have the option of looking at two or more Nsight Systems results files in separate panes. To do so, open each in a tab. Grab one of the tabs and undock:

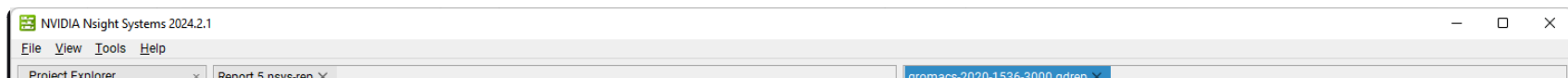


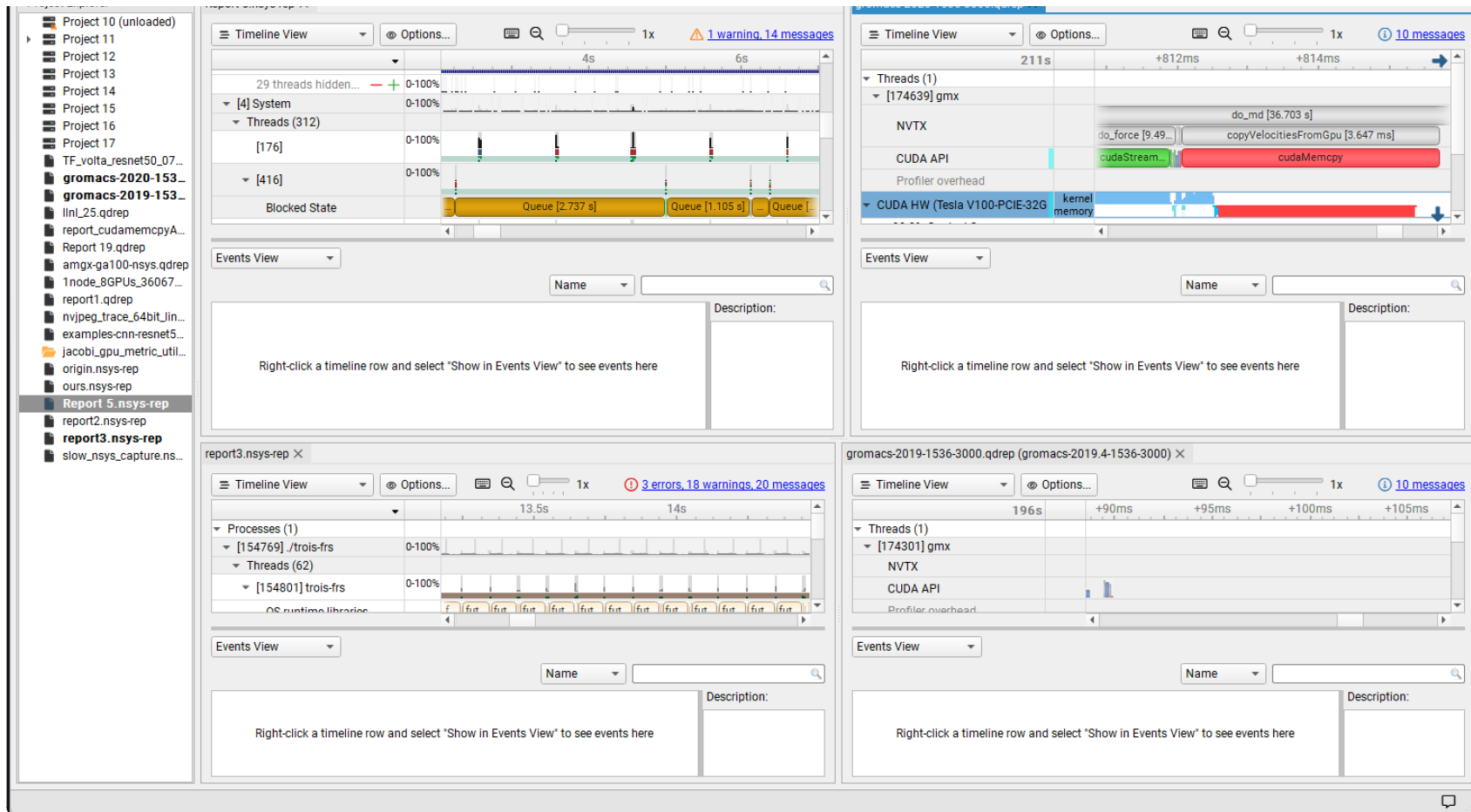


When you hover with the cursor in the middle of the GUI, you will see options for where to dock the pane:



Multiple reports can be docked in the window.

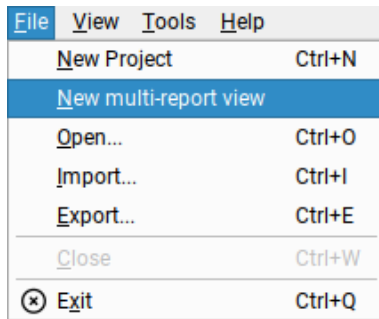




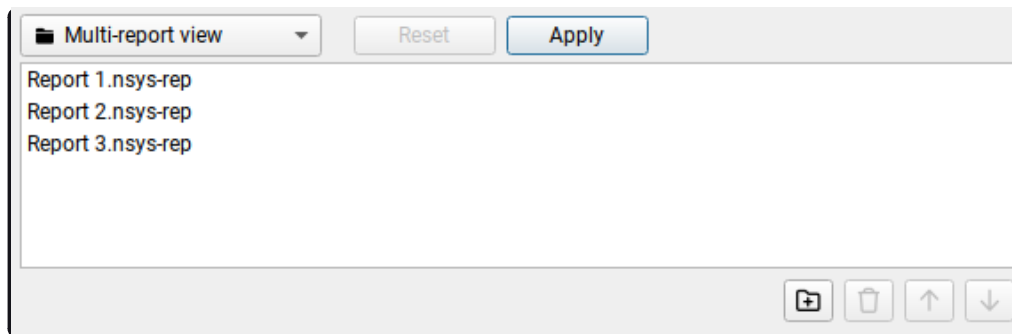
Viewing Multiple Reports in the Same Timeline#

You can open several reports in a single timeline. This could be done using one of these methods:

- **File > Open...** in the main menu, and select several report files.
- **File > New multi-report view** in the main menu, add report files that you want to open in the Multi-report view, and click the “Apply” button.



Multi-report view contains simple editor that allows to add/remove some report files and will load them all on a single timeline after applying that set of reports.



When reports are loaded, one can use the View Selector to open the Multi-report view again, change the set of reports, and click on “Apply” button to reload the timeline with the new set of reports.

The selected set of reports can be saved as a Multi-report view document and could be opened later to load the same set again.

Time Synchronization#

When multiple reports are loaded into a single timeline, timestamps between them need to be adjusted, such that events that happened at the same time appear to be aligned.

Nsight Systems can automatically adjust timestamps based on **UTC time** recorded around the collection start time. This method is used by default when other more precise methods are not available. This time can be seen as UTC

time at `t=0` in the *Analysis Summary* page of the report file. Refer to your OS documentation to learn how to sync the software clock using the Network Time Protocol (NTP). NTP-based time synchronization is not very precise, with the typical errors on the scale of one to tens of milliseconds.

Reports collected on the same physical machine can use synchronization based on **Timestamp Counter (TSC) values**. These are platform-specific counters, typically accessed in user space applications using the RDTSC instruction on x86_64 architecture, or by reading the CNTVCT register on Arm64. Their values converted to nanoseconds can be seen as `TSC value at t=0` in the *Analysis Summary* page of the report file. Reports synchronized using TSC values can be aligned with nanoseconds-level precision.

TSC-based time synchronization is activated automatically, when Nsight Systems detects that reports come from same target and that the same TSC value corresponds to very close UTC times. Targets are considered to be the same when either explicitly set environment variables `NSYS_HW_ID` are the same for both reports or when target hostnames are the same and `NSYS_HW_ID` is not set for either target. The difference between UTC and TSC time offsets must be below 1 second to choose TSC-based time synchronization.

To find out which synchronization method was used, navigate to the *Analysis Summary* tab of an added report and check the `Report alignment source` property of a target. Note, that the first report won't have this parameter.

Target

Target name	9a1630ecdd6a
Local time at t=0	2021-07-02T12:01:57.310Z
UTC time at t=0	2021-07-02T12:01:57.310Z
TSC value at t=0	1041856117291223
Report alignment source	TSC

Target

Target name	9e2247e584e1
Local time at t=0	2021-07-02T12:01:57.311Z
UTC time at t=0	2021-07-02T12:01:57.311Z
TSC value at t=0	1041856118165144
Report alignment source	UTC

When loading multiple reports into a single timeline, it is always advisable to first check that time synchronization looks correct, by zooming into synchronization or communication events that are expected to be aligned.

Timeline Hierarchy#

When reports are added to the same timeline Nsight Systems will automatically line them up by timestamps as described above. If you want Nsight Systems to also recognize matching process or hardware information, you will need to set environment variables `NSYS_SYSTEM_ID` and `NSYS_HW_ID` as shown below at the time of report collection (such as when using the “`nsys profile ...`” command).

When loading a pair of given report files into the same timeline, they will be merged in one of the following configurations:

- **Different hardware** — is used when reports are coming from different physical machines, and no hardware resources are shared in these reports. This mode is used when neither `NSYS_HW_ID` or `NSYS_SYSTEM_ID` is set and target hostnames are different or absent, and can be additionally signalled by specifying different `NSYS_HW_ID` values.
- **Different systems, same hardware** — is used when reports are collected on different virtual machines (VMs) or containers on the same physical machine. To activate this mode, specify the same value of `NSYS_HW_ID` when collecting the reports.
- **Same system** — is used when reports are collected within the same operating system (or container) environment. In this mode a process identifier (PID) 100 will refer to the same process in both reports. To manually activate this mode, specify the same value of `NSYS_SYSTEM_ID` when collecting the reports. This mode is automatically selected when target hostnames are the same and neither `NSYS_HW_ID` or `NSYS_SYSTEM_ID` is provided.

The following diagrams demonstrate typical cases:

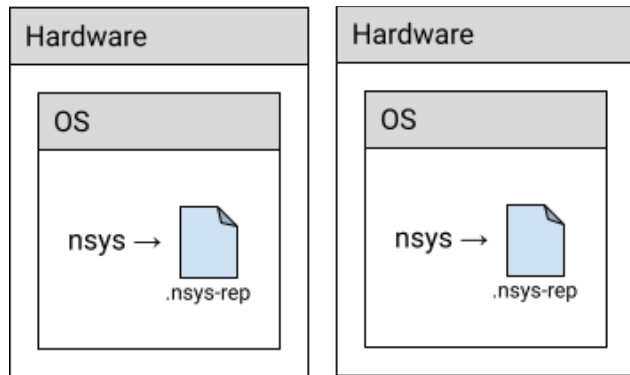


Fig 1. Different hardware (default mode)

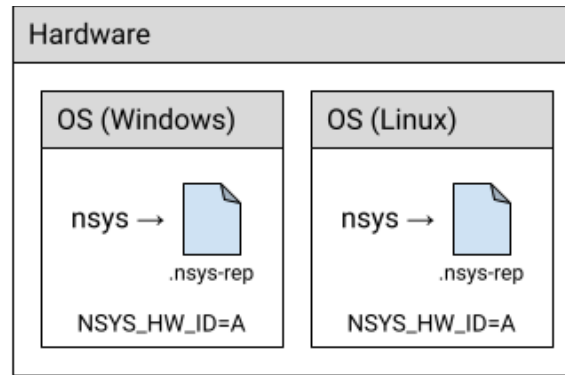


Fig 2. Same hardware, different systems (VMs)

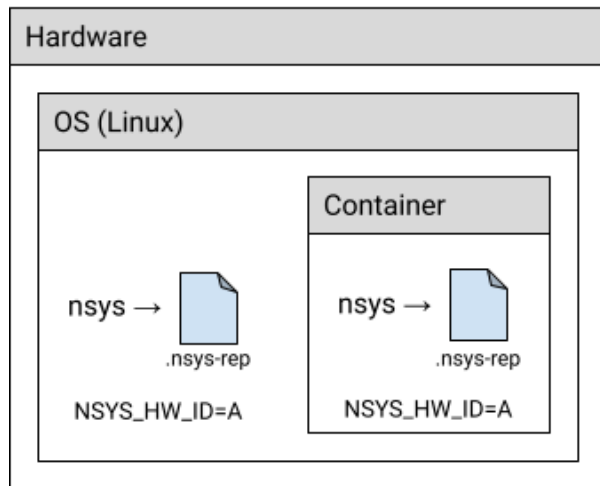


Fig 3. Same hardware, different systems (host and container)

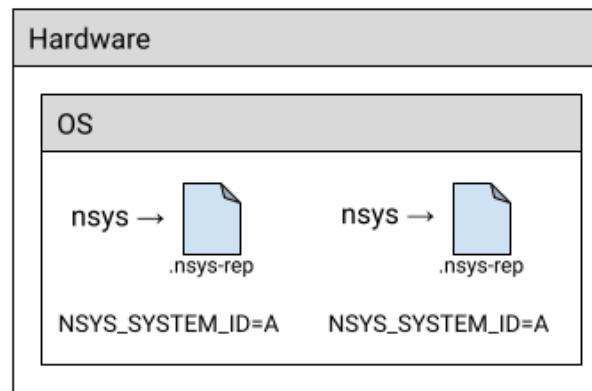


Fig 4. Same system

Example: MPI#

A typical scenario is when a computing job is run using one of the MPI implementations. Each instance of the app can be profiled separately, resulting in multiple report files. For example:

```
# Run MPI job without the profiler:
mpirun <mpirun-options> ./myApp
```

Run MPI job and profile each instance of the application:

```
mpirun <mpirun-options> nsys profile -o report-%p <nsys-options> ./myApp
```

When each MPI rank runs on a different node, the command above works fine, since the default pairing mode (different hardware) will be used.

When all MPI ranks run the localhost only, use this command (value “A” was chosen arbitrarily, it can be any non-empty string):

```
NSYS_SYSTEM_ID=A mpirun <mpirun-options> nsys profile -o report-%p <nsys -options> ./myApp
```

For convenience, the MPI rank can be encoded into the report filename. For Open MPI, use the following command to create report files based on the global rank value:

```
mpirun <mpirun-options> nsys profile -o report-%q{OMPI_COMM_WORLD_RANK}<nsys -options> ./myApp
```

MPICH-based implementations set the environment variable PMI_RANK and Slurm (s run) provides the global MPI rank in SLURM_PROCID.

Limitations on Syncing Multiple Reports in Timeline#

- Only report files collected with Nsight Systems version 2021.3 and newer are fully supported.
- Sequential reports collected in a single CLI profiling session cannot be loaded into a single timeline yet.

Add-on Graphs - Flame Graph#

The generation of Flame Graphs from Nsight Systems reports is not a built-in feature, but it is possible to create such graphs from Nsight Systems reports with the script `stackcollapse_nsys.py` located at `<nsys-install-dir>/<host-folder>/Scripts/Flamegraph`. There is also a `README.md` file at that location with additional usage details.

Requirements:

- `flamegraph.pl` from [Brendan Gregg's FlameGraph github](#),

- Perl

Usage

Generating flamegraph from Nsight Systems report file on Linux:

```
python3 stackcollapse_nsys.py report.nsys-rep | ./flamegraph.pl > result_flamegraph.svg
```

Generating flamegraph from Nsight Systems report file on Windows:

```
PowerShell -Command "python stackcollapse_nsys.py report.nsys-rep | perl flamegraph.pl > result_flamegraph.svg"
```

The script exports the report to SQLite, queries the CPU samples and passes them as input to flamegraph.pl.

Parameters

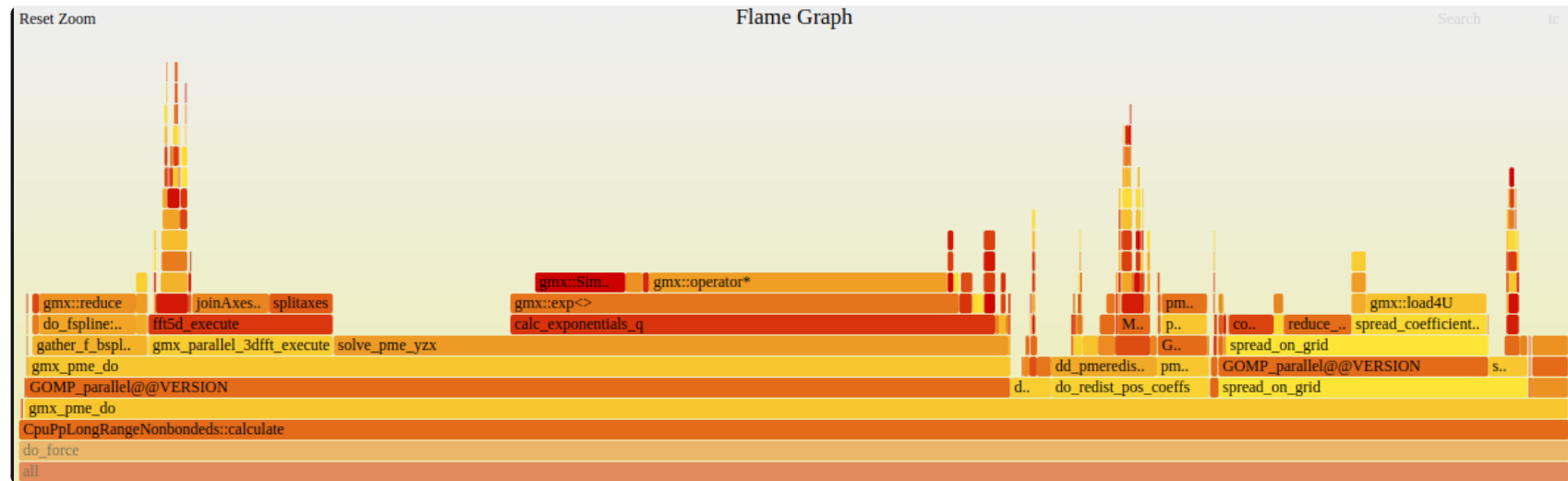
The following parameters can be passed to the script:

Short	Long	Default	Switch Description
	--nsys	Current Nsight Systems CLI installation location	Path to the Nsight Systems CLI directory (e.g., /opt/nvidia/nsight-systems/2022.4.1/target-linux-x64).
-o	--out	Output is written to stdout	Path to a result file containing a data suitable for flamegraph.pl.
	--full_function_names	False	Use full function names with return type, arguments and expanded templates, if available.

Note

By default, the script tries to shorten function definitions (eliminating return type, arguments and templates). In some complex cases shortening may fail and return a full function definition. To disable shortening defining `--full_function_names=False` argument can be used.

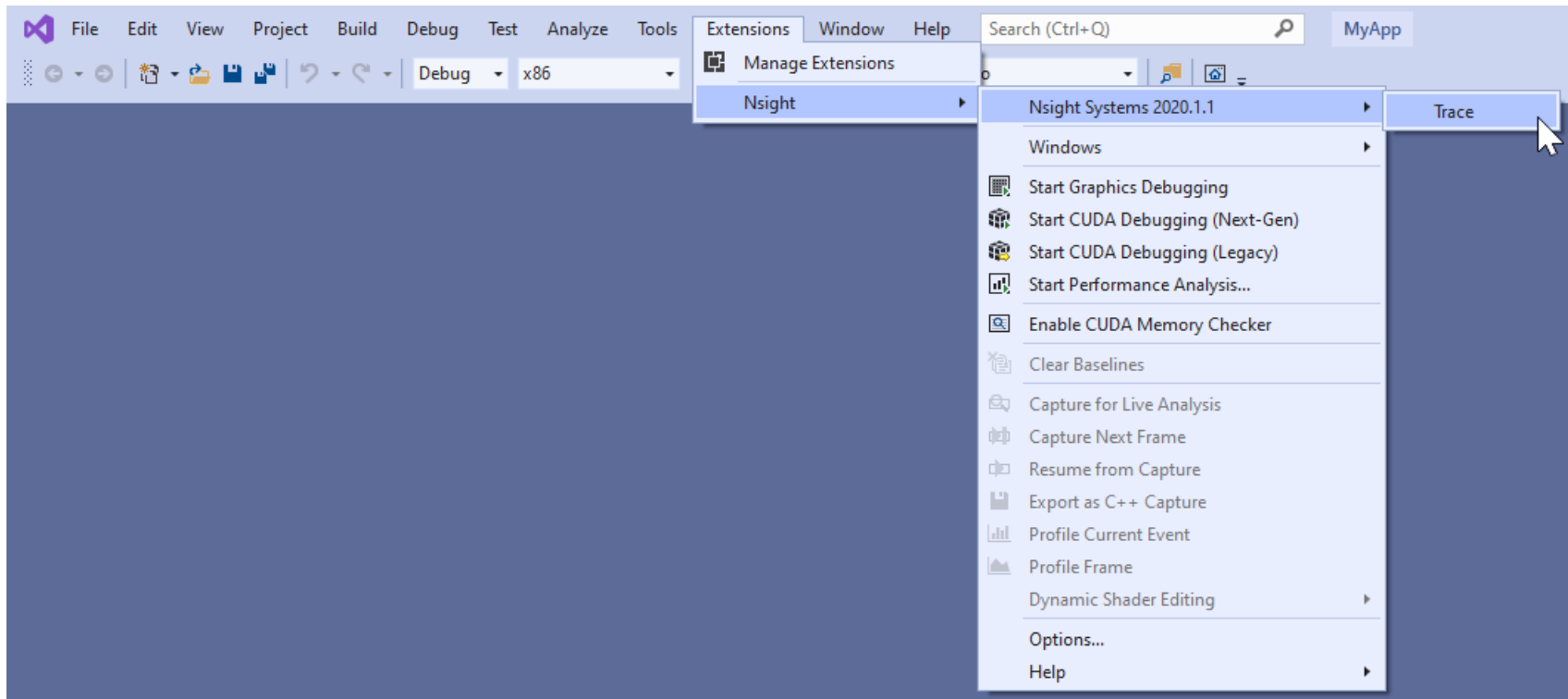
Here is an example of a Flame Graph generated from an Nsight Systems report. The program was a debug build of GROMACS, running on two ranks, each running two OpenMP threads.



Visual Studio Integration#

NVIDIA Nsight Integration is a Visual Studio extension that allows you to access the power of Nsight Systems from within Visual Studio.

When Nsight Systems is installed along with NVIDIA Nsight Integration, Nsight Systems activities will appear under the NVIDIA Nsight menu in the Visual Studio menu bar. These activities launch Nsight Systems with the current project settings and executable.



Selecting the “Trace” command will launch Nsight Systems, create a new Nsight Systems project and apply settings from the current Visual Studio project:

- Target application path
- Command line parameters
- Working folder

If the “Trace” command has already been used with this Visual Studio project then Nsight Systems will load the respective Nsight Systems project and any previously captured trace sessions will be available for review using the Nsight Systems project explorer tree.

For more information about using Nsight Systems from within Visual Studio, please visit

- [NVIDIA Nsight Integration Overview](#)
- [NVIDIA Nsight Integration User Guide](#)

Troubleshooting#

General Troubleshooting#

Profiling

If the profiler behaves unexpectedly during the profiling session, or the profiling session fails to start, try the following steps:

- Close the host application.
- Restart the target device.
- Start the host application and connect to the target device.

Nsight Systems uses a settings file (`NVIDIA Nsight Systems.ini`) on the host to store information about loaded projects, report files, window layout configuration, etc. Location of the settings file is described in the **Help** → **About** dialog. Deleting the settings file will restore Nsight Systems to a fresh state, but all projects and reports will disappear from the Project Explorer.

Profiling Games

In launcher-based platforms (like Steam), if you attempt to run the game executable directly from Nsight Systems the game will detect that the launcher is missing. It will therefore launch the launcher and then self-terminate.

To avoid this, Nsight Systems on Windows automatically attaches to child processes spawned by the target process that Nsight Systems launched. Instead of setting Nsight Systems to launch the game, set Nsight Systems to launch the game platform client, and use the client GUI to launch the game.

Nsight Systems is configured to ignore the game platform client and launcher apps, and will attach to the game executable that the client launches.

An example workflow:

1. Verify the Steam client is not running. Select the Quit command to terminate Steam if it is running.

2. Configure Nsight Systems to launch the Steam client with a manual collection option. It is recommended you check the hotkey checkbox to begin data collection from within the game without requiring you to switching window focus.
3. Click Start. Nsight Systems will launch the Steam client.
4. Use Steam GUI to launch the game.
5. When the game is running and you have reached the scene you want to profile, press the F12 hotkey to start Nsight Systems data collection. Let the game continue running while Nsight Systems collects its profiling data (typically 10-60 seconds, or however long is relevant for you). Press F12 again to end the collection.

Environment Variables

By default, Nsight Systems writes temporary files to /tmp directory. If you are using a system that does not allow writing to /tmp or where the /tmp directory has limited storage you can use the TMPDIR environment variable to set a different location. An example:

```
TMPDIR=/testdata ./bin/nsys profile -t cuda matrixMul
```

Environment variable control support for Windows target trace is not available, but there is a quick workaround:

- Create a batch file that sets the env vars and launches your application.
- Set Nsight Systems to launch the batch file as its target; i.e., set the project settings target path to the path of batch file.
- Start the trace. Nsight Systems will launch the batch file in a new cmd instance and trace any child process it launches. In fact, it will trace the whole process tree whose root is the cmd running your batch file.

WebGL Testing

Nsight Systems cannot profile using the default Chrome launch command. To profile WebGL please follow the following command structure:

```
"C:\Program Files (x86)\Google\Chrome\Application\chrome.exe"  
--inprocess-gpu --no-sandbox --disable-gpu-watchdog --use-angle=gl  
https://webgl.samples.org/aquarium/aquarium.html
```

Common Issues with QNX Targets

- Make sure that `trace logger` utility is available and can be run on the target.
- Make sure that `/tmp` directory is accessible and supports sub-directories.
- When switching between Nsight Systems versions, processes related to the previous version, including profiled applications forked by the daemon, must be killed before the new version is used. If you experience issues after switching between Nsight Systems versions, try rebooting the target.

CLI Troubleshooting#

.nsys-rep file will not load

If you have collected a report file using the CLI and the report will not open in the GUI, check to see that your GUI version is the same or greater than the CLI version you used. If it is not, download a new version of the Nsight Systems GUI and you will be able to load and visualize your report.

This situation occurs most frequently when you update Nsight Systems using a CLI only package, such as the package available from the NVIDIA HPC SDK.

.nsys-rep file not generated

The CLI initially generates a `.qdstrm` file. The `.qdstrm` file is an intermediate result file, not intended for multiple imports. It needs to be processed. Usually this happens automatically. If it does not, you can use the standalone `QdstrmImporter` utility to generate an optimized `.nsys-rep` file. You can then use this file to visualize locally, to open the result on a different machine, or for sharing results with teammates.

The CLI and `QdstrmImporter` versions must match to convert a `.qdstrm` file into a `.nsys-rep` file. This `.nsys-rep` file can then be opened in the same version or more recent versions of the GUI.

To run `QdstrmImporter` on the host system, find the `QdstrmImporter` binary in the `Host-x86_64` directory in your installation. `QdstrmImporter` is available for all host platforms. See options below.

To run `QdstrmImporter` on the target system, copy the `Linux Host-x86_64` directory to the target Linux system or

install Nsight Systems for Linux host directly on the target. The Windows or macOS host QdstrmlImporter will not work on a Linux Target. See options below.

Short	Long	Parameter	Description
-h	--help		Help message providing information about available options and their parameters.
-v	--version		Output QdstrmlImporter version information
-i	--input-file	filename or path	Import .qdstrm file from this location.
-o	--output-file	filename or path	Provide a different file name or path for the resulting .nsys-rep file. Default is the same name and path as the .qdstrm file.

Launch Processes in Stopped State[#]

In many cases, it is important to profile an application from the very beginning of its execution. When launching processes, Nsight Systems takes care of it by making sure that the profiling session is fully initialized before making the `exec ( )` system call on Linux.

If the process launch capabilities of Nsight Systems are not sufficient, the application should be launched manually, and the profiler should be configured to attach to the already launched process. One approach would be to call `sleep ( )` somewhere early in the application code, which would provide time for the user to attach to the process in Nsight Systems Embedded Platforms Edition, but there are two other more convenient mechanisms that can be used on Linux, without the need to recompile the application. (Note that the rest of this section is only applicable to Linux-based target devices.)

Both mechanisms ensure that between the time the process is created (and therefore its PID is known) and the time

any of the application's code is called, the process is stopped and waits for a signal to be delivered before continuing.

LD_PRELOAD#

The first mechanism uses LD_PRELOAD environment variable. It only works with dynamically linked binaries, since static binaries do not invoke the runtime linker, and therefore are not affected by the LD_PRELOAD environment variable.

- For ARMv7 binaries, preload
`/opt/nvidia/nsight_systems/libLauncher32.so`
- Otherwise if running from host, preload
`/opt/nvidia/nsight_systems/libLauncher64.so`
- Otherwise if running from CLI, preload
`[installation_directory]/libLauncher64.so`

The most common way to do that is to specify the environment variable as part of the process launch command, for example:

```
$ LD_PRELOAD=/opt/nvidia/nsight_systems/libLauncher64.so ./my-aarch64-binary --arguments
```

When loaded, this library will send itself a SIGSTOP signal, which is equivalent to typing `Ctr l+Z` in the terminal. The process is now a background job, and you can use standard commands like `jobs`, `fg` and `bg` to control them. Use `jobs -l` to see the PID of the launched process.

When attaching to a stopped process, Nsight Systems will send SIGCONT signal, which is equivalent to using the `bg` command.

Launcher#

The second mechanism can be used with any binary. Use `[installation_directory]/launcher` to launch

your application, for example:

```
$ /opt/nvidia/nsight_systems/launcher ./my-binary --arguments
```

The process will be launched, daemonized, and wait for SIGUSR1 signal. After attaching to the process with Nsight Systems, the user needs to manually resume execution of the process from command line:

```
$ pkill -USR1 launcher
```

Note

Note that `pkill` will send the signal to any process with the matching name. If that is not desirable, use `kill` to send it to a specific process. The standard output and error streams are redirected to `/tmp/stdout_<PID>.txt` and `/tmp/stderr_<PID>.txt`.

The launcher mechanism is more complex and less automated than the `LD_PRELOAD` option, but gives more control to the user.

GUI Troubleshooting#

Empty or Black Pages in Analysis or Diagnostics Summary#

If the **Analysis Summary** or **Diagnostics Summary** pages appear empty or black when running Nsight Systems, this may be caused by rendering issues, often related to drivers for OpenGL or Vulkan.

To resolve this, try running Nsight Systems with the following command:

```
QTWEBENGINE_CHROMIUM_FLAGS="--no-sandbox" QMLSCENE_DEVICE=softwarecontext  
[installation_path]/host-linux-[arch]/nsys-ui
```

xcb-cursor0 or libxcb-cursor0 is needed to load the Qt xcb platform plugin#

If you encounter the following error, you may be missing the required *xcb-cursor* package:

qt.qpa.plugin: From 6.5.0, xcb-cursor0 or libxcb-cursor0 is needed to load the Qt xcb platform plugin.

This issue typically occurs on RHEL but may also affect other distributions. To resolve it, install the required *xcb-cursor* package based on your OS:

- **RHEL/CentOS/Fedora:**

```
sudo dnf install -y xcb-util-cursor
```

- **OpenSUSE:**

```
sudo dnf install -y xcb-util-cursor
```

- **Debian/Ubuntu:**

```
sudo apt-get install -y libxcb-cursor0
```

Other Libraries Loading Errors#

If opening the Nsight Systems Linux GUI fails with one of the following errors, you may be missing some required libraries:

This application failed to start because it could not find or load the Qt platform plugin "xcb" in "". Available platform plugins are: xcb. Reinstalling the application may fix this problem.

or

error while loading shared libraries: [library_name]: cannot open shared object file: No such file or directory

Ubuntu and CentOS With Root Privileges#

- Launch the following command, which will install all the required libraries in system directories:

```
[installation_path]/host-linux-[arch]/Scripts/DependenciesInstaller/install-dependencies.sh
```

- Launch the Linux GUI as usual.

Ubuntu and CentOS Without Root Privileges#

- Choose the directory where dependencies will be installed (`dependencies_path`). This directory should be writeable for the current user.
- Launch the following command (if it has already been run, move to the next step), which will install all the required libraries in `[dependencies_path]`:

```
[installation_path]/host-linux-[arch]/Scripts/DependenciesInstaller/install-dependencies-without-root.sh  
[dependencies_path]
```

- Further, use the following command to launch the Linux GUI:

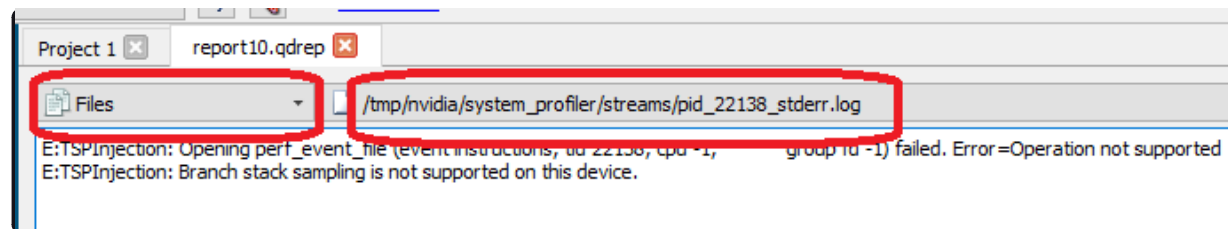
```
source [installation_path]/host-linux-[arch]/Scripts/DependenciesInstaller/setup-dependencies-environment.sh  
[dependencies_path] && [installation_path]/host-linux-[arch]/nsys-ui
```

Other platforms, or if the previous steps did not help#

Launch Nsight Systems using the following command line to determine which libraries are missing and install them.

```
$ QT_DEBUG_PLUGINS=1 [installation_path]/host-linux-[arch]/nsys-ui
```

If the workload does not run when launched via Nsight Systems or the timeline is empty, check the `stderr.log` and `stdout.log` (click on drop-down menu showing **Timeline View** and click on **Files**) to see the errors encountered by the app.



Symbol Resolution#

If stack trace information is missing symbols and you have a symbol file, you can manually re-resolve using the ResolveSymbols utility. This can be done by right-clicking the report file in the Project Explorer window and selecting “Resolve Symbols...”.

Alternatively, you can find the utility as a separate executable in the [installation_path]\Host directory. This utility works with ELF format files, with Windows PDB directories and symbol servers, or with files where each line is in the format <start><length><name>.

Short	Long	Argument	Description
-h	--help		Help message providing information about available options.
-l	--process-list		Print global process IDs list
-s	--sym-file	filename	Path to symbol file
-b	--base-addr	address	If set then <start> in symbol file is treated as relative address starting from this base address
-p	--global-pid	pid	Which process in the report should be resolved. May be omitted if there is only one process in the report.
-f	--force		This option forces use of a given symbol file.
-i	--report	filename	Path to the report with unresolved symbols.
-o	--output	filename	Path and name of the output file. If it is omitted then “resolved” suffix is added to the original filename.

Short	Long	Argument	Description
-d	--directories	directory paths	List of symbol folder paths, separated by semi-colon characters. Available only on Windows.
-v	--servers	server URLs	List of symbol servers that uses the same format as <code>_NT_SYMBOL_PATH</code> environment variable, i.e. <code>srv*<LocalStore>*<SymbolServerURL></code> . Available only on Windows.
-n	--ignore-nt-sym-path		Ignore the symbol locations stored in the <code>_NT_SYMBOL_PATH</code> environment variable. Available only on Windows.

Broken Backtraces on Tegra[#]

In Nsight Systems Embedded Platforms Edition, in the symbols table there is a special entry called **Broken backtraces**. This entry is used to denote the point in the call chain where the unwinding algorithms used by Nsight Systems could not determine what is the next (caller) function.

Broken backtraces happen because there is no information related to the current function that the unwinding algorithms can use. In the Top-Down view, these functions are immediate children of the Broken backtraces row.

One can eliminate broken backtraces by modifying the build system to provide at least one kind of unwind information. The types of unwind information, used by the algorithms in Nsight Systems, include the following:

For ARMv7 binaries:

- DWARF information in ELF sections: `.debug_frame`, `.zdebug_frame`, `.eh_frame`, `.eh_frame_hdr`. This information is the most precise. `.zdebug_frame` is a compressed version of `.debug_frame`, so at most one of them is typically present. `.eh_frame_hdr` is a companion section for `.eh_frame` and might be absent.

Compiler flag: `-g`.

- Exception handling information in EHABI format provided in `.ARM.exidx` and `.ARM.extab` ELF sections. `.ARM.extab` might be absent if all information is compact enough to be encoded into `.ARM.exidx`.

Compiler flag: `-funwind-tables`.

- Frame pointers (built into the `.text` section).

Compiler flag: `-fno-omit-frame-pointer`.

For Aarch64 binaries:

- DWARF information in ELF sections: `.debug_frame`, `.zdebug_frame`, `.eh_frame`, `.eh_frame_hdr`. See additional comments above.

Compiler flag: `-g`.

- Frame pointers (built into the `.text` section).

Compiler flag: `-fno-omit-frame-pointer`.

The following ELF sections should be considered empty if they have size of 4 bytes: `.debug_frame`, `.eh_frame`, `.ARM.exidx`. In this case, these sections only contain termination records and no useful information.

For GCC, use the following compiler invocation to see which compiler flags are enabled in your toolchain by default (for example, to check if `-funwind-tables` is enabled by default):

```
$ gcc -Q --help=common
```

For GCC and Clang, add `###` to the compiler invocation command to see which compiler flags are actually being used.

Since EHABI and DWARF information is compiled on per-unit basis (every `.cpp` or `.c` file, as well as every static library, can be built with or without this information), presence of the ELF sections does not guarantee that every function has necessary unwind information.

Frame pointers are required by the Aarch64 Procedure Call Standard. Adding frame pointers slows down execution

time, but in most cases the difference is negligible.

Debug Versions of ELF Files#

Often, after a binary is built, especially if it is built with debug information (`-g` compiler flag), it gets stripped before deploying or installing. In this case, ELF sections that contain useful information, such as non-export function names or unwind information, can get stripped as well.

One solution is to deploy or install the original unstripped library instead of the stripped one, but in many cases this would be inconvenient. Nsight Systems can use missing information from alternative locations.

For target devices with Ubuntu, see [Debug Symbol Packages](#). These packages typically install debug ELF files with `/usr/lib/debug` prefix. Nsight Systems can find debug libraries there, and if it matches the original library (e.g., the built-in `BuildID` is the same), it will be picked up and used to provide symbol names and unwind information.

Many packages have debug companions in the same repository and can be directly installed with APT (`apt-get`). Look for packages with the `-dbg` suffix. For other packages, refer to the [Debug Symbol Packages](#) wiki page on how to add the debs package repository. After setting up the repository and running `apt-get update`, look for packages with `-dbg` suffix.

To verify that a debug version of a library has been picked up and downloaded from the target device, look in the **Module Summary** section of **Analysis Summary**:

Module summary

Module name	Address	CPU time
[kernel.kallsyms]	0xffffffffc000080000- 0xffffffffc001471010	53.46%
/lib/aarch64-linux-gnu/libc-2.23.so	0x7f7ebad000-0x7f7ecda000	26.04%
/usr/lib/debug/lib/aarch64-linux-gnu/libc-2.23.so		

Logging#

To enable logging on the host, refer to this config file:

host-linux-x64/nvlog.config.template

When reporting any bugs please include the build version number as described in the **Help** → **About** dialog. If possible, attach log files and report (.nsys-rep) files, as they already contain necessary version information.

Verbose Remote Logging on Linux Targets#

Verbose logging is available when connecting to a Linux-based device from the GUI on the host. This extra debug information is not available when launching via the command line. Nsight Systems installs its executable and library files into the following directory:

/opt/nvidia/nsight_systems/

To enable verbose logging on the target device, when launched from the host, follow these steps:

1. Close the host application.
2. Restart the target device.
3. Place nvlog.config from host directory to the /opt/nvidia/nsight_systems directory on target.
4. From SSH console, launch the following command:

```
sudo /opt/nvidia/nsight_systems/nsys --daemon --debug
```
5. Start the host application and connect to the target device.

Logs on the target devices are collected into this file (if enabled):

nsys.log

in the directory where nsys command was launched.

Please note that in some cases, debug logging can significantly slow down the profiler.

Verbose CLI Logging on Linux Targets#

To enable verbose logging of the Nsight Systems CLI and the target application's injection behavior:

1. In the target-linux-x64 directory, rename thenvlog.config.template file to nvlog.config.
2. Inside that file, change the line:

\$ nsys-ui.log

to:

\$ nsys-agent.log
3. Run a collection and the target-linux-x64 directory should include a file named nsys-agent.log.

Note

In some cases, debug logging can significantly slow down the profiler.

Verbose Logging on Windows Targets<#>

Verbose logging is available when connecting to a Windows-based device from the GUI on the host. Nsight Systems installs its executable and library files into the following directory by default:

C:\Program Files\NVIDIA Corporation\Nsight Systems 2023.3

To enable verbose logging on the target device, when launched from the host, follow these steps:

1. Close the host application.
2. Terminate the nsys process.
3. Place nvlog.config from host directory next to Nsight Systems Windows agent on the target device.
 - Local Windows target:
C:\Program Files\NVIDIA Corporation\Nsight Systems 2023.3\target-windows-x64
 - Remote Windows target:

C:\Users\<user name>\AppData\Local\Temp\nvidia\nsight_systems

4. Start the host application and connect to the target device.

Logs on the target devices are collected into this file (if enabled):

nsight-sys.log

in the same directory as Nsight Systems Windows agent.

Note

In some cases, debug logging can significantly slow down the profiler.

Other Resources#

Looking for information to help you use Nsight Systems the most effectively? Here are some more resources you might want to review:

Training Seminars#

NVIDIA Deep Learning Institute Training - Self-Paced Online Course - [Nsight Analysis System: Build Custom Python Analysis Scripts](#)

NVIDIA Deep Learning Institute Training - Self-Paced Online Course - [Optimizing CUDA Machine Learning Codes With Nsight Profiling Tools](#)

CUDA Developer Tools YouTube Channel - [Intro to NVIDIA Nsight Systems](#)

2018 NCSA Blue Waters Webinar - Video Only - [Introduction to NVIDIA Nsight Systems](#)

Blog Posts#

NVIDIA developer blogs, these are longer form, technical pieces written by tool and domain experts.

- 2021 : [Optimizing DX12 Resource Uploads to the GPU Using CPU-Visible VRAM](#)

- 2019 : [Migrating to NVIDIA Nsight Tools from NVVP and nvprof](#)
- 2019 : [Transitioning to Nsight Systems from NVIDIA Visual Profiler / nvprof](#)
- 2019 : [NVIDIA Nsight Systems Add Vulkan Support](#)
- 2019 : [TensorFlow Performance Logging Plugin nvtx-plugins-tf Goes Public](#)
- 2020 : [Understanding the Visualization of Overhead and Latency in Nsight Systems](#)
- 2021 : [Optimizing DX12 Resource Uploads to the GPU Using CPU-Visible VRAM](#)

Feature Videos#

Short videos, only a minute or two, to introduce new features.

- [OpenMP Trace Feature Spotlight](#)
- [Command Line Sessions Video Spotlight](#)
- [Direct3D11 Feature Spotlight](#)
- [Vulkan Trace](#)
- [Statistics Driven Profiling](#)
- [Analyzing NCCL Usage with NVIDIA Nsight Systems](#)

Conference Presentations#

- GTC 2025 - [Optimizing Multi-Language Scientific Simulations: A Grace Superchip Case Study](#)
- GTC 2024 - [Achieving Higher Performance From Your Data Center and Cloud Application](#)
- Jetson Edge AI Developer Days 2023 - [Getting the Most Out of Your Jetson Orin Using NVIDIA Nsight Developer Tools](#)
- GTC 2023 - [Optimizing at Scale: Investigating Hidden Bottlenecks for Multi-Node Workloads](#)

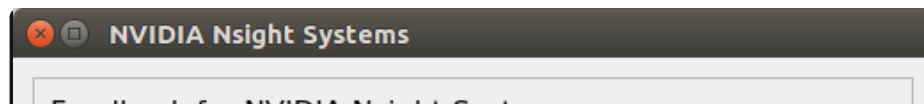
- GTC 2023 - [Optimize Multi-Node System Workloads With NVIDIA Nsight Systems](#)
- GTC 2023 - [Ray-Tracing Development using NVIDIA Nsight Graphics and NVIDIA Nsight Systems](#)
- GTC 2022 - [Killing Cloud Monsters Has Never Been Smoother](#)
- GTC 2022 - [Optimizing Communication with Nsight Systems Network Profiling](#)
- GTC 2022 - [Optimizing Vulkan 1.3 Applications with Nsight Graphics & Nsight Systems](#)
- GTC 2021 - [Tuning GPU Network and Memory Usage in Apache Spark](#)
- GTC 2020 - [Rebalancing the Load: Profile-Guided Optimization of the NAMD Molecular Dynamics Program for Modern GPUs using Nsight Systems](#)
- GTC 2020 - [Scaling the Transformer Model Implementation in PyTorch Across Multiple Nodes](#)
- GTC 2019 - [Using Nsight Tools to Optimize the NAMD Molecular Dynamics Simulation Program](#)
- GTC 2019 - [Optimizing Facebook AI Workloads for NVIDIA GPUs](#)
- GTC 2018 - [Optimizing HPC Simulation and Visualization Codes Using NVIDIA Nsight Systems](#)
- GTC 2018 - Israel - [Boost DNN Training Performance using NVIDIA Tools](#)
- Siggraph 2018 - [Taming the Beast; Using NVIDIA Tools to Unlock Hidden GPU Performance](#)

For More Support#

To file a bug report or to ask a question on the Nsight Systems forums, you will need to register with the NVIDIA Developer Program. See the [FAQ](#). You do not need to register to read the forums.

After that, you can access Nsight Systems [Forums](#) and the [NVIDIA Bug Tracking System](#).

To submit feedback directly from the GUI, go to **Help->Send Feedback** and fill out the form. Enter your email address if you would like to hear back from the Nsight Systems team.



Feedback for NVIDIA Insight Systems

Feature Suggestion ▼

Comments:

Please enter your feedback here...

▶ ☒ Include System Info

▶ ☒ Include Screenshots

▶ **Attach Additional Files:**

Contact Information:

Name:

Email:

By providing your email, you agree that NVIDIA may contact you regarding this feedback. Information you upload will only be used for the purpose of improving NVIDIA software. Please do not include any personal data in your uploads. If you have provided your email, you may request to delete your feedback at devtools-support@nvidia.com at any time.

Send

✕ Cancel